



MYTHBOUND

H046 Computer Science Project
Centre Number: [REDACTED]
Candidate Name: Mohammed Basaad
Candidate Number: [REDACTED]



UNIVERSITY COLLEGE BIRMINGHAM

Contents

Analysis	6
Problem identification	6
Problem overview	6
Computational Thinking/Methods.....	6
Stakeholders	8
Questionnaire	9
Response 1 – Alex, 15 years old	11
Response 2 – Jamie, 17 years old	12
Response 3 – Sara, 14 years old	13
Analysis	15
Research.....	19
Explanation of the different Pegi ratings.....	19
Technical interview	31
Essential Features.....	33
Limitations	35
Hardware and Software requirements	36
Hardware requirements	36
Software requirements.....	37
Success criteria	37
Design	40
Decomposition	40
Overview.....	40
Why top-down design?	40
Data flow diagram	43
Usability features.....	46
Classes	56
Justification of Classes, Variables and Data structures	64
Algorithms	81
Iterative test data.....	89
Scenes	89
Development	91
Assets	91
Constructing the main menu.....	93

Test #1	97
Issue with performance	100
Removing font attribute	105
Test #2	109
Interview with the stakeholders	110
These are the success criteria that were completed with this prototype:	110
Building the practise arena	111
Test #1	115
Image not found error:	115
Test #2	116
Stretch error	116
Test #3	117
Generalizing stages	118
Test #4	123
Creating the character and animation classes	124
Debugging	132
Adding attacks and jumps!	137
Finishing touches	148
Abnormal jumping bug	152
Interview with the stakeholders	153
Implementing a local multiplayer mode	154
Creating a settings menu	154
Testing the settings menu	164
Creating the 2-player mode	166
Testing the 2-player mode	172
Attack frames issue	173
Stage selection menu	175
Testing the stage selection menu	178
Menu glitches	179
Creating a character selection menu	180
Testing the character selection menu	188
Character sliding bug	191
More accurate attacks	196
Creating the HUD	197
Testing the HUD	201
Improving the stage selection menu	206
Samurai character	218

Interview with the stakeholders	222
Implementing LAN mode	223
LAN game class.....	224
Network class	225
Server class	227
Implementing the rest of the LAN game class	230
Testing the LAN game	233
Adding attacks and game logic	236
Testing game logic.....	238
Adding attacks	238
Testing attacks	239
Rising attack edges	242
Testing game logic.....	246
More menus!.....	250
Stage selection	250
Waiting scene	251
Testing the waiting scene	254
Lan game class – note.....	255
Character selection menu – LAN version	255
Testing the character selection menu	259
Editing the lan game class	260
Testing the lan game class	261
Graceful error handling	262
Testing error handling	268
The joining menu	270
Testing the broadcasting.....	274
Testing the joining menu.....	277
The ending banner.....	278
Testing the ending banner	279
Final edits	280
Interview with the stakeholders	281
Evaluation	281
Final results of all tests	282
Usability features in the game.....	293
Final interview with the stakeholders.....	299
Evaluating the success criteria.....	301
Maintenance	305

Limitations	306
Summary	307
Bibliography	309

Analysis

Problem identification

Problem overview

Using the programming language Python and the libraries pygame-ce and socket, I will create a 2D platform fighting game, inspired by super smash bros. The main problem I will be solving within the market of 2D fighters, the lack of accessible free 2D fighter games that are fun and can run-on low-end devices. My project will address this gap by being a free, lightweight and fun alternative with a gentle learning curve that will suit my stakeholders. The stakeholders for my project will be young teens, as they are the main demographic I aim for.

The game will need to have a start-up menu, with multiple different options for playing such as LAN or 2 player on the same computer. The game will need to keep track of the rounds played (max of 3) and decide the winner. The health of both players will also need to be tracked and displayed at the top of the screen.

Computational Thinking/Methods

The problem of making this game is suited to being solved with computational methods as many things can be abstracted from it like realistic graphics, animations, movement etc, also it is suited to being split up into smaller problems like characters, menu and solved using algorithms that allow the computer to display the different features such as character selection, map selection etc. Thinking concurrently is also needed to solve the multiplayer problem as multi-threading will be crucial in the server script to handle the different character movements and such.

Thinking abstractly

Using a computational method called abstraction, I will reduce the complexity of my project to ensure that this game can run on lower-end devices and be free of lag so that I can finish the project. Abstraction means removing unnecessary details and only focusing on key features. I recognise that I will be working on this project without a team, so the project mustn't be too ambitious so that it can be finished.

Therefore, I will abstract features like

- Remove realistic graphics to run on lower end hardware
- No need for a complex physics engine, simplified knockback rules, hit detection and gravity
- Focus on building a run and engaging combat system
- Characters could be abstracted into stickmen or low-res assets to reduce the graphical processing power needed to run the game
- I will limit feature creep so that I can finish this project on time and make it playable on low end devices

This game must run on lower-end devices, as people who end up playing this as a free alternative will also likely not have or will be unwilling to spend money on a better-specified computer.

Thinking ahead

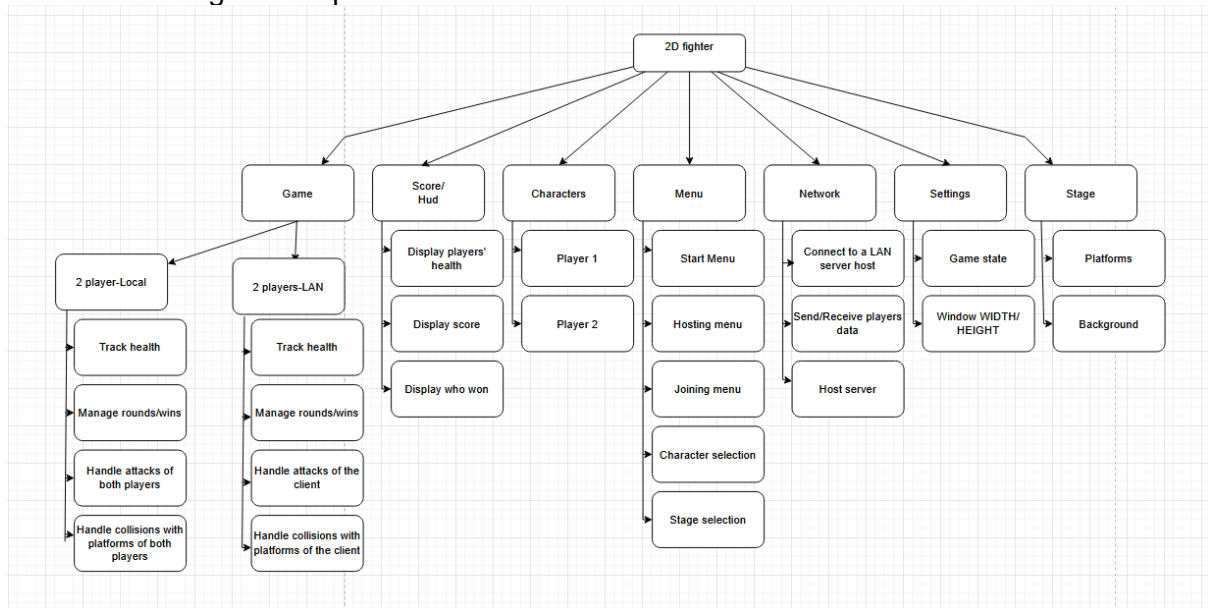
Thinking ahead will also be crucial to my project as it will allow me to plan my project easier because I would have already considered the different features needed and how they will be used.

By thinking ahead:

- I know that my game will multiplayer, which is why it needs to be modular, since some people will be playing on the same device. The module that hosts/connects to a server won't be ran when the 2-player option is selected in the menu
- I know that the OOP paradigm is most suited to this, as having separate classes and functions will make the game code neater for future updates, easier to debug and reusable for future projects
- I will manage error handling effectively to reduce the number of crashes, for example when a network connection fails, the game will exit rather than crash

Thinking procedurally

By thinking procedurally, I can split the main problem of creating my game into separate categories. This will make it easier to plan the project and write code for different subroutines that will work together to produce the correct result



Game: Depending if it is a 2 player local game or on a LAN connection, it will either handle collisions between both players or just the client. It will also track the health of the players and manage the rounds/wins so they can be displayed by the score module.

Score/HUD: This will display the players' health, score onto the screen. It will also display who won at the end of the match

Characters: This will contain many base characters for the user to choose from. The player 1 and 2 classes will be a subclass of this as there are some attributes and methods that will be common against all characters such as taking damage. However, some functions will be overridden like the attack function as that will have different animations based on the character.

Menu: This will be responsible for drawing the start, hosting and joining menu, So that the players can select between the different game modes and characters.

Network: This module will connect to the host if there is one and will send players data and receive the other player's information too. This info will include attributes like position, current animation, animation frame.

Settings: This will store current attributes like key binds for the different controls which will be needed in all the game modes. It will store the width and height of the game.

Stage: This will contain the different kinds of stages. Each stage will have a background and different platforms to be displayed onto the screen.

Thinking logically

Thinking logically will be essential to write all the algorithms in the implementation. For example, in the start menu, a decision will need to be made to select between the different options (Local, LAN, host, join) based on what the user clicks or doesn't in which case the decision will be to do nothing. To determine whether the player needs to move, my program will check whether any key has been pressed and decide. For example, if the user clicks space, the character will jump.

Thinking concurrently

As my game will feature multiplayer, thinking concurrently will be used in the server to have a separate thread to manage the connection to the two players simultaneously. This is needed so that they can play with each other at the same time.

Visualisation

I will also use a computational method called visualisation. This means to present raw data results in a more engaging way that is more easily understood. Visualisation will be used to solve the problem of not knowing exactly what my stakeholders want, more specifically, this will be used in presenting the raw data taken from the questionnaire. This way, I will better understand what my stakeholders want from this project and be able to develop it well.

Stakeholders

The research suggests that the common age group that plays fighter games today is 35-44¹. This is why this genre of games is known as an aging genre, and it may happen because veteran players adapt quickly to new controls, and when a new fighting game is released, new players struggle to learn to master the game and often get discouraged¹. This is why I want to design my game for newer players, to revive the genre, and to invite young teens to join in on the fun, too. By implementing design choices like the simplified controls and offering local and LAN modes, the barrier for entry will be lowered thus inviting my target users to engage with the genre. Therefore, the target audience for my project will be teens between the ages of 13 and 19. Typically, teens wouldn't want/have money to spend on buying games, which could also be another reason for why the age group playing fighter games is 35-44, as adults will typically have more disposable income to spend on games. The fighter games today are quite expensive, this is why I will be making my game free. My stakeholders will be my three friends: Alex, Jamie and Sara. Alex is 15, Jamie is 17 and Sara is 14, as they all have an interest in how my game will be developed as they are the target audience and wish to play it after development. My assessor will also be a stakeholder; however, his view won't be included in the scope of the project. I will also be a stakeholder in this project as I am the developer, and I want to be able to play an enjoyable game afterwards.

The needs of the stakeholders can now be summarised into a list:

- The game must run on low-end hardware
- Free to access
- Easy to learn controls
- Fun multiplayer/local experience

¹ <https://www.gamedeveloper.com/design/fighting-games-a-great-yet-aging-genre>

To consolidate the needs of the stakeholders, it is necessary to have feedback from the stakeholders themselves. This feedback could also show any more needs that were missed in the original list. Using visualisation, I will also organise this data in charts to be able to analyse trends and see what features the stakeholders want to be included. I have chosen a questionnaire rather than an interview because of the limited options, most of the answers will be picked from a set of responses which will allow me to analyse the answers better and use visualisation easier. The open endedness of an interview makes the results hard to visualize. I will still use questions that have more wordy answers for topics that are subjective such as improvements on games they have previously played or if there is anything else they would like to add.

Questionnaire

1. How often do you play video games?
 - Daily / A few times a week / Rarely / Never
2. Which devices do you use to play games? (Tick all that apply)
 - PC / Laptop / Console (e.g. PlayStation, Xbox, Switch) / Mobile / Tablet
3. What genres of games do you enjoy the most?
 - Fighting / Platformers / Shooters / RPG / Puzzle / Sports / Other (please specify)
4. Have you ever played games like Super Smash Bros or Brawlhalla?
 - Yes / No
 - If yes, what do you like or dislike about them?
5. What matters more to you in a game?
 - Smooth gameplay and performance
 - High-end graphics
 - A balance of both
6. Would you be happy playing a game with simple 2D graphics if it runs smoothly on your device?
 - Yes / No / Maybe
7. Do you prefer games that are:
 - Easy to learn but hard to master
 - Challenging from the start
 - Somewhere in between
8. How important is a tutorial or beginner-friendly controls in a fighting game?

- Very important / Somewhat important / Not important

9. Which do you prefer?

- Local multiplayer (on the same device)
- Online multiplayer (over the internet)
- Both equally
- Single player only

10. What problems have you experienced with multiplayer games? (Open-ended)

11. Would you be more likely to play a game if it were:

- Free
- Cheap (under £5)
- I don't mind paying if the game is good

12. Do you have access to a high-spec gaming PC?

- Yes / No / Not sure

13. What features would you like to see in a 2D fighting game? (Tick any)

- Variety of characters
- Power-ups
- Online matchmaking
- Stage variety
- Custom game rules
- Simple controls
- Ranking system
- Other: _____

14. How important are these features to you? (1 = Not important, 5 = Very important)

- Fun combat mechanics
- Easy-to-learn controls
- Smooth performance
- Multiplayer support
- Appealing visuals

15. Would you be willing to test a prototype of this game in future?

- Yes / No / Maybe

16. What name sounds the best for a 2D fighter?
 - Mythbound
 - Combo king
 - Fury league
17. Any other thoughts or suggestions? (Open-ended)

Question choices reasoning

I have chosen to start off by asking them what platform they use to play games and the genres of games they like. This will help me find out the preferred type of controls that I should code for in my project and also whether young teens are already playing fighters or not, which will help me in deciding whether to make the game more beginner friendly by implementing more simplified mechanics. The questionnaire also includes a question about the previous fighter games they have played which gives me a chance to ask about improvements. This will help massively as I can then tailor my project to their needs by building up on the mistakes of popular games. I will then ask them about the graphics and controls requirements to so that I can use abstraction appropriately to keep or remove high quality graphics. It will also help in the design of the controls which are arguably one of the most important features of a fighter game. Complex controls and attacks or simplified easy to learn controls can significantly impact user satisfaction and enjoyment. I will also then ask them whether they have a high spec PC and about the importance of features like stage variety and power ups. If the majority do not have a high spec PC, then designing a game that has great graphics will just make the game experience unenjoyable because of lag. By asking these questions, I am thinking ahead about the design of the project and how that will affect my stakeholders' view of the project. I will now ask my stakeholders to fill out the questionnaire and process their responses.

Response 1 – Alex, 15 years old

1. **How often do you play video games?**
Daily
2. **Which devices do you use to play games?**
Laptop, Console, Mobile
3. **What genres of games do you enjoy the most?**
Fighting, Platformers, Shooters
4. **Have you ever played games like Super Smash Bros or Brawlhalla?**
Yes
5. **If yes, what do you like or dislike about them?**
Like: fast-paced action, cool characters; Dislike: can be too hard to learn at first
6. **What matters more to you in a game?**
Smooth gameplay and performance
7. **Would you be happy playing a game with simple 2D graphics if it runs smoothly on your device?**
Yes
8. **Do you prefer games that are:**
Easy to learn but hard to master
9. **How important is a tutorial or beginner-friendly controls in a fighting game?**
Very important

10. **Which do you prefer?**
Both equally (Local and Online multiplayer)
11. **What problems have you experienced with multiplayer games?**
Lag and long matchmaking times
12. **Would you be more likely to play a game if it were:**
Free
13. **Do you have access to a high-spec gaming PC?**
No
14. **What features would you like to see in a 2D fighting game?**
✓ Variety of characters
✓ Stage variety
✓ Simple controls
✓ Power-ups
15. **How important are these features to you?**
• Fun combat mechanics: **5**
• Easy-to-learn controls: **5**
• Smooth performance: **5**
• Multiplayer support: **5**
• Appealing visuals: **3**
16. **Would you be willing to test a prototype of this game in future?**
Yes
17. **What name sounds the best for a 2D fighter?**
Mythbound
18. **Any other thoughts or suggestions?**
No

Response 2 – Jamie, 17 years old

1. **How often do you play video games?**
A few times a week
2. **Which devices do you use to play games?**
PC, Mobile
3. **What genres of games do you enjoy the most?**
RPG, Fighting, Puzzle
4. **Have you ever played games like Super Smash Bros or Brawlhalla?**
Yes
5. **If yes, what do you like or dislike about them?**
They're fun, but combos are hard to pull off unless you practice a lot. I like Brawlhalla because its free so all of my friends have it but super smash bros is really expensive so I can't play it with all of my friends.
6. **What matters more to you in a game?**
A balance of both (graphics and performance)

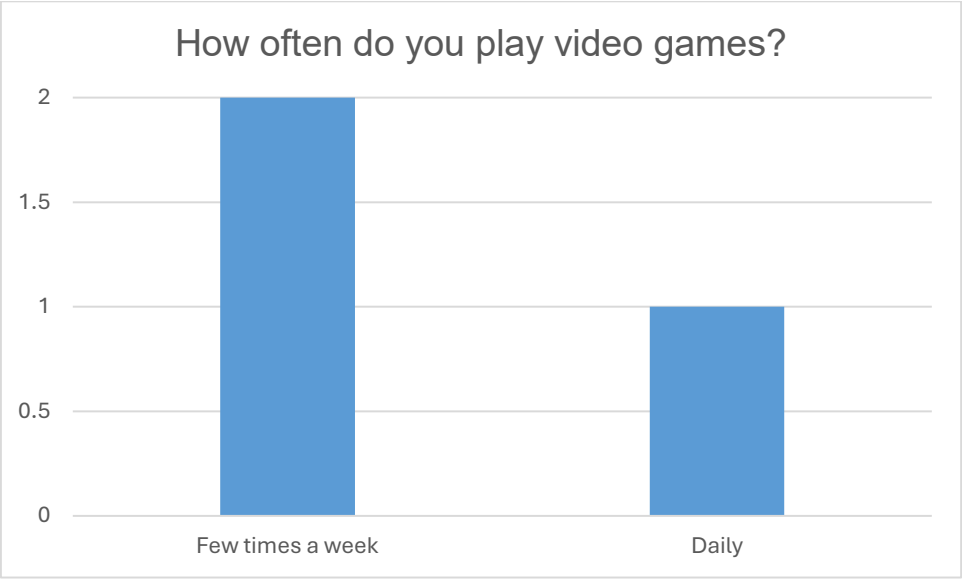
7. **Would you be happy playing a game with simple 2D graphics if it runs smoothly on your device?**
Maybe
8. **Do you prefer games that are:**
Somewhere in between
9. **How important is a tutorial or beginner-friendly controls in a fighting game?**
Somewhat important
10. **Which do you prefer?**
Online multiplayer
11. **What problems have you experienced with multiplayer games?**
Bad servers or unfair matchups
12. **Would you be more likely to play a game if it were:**
Free
13. **Do you have access to a high-spec gaming PC?**
Not sure
14. **What features would you like to see in a 2D fighting game?**
 - ✓ Power-ups
 - ✓ Ranking system
 - ✓ Online matchmaking
 - ✓ Custom game rules
15. **How important are these features to you?**
 - Fun combat mechanics: **4**
 - **Easy-to-learn controls: 4**
 - Smooth performance: **5**
 - Multiplayer support: **5**
 - Appealing visuals: **3**
16. **Would you be willing to test a prototype of this game in future?**
Maybe
17. **What name sounds the best for a 2D fighter?**
Mythbound
18. **Any other thoughts or suggestions?**
Nah.

Response 3 – Sara, 14 years old

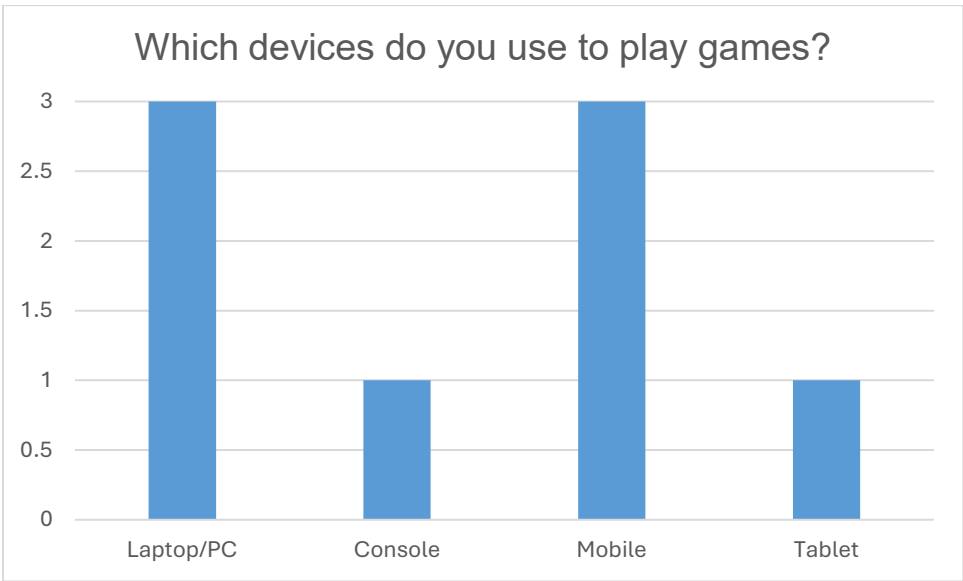
1. **How often do you play video games?**
A few times a week
2. **Which devices do you use to play games?**
PC, Tablet, Mobile
3. **What genres of games do you enjoy the most?**
Platformers, Puzzle
4. **Have you ever played games like Super Smash Bros or Brawlhalla?**
Only Brawlhalla

5. **If yes, what do you like or dislike about them?**
I liked how many different characters there were, but the controls were hard to learn.
6. **What matters more to you in a game?**
Smooth gameplay and performance
7. **Would you be happy playing a game with simple 2D graphics if it runs smoothly on your device?**
Yes
8. **Do you prefer games that are:**
Easy to learn but hard to master
9. **How important is a tutorial or beginner-friendly controls in a fighting game?**
Very important
10. **Which do you prefer?**
Local multiplayer
11. **What problems have you experienced with multiplayer games?**
Too complicated to set up
12. **Would you be more likely to play a game if it were:**
Free
13. **Do you have access to a high-spec gaming PC?**
No
14. **What features would you like to see in a 2D fighting game?**
 - ✓ Simple controls
 - ✓ Power-ups
 - ✓ Stage variety
15. **How important are these features to you?**
 - Fun combat mechanics: **4**
 - Easy-to-learn controls: **5**
 - Smooth performance: **5**
 - Multiplayer support: **2**
 - Appealing visuals: **3**
16. **Would you be willing to test a prototype of this game in future?**
Yes
17. **What name sounds the best for a 2D fighter?**
Mythbound
18. **Any other thoughts or suggestions?**
Add a practice area please.

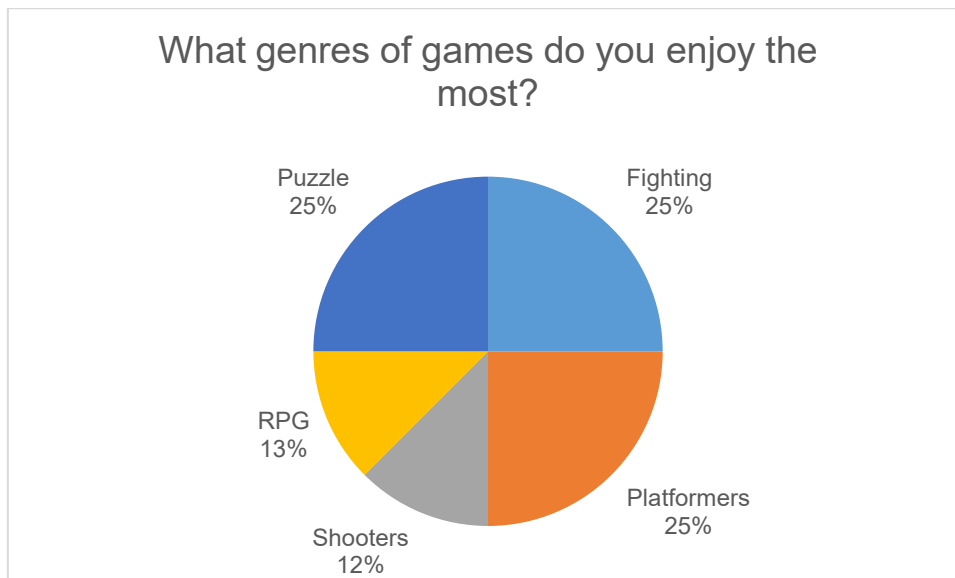
Analysis



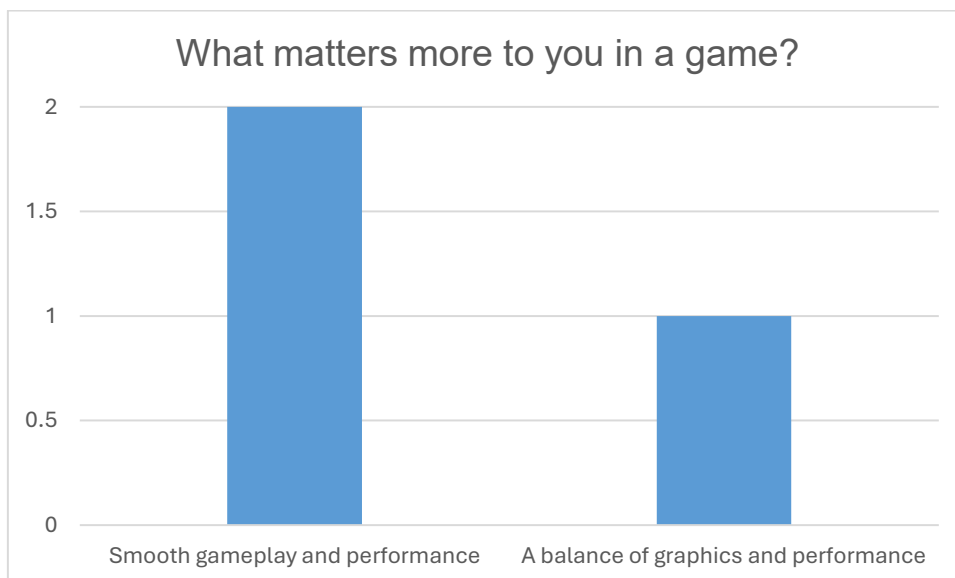
All of responses stated they play games at least A few times a week with one saying daily, all the stakeholders also responded to the second question with a PC or laptop being one of the devices used to play games. These two pieces of information shows that our project is suitable as all the stakeholders play games frequently and on PC.



While mobile is also common between the stakeholders, PC will be the main device I am targeting with my project as I am most familiar with the language, python. Porting over the entire game to mobile requires me to learn a completely different language and will take away focus from my main project.



2 of my stakeholders have also stated that the game genre “fighting” is among the genres they enjoy the most. While this is a broad genre as many games do have some sort of fighting, it does show that stakeholders have a very high chance of responding well to a new 2D fighter game. While Sara hasn’t explicitly stated Fighting, she has mentioned platformers. I will implement this into my project as it is quite suitable for a fighting game, as evidenced by Brawlhalla which is one of the most popular 2D platformer fighter games in history². All stakeholders have also played Super Smash Bros or Brawlhalla. An interesting but expected discovery is that all stakeholders mentioned controls being too hard to learn when asked for the like and dislikes of these games. While, clearly, Brawlhalla and SSB Ult are very successful games (as evidenced by their sales and player base)²³ this highlights the disadvantages of the games and the need to improve them, which can be done with my project, by having simpler controls.

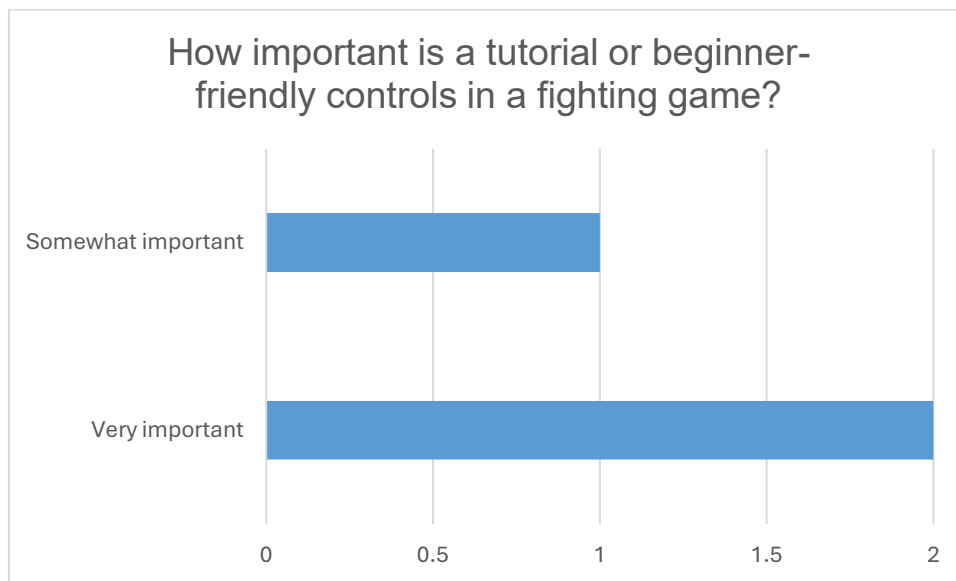


The majority of the stakeholders preferred smooth gameplay and performance which is arguably more important in a fighter game, as the combat is typically fast paced so performance is

² <https://en.wikipedia.org/wiki/Brawlhalla>

³ https://vgsales.fandom.com/wiki/Super_Smash_Bros.

crucial. Therefore, the graphics in my project won't be hyper realistic but will look good enough to enjoy, so that low spec computers can still have smooth gameplay.



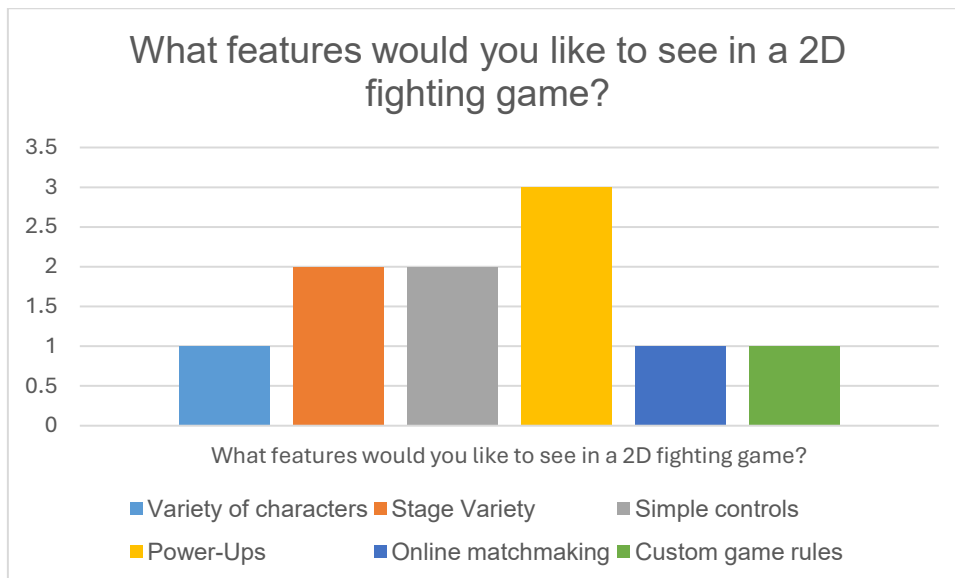
Since two of my stakeholders have agreed that beginner-friendly controls are very important. Easy-to-learn controls have also scored very highly on Q15. This could add to the evidence that younger teens entering the 2D fighter's genre feel overwhelmed because of the complicated controls or lack of a tutorial⁴. I will solve this issue by making my game have simple controls to not overwhelm my target audience.

Both Alex and Sara prefer Local multiplayer, while Jamie prefers Online multiplayer. Online multiplayer could be a potential limitation as I don't have access to servers that can manage the connections of everybody playing. They are very costly to purchase/rent also. However, this could be solved with the use of Hamachi or other networking software that allows your computer to connect to another person's Ip address, thus imitating being on their network and allowing you to play local multiplayer games.

Sara has also stated that the problems she is experiencing with multiplayer games are too complicated to set up, which is why the UI for my game will be simple and easy to use. Alex's problem of long matchmaking times and lag could also possibly solved with the host server running on a separate thread on the host computer. All stakeholders have also stated that they would be more likely to play a game if it were free which makes sense as they are young teens that are less likely to have disposable income to spend on games. My game will be free to all which would appeal to my target audience and stakeholders.

Alex and Sara both said they don't have access to a high-spec gaming PC and all responses ranked appealing visuals 3/5 on how important they are. This is why I will make my game have simple graphics as its unnecessary and will impact smooth performance which was ranked 5/5 importance across the board.

⁴ <https://www.gamedeveloper.com/design/fighting-games-a-great-yet-aging-genre>



The feature with the most votes is power ups followed by stage variety and simple controls. These are the features that, by abstraction, I will focus on the most out of the other 6 that were voted. Online matchmaking is not possible as I do not have access to online servers capable of efficient parallelism.

In summary, the list of my stakeholders needs can now be finalised into:

- The game must run on low-end hardware
- Free to access
- Easy to learn controls
- Fun local experience
- Power-Ups
- Stage variety
- Simple controls
- Variety of characters
- A tutorial
- Practice arena
- Smooth performance
- Runs on PC

Research

Explanation of the different PEGI ratings:



Pegi 3: Games with this rating are considered suitable for all age groups as there may only be very mild and unrealistic violence in a child-like setting. Nudity may be present but only in a completely natural and non-sexual manner, for example during breastfeeding.

Pegi 7: Games may contain unrealistic violence, often directed towards fantasy characters, violence towards human characters will be unrealistic and undetailed, of a minor nature, or only implied. They may also contain sounds that may be scary or frightening to children and may have in-game purchases.

Pegi 12: Games may contain more detailed and realistic-looking violence towards fantasy characters. There may be moderate violence towards human characters. There must not be any sight of blood or injuries, or an emphasis on pain, during violence towards humans. There may be moderate horror sequences such as characters in danger or sight of injuries and dead bodies, provided these are not too gory or graphic.

Pegi 16: Games may contain more realistic and sustained violence against human characters, including sight of blood and injuries. There may also be intense and sustained horror sequences or strong gory images.

Pegi 18: Games can contain very strong content and are only suitable for adults. This could include torture and infliction of severe pain and injury to human characters.

These explanations are taken directly from <https://gamesratingauthority.org.uk/RatingBoard/ratings-system> and shortened.

My game will be Pegi 12, as it will include semi detailed violence towards human characters in the fighting rounds. There won't be any blood as the asset packs I will use won't have any. Pegi

7 doesn't allow deaths where as Pegi 12 does. My game will have deat animations that aren't detailed at the end of a round.

Super Smash Bros. Ultimate Nintendo Switch

Ref: 166095



Screenshot of the Nintendo switch game "Super Smash Bros. Ultimate". The picture shows the MSRP and front cover of the game.

Nintendo is a games company known to price their games very highly as shown in the screenshot above, which can be an issue for people who aren't willing to spend money on games or don't have the money to spend on games, such as young teens. We can also infer that this applies to the majority of people as Super Smash Bros Ultimate has sold 36 million copies⁵ while Brawlhalla which is a free 2D platformer fighter has over 100 million players⁶, although admittedly it released just a year before. Brawlhalla also boasts 80 million players on

⁵ https://vgssales.fandom.com/wiki/Super_Smash_Bros.

⁶ <https://en.wikipedia.org/wiki/Brawlhalla>

steam alone³. In order to appeal to a larger and younger audience, I will make my game free for all to play and enjoy.



However, a feature that I quite like of SSB Ult. is the wide variety of characters from different game verses, all with their own fighting styles, for users to play and enjoy. Following the requirements from the analysis, I want to implement this feature, however not to the same scale. Nintendo can have all these different characters from franchises because of different copyright agreements. While I won't be able to implement the same 101 characters, I can still ensure my game has several different characters to play. This will also make sure to entertain my target audience even when playing the game multiple times as there will be different fighting styles to learn and master.

Street fighter 6 is a very popular 2D fighter game, as evidenced by the majority of reviews (36,221) being "very positive". You can see in the screenshot below the animations of some of

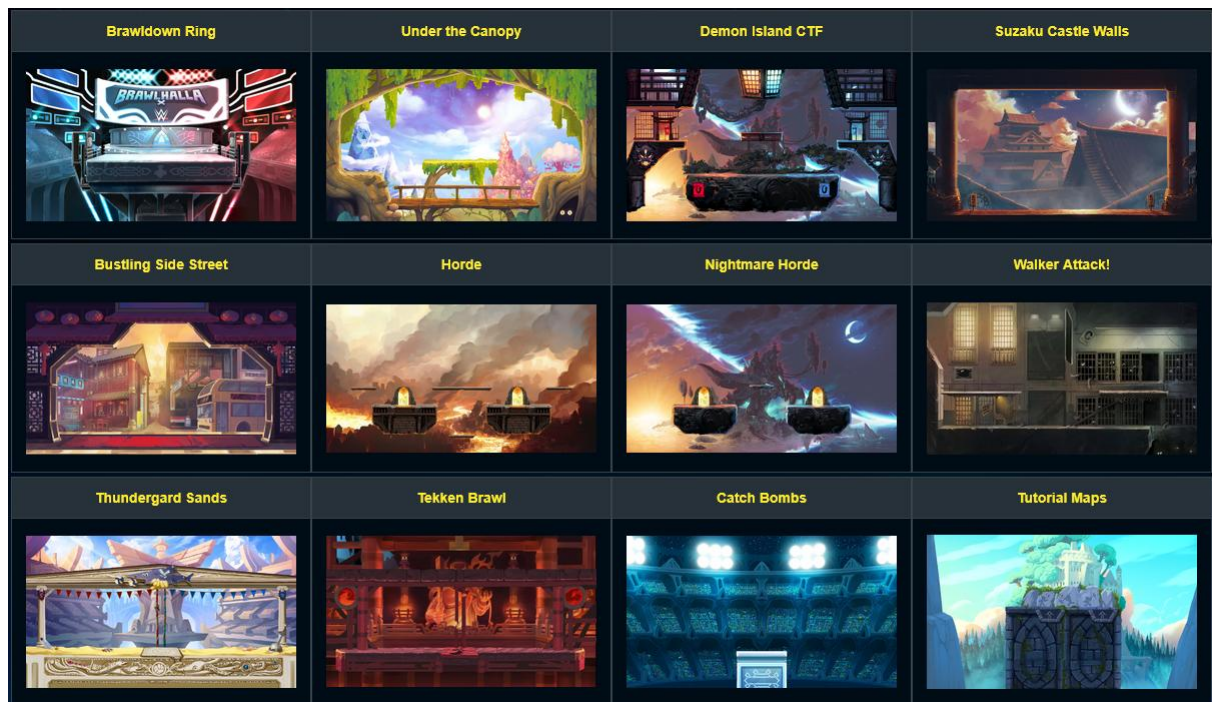
the attacks performed by the characters. I will incorporate animations into my own game to make it feel more realistic and add to the visual element of the game.



A particular design feature that I like and will implement into my own game are the health bars being at the top to the right and left of the timer that is in the middle. This makes it very easy to check yours or your opponent's health and the time left to plan any strategies.



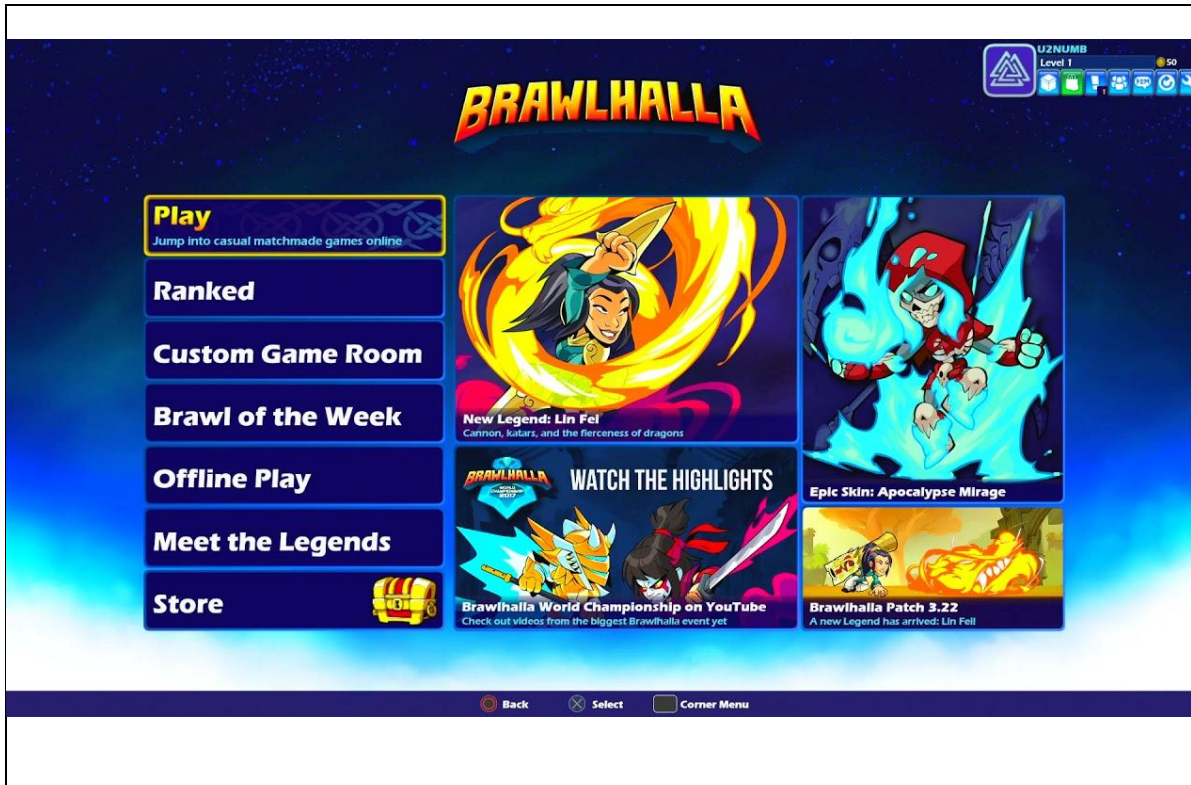
Each match in street fighter 6 follows a best of three system, which I like as it's the perfect balance between too much and too little. The player that wins 2 rounds wins the game, and if there is a draw then the 3rd round is played. Any more than 3 rounds, the players may just get bored and any less, it will feel like it's over too quickly. This system is a perfect balance, as its just enough for people to feel satisfied with the game and not get bored of repetitive matches.



Above is a screenshot of just some of the 2D playable maps that are called “realms” in Brawlhalla. I like this feature as it can make the game fun and not just repetitive. However, because of the size of Brawlhalla maps, the game will automatically zoom in at times on certain attacks so that the player can see their characters better. I will make my maps smaller so the game won’t need to be zoomed in to see the characters, as it can be distracting for my players. The map selection will also be picked before the start of each game.



Street fighter 6 menu, it looks crowded and overwhelming, which can make it difficult to navigate and setup. This is one of the complaints of my stakeholders too, on the other hand, the Brawlhalla menu is much simpler and easy to navigate.



In my own game, rather than having the menu on the left, I will have it in the centre, as my game will be new and not have any news to display. This will simplify the set-up process and be aesthetically pleasing.

Normal Attacks					
Standing Light Punch 5 Frame Startup 2 [3] Active Frames 7 [9] Recovery Frames +5 [+2] On Hit 0 [-3] On Block 0 Details Click to view frame-by-frame of hitbox image 1+ Details	Standing Medium Punch 8 Frame Startup 4 Active Frames 14 Recovery Frames +6 On Hit +2 On Block 25 Total Frames 540 Damage Attack Type: High Cancelable: -- Note: --	Standing Heavy Punch 5, 7, 12 Frame Startup 1, 1, 3 Active Frames 20 Recovery Frames +1 On Hit -3 On Block 34 Total Frames 90, 90, 430 Damage Attack Type: High Cancelable: Second Hit Only: Special, Super Note: --	Standing Light Kick 5 Frame Startup 2 Active Frames 11 Recovery Frames +3 On Hit -1 On Block 17 Total Frames 270 Damage Attack Type: High Cancelable: Chain, Special, Super Note: --	Standing Medium Kick 9 Frame Startup 3 Active Frames 17 (18 on whiff) Recovery Frames +2 On Hit -3 On Block 29 (30 on Whiff) Total Frames 540 Damage Attack Type: High Cancelable: Special, Super Note: Whiffed reversible after active frames on June 2025 patch.	Standing Heavy Kick 15 Frame Startup 3 Active Frames 21 Recovery Frames +3 On Hit -5 On Block 38 Total Frames 720 Damage Attack Type: High Cancelable: Special, Super Note: Crumple of +71 on Punish Counter: Limited Juggle: State on end-of-hit. As of December 2024 patch: "Reduced the hurtbox on the front leg during the attack's start up" was reported in the official patch notes... However, this was never implemented in the "Punish" and "Leg" frames. S. D. Additionally, the hurtbox on the back leg is not implemented in the "Punish" and "Leg" frames.

A feature I don't like in street fighter 6 is the number of attacks per character. It makes the game very complex and hard to pick up and play. For example, a character called Jamie has a total of 48 different variations for 6 different attacks. As I am a solo developer, I will not be able to implement that many types of attacks for each character which is why I will keep it simple and do 2 attacks per character, heavy and light. Furthermore, having that many different types of attacks will overwhelm starting players who are new to the genre, which this game is meant to be catered towards.



I also think that the character selection menu is very appealing in Brawlhalla, all of the characters are at the middle top, and it shows you and the person you're playing with a preview of the character you are hovering over. I will be adding this type of menu to my own game.



All three games have change-able key binds. I will be implementing this into my own game to make it more user and beginner friendly. This will enhance ergonomics as default key binds may not be comfortable for some players.



I like Brawlhalla's menu for customizing controls because it is easier for me, a solo developer, to implement. By focusing on keyboard controls and a simple menu, it is more likely that I will finish my project in the time given to me. Implementing controller support is something that I am certainly not familiar with and may take too much time to code.

From the analysis of frame data of Brawlhalla, Street fighter 6 and SSB, I have gathered the following information⁷⁸:

- All three games are capped at 60 FPS.
- All three games have recovery frames after jumps and attacks that range depending on the damage of the attack.

Street fighter 6:

- Jumping animations consist of 49 frames: 4 pre-jump frames and 45 jumping frames
- Normal attacks, on average, consist of 26.5 frames

Super smash bros ultimate:

- Jumping animations consist of 3 frames but characters jump much higher, so a jump will last for more than a second.
- Normal attacks, on average, consist of 30.75 frames

Brawlhalla:

- Jumping animations consist of 15 frames⁹, usually lasts for about 1.5 seconds, from in-game testing.
- Normal attacks with most weapons have around 25 total frames

Analysis on next page.

⁷ <https://ultimateframedata.com>

⁸

https://docs.google.com/spreadsheets/d/1pnXI_TvDJPeU0ayFbJzvYf7AmrkUcBOUuDegJ5kP7zk/edit?usp=sharing

⁹ <https://brawlhalla.fandom.com/wiki/Movement>

From this analysis, I have decided to also cap my game at 60 frames per second. This is because in fighting games every frame matter, as players take everything into account when playing. Recovery frames, for example, could be the difference between you getting the last hit and winning or losing because you can't attack. To not complicate how long attacks will last for and how the maths will work with higher frame rates, 60 frames is perfect as it is a good standard and measure for all attacks/jumps. Also, as my stakeholders don't have a high spec PC, higher frame rates may be unstable and fluctuate greatly or may give an unfair advantage to people with greater budgets to spend on a PC.

My game will also have recovery frames which are frames that the character recovers in and is unable to attack. This will add some delay and not let the user just spam attacks. I will also add to my game knockback functionality like street fighter 6, to kind of simulate a real fighting environment and not allow users to repeatedly just attack one another until the round ends, it will force users to time attacks properly as you would in any of the other popular and successful titles. This won't be too complicated as the knockbacks won't have a long recovery time, it will be just enough to feel rewarding for the person inflicting it and make the opponent want to fight back and enjoy the game.

The characters' jumps will also last for 1.25 seconds as 2 of the games that I am researching have jumps that last around that time. It will also allow for the characters to jump high enough to be considered an actual jump. Street fighter 6 jumps last a very short time as the jumps aren't very high, they feel quite fast paced too which is why I think 1.25 seconds is a good time. My characters light attacks will also last for around 30 frames (half a second) depending on the assets that I use.

Technical interview

To have a definitive list of the design choices I mentioned above, I will conduct an interview with my stakeholders. This interview will be more technical and focus on specific features of the game such as where the health bar will be.

Interview plan:

Menu:

- Would you like the start menu buttons to be in the centre?
- Would you like a solo practise arena?

Game:

- Would you like to have the ability to pause the game?
- Where would you like the health bars to be?
- How much time should each round be?
- Do you want an animation at the end of the match displaying who won?
- How many rounds should there be?
- Would you like to move the character with A and D or arrow keys?
- Would you like to be able to jump with the space key?
- Would you like the characters to be drawn using pixel art?
- Do you want the option to change key binds?
- Do you want animations for each type of movement (running, jumping, attacking?)

- Who should pick the stage (the person hosting or joining)?

Combat System:

- Do you think that attacks should be cancellable once they have started?
- How many types of attacks should each character have?
 - o What keys should they be activated by?
- Should holding an attack make the character do a different attack or a heavy version?
- How much health should every character have?
- How much damage should each attack do?

Interview results:

Key : Alex = Red, Jamie = Green, Sara = Blue

Menu:

- Would you like the start menu buttons to be in the centre?
Yes. Yes. Yes.
- Would you like a solo practise arena?
Yes, that would be a nice place to try out characters. Yes. Yes!

Game:

- Would you like to have the ability to pause the game?
No that would be cheating. No. No.
- Where would you like the health bars to be?
Top in the middle. At the right and left top corners. Middle top.
- How much time should each round be?
90 seconds. A minute and a half. 90 seconds.
- Do you want an animation at the end of the match displaying who won?
Yeah, like a banner in the middle of the screen with the name at the top. Yes.
Yep, with the picture of the character in the middle of the screen
- How many rounds should there be?
Three. Three. Three.
- Would you like to move the character with A and D or arrow keys?
A and D. A and D. A and D.
- Would you like to be able to jump with the space key?
No, the W key is better. The W key is more suitable. I think the W key is more comfortable as I'm already using A and D which are right next to it.
- Would you like the characters to be drawn using pixel art?
Yes, I don't mind. Yeah. Yes.
- Do you want the option to change key binds?
Yes. Yes. Yes.
- Do you want animations for each type of movement (running, jumping, attacking?)
Yes. Yes. Yes.
- Who should pick the stage (the person hosting or joining)?
Hosting. Hosting. Hosting.

Combat System:

- Do you think that attacks should be cancellable once they have started?
No. No as players would then be forced to time their attacks correctly. No
- How many types of attacks should each character have?
2, a light and heavy attack. 2 to keep it simple. 2.
 - o What keys should they be activated by?
P and L. P and L. P and L.
- Should holding an attack make the character do a different attack or a heavy version?
No they should be fixed. No. No.
- How much health should every character have?
100 is a nice number. 100? 100.
- How much damage should each attack do?
Ranged attacks should deal less but the heavier attacks should deal more but nothing over 50. I don't know. It depends on the attack and character, but it should be balanced.

Essential Features

After consulting my stakeholders, I have made a list of the features based on what they want to see in my project:

Essential feature	Justification
Interactive start menu	Allows the user to navigate through the different menus. It will allow the user to change their key binds in the settings or choose between hosting an online game or joining one. They may also want to play a local 2-player game which an interactive start menu will make possible.
Multiplayer LAN connectivity	This will give the option for users to play with each other on the same network. Multiplayer support was also rated 5 in the first questionnaire. Some kind of multiplayer support is crucial in a fighting game as evidenced by the fact that all of the successful games in my research have multiplayer support.
Pixel art characters that play animations when performing actions.	Pixel art looks good but is also able to perform well on low end PCs. My stakeholders like pixel art and don't have high spec PCs. The characters should also be able to play idle, run, jump and attack animations. This wont be

	computationally expensive as images will only need to be switched at the right times. However, this will add to the game visuals and enhance player experience by having responsive animations.
The character to move if A or D keys are pressed and jump with the W key	The default configuration will be the A and D keys as they are on the left side of the keyboard. This makes perfect sense as most gamers are right-handed which means that the mouse which is needed for more precise control should be used by the dominant hand, which leaves the left hand free for the left side of the keyboard for movement. However, key binds will also be changeable. The W key will be used for jump, it is a convenient button to press as you can move and jump at the same time (Middle finger on W, index on D, ring finger on A).
Changeable key binds	All stakeholders wanted this feature in the technical interview. Also, having the ability to change the controls is important for players that are not comfortable with the default A, D (movement) and P,L(attacks).
Game will feature two types of attacks, heavy and light. Controlled with P and L.	The stakeholders prefer two different types of attacks. This is good as it keeps the controls simple. P and L are good choices because they are far away from the movement keys so the user can move and attack at the same time. The mouse will not be needed in my game so having the movement and attack keys on opposite sides of the keyboard will be comfortable for the players to press. L will be for a light attack, this makes sense as this attack will be used more frequently and is therefore the attack that will be controlled by the index finger on the right hand. P, for the heavy attack, can be controlled by the middle
Keyboard compatibility	All my stakeholders have a PC/Laptop in common, so keyboard compatibility is key for everyone to be able to play the game.
2 Players on the same device can play the game together, using arrow keys and WASD keys. Player 1 will still use	This will allow users without a network connection to play with their friends on the same PC. Playing on the same keyboard can

P and L for attacks, while player 2 will use enter and shift.	feel cramped at times, which is why the arrow keys will be used or player 2's movement as it's on the far-right side of the WASD keys which are used for player 1. Enter and shift usually are quite big keys which will be more comfortable to press, the size of the keys makes up for them being very close to P and L.
Variety of stages	This is needed to add variety to the game and is seen in all of the successful games I have researched. Majority of stakeholders have also stated that a variety of stages is preferred in a 2D fighting game.
Option to be a host or join a game	This is essential as I don't have access to servers, so users will host from their own PCs to play with friends on the same network or by joining friend's host.
Practise arena	For players to practise their attacks and try out different characters. Stakeholders have also confirmed this feature in the technical interview.
Name of 'Mythbound'	I like the sound of the name, and all stakeholders have chosen this name in the first questionnaire.
The person hosting should pick the stage	This feature was decided by the stakeholders, a voting system would not be suitable as there are 2 players so draws could happen. Also, it is fitting that the person hosting the fight should pick the stage.

Limitations

Limitation	Why is this a limitation?	Potential impact of these limitations?	Potential workaround?
LAN multiplayer	My lack of expertise and familiarity with server-client models and multithreading.	A potential impact of this is that there could be lag or a lack of multiplayer functionality completely.	I can watch online tutorials about LAN server-client python models and try to adapt it to my own game.
Variety of characters	I not a particularly good artist and in the time that I have which is less than a year, I doubt it will be enough to completely learn how to make 2D pixel assets properly. I will have to research free assets to use which may be of lower quality	This could lead to poor animation quality and there being less characters. Lack of characters or good animations could also make the combat system feel bad and clunky.	I could research online for copyright-free assets that use that also include animations on sites like itch.io.

Online matchmaking (playing with people across the earth)	I am the only programmer working on this game and have less than year with limited resources. I won't be able to pay for a server to run a script for online matchmaking and game managing. This server will need to be very high-spec and be able to run parallelism effectively. Cost of such servers are too high for me to use.	Users won't be able to play with people that aren't in their house.	Players can use Hamachi (or similar software) to imitate a LAN connection.
Stage variety	As I'm not very good at drawing, I won't be able to draw the maps and backgrounds myself. I will have to look online for assets.	The maps may not be visually impressive which could deduct from the overall graphics	I could use AI to generate stage backgrounds or search online for copyright - free assets to use.
Hardware	There will be many things running and being processed, like collision detection, server handling, processing of the players movement, the background etc. The hardware in the PC may not be able to keep up especially as most of my stakeholders don't have a high spec PC	There may be lag when playing which will make the game less fun and negatively impact the overall game experience	I can make sure to use lower resolution assets so that computer will have to render less when drawing each frame.

Hardware and Software requirements

Hardware requirements

Minimum hardware requirements	
-------------------------------	--

Monitor	To display the game
Mouse	To navigate the menu
Keyboard	To control the player
2 Ghz dual core processor	These are the minimum requirements to run python. Dual cores are also needed for the hosting functionality in LAN multiplayer.
2 GB RAM	Minimum requirements for python
10GB + 1 GB of secondary storage	10Gb are the minimum requirements for installing python, 1 GB will be enough to install pygame-ce and store the assets and python files for the game

Software requirements

Python 3.x	So that the game files can be interpreted and executed. This also includes socket, so no need to install it separately
Pygame-ce	Library is needed to run the game

Success criteria

#	Success Criteria	Justification	Evidence for testing
1	In rounds, the players must be able to control their characters movements using A and D (left and right movement).	This is needed so that users can move their characters. A and D are very common for moving left and right across nearly all games. <i>Stakeholders have also confirmed this in the technical interview.</i>	Video of the character responding correctly to the key presses.
2	When pressing the W key in a LAN game, the character should start the jump animation and move up and then down. The animation should be playing the entire time and for a total of 1.25 seconds.	Every fighting game has a jump functionality; this will be needed to dodge attacks and traverse the stage as the stages will have different raised platforms. The W key is perfect as A and D will be used by the index and ring finger which leaves the middle finger free for the W key.	Video of the character responding correctly to the key presses.

		<i>Stakeholders have also confirmed this in the technical interview.</i>	
3	Each character will have two attacks controlled by P and L. The heavy attack will be L while the light attack will be P.	This allows for simple controls that are also fun to use, having only one attack would be too basic and get repetitive. Players can use these attacks to strategize and execute them at the right times. <i>Stakeholders have also confirmed this in the technical interview.</i>	Video of the character responding correctly to the key presses.
4	There must be a start menu with clickable buttons: 'Play', 'Settings' and 'Quit'.	This will allow the users to navigate between the different menus to play the game. <i>Stakeholders have also confirmed this in the technical interview.</i>	A screenshot of the main menu.
5	There must be a settings menu accessible from the start of the game, that allows the players to change their key binds.	Changing key binds will make players feel more comfortable as they may not like the default keys. <i>Stakeholders have also confirmed this in the technical interview.</i>	A screenshot of the settings menu.
6	There must be a playing menu that allows users to access and play the different play modes. The buttons should include, "Practise", "2-player", "join" and "host".	This will allow the users to navigate between the different menus to play the game. <i>Stakeholders have also confirmed this in the technical interview.</i>	A screenshot of the play menu.
7	The characters must have Animations for all kinds of movement (running, jumping, attacking) that are shown once the character performs them.	This will add to the visual element of the game, making it feel more refined. Having a character that will just seem to	Videos of the characters animations in practise mode.

		stand still while attacking will take away from the user experience. <i>Stakeholders have also confirmed this in the technical interview.</i>	
8	A HUD at the top of the screen must show each player's health (100) and display a timer in the middle which is 90 seconds per round.	Placing the health bars and timer at the top is perfect as it will allow players to see how much health and time, they have left so that they can plan their next move accordingly. <i>Stakeholders have also confirmed this in the technical interview.</i>	Screenshot of a in-game round, showing the HUD at the top of the screen.
9	Before each round there must be a character selection menu for the players to select their characters.	Necessary so that the players can select a character that they wish to play.	Screenshot of the character selection menu.
10	The game must also include a practise arena which allows players to test out characters without any opponents or timer.	This is a great feature suggested by a stakeholder that will allow players to learn attacks and try out different characters.	Video of the practise arena being accessible from the menu and the character playing in the practise arena.
11	There must be a stage selection menu for compatible modes like practise, host and local.	This feature will allow users to pick and play on the stages they want. It will make the game more fun and engaging. All solutions researched had this feature. <i>Stakeholders have also confirmed that only hosts, in online games, should be able to pick the stage in the interview.</i>	Screenshot of the stage selection menu.
12	The game must support LAN multiplayer, allowing users to connect	Part of the essential features, to allow players to play with	A screenshot of the code and a video of the feature working.

	and play with each other from the same network.	their friends on a local network.	
13	In the case of network failures, the game must not crash and instead handle it gracefully by returning to the main menu.	This is good programming practise and will improve the user experience.	Video of a network failure and the game handling it gracefully
14	2-player on the same device must also be supported. This should be accessible in the play menu.	This provides a fun experience for players and allows them to play with their friends on the same keyboard if they don't have a Wi-Fi connection.	A screenshot of the code and a video of the feature working.
15	At the end of each game, there has to be an ending banner that displays who won, for 3 seconds.	Instead of abruptly ending the game, I can display who won the game for 3 seconds. 3 seconds is not too long to feel annoying, but not too short that you can't see who won clearly.	A screenshot of the ending banner.

Design

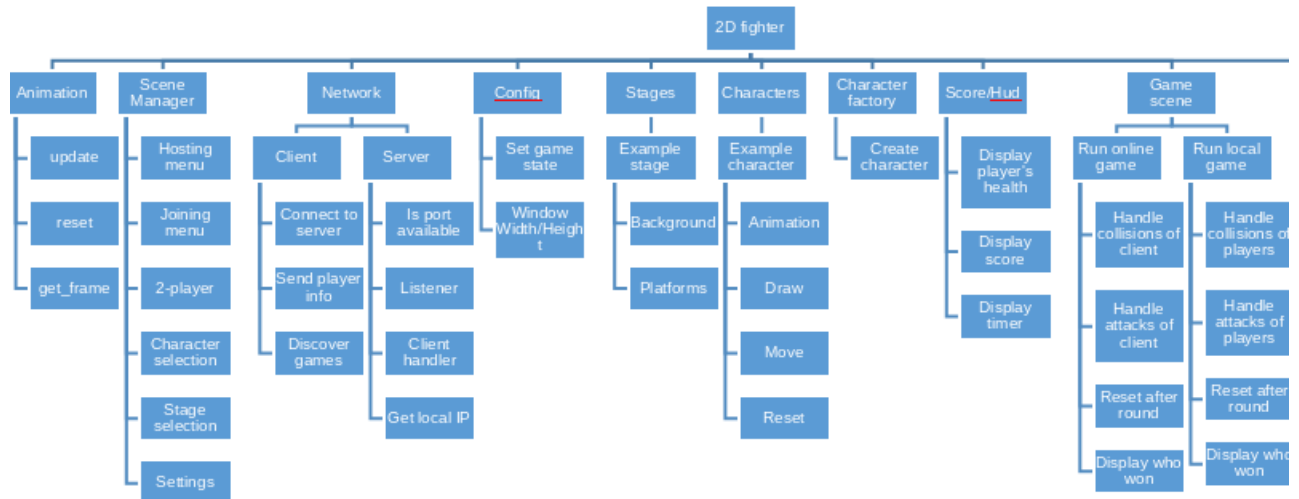
Decomposition

Overview

As I'm not able to fit all of the subproblems here, I will split them up and first show the overview of the game.

Why top-down design?

Stepwise refinement or top-down design will allow me to organise all the code and algorithms but splitting it up into modules and sub modules. This will be very helpful later in the development stage, as I can just follow the design and implement all of the algorithms without thinking about how all the modules will fit together. By splitting up the problem into smaller pieces, this will help in the future updates as all the logic will be split up and easy to follow and update.



Animation : This class will be used in animating the characters by flipping through a list of images for certain actions like 'idle' or 'run'.

Update: This procedure will update the current frame index (so that the image changes) depending whether enough time has passed since the last frame.

Reset: This procedure will simply reset its attributes so that it can be reused. Idle animations won't need to do this as they will loop but attacks will need to be resetted after each use.

Get frame: This will simply return the current image that should be used to draw the character

Menu

Hosting menu: This will get a name inputted by the user and run a server thread that will run the listener subroutine. Once the server connects 2 players, this hosting menu will stop running and switch the game state.

Joining menu: This will get the outputs from the discover games subroutine and display the different options to join onto the screen. Once the user clicks an option, the client should connect to the server and the game state will be switched.

2-player: Once clicked the game state should change to the stage selection menu

Character selection: This should wait for the 2 players to select and confirm their characters and then the game state should be changed to point to the stage selection

Network

Client

Connect to server: This should connect to the socket at the IP address and port that will be passed in as parameters.

Send player info: This function will serialize the player data (position, animation, animation frame, opponent's health) passed in as parameters and send it to the server. It will deserialize and return the other player's data.

Discover games: This function will define a UDP socket on a random port and send out requests to discover games. Responses will be collected and appended to a list which is returned after 3 seconds of waiting for responses.

Server

Is port available: This will attempt to connect to a certain port and return true if the port is free and false if it is not.

Listener: This function will listen on the same port that discover games is using for requests. It will then respond with a name, IP and port for the client to connect to.

Get local Ip: This function will simply return the Ip address of the device running it and will be essential to the server. When hosting, the server will bind to the socket at the IP and port (from trying ports with is port available).

Config

Set game state: This procedure will just set the game state to the parameter that is passed into it

Window width/height: These are global variables that will be used to position the different menu buttons and such

Colours: These different variables will have the colour themes for buttons in the form of (RRR,GGG,BBB)

Stages

Example stage: Example stage class, each type of stage will have its own class

Background: This attribute will hold the background image for the stage

Platforms: This attribute will hold the image/s for the platform/s.

Characters

Example character: Example stage class, each type of character will have its own base class

Animation: This subroutine will handle its own image as its switching frame by frame through the sprite sheets.

Move: This subroutine will take the keys pressed as input and switch the direction that the character is moving in accordingly. It won't change the actual position as that will be handled by draw.

Draw: This subroutine will take the screen as a parameter and call animation and move subroutines within it before drawing the character onto the screen

Reset: This will be responsible for resetting all attributes before the start of each round.

Character Factory

Create character: This will take in the chosen characters in the character selection screen and return the character objects.

Hud

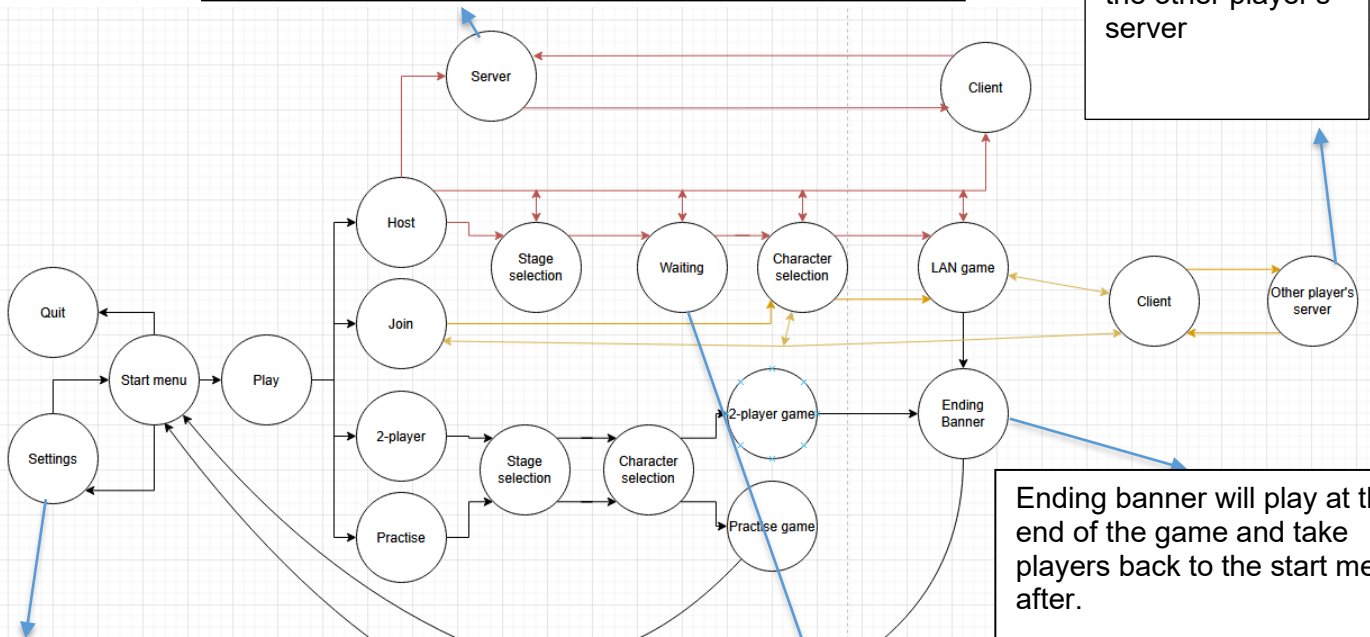
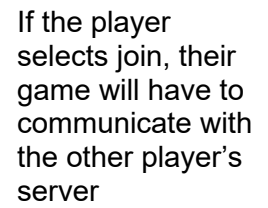
Display players' health: This subroutine will be read the health from the player class and display it as a part of a full health bar.

Display score: This subroutine will display the score for the players.

Display timer: This subroutine will display the timer in the top middle part of the screen.

Data flow diagram

Below is how the user will navigate through the game, this will help me in the development stage by providing a clear path for me to route all the different game states.



Ending banner will play at the end of the game and take players back to the start menu after.

Similarly, player 2 will need to communicate to the host server on player 1 character selection and during the LAN game. Communication info will include stage selection for drawing later, active character selection, host position, animation frame etc.

The host will need to constantly communicate with the server for sending stage selection info, updates on if someone joins the server, player 2's character selection and during the game(player 2 position, animation, animation frame etc).

Information that needs to be routed through the different modules

Start menu:

- Return the game state if button clicked, otherwise return start

Play menu:

- Return the game state if button clicked, otherwise return play

Settings :

- Allows user to change keybinds

Host:

- Startup server thread and client then switch game state to stage selection

Join (needs **network class**):

- Tell network to discover
- Display games
- Allow user to pick game
- Connect to the game's server
- Switch game state to character selection

Stage selection (needs **network class** and **playmode**):

- Draw the menu
- Allow the user to click a stage
- save the stage selection
- Change game state to waiting or character selection based on playmode variable

Waiting (needs **stage** and **network class**):

- Draw stage
- Tell server to listen for connections and send game data (stage selection, ip, port)
- Once accepted then change game state to character selection

Character selection (needs **network class** and **playmode**):

- Send data back and forth between the server (this includes mouse positions, character selections, confirmation data)
- Data will be used by character selection menu class to draw the appropriate characters
- Once both confirmed, change game state to LAN game or 2-player or practise based on playmode variable

LAN game (needs **network class** and **players' info(selections)** and **stage**):

- Send data back and forth (player position, animation, animation frame, health)

- Check number of rounds
- Check scoring
- Switch game state to ending banner

Ending banner (needs **winning player info** and **network class**):

- Draw winning banner
- End network/server
- Go back to start menu

2-player ():

- If this is selected
- change game state to Stage selection

2-player game (needs **network class** and **player's info(selections)** and **stage**):

- Check number of rounds
- Check scoring
- Switch game state to ending banner

Practise game (needs **player's info(selection)** and **stage**):

- Allow user to practise
- if user selects character selection menu
- switch game state to character selection
- if user selects quit
- change game state to main menu

Usability features

Mythbound

Game title in big letters, this makes it clear what game the user is playing. The font of the title is visually appealing.

Play

Settings

Quit

Menu with buttons in the centre as confirmed by the stakeholders in the technical interview. This is also a generic menu choice in games, which makes it very clear how to navigate the game to gamers, but also newcomers as its directly in the centre.

3 menu buttons to keep it simple and clear. This will allow users to navigate to the play section where they can play with friends or practise. They can also change their key binds in the settings or quit the game.

PLAY MENU

Practise

2-player

LAN Join

LAN Host

To meet the success criteria, the game must have practise, 2 player and LAN modes. I have split these into 4 buttons as a player can either host or join a LAN game.

The buttons are centered in the middle to make it clear and easy to read and press the buttons. This is much like Brawlhalla's menu; however they are centered as it's easier to look at the middle of the monitor than the side. This is also unlike street fighter 6's cluttered menu.

Stage choice 1

Stage choice 2

Stage choice 3

Players will be able to select from previews of stages that will be smaller versions of the stages. This will allow players to see the stage and decide if they like it enough to play on it. Stages will be organized horizontally and at the top as initially there will be 3 stages. This will give the illusion of more choices as if they are organized vertically starting from the left, the rest of the screen will look empty, similar to if they were organized in the middle vertically.

This is the intermission waiting screen that runs between the stage and character selection menus if you are hosting.

Bottom left corner to show the background which will be the background of the chosen stage.

Waiting...

'Waiting' will be in big letters so that its clear the host should wait for a player to join.

CHARACTER

Character boxes will make it easy to see the faces of the characters and act as a mini preview.

This menu is designed to be similar to the brawlhalla menu, at the bottom it will show the character that is selected, with the profile boxes for selection at the top

Char
acter

Char
acter

Char
acter

Char
acter

Char
acter

Char
acter

Confirm

Player 1

Confirm buttons will allow the players to confirm their choices. These are placed on top of the characters so that they are easy to access and clearly correspond to each player.

Confirm

Player 2

GAME

If the health bars were placed on the characters, it would make them hard to see and easily confusable with the enemy. This placement keeps them out of the way but still easy to glance at during a round.

P1 - Health

Time

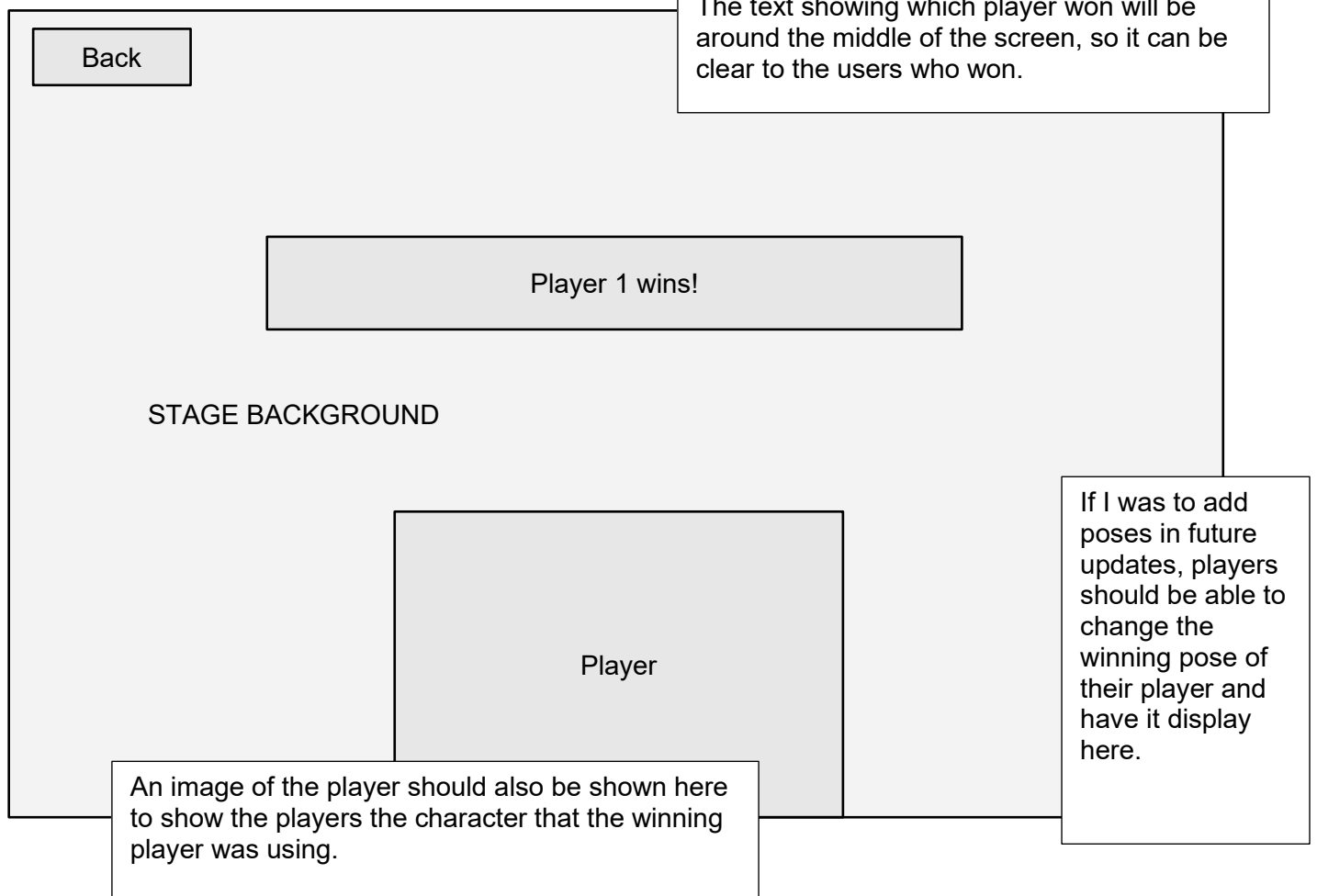
P2 - Health

The time and health will be on the top so that its easy to see how much time and health you have left. It also allows you to see how much health your opponent has. (*Decided by the stakeholders too*)

BACKGROUND

PLATFORMS

ENDING BANNER



PRACTISE

Exit

Character selection

Character selection button will take the player to the character selection menu where they can change their character.

The exit button will allow players to exit to the start menu if they want to quit or start a game after practising.

Buttons placed in the top left corner to not take away screen space from the game.

Background

PLATFORMS

Player 1 and 2's controls are in different columns so that its clear for the player/s which controls belong to who.

Player 1 move left

Player 1 move right

Player 1 jump

Player 1 heavy attack

Player 1 light attack

Player 2 move left

Player 2 move right

Player 2 jump

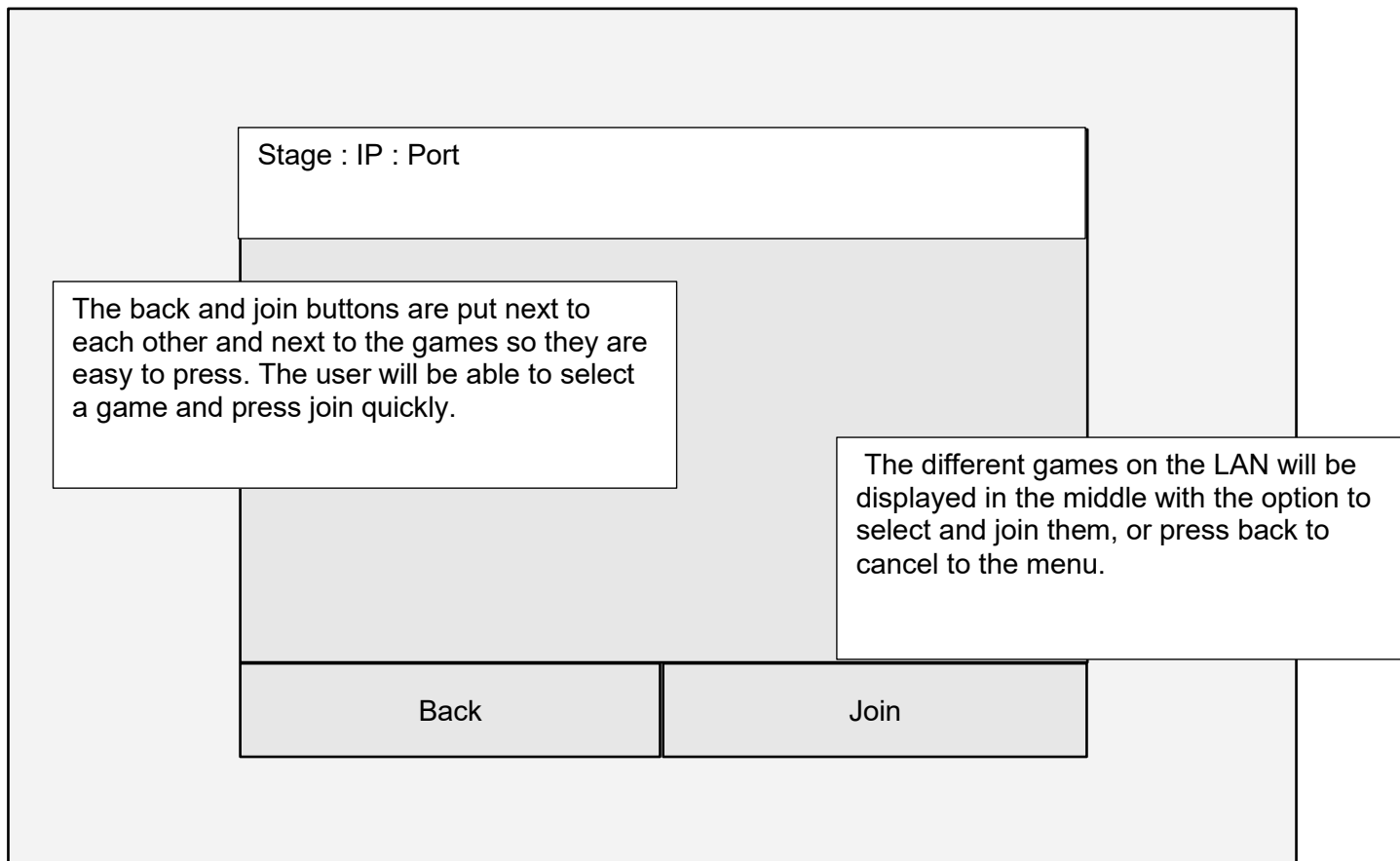
Player 2 heavy attack

Player 2 light attack

Back

The back button is placed at the bottom in the middle as it will be easy to press. It also is separated from the key binds so it is easy to find and press.

JOINING MENU



Classes

StageSelection
attributes
manager : SceneManager network : network class server : server class font : pygame font
operations
update draw

SceneManager
attributes current_scene : scene class network : network class server : server class
operations change_scene update draw

WaitingScene
attributes manager : SceneManager network : network class server : server class font : pygame font
operations update draw

SettingScene
attributes manager : SceneManager network : network class server : server class font : pygame font
operations update draw

prompt_handler	
attributes	
prompts : array	
operations	
create_prompt	
handle	

prompt_handler	
attributes	
prompts : array	
operations	
create_prompt	
handle	

Stage	
attributes	
background	
platforms : array	
operations	
draw	

CharacterSelection
attributes manager : SceneManager network : network class server : server class selected_characters : Array mode : string font : pygame font
operations update draw

Character factory
attributes
operations create

Character
attributes image : image file rect : pygame rect character_type: boolean animations : dictionary health : integer invincible : boolean on_ground : boolean attacking : boolean animation_state : string current_animation : animation knockback: Boolean recovery : Boolean direction : pygame math vector gravity : integer speed : integer facing_left : boolean spawn_point: tuple
operations draw take_damage update move

prompt
attributes time : integer image : pygame surface rect : pygame rect textS : pygame surface textR : pygame rect
operations draw update

GameScene
attributes manager : SceneManager server : server class network : network class game : Game mode : string font : pygame font
operations update draw

PlayMenuScene
attributes manager : SceneManager network : network class server : server class font : pygame font
operations update draw

Hud	
attributes	
player1 : Character	
player2 : Character	
timerSurface : pygame Surface	
timerRect : pygame rect	
healthbarSurface: pygame surface	
healthbarRect: pygame rect	
health2Surface : pygame surface	
health2Rect: pygame rect	
health1Surface : pygame surface	
health1Rect: pygame rect	
operations	
draw	

MenuScene	
attributes	
manager : SceneManager	
network : network class	
server : server class	
font : pygame font	
operations	
update	
draw	

Server
attributes socket IP:string port : integer addr : tuple status : boolean listening: boolean
operations connect shutdown handle_client_thread get_local_ip listener is_port_available

Justification of Classes, Variables and Data structures

Type	Name	Data type	Justification
Class	Animation	N/A	This class will separate the animation logic from the rest of the game. It will also make it easier to customize animations to characters and some attacks, for example may need to run for longer. With the simple change of parameter this can be done. This will make the essential feature of character animations possible too.
Variable	frames	array	This array will store all the images of the character in an animation that need to be drawn so that animations work properly.
Variable	frame_duration	Integer	This integer will store , in milliseconds, the amount of time that each frame should take. If you multiply this by the amount of frames there are, the result of this will be the total animation time. As pygame works on a frame by frame basis, its more compatible to measure how long each frame will take rather than the whole animation, in order to get a smooth fully working animated character.
Variable	loop	Boolean	This Boolean will store whether the animation should be looped or not. Idle animations will have this set to true as idle animations need to be looped.
Variable	index	Integer	This variable will store the index of the image of the character that should be drawn.
Variable	timer	Float	This variable will be used to record the time elapsed since the last frame, so that the index can be incremented.
Variable	done	boolean	This will be returned in the update function if the animation has finished. This is so that attacks don't loop forever and actually finish.

Variable	invincibility	Array	This array will store the indexes of frames that the character should be invincible in. The update function will use this to return whether or not the character should be invincible.
Variable	recovery	Array	This array will store the indexes of frames that the character should be recovering in. The update function will use this to return whether or not the character should be recovering.
Variable	attacking	Array	This array will store the indexes of frames that the character should be attacking in. The update function will use this to return whether or not the character should be attacking.
Procedure	reset	N/A	This procedure will simply reset the attributes so that the animation can be reused. This will be needed for attack animations as they can't be looped so they need a way to reset the attributes before the launch of the next attack.
Function	get_frame	N/A	This function will return the current image of the character that needs to be drawn.
Function	update	N/A	This function will increment the index attribute based on whether enough time has passed. In order to make an animation last for a certain amount of time.
Class	Character	N/A	This character class will draw the character, allow it to be moved by the player, update the images for the appropriate animation, check collisions with platforms(so player doesn't fall through the screen) and allow the player to fight the opponent. It is necessary as it will split the character logic from the rest of the game and make it easier to debug and write.

Variable	character_type	string	This will hold the type of character that the user selected so that it can be passed to the opponent via the server. This will allow the players' games to draw their opponents and opponent's attacks/movements correctly.
Variable	animations	Dictionary	Python dictionaries will be very useful, as now I can store an entire sprite sheet within one tag for example animations['idle'] will contain a list of all the images that need to be drawn for the idle animation. Very convenient and will be useful in the animation logic algorithms as I won't have to use a lot of if statements.
Variable	health	Integer	This variable will store the health for the character, which will be used in the HUD and sending to the opponent via server.
Variable	Invincible	Boolean	This will be set to true during certain frames of some animations, making the character invincible for a very short period. These are called eye-frames and are needed for fairness. If the character is performing a heavy attack, then towards the end of the attack, the character should be invincible to allow the finishing of the attack without any knockback.
Variable	On_ground	Boolean	This will be used for jumping logic, and won't allow the user to jump when in the air (that would be flying which of course should not be allowed). This will be set true as soon as the character collides with any platform's top.
Variable	Attacking	Boolean	This will be set true during the frames of an animation where the character is actually attacking, needed so that during these frames if the player's collides, then

			the other player should take damage.
Variable	Animation_state	String	This will hold the actual state that the player is in like 'idle' or 'jump'. Needed so that the animations dictionary can be accessed with this variable rather than many if statements which writing the animation logic easier.
Variable	current_animation	Animation	This will hold the current animation that is running on the screen. This variable is very important as it will be changed in the move class and its functions will be called in the update to actually switch the character's images to animate the character.
Variable	Direction	Pygame math vector	Pygame math vector is important and different from tuples. When these vectors are multiplied by n , the values inside are multiplied by n , rather than just returning n copies of that tuple. Its also compatible with changing the positions of rects with these vectors. This makes it easy to change the positions of players when they press a move button and simplifies the logic rather than adding the x and y values separately.
Variable	Gravity	Integer	This value will constantly increase (until it reaches max_fall_speed) and be applied to the y value of the player. With this variable, the jumping mechanics will look more natural as gravity will be set to a negative amount and gradually increase, resulting in a much smoother jump. At the same time, it will also simplify jumping as now the player doesn't need to be gradually moved every frame manually.
Variable	knockback	Boolean	This Boolean value will be activated if invincible is set to False and will be based

			on chance when the character is hit by the opponent. If it is activated then the player won't be able to move or attack for a very short period.
Variable	recovery	Boolean	This value will be set to true after attacks. When it is set to true, the character won't be able to attack for a short period of time. This is for fairness so that players can't just spam attacks.
Variable	Max_fall_speed	Integer	This will be the maximum fall speed that the player can experience and will be used as limiter for gravity. This is needed so that the player doesn't just phase through platforms and go beyond the screen.
Variable	Speed	Integer	This will set the speed (pixels per second) at which the player will move left and right. Having this as a variable will make it easier to apply in the movement section. This also may make it easier in future updates if I want to add power ups, as one variable will have to be changed for a speed powerup.
Variable	Facing_left	Boolean	This will be set to true as soon as the player clicks the left movement button and set to false if the right movement button is clicked. This is needed in so that images can be appropriately flipped vertically to match the way that the character should be facing before being drawn.
Variable	Spawn_point	Tuple	This will store the position at which the player will spawn in the actual game. The x value may be subtracted from the window width depending if the character is a player 1 or 2. This is needed in practise as well, so that as soon as the player falls out of screen, they can be instantly spawned back. They shouldn't be able to

			die in practise, as it is purely for practising attacks and characters
Procedure	Draw	N/A	This will draw the player onto the screen by using the image and rect, so that the user can interact with the game and see their character.
Procedure	Take_damage	N/A	When an opponent attacks the player, health will be deducted using this method. This will allow the game progress and eventually , when the health is 0, have a player win.
Procedure	Update	N/A	This will handle the animations and switch images frame by frame, so that the user can see their player responding to controls which will add to the visual effect of the game. It will also handle the invincible attribute, setting that to true at the appropriate animation frames.
Procedure	Set_animation_state	N/A	Using the various character attributes, this procedure will set the animation_state appropriately. For example if on_ground is true then Jump animation will no longer be played. This will be called in the update procedure and aid that method in handling the animations.
Procedure	Reset	N/A	This will reset all the characters attributes to the default values so a new round can take place which will allow the best of three scoring system to take place.
Procedure	Move	N/A	By responding to user key presses, this will handle all player movement and collisions with platforms. This will also be used to set some attributes like direction, on_ground and facing_left.
Class	Server	N/A	This class will be used when a user decides to host. It will handle all

			communications between 2 players in a LAN game which is why its crucial to this game's most important feature, LAN multiplayer. Its important that this class is separated from the rest of the logic as it is its own entity that will run on a thread in the background.
Variable	Socket	Socket	This is a socket that will take up a specific port on the local IP address. It will bind to this socket when the server is ran and listen for incoming connections (players). This variable is important for settings the protocols for how communication will be handled.
Variable	IP	String	This will contain the IP address the socket will be at. Needed for the socket to be binded. It may also be changed from the default value with a separate function (get_local_ip)which is why it needs to be an attribute.
Variable	Port	Integer	This is the port for the socket. Needed for the socket to be binded. It may also be changed from the default value with a separate function (is_port_available) which is why it needs to be an attribute.
Variable	Status	Boolean	This will contain a Boolean value pertaining to whether the server is still connected to the socket or not. This will be used in handling errors with server connections to gracefully handle these errors to improve user experience.
Variable	Listening	Boolean	As the listener procedure will be running on a separate thread, this variable is needed to be able to close that thread, the main thread (server) will set this to true when in the waiting screen and false when a person has joined

			as the server will be full. UDP will be used as that is what the discover games function will be using to send out requests. Broadcasting can be done with UDP which is why its being used. Broadcasting is essential for discovering online games on the LAN.
Function	Connect	N/A	This function will bind the socket to the addr to start the server up if the status is set to false. This socket will use the TCP protocol as it offers reliable delivery with the order of packets being guaranteed to arrive in the order they're sent. This is essential for animations as animations that are out of sync will look laggy and will ruin game visuals, which is more likely to happen with the UDP protocol.
Procedure	Shutdown	N/A	This will shut the server down when the game is finished, ensuring that the server is not blocking the socket from being used by the same user again.
Procedure	Handle_client_thread	N/a	This will handle the communications between the players in a game (by sending player data back and forth).
Procedure	Get_local_ip	N/A	This will get the current devices ip address so that the server can bind to it at a specific port.
Procedure	Listener	N/A	This procedure will send out server/game details(stage, ip, port) for players to join during the waiting screen.
Class	Network	N/A	This class is essential for the LAN multiplayer feature of the game. It will allow users to connect to the server, the game will then send player data and receive opponent data through this class. Also, this class will be used in the joining menu to discover games on the LAN.

Variable	Socket	Socket	This will define protocols which the game will use to connect and communicate with the server. My game will run on TCP as that offers reliable delivery and the order at which the packets are sent are guaranteed to be received that way. This is essential for animations as out of sync animations ruin game visuals and will look laggy.
Variable	IP	String	This will contain the IP address that the server is running on. Needed in order to connect to it. It may also be changed from the default value with a separate function discover games or in the joining menu which is why it needs to be an attribute.
Variable	Status	Boolean	This will contain a Boolean value pertaining to whether the client is still containing to the server or not. This will be used in handling errors with server connections to gracefully handle these errors to improve user experience.
Variable	Addr	Tuple	This will be used to connect to the server. Contains the IP and Port and needs to be in tuple form.
Variable	Port	Integer	This is the port for the server. Needed to connect to the server. It may also be changed from the default value with a separate function discover games or in the joining menu which is why it needs to be an attribute.
Variable	Discovering	Boolean	This will be needed in the joining menu to signal to the network class when to stop the discover games thread (when user joins a game or leaves menu).
Variable	Discovered	List	Contains a list of all current games' info on the LAN, to display on the joining menu. To allow players to join another person's game.

Procedure	Connect	N/A	This procedure will connect the socket to the server so that the user can play with others on the LAN.
Function	Send	N/A	This function will serialize and encode data to send to the server, Then it will decode and deserialize the response which may be the opponents character data.
Procedure	Reset	N/A	This will reset the connection and will be called in the connect before connecting to remove the possibility of any errors happening from an already connected client when playing game after game.
Procedure	Shutdown	N/A	This will shutdown the connection at the end of a game. So that the network doesn't keep running and produce an error when the server shuts down.
Procedure	Discover_games	N/A	This will broadcast a game discovery request to all devices on the network and edit a shared list to store the games' info. It will also run in a separate thread as it may cause freezing if it runs in the main game thread because of delay while waiting for server responses. This will allow the player to join a LAN game in the joining menu.
Class	Hud	N/A	The player hud will allow both players in games to view their health and see how much time is left in each round.
Variable	Player1	character	Classes are mutable meaning the data they hold can be changed during runtime and it will save across the program in python. Passing the entire player class will make it easier to instantly get access to the player health from within the HUD class rather than passing it as a parameter every time, the health needs to be displayed onto the screen (every frame during the

			round). It will reduce complexity and making retrieving the health stats faster.
Variable	Player2	character	Classes are mutable meaning the data they hold can be changed during runtime and it will save across the program in python. Passing the entire player class will make it easier to instantly get access to the player health from within the HUD class rather than passing it as a parameter every time, the health needs to be displayed onto the screen (every frame during the round). It will reduce complexity and making retrieving the health stats faster.
Variable	timerSurface	Pygame surface	This will be the rendered text displaying the 90 second timer in rounds. This will be displayed in the mid top of the screen in between the health bars. <i>This image will be a surface for pygame to draw. Pygame is a module that works with surfaces and rects to draw things onto the screen.</i>
Variable	timerRect	Pygame rect	This rect will be made based off the rendered text. <i>This rect will be needed for pygame to draw the surface onto. Pygame is a module that works with surfaces and rects to draw things onto the screen.</i>
Variable	healthbarSurface	Pygame surface	This will contain the health bar surface for the two players. Two health bar surface attributes aren't needed as this can be reused for both players. <i>This image will be a surface for pygame to draw. Pygame is a module that works with surfaces and</i>

			<i>rects to draw things onto the screen.</i>
Variable	healthbarRect1	Pygame rect	This rect will be an attribute as the underlying healthbar won't need to be resized, only the health rect will be. Two of them are also needed as rects can't be reused for the two players because they contain the position. <i>This rect will be needed for pygame to draw the surface onto. Pygame is a module that works with surfaces and rects to draw things onto the screen.</i>
Variable	healthbarRect2	Pygame rect	This rect will be an attribute as the underlying healthbar won't need to be resized, only the health rect will be. Two of them are also needed as rects can't be reused for the two players because they contain the position. <i>This rect will be needed for pygame to draw the surface onto. Pygame is a module that works with surfaces and rects to draw things onto the screen.</i>
Variable	healthSurface	Pygame surface	This will contain the surface for the health that will need to be drawn onto the health bar. Surfaces can be reused so two aren't needed. The rects can't be defined as attributes as the health surface will constantly be resized. It will also be easier to make the rects in the draw function and use them for drawing onto the screen <i>This image will be a surface for pygame to draw. Pygame is a module that works with surfaces and rects to draw things onto the screen.</i>
Procedure	draw	N/A	<i>This will draw the entire health bar onto the screen so players</i>

			<i>can see health and time left in each round.</i>
Class	Stage	N/A	This is an example class of a stage. Each stage will need its own background and platforms for the players to play on. Making a separate class for this makes it easier to display each one in the stage selection and follows the OOP that I am using for this project.
Variable	Background	Pygame surface	This background surface will be <i>This image will be a surface for pygame to draw. Pygame is a module that works with surfaces and rects to draw things onto the screen.</i>
Variable	BackgroundRect	Pygame rect	<i>This rect will be needed for pygame to draw the surface onto. Pygame is a module that works with surfaces and rects to draw things onto the screen.</i>
Variable	platforms	List	This list will contain the platforms which will need to be iterated through to check for player collisions. Having the platforms in a list makes them all easily accessible for checking collisions. Also, stages could have different numbers of platforms, putting them all in a list will make the draw procedure suitable for all types of stages.
Procedure	draw	N/A	This will draw the stage background and all the platforms onto the screen so the players can see and play the game.
Class	Character_factory	N/A	This character factory will be used to create instances of characters for the player to use. Having this as a separate class is very useful as in the character selection, it will make the code neater and allow room for scalability in terms of more characters in future updates. This class will

			separate the character select logic from the menus, making code easier to read and organize.
Function	Create	N/A	This function will return the character that corresponds to the string passed into the parameter. This function will make it easy to add more characters in the game in later updates as I can add more cases/if statements easily to point to character classes.
Class	SceneManager	N/A	This class will be used to manage all the different menus and game states. It will run in the main loop and simplify a lot of the logic to do with game states. Having a main class handle all the scene switching rather than a match case with a game state variable is much neater and will make all the scenes easier to link together and route.
Variable	Current_scene	Scene class	This will hold the current scene which will be a class so that it can call the scene's update and draw procedures to display the scene onto the screen for the player to interact with.
Variable	network	Network class	This will hold the instance of the network class so that it can be passed into any scenes that need it like the waiting, joining, game
Variable	server	Server class	This will hold the instance of the server class so that it can be passed into any scenes that need it like the waiting, joining, game.
Procedure	Change_scene	N/A	This will replace the game state match case and will make it much simpler to decide when to switch scenes and what scene to switch to. This can be assigned to certain buttons on press and it will change scenes to another scene.
Procedure	Update	N/A	Every scene class will have an update function which will be called on the current scene variable in the main

			game loop. This is how all the scenes will run the logic needed for the game. For example, pressing a button will be handled in the update procedure where as, drawing a button will be handled by the draw function. This separates what the UI into what the user sees and what happens behind that. Which makes it easier to categorise algorithms.
Procedure	Draw	N/A	Every scene class will have a draw procedure which will be responsible for drawing everything that needs to be drawn onto the screen.
Variable	mode	string	The game scene class will require this variable, specifying whether it is a practise 2-player or LAN game mode. This will help the game class run the appropriate logic as these game modes will require different code. For example only the LAN game mode needs the connection/network handling logic while the practise mode doesn't need a HUD or a connection to the server. The 2-player just needs the HUD and the specified controls in the settings.
Variable	font	Pygame font	Every scene will need this to display text on the screen. Some fonts may be different to others
Variable	Game	Game class	Game scene will need this class will handle all the logic to do with the actual game, like managing the HUD, connection the server and will call all of the subroutines in the correct order, needed for the user to actually experience playing the game.
Variable	PROGRAM_DATA	Dictionary	This dictionary will contain any data that needs to be across the whole program. Dictionaries are mutable which means that are

			changeable during runtime Storing variables in this dictionary will make it easier to access important variables. For example delta time will be stored in this, which is the time elapsed since the last frame. This will be multiplied by any amount of pixel movement on screen to ensure that the speed of the pixels is the same across different frame rates. This is essential for fairness, as without it people with higher frames will move faster in game. Also, the player controls will be stored in this dictionary for easy access to in the character functions and the settings.
--	--	--	---

Server

Function connect

try

Socket.bind((IP,0)) *#assigns the socket to an #IP address and OS decides a free port*

Socket.listen(2) *#accepts TCP connections up to #2*

Return True

Except e as error

Print("Hosting error ", e) *#prints the actual error details to the console*

Return False *#function returns false so that it can be retried and waiting scene doesn't start*

endfunction

Justification

When the user clicks the host button, this function will run. If it is successful and returns True then the scene will be changed to the waiting scene. Otherwise, an error will be printed and false will be returned so the user returns to the play menu. This function will be crucial as it will set up the server, so that the user can play the LAN mode.

Procedure handle_client_thread (conn,)

TO BE UPDATED WITH SERVER LOGIC

Endprocedure

Justification

This procedure will run in parallel with the rest of the game for the host. It should handle all communications between the server and a player.

```

try

    udpSock.connect(('10.255.255',1)) #connects to any a server on any ip and port, doesn't
    matter (doesn't need to be reachable)

    IP = s.getsockname()[0] #retrieves (IP,port)that OS uses to send packets

Except error

    IP = False

Return IP

endfunction

```

Justification

This function will connect to any ip and port and get the IP by using the getsockname method to retrieve a tuple containing the IP and port. This function will be useful to determine where to set up the server. You need an IP to host a LAN multiplayer game on.

Procedure listener(stage)

```

udpSock = socket(IPv4, UDP)      #allows IPv4 addresses and UDP connection

```

```

udpSock.bind(('',9999)) # assigns socket to all IP addresses on a device and port 9999

```

```

while listening == True then

```

```

    data, addr = udpSock.recvfrom(1024) #receives a UDP packet data and the addr #which #contains IP
    and port (used to send a reply)

```

```

    if decode(data) == "GAME_DISCOVERY_REQUEST_mythbound" then #checks if #data comes from
    another Mythbound player

```

```

        response = "STAGE:" + stage + ":IP:" + IP + ":PORT:" + str(port)      #prepares game info

```

```

        udpSock.sendto(encode(response), addr) # sends the game info back to the #Mythbound
        client

```

```

endprocedure

```

This procedure will need to run on a separate thread as the recvfrom will wait until it receives a UDP packet. If it was running in the main game thread then the program will stall and become unresponsive. The while loop allows the thread to safely end once listening is set to False, this is called a shared variable that is controlled outside of this thread.

Network

Procedure discover_games()

```
udpSock = socket(IPv4, UDP)      #allows IPv4 addresses and UDP connection

udpSock.settimeout(3) # does a socket time out error if 3 seconds pass with no response

udpSock.allowBroadcast() #allows the socket to broadcast to all devices on a network

udpSock.bind(('',0)) #OS assigns available free port

message = encode("GAME_DISCOVERY_REQUEST_mythbound")

udpSock.sendto(message, (BROADCAST, 9999)) # sends the discovery message to all devices on
a network on port 9999

start = time() #current time

Discovered = [] #this attribute on the network class is set to empty, to not repeat servers

While time() – start < 3 then #to get multiple responses from multiple available games, a while loop
is needed

    Try

        Data,addr = udpSock.recvfrom(1024) #receives a UDP packet data and the addr
        #which #contains IP and port (used to send a reply)

        Discovered.append((Data,addr))

    Except socket.timeout

        Pass
```

Endprocedure

Justification

This procedure will need to run on a separate thread as the recvfrom will wait until it receives a UDP packet. If it was running in the main game thread then the program will stall and become unresponsive. This procedure does what is needed by editing a list with games' info that are running on the LAN, by sending a game discovery request and waiting for responses from servers.

Character

Take damage

```
Procedure take_damage(damage, player2)
```

```
    If invincible == False and player2.rect.collidewith(rect) and  
    player2.attacking == True then
```

```
        health = max(health - damage, 0 )
```

```
        R = random.number(0,5) #random number between 0 and  
4 inclusive
```

```
        If R == 0 then #1 in 4 chance of a knockback
```

```
            Knockback = True #sets the character's knockback  
variable
```

```
    Endif
```

```
Endprocedure
```

Justification

This procedure will work out if the character is not invincible then the damage from the attack will be taken away from the character's health. It will then pick the biggest value out of the reduced health or 0, this is to ensure that health doesn't go into negative values. Which is illogical and may cause bugs in the HUD when calculating the length of the health. This procedure will ensure fair attack registering, which is crucial for good fighting mechanics.

Update

Procedure move (platforms)

Justification

This update procedure is needed for

```
Procedure update()
```

```
    U = current_animation.update()
```

```
    If U != 'finished' then
```

```
        invincibility = U[0]
```

```
        recovery = U[1]
```

```
        attacking = U[2]
```

```
    current_animation = animations[animation_state]
```

```
    image = current_animation.get_frame()
```

```
    if facing_left == True then
```

```
        image = verticalFlip(facing_left)
```

```
Endprocedure
```

```

    gravity +=10 #applies gravity each frame to pull the player down after a
#jump

    #Player movement only allowed if not attacking, not in recovery and not
#in knockback

    If animation_state != "light" and animation_state != "heavy" and knockback != True
and recovery != True then

        If Key[PROGRAM_DATA["PLAYER1_RIGHT"]] then

            direction = pygame.math.Vector2(1,0) #direction set to right

            facing_left = False #character is facing right

            animation_state = "run" #signal to start run animation

            moving_right = True #store movement direction for collision

#handling

        Elif Key[PROGRAM_DATA["PLAYER1_LEFT"]] then

            direction = pygame.math.Vector2(-1,0) #direction set to left

            facing_left = True #character is facing left

            animation_state = "run" #signal to start run animation

            moving_right = False #store movement direction for collision

#handling

        Else

            direction = pygame.math.Vector2(0,0) # no movement

            animation_state = "idle" # signal to start idle animation

        endif

        #jump logic

        If Key[PROGRAM_DATA["PLAYER1_JUMP"]] and on_ground == True then

            gravity = -150 #apply upward force for jumping

            on_ground = False #player no longer on ground

            animation_state = "jump" #signal to start jump animation

        endif

        #apply horizontal movement

```

```

Rect.center += direction * speed * PROGRAM_DATA["dt"]

endif

#hozional collisions

For platform in platforms

    If rect.collidewith(platform) then

        If moving_right == True then

            Rect.right = platform.left # snaps position to the left
edge if player hits it

        else

            rect.left = platform.right # snaps position to the right
edge if player hits it

        endif

    endif

endif

Rect.centery += Min(max_fall_speed, gravity) * PROGRAM_DATA["dt"] #gravity
#needs to occur at all times despite attacks

#vertical collisions

For platform in platforms

    If rect.collidewith(platform) then

        If gravity < 0 then

            Rect.top = platform.bottom #snaps position to the bottom
edge #of a platform if player hits underside

        else

            rect.bottom = platform.top #snaps position to top of a
platform #if player lands on the platform

            on_ground = True # allows jumping again

        endif

    endif

endif

Endprocedure

```

Justification

This procedure will handle the moving of players by considering user key presses and set the animation state appropriately. It will also check the collisions with platforms so players don't just fall through the screen. This method is crucial for completing the first 3 success criteria and many of the essential features like changeable key binds and animated pixel art characters.

Animation

update

Function update (dt)

*# If animation finished,
return immediately*

If done == True then

Return 'finished'

Endif

#increment the timer by dt (time elapsed since last frame)

timer += dt

if enough time has passed to show next frame

if timer >= frame_duration then

#reset timer

timer = 0

#move to next frame

index +=1

#if index is past the last frame

if index >= frames.length() then

#if the animation should loop then reset to 0

if loop == True then

index = 0

else

#otherwise keep the animation on the last frame and mark as done

index = frames.length() -1

done = True

endif

endif

endif

return whether the current frame is inside the special frame sets

return [index in invincibility, index in recovery, index in attacking]

Endfunction

This function will handle when to switch images based on the time elapsed since the last frame and how long each frame should take. This function is very important to achieving success criteria #2 and #6. By counting the frames in the jump sprite sheets, I can divide 1.25 seconds by the number and then divide the result by 1000 to get how many milliseconds each frame should be drawn for, which will be inputted as a parameter into the animation class for this function to use.

Iterative test data

Scenes

For each scene, I test out the scene's primary functions (given that it does not rely on external unwritten code). If it does rely on external unwritten code, I will abstract the idea and create a mini testing simulation, outputting the results to the console. For example, in order not to test modules unnecessarily, the game scene's main function, while in development, will just repeatedly output "Game logic code here" to the console.

Main menu

Test data	Expected outcome
Mouse click on empty space (not a button)	Nothing
Mouse click on "Play" button	Change scene to play menu
Mouse click on "Settings" button	Change scene to settings menu
Mouse click on "Quit" button	Quit python and pygame

Initially, in the menu code, I won't have written out the code for displaying the different game modes. So, I will be, initially, outputting the mode as a string to the console so that I know that it is correctly executing the block of code (that I can then insert game logic function calls in).

Scene Manager

Test data	Expected outcome
change scene function	Output to the console "change scene to (scene name)". This should also switch the update and draw functions to those scenes' update and draw functions.

Play menu scene

Test data	Expected outcome
Mouse click on "Practise" button	Change scene to game scene
Mouse click on "2-player" button	Change scene to game scene (local mode)
Mouse click on "LAN join" button	Change scene to joining scene
Mouse click on "LAN host" button	Change scene to waiting scene

Setting scene

Test data	Expected outcome
Mouse click on player 1 controls	Log the next key
Change to player 1 controls	Change the appropriate PROGRAM_DATA controls
Mouse click on player 2 controls	Log the next key
Change to player 2 controls	Change the appropriate PROGRAM_DATA controls
Mouse click on "back" button	Return to main menu scene

Joining menu

Test data	Expected outcome
Mouse click on "back" button	Return to play menu scene
Mouse click on "join" button	change scene to game scene (joining mode), and games currently active displayed with correct details.

Drag scroll bar	The game choices should move up when scrolling down, and move down when scrolling up to be able to display all LAN games
-----------------	--

Stage selection menu

Test data	Expected outcome
Mouse click on first stage	Change scene to character selection or waiting scene, depending on whether the user has selected 2-player, practise or host.
Mouse click on second stage	Change scene to character selection or waiting scene, depending on whether the user has selected 2-player, practise or host.
Mouse click on third stage	Change scene to character selection or waiting scene, depending on whether the user has selected 2-player, practise or host.

Waiting Scene

For this scene, I will just be testing the visual aspect, as most of the code will be in the network class which will just be tested separately.

Character Selection scene

Test data	Expected outcome
Mouse click on characters	Show character at the bottom
Mouse click on confirm	Confirm choice, and change scene if both players confirm
Visual test on the characters' images (to ensure they are being displayed properly)	There should be appropriately sized previews at the bottom, and the characters face as the selections

Game Scene

Server

Test data	Expected outcome
Clients' requests sent to the server	Appropriate data should be sent back, such as enemy info (animation frame, health, choice etc) via the handle client thread function, which should run on a separate thread.
On server connect	Should bind to a socket on a free port, and run on a separate thread.

Network

Test data	Expected outcome
On connect	Should connect to the server
Discover games	Should return list of games on current LAN network

Stage

Test data	Expected outcome
-----------	------------------

Draw function is called in game scene	To draw the arena onto the screen, including the background and any platforms correctly.
---------------------------------------	--

Character

In addition to the test data below, I will also have to test out the values of speed, max fall speed, gravity etc. This is to ensure that the characters aren't too fast, don't fall or jump too fast/high, so the game is playable and enjoyable.

Test data	Expected outcome
"W"	Character should move upwards (jump) and change animation
"A"	Character should be facing left and move
"D"	Character should be facing right and move
Controls for attacks have been clicked	Character should start animation for that attack
Opponent attacks hitbox(rect)	Health should be lowered on the health bar

HUD

Test data	Expected outcome
While game scene active	Should draw the health bars, timers, current health on to the screen
Any player health is lowered	Effect should be shown on the relevant health bar (if player 1 health is lowered then the health on player 1's health bar should be lowered)

Character factory

Test data	Expected outcome
Press character selection confirm buttons	Create a character object and return it

Ending banner

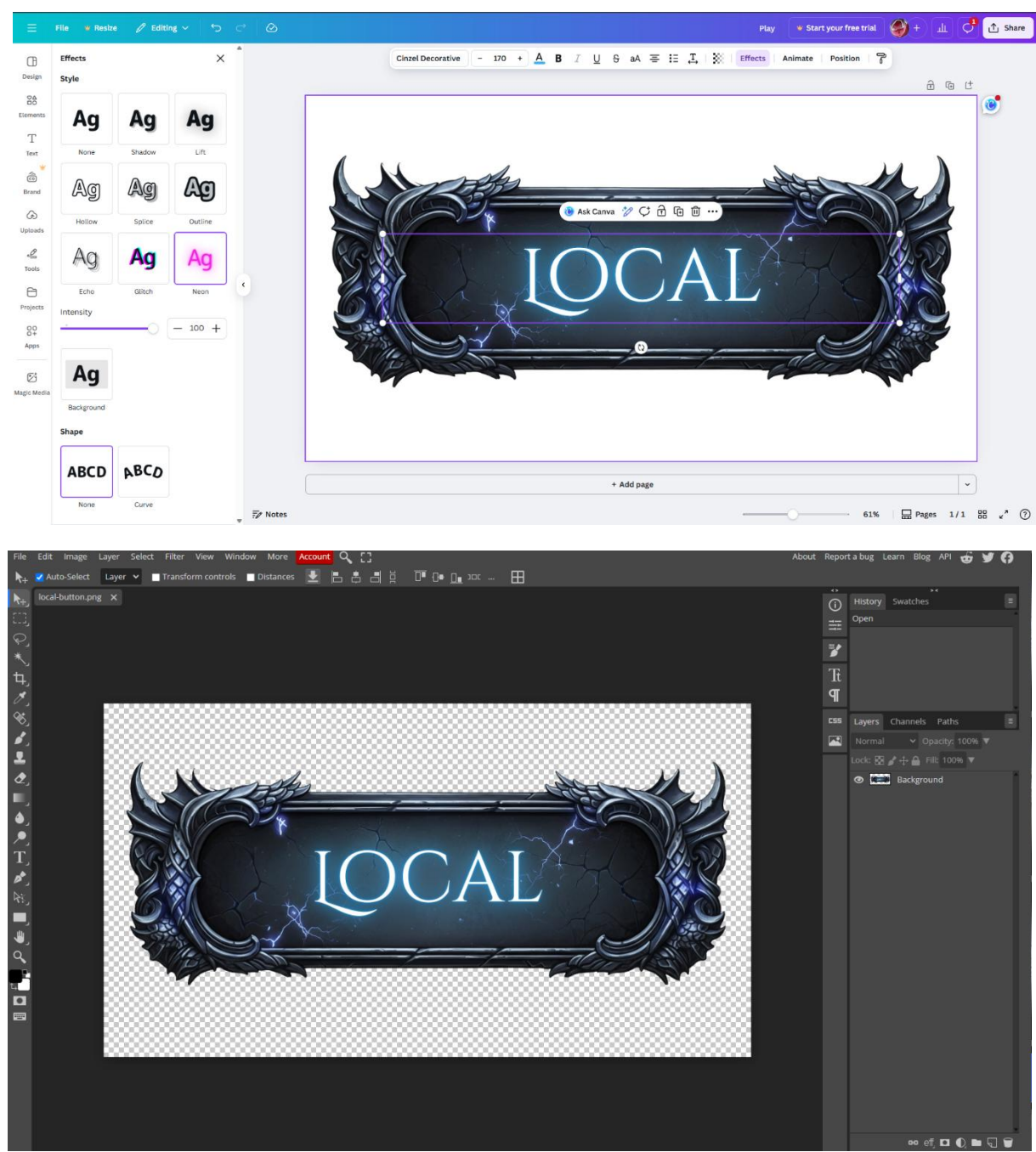
Test data	Expected outcome
A win in local or LAN mode.	Ending banner should be displayed with the character centered at the middle bottom and the text clearly showing which player won in the middle.

Development

Assets

For assets, I will be using Google's imagen 4 ultra to create backgrounds, stages, platforms. I will then be editing some of these in Canva (like buttons), to add text and then remove the background in photopea (free photoshop on web). For platforms, I just get the image from

imagen and edit it to remove background in photoshop. An example of how I make buttons is below:



For character assets, I will be searching on a website called itch.io for free copyright asset packs, that include two attacks, a jump, idle and run animation. As those are the one of the essential features of my game.

Constructing the main menu

I will create the scene manager class first, as that will be used across the entire program for handling each menu, game etc.

```
class SceneManager:
    def __init__(self, network, server, font, sWidth, sHeight):
        self.network = network
        self.server = server
        self.font = font
        #Screen dimensions needed for relative sprite placement
        self.sDimensions = (sWidth, sHeight)
        #Sets menu menu as the first scene
        self.current_scene = MenuScene(self, self.font, self.sDimensions)

    def change_scene(self, new_scene, *args):
        self.current_scene = new_scene(*args)

    def handle_events(self, events):
        #Each scene will have the parent handle events function, that can be m
        self.current_scene.handle_events(events)

    def update(self, dt, mpos, mbut, keys, keys_j):
        #Each scene may need keys and mouse buttons pressed for logic
        self.current_scene.update(dt, mpos, mbut, keys, keys_j)

    def draw(self, screen):
        #Calls draw function of current scene
        self.current_scene.draw(screen)

    def quit(self):
        pygame.quit()
        quit()
```

Screen dimensions are needed for relative placing of buttons and such. Current scene attribute is initialised as the main menu, because that's what the user will start out with to be able to select different playmodes

The handle events, update and draw functions will run every frame. They have been split like this for organisation and makes it easier to make the game better and more complex. Handle events will take care of pygame events such as a user trying to close the game. Update will run all the behind the scene's logic and will usually be the complicated part. Draw will simply be used to output all the sprites of that scene onto the screen.

Change scene will be called to change all that is happening on screen. This way is much neater than game states and switch statements. Once a scene is changed, that scene's handle, update and draw functions will be running.

This may be used in the future to reset all network connections and shut the game down in case of a crash or network failure.

```
class scene:
    def __init__(self, manager, font, sDimensions):
        self.manager = manager
        self.font = font
        self.sDimensions = sDimensions

    def handle_events(self, events):
        #Basic inherited function for all scenes to check whether user closes the window
        for event in events:
            if event.type == pygame.QUIT:
                pygame.quit()
                quit()
```

This class is new and will be used as a parent class for all scenes, because of its usefulness .

Most scenes, won't have to override the handle events function, which is why it's perfect to use in the parent class. Also, all classes will have the main 3 attributes be the same which is the manager, font and screen dimensions attributes. This will save a lot of unnecessary rewriting of code for every scene.

To construct the menu that was designed in the usability features, I will need to program buttons that will change scenes via the scene manager. However, copy pasting the same button code repeatedly for all the buttons in the game is inefficient. I didn't realise this in design. Therefore, I will add a button class that will be reusable and applicable to every button in the game.

The size text and function will be specified upon creation of the object, to make the class more reusable and applicable to every button. Position in pygame is very customizable via keyword arguments in the parameters rect, which is why passing them in as ****kwargs** is very useful and saves a lot of unnecessary if statements to create a button where the top left of it is positioned in a certain place, for example.

```
class Button: 7 usages
    def __init__(self, size, text, function, **rect_kwargs):
        """rect_kwargs will be needed to be able to place different points on the rect in different positions
        # example: need to place center on (x,y) or the topleft on (x,y)
        self.function = function

        self.image = pygame.image.load(f"assets/{text}-button.png").convert_alpha()
        #conver alpha needed for png images for faster blitting
        self.image = pygame.transform.scale(self.image, size) #sizes the image down

        self.image6 = pygame.image.load(f"assets/{text}-button-g.png").convert_alpha()
        self.image6 = pygame.transform.scale(self.image6, size)

        self.rect = self.image.get_frect(**rect_kwargs)
        self.rect6 = self.image6.get_frect(**rect_kwargs)

        # used to change current image when hovering
        self.currentI = self.image
        self.currentR = self.rect
```

The current image attribute is needed to specify which version will be drawn each frame, explained more in the update function.

Each button will have two versions of it, one will have the text highlighted in gold. While the other, will be the usual cool blue. This will allow for a very cool light up visual effect when hovering over the button with the mouse. The logic is explained in the update function.

```
def update(self, mpos, mbut):
    #Checks if mouse is hovering over it
    if self.rectG.collidepoint(mpos):
        #Sets image to gold, if hover
        self.currentI = self.imageG
        self.currentR = self.rectG
        #If mouse press, button function is called
        if mbut[0]:
            self.function()
    else:
        #Sets image to normal, if not hover
        self.currentI = self.image
        self.currentR = self.rect
```

Pygame's inbuilt rect function checks if the mouse position is inside the rect of the button, if it is then the attribute for the current image is changed to the gold text version. Furthermore if it is pressed, then the function is executed. Else, it will just set this back to the normal version. This function will be executed each frame which is why the else is necessary.

```
def draw(self, screen):
    #Draws current button image to screen
    screen.blit(self.currentI, self.currentR)
```

This will simply draw the current button image to the screen.


```

import pygame
from scene import SceneManager
from networking import server, network

#Initilize pygame
pygame.init()

#Essential program data
PROGRAM_DATA = {
    "width": 1280,
    "height": 720,
    "dt": 0,
}

#Sets screen size and creates a clock object
screen = pygame.display.set_mode((PROGRAM_DATA["width"], PROGRAM_DATA["height"]))
clock = pygame.time.Clock()

#creates server, network and SceneManager objects
s = server()
n = network()
SM = SceneManager(network=n, server=s,
                  font=pygame.font.Font("assets/Cinzel-Regular.otf", 24),
                  sWidth=PROGRAM_DATA["width"], sHeight=PROGRAM_DATA["height"])

running = True
while running:
    #Retrieves all events for scene manager to handle
    events = pygame.event.get()
    screen.fill((0,0,0))

    #Retrieves mouse position, buttons pressed, state of all keys
    mpos = pygame.mouse.get_pos()
    mbut = pygame.mouse.get_pressed()
    keys = pygame.key.get_pressed()
    keys_j = pygame.key.get_just_pressed()

    #Scene manager takes care of handling events, logic and drawing sprites to the screen
    SM.handle_events(events)
    SM.update(PROGRAM_DATA["dt"], mpos, mbut, keys, keys_j)
    SM.draw(screen)

    #time elapsed since last frame (used for fair movement speed)
    PROGRAM_DATA["dt"] = clock.tick()/1000

    #Updates display
    pygame.display.flip()

```

These lines create the screen and viewport, using the resolution specified in program data. Which all the sprites will be drawn onto. A Clock object is also created which will be used to workout the time elapsed since the last frame (ensuring characters move at the same speeds no matter the fps) for fairness in LAN games.

Explained and justified in the design, program data is made here in the main file for its data to be passed into the scene manager.

There will only be one instance of these objects running the whole time, that the game is running. Which is why they are created in the main file. The network will be responsible for joining games, while the server for hosting games.

User input is retrieved here. This includes all the main forms of input (mouse and keyboard), just pressed refers to a single keystroke while get pressed will return true for any key that is held down. Just pressed keys can be used for attacks that can't be spammed, while pressed can be used for movement.

Each frame, events will be retrieved to be handled by the scene manager. These events can include key presses or requests to close the game. The screen is filled with black to reset any drawing from the previous frame.

Delta time or "dt" is the time elapsed since the last frame in seconds. This is multiplied by movement speed (pixels you want the character to move in a second) to ensure that no matter the framerate, the character moves at the same speed. This can be used on any moving object too. The flip function called at the end just updates the display and presents the drawings to the screen.

Test #1

What I'm testing	Input data	Why am I testing this?	Expected outcome	Test results
Button visuals/sizes	N/A	Menu button have to be readable and	For the buttons to be big enough to click	Buttons' text was small, and the text looked

		accessible to click and navigate easily.	and easily readable.	very grainy. Even though, the mouse wasn't on the button, It was still coloured gold.
Does the play button respond to click	Mouse click on button	Menu needs be navigable for users to play.	The play button should take the user to the play menu scene which should be blank.	As expected
Does settings button respond	Mouse click on button	Menu needs be navigable for users to play.	Console should output "settings"	As expected
Does the quit button respond	Mouse click on button	Menu needs be navigable for users to play.	Game should quit	As expected

Mouse was below the button (screenshot doesn't show the mouse), however the button is still gold. The buttons are quite small and the text looks stretched.



```

class Button: 7 usages  alenthusiast7 *
    def __init__(self, size, text, function, **rect_kwargs):  alenthusiast7 *
        """rect_kwargs will be needed to be able to place different points on the rect in different positions
        # example: need to place center on (x,y) or the topleft on (x,y)
        self.function = function

        self.image = pygame.image.load(f"assets/{text}-button.png").convert_alpha()
        #conver alpha needed for png images for faster blitting
        croppedImage = self.image.subsurface(self.image.get_bounding_rect())
        self.image = pygame.transform.smoothscale(croppedImage, size) #sizes the image down

        self.image6 = pygame.image.load(f"assets/{text}-button-g.png").convert_alpha()
        croppedImage6 = self.image6.subsurface(self.image6.get_bounding_rect())
        self.image6 = pygame.transform.smoothscale(croppedImage6, size)

        self.rect = self.image.get_frect(**rect_kwargs)
        self.rect6 = self.image6.get_frect(**rect_kwargs)

        # used to change current image when hovering
        self.currentI = self.image
        self.currentR = self.rect

```

The issues with buttons being highlighted when the mouse wasn't hovering on them was solved as the transparent pixels extended the rect of the button unnecessarily, so removing them solved that issue too.

To solve the weird sizing issues, I used the get bounding rect function to return a rect with no transparent pixels, then used the subsurface to convert it back to a surface so it can be used as an image. Removing the transparent pixels allowed scaling the button to have an accurate effect on the actual size. Smooth scale removes the grainy effect at the cost of performance. The menus take longer to load.

The buttons look much better now, including the text:



Issue with performance

There is now a problem with performance. The smooth scale function, while it does solve the grainy stretched text look, takes significantly longer to load the menu. Therefore, I will need to load and smooth scale and size all assets appropriately prior to loading the executing the code for the game. To do this, I will create a new class called assets, responsible for loading and then holding fully processed, ready to use images. This should speed loading times up significantly and will help later in development when I use more and more assets.

```

class assets:
    def __init__(self, screenSize, buttonSize):
        #Sets base to absolute path of the py file
        self.base = os.path.dirname(os.path.abspath(__file__))
        #Assets folder path
        self.path = os.path.join(self.base, "assets")

        #Backgrounds,buttons folders path
        self.backgrounds_path = os.path.join(self.path, "backgrounds")
        self.buttons_path = os.path.join(self.path, "buttons")
        self.loading_path = os.path.join(self.path, "loading")
        self.fonts_path = os.path.join(self.path, "fonts")

        #Specifies screen size and button size for image scaling
        self.screenSize = screenSize
        self.buttonSize = buttonSize

        #assets dictionaries seperated by type eg backgrounds, buttons
        self.backgrounds = {}
        self.buttons = {}
        self.fonts = {}

        #Total number of images that need to be loaded, progress check (for load
        #excludes loading screen assets, as they wont be loaded in the load meth
        self.counter = self.count(self.path) - self.count(self.loading_path)
        self.progressCounter = 0

        self.loaded = False

```

These two attributes will set the path of the assets folder, depending on where the assets.py file is within the computer. This will make the code compatible with any computer, given that the game files haven't been moved around. `__file__` returns the path to the current file, `abspath` ensures that it's an absolute path (as in starts at the C drive). `Dirname` just returns makes the path point to the directory its in, rather than the file.

For better organisation, the images have been split up by type into different folders. Note: loading assets will be the very first things loaded in the game, therefore, they require their own separate folder for them to be loaded independently. **This way of organisation also allows me to customize size of buttons and backgrounds easily, rather than mess around with splicing file names.**

The screen size and button size being attributes allows me to use them to scale each button and background appropriately, this can also be easily specified upon creation of the class/object.

Dictionaries are brilliant. They allow easy access to images.

Once the load function is finished, the loaded Boolean is set to true. This signals to the loading screen to change scene into the main menu.

The counter and progress counter are used in determining the size of the loading bar in the loading screen, to let users know how much time is left (approx.) until they can play. The count method counts how many files in a directory (explained later), the total assets that need to be loaded in the loading screen don't include the loading screen assets themselves, which is why there is a subtraction of the number of loading screen assets.

This ratio function will be used in the loading screen to determine how far along the loading bar will be.

```
def ratio(self):  
    #Returns loading progress as a ratio to be used for loading bar width  
    return self.progressCounter/self.counter
```

```
def status(self):  
    #Checks if all assets are loaded  
    return self.loaded
```

Status will also be used in the loading screen to determine whether to switch scenes. The difference between this and ratio is that, this will simply return whether the load function is finished. So, if there are assets counted that aren't (not meant to) be loaded, the loading screen won't be stuck forever.

```
def count(self, path):  
    #Sets file count to 0  
    counter = 0  
    #For each entry in the folder  
    for file in os.listdir(path):  
        #Check if its a file  
        if os.path.isfile(os.path.join(path,file)):  
            #Increment counter if it is a file  
            counter+=1  
        else:  
            #If its a folder recurse and count how many in that folder to add  
            counter += self.count(os.path.join(path, file))  
    #Once its finished counting everything in asset folder, return counter  
    return counter
```

This function uses recursion to count how many files there are in any folder. It works by iterating through each file and recursing if it's a folder, by going inside that folder and counting and so on. This is necessary for the loading screen as it will automatically count how many assets need to be loaded so it can be used for the competition status.

I will now create the rest of the methods for the assets class:

```

def load(self):
    #Goes through each file in backgrounds folder
    for file in os.listdir(self.backgrounds_path):
        #if image is png
        if file[-3:] == "png":
            #Allows transparent pixels
            image = pygame.image.load(os.path.join(self.backgrounds_path, file)).convert_alpha()
        else:
            image = pygame.image.load(os.path.join(self.backgrounds_path, file))
            image = pygame.transform.smoothscale(image, self.screenSize)

        #Removes file extension for easy storing
        name = file[:file.find(".")]
        #Stores each image in dictionary
        self.backgrounds[name] = image

        self.progressCounter+=1 #Increases loaded images counter by 1

    for file in os.listdir(self.buttons_path):
        #All buttons are png
        #Allows transparent pixels, then removes them
        image = pygame.image.load(os.path.join(self.buttons_path, file)).convert_alpha()
        image = image.subsurface(image.get_bounding_rect())
        #Scales using button size
        image = pygame.transform.smoothscale(image, self.buttonSize)

        name = file[:file.find(".")]
        self.buttons[name] = image

        self.progressCounter+=1

    for file in os.listdir(self.fonts_path):
        font = pygame.font.Font(os.path.join(self.fonts_path, file), 16)
        name = file[:file.find(".")]
        self.fonts[name] = font

        self.progressCounter+=1

    self.loaded = True

```

For loop is used to loop through each file in the backgrounds folder, looking for background images to load, if a background is a png, convert alpha is used to allow transparent pixels (allows for faster rendering, since they aren't a colour).

The file extension is spliced out of the name so it can be used in the dictionary as a key for the image. This saves the hassle of finding different file extensions when searching for the images. The progress counter is increased by one for each asset loaded.

All buttons need transparent pixels removed to fix sizing issues, and smooth scale for the grainy image issues.

Another reason for different folders for each type of asset is that so each one can be loaded in correctly and easily. I can loop over every file in a fonts folder, knowing each one is a font and loading it in correctly using pygame, rather than messing around with splicing file extensions and if statements for each file extension.

The loading function is finished, so loaded is set to true. At this point, the thread would be ended too (explained in loading screen code)

```
def getButton(self, text):
    #Returns normal button and glowing button
    return self.buttons[text+"-button"], self.buttons[text+"-button-g"]

def getBackground(self, name):
    return self.backgrounds[name]
```

Each button has specified text on it, which is included in the name. The variant can also easily be accessed with a "-g" at the end. Each background also has a name, which allows easy access through a dictionary and a getter function.

```
def getLoadingScreen(self):
    #Returns processed loading screen assets, for displaying to the user
    #Removes any transparent pixels, ready for scaling
    LoadingScreenBackground = pygame.image.load(os.path.join(self.loading_path, "loading_screen.png")).convert_alpha()
    LoadingScreenBackground = pygame.transform.scale(LoadingScreenBackground, self.screenSize)

    LoadingBar = pygame.image.load(os.path.join(self.loading_path, "loading_bar.png")).convert_alpha()
    LoadingBar = LoadingBar.subsurface(LoadingBar.get_bounding_rect())

    Bar = pygame.image.load(os.path.join(self.loading_path, "gold_bar.png")).convert_alpha()
    Bar = Bar.subsurface(Bar.get_bounding_rect())

    return LoadingScreenBackground, LoadingBar, Bar
```

Scale is used instead of smoothscale, as the loading screen needs to be loaded quickly for it to be used and not stall the game.

Loading bar and Bar images are loaded in and stripped of the transparent pixels by get bounding rect, then converted back into a surface by creating a subsurface for it to be drawn later.

Processing the loading screen assets needs to be separate from loading other assets, as it will be one of the first tasks completed by the game, before loading assets.


```

class Button:
    def __init__(self, image, imageG, function, **rect_kwargs):
        #Sets function attribute to the action, called on press
        self.function = function

        #Sets button/glow button
        self.image = image
        self.imageG = imageG

        #Passes the arguments to create a rect object, more accurate button placing
        self.rect = self.image.get_frect(**rect_kwargs)
        self.rectG = self.imageG.get_frect(**rect_kwargs)

        #Current image (either gold or normal) for when mouse is hovering over it
        self.currentI = self.image
        self.currentR = self.rect

```

Setting image attributes in buttons is simpler now, as they are simply passed in as a parameter. They have also been perfectly sized in the assets class too, as resizing can stall the program.

Removing font attribute

As I was writing the code, I realised that each scene does not need an entire separate font attribute, as this can be accessed through the assets class. However, each scene doesn't have an assets attribute. It would make sense for each scene to have one as they all need images for backgrounds, buttons, characters etc. Therefore, I added an assets attribute and removed the fonts attribute.

```

class LoadingScene(scene):
    def __init__(self, manager, assets, sSize):
        super().__init__(manager, assets, sSize)
        #retrieves loading screen assets
        self.LoadingScreen, self.LoadingBar, self.bar_image = self.assets.getLoadingScreen()

        self.LoadingScreenRect = self.LoadingScreen.get_frect(topleft = (0,0))

        self.LoadingBar = pygame.transform.smoothscale(self.LoadingBar, (2.66*200, 200))
        self.LoadingBarRect = self.LoadingBar.get_frect(center = (self.sSize[0]/2, self.sSize[1]/2))

        #Bar is scaled to fit within loading bar, max width stored
        self.bar_image = pygame.transform.smoothscale(self.bar_image, (1.95/3*2.66*200, 200/7))
        self.BAR_MAXWIDTH = self.bar_image.get_width() #constant
        self.bar_width = self.BAR_MAXWIDTH
        self.barRect = self.bar_image.get_frect(center=(self.sSize[0]/2, self.sSize[1]/2+4)) #slight offset

        #Loading needs to be completed in a seperate thread to avoid freezing
        self.thread = threading.Thread(target= self.assets.load, daemon=True)
        self.thread.start()

```

A separate thread is started to run the assets class loading function. This is required to run alongside the program to prevent the game stalling and unresponsiveness. This won't cause any race conditions (data structures being edited simultaneously) as this thread is the only process editing the dictionaries. The status and ratio functions are simply retrieving the state of the variables.

After the images are retrieved, the background and loading bar are saved as attributes to be used in the draw function. All sizing was done with trial and error. The max bar width refers to the maximum size in the line above, this is saved to be multiplied by the proportion of assets loaded to get the moving loading bar effect.

```

def update(self, dt, mpos, mbut, keys, keys_j):
    #If assets are loaded, scene is changed to main menu
    if self.assets.status() == True:
        self.manager.change_scene(MenuScene, self.manager, self.assets, self.sSize)

    #Multiplies max bar width by ratio of assets loaded
    ratio = self.assets.ratio()
    self.bar_width = ratio*self.BAR_MAXWIDTH

    def draw(self, screen):
        screen.blit(self.LoadingScreen, self.LoadingScreenRect)
        screen.blit(self.LoadingBar, self.LoadingBarRect)
        #Third parameter specifies the area of the rect to be drawn
        #Start at (0,0) topleft and go across to (bar width, 0) and down to (bar width, bar height)
        screen.blit(self.bar_image, self.barRect, (0,0, self.bar_width, self.bar_image.get_height()))

```

Bar width multiplied by proportion of assets loaded to get moving bar effect.

This will be ran every frame, to check whether assets have been loaded. If they have the scene will be changed to the menu scene.

Loading screen background drawn first, then the loading bar, then the bar.

This is where the bar width attribute is used, to specify how much how the original image is drawn, by using the third parameter of the blit function

```

class MenuScene(scene):
    def __init__(self, manager, assets, sSize):
        super().__init__(manager, assets, sSize)

        #Sets menu background
        self.background = self.assets.getBackground("start_menu")
        self.backgroundRect= self.background.get_frect(topleft = (0,0))

        #Creates all the main menu buttons
        #Retrieves button images from assets
        play, playG = self.assets.getButton("play")
        settings, settingsG = self.assets.getButton("settings")
        quit, quitG = self.assets.getButton("quit")
        self.playButton = Button(play, playG,
                                lambda: self.manager.change_scene(PlayMenuScene,self.manager,self.assets, self.sSize),
                                center=(self.sSize[0]/2, self.sSize[1]*3/8))

        self.settingsButton = Button(settings, settingsG,
                                    lambda:print("settings"),
                                    center=(self.sSize[0]/2, self.sSize[1]*4/8))

        self.quitButton = Button(quit, quitG,
                                self.manager.quit,
                                center=(self.sSize[0]/2, self.sSize[1]*5/8))

```

The button and background images are now retrieved using the new assets attribute and methods.

Now, finally, a working assets class with a loading scene. I can now easily create the join/play menu using a very similar framework to the main menu scene.

```
class PlayMenuScene(scene):
    def __init__(self, manager, assets, sSize):
        super().__init__(manager, assets, sSize)

        self.background = self.assets.getBackground("start_menu")
        self.backgroundRect = self.background.get_frect(topleft = (0,0))

        practise, practiseG = self.assets.getButton("practise")
        local, localG = self.assets.getButton("local")
        join, joinG = self.assets.getButton("join")
        host, hostG = self.assets.getButton("host")
        back, backG = self.assets.getButton("back")
        self.practiseButton = Button(practise, practiseG, lambda: print("practise"), center=(self.sSize[0]/2, self.sSize[1]*2.5/8))
        self.localButton = Button(local, localG, lambda: print("local"), center=(self.sSize[0]/2, self.sSize[1]*3.5/8))
        self.joinButton = Button(join, joinG, lambda: print("join"), center=(self.sSize[0]/2, self.sSize[1]*4.5/8))
        self.hostButton = Button(host, hostG, lambda: print("host"), center=(self.sSize[0]/2, self.sSize[1]*5.5/8))
        self.backButton = Button(back, backG, lambda: self.manager.change_scene(MenuScene, self.manager, self.assets, self.sSize),
                                center=(self.sSize[0]/2, self.sSize[1]*7/8))
```

In the play menu scene constructor above, I just retrieved each background and button image using the assets class, making sure to use two variables for the buttons (one for the glowing variant). I then passed the images in as parameters to the button class and instantiated an object for each button. You can see multiple uses of lambda; this allows to essentially declare a mini function so I can pass it in as a parameter. I repositioned the centers of the buttons using trial and error and some basic fraction work by dividing the screen laterally by 8. I also added a back button which is needed to go back to the main menu.

In the update and draw functions of the play menu scene, I just call each button's update functions to ensure they perform an action when pressed and drew them in the draw function.

```
def update(self, dt, mpos, mbut, keys, keys_j):
    self.practiseButton.update(mpos, mbut)
    self.localButton.update(mpos, mbut)
    self.joinButton.update(mpos, mbut)
    self.hostButton.update(mpos, mbut)
    self.backButton.update(mpos, mbut)

def draw(self, screen):
    screen.blit(self.background, self.backgroundRect)
    self.practiseButton.draw(screen)
    self.localButton.draw(screen)
    self.joinButton.draw(screen)
    self.hostButton.draw(screen)
    self.backButton.draw(screen)
```

Test #2

I will now test my game again and show the results of my tests below:

What I'm testing	Input data	Why am I testing this?	Expected outcome	Test results
Assets				
Is the bar moving across the loading screen.	Starting up the game	This should work to show the user how long left until the game is started up.	Bar should be moving across to show progress in the loading screen.	As expected
Are the assets being loaded	Starting up the game	Without assets, the game would not work.	Assets should be loaded and look good in the menu, including glow variants of buttons.	As expected
Is moving in between menus instant	Starting up the game	Removing delay and lag is one of the essential features and will help users to enjoy the game.	Clicking the play button should instantly load up the join menu with no delays.	As expected
Play menu				
Does pressing back button work in the play menu take me back to the main menu	Mouse press on the button	It will give users and interactive menu experience.	Take user to play menu.	As expected
Does pressing the practise button print "practise" to the console	Mouse press on the button	Menu needs to be navigable for users to play.	Output "practise" to console.	As expected
Does pressing the local button print "local" to the console	Mouse press on the button	Menu needs to be navigable for users to play.	Output "local" to console.	As expected
Does pressing the join button print "join" to the console	Mouse press on the button	Menu needs to be navigable for users to play.	Output "join" to console.	As expected

Does pressing host button print "host" to the console	Mouse press on the button	Menu needs to be navigable for users to play.	Output "host" to console.	As expected
Does pressing back button change menus to the main menu	Mouse press on the button	Menu needs to be navigable for users to play.	Return back to main menu.	As expected

A video of the results is embedded below:



Interview with the stakeholders

These are the success criteria that were completed with this prototype:

#	Success Criteria	Justification	Evidence for testing
4	There must be a start menu with clickable buttons: 'Play', 'Settings' and 'Quit'.	This will allow the users to navigate between the different menus to play the game. <i>Stakeholders have also confirmed this in the technical interview.</i>	A screenshot of the main menu.
6	There must be a playing menu that allows users to access and play the different play modes. The buttons should include, "Practise", "2-player", "join" and "host".	This will allow the users to navigate between the different menus to play the game. <i>Stakeholders have also confirmed this in the technical interview.</i>	A screenshot of the play menu.

Me:

I have completed the success criteria that I set out to complete this iteration. I have programmed a fully interactive start and play menu. Although some of the buttons only lead to an output in the console, I hope they will be resolved in future prototypes. With this prototype, I

have built the foundations which will lead nicely onto the development of other features in the game. I thought a loading screen was a necessary addition to prevent the program stalling upon start up while loading a large amount of assets. I tried to make it visually pleasing and have a somewhat professional look.

Summary of what the stakeholders said:

The UI design is clean and professional, and the inclusion of a loading screen was an unexpected but welcome detail that adds polish to the application. Navigation between the menus works smoothly as intended. However, while the structure is there, I am eager to see the actual gameplay mechanics implemented, as currently many buttons only output to the console. It is a solid foundation, but the next iteration needs to focus on functionality over aesthetics.

Considering this interview I have concluded I will start developing the actual game. The networking will come later; however, I want to build a practise arena at least.

Building the practise arena

To have a working practise arena, I will have to abstract out making a stages and character selection menu for now. These can be coded at a future date. Core gameplay is crucial to get started on as it may take too long and cause me to miss the deadline if I make all the menus first and leave it till last.

However, looking ahead I will need to tidy the assets class a little bit by splitting the quite large load function into smaller bits:

```
def load(self):  
    self.load_backgrounds()  
    self.load_buttons()  
    self.load_fonts()  
    self.load_stages()  
  
    self.loaded = True
```

The only change is load stages function, the rest are the exact same except they have just been enclosed in some functions. A practise arena will need a stage and character to get initially working. Within this iteration, I will aim to finish both a working character and stage. Below is the load_stages function:

```
def load_stages(self):
    self.load_volcano_valley()
```

Quite a small function. Hopefully it will grow as I add more stages. Each stage has may have different number of platforms so they will need their own customized functions. All stages will have the same naming convention which is to have no capital letters and underscores between words in their names. This will be used everywhere and will help immensely in error checking.

```
class PractiseScene(scene):
    def __init__(self, manager, assets, sSize, stage, character):
        super().__init__(manager, assets, sSize)
        self.stage = self.assets.getBackground(stage)
        self.stageR = self.stage.get_frect(topleft = (0,0))
        self.platform = self.assets.getPlatform(stage, 1)
        self.platformR = self.platform.get_frect(midbottom = (sSize[0]/2, sSize[1]))

    def update(self, dt, mpos, mbut, mbut_j, keys, keys_j):
        pass

    def draw(self, screen):
        screen.blit(self.stage, self.stageR)
        screen.blit(self.platform, self.platformR)
```

The stage is retrieved using the assets get background function. The one volcano valley platform is also fetched and its rect is made

This is just some temporary code to get an arena working, I


```

def load_volcano_valley(self):
    #The path for this function is the volcano valley folder inside the stages folder
    path = os.path.join(self.stages_path, "volcano_valley")

    #Goes through each platform
    for file in os.listdir(path):
        #Removes file extension for easy storing
        name = file[:file.find(".")]
        platform = pygame.image.load(os.path.join(self.stages_path, file)).convert_alpha()
        platform = platform.subsurface(platform.get_bounding_rect()) #removes transparent pixels

        if name == "platform1":
            ratio = 2.09 #width by height

            platform = pygame.transform.smoothscale(platform, (self.screenSize[1]/2/ratio, self.screenSize[1]/2)) #half the screen

    #Stores each platform in dictionary
    self.platforms[name+"_volcano_valley"] = platform

    self.progressCounter+=1 #Increases loaded images counter by 1

```

As each platform will have different ratio scaling, I will have to use if statements and number each platform. If I used one ratio for all platforms, some would look stretched and not right. All the load volcano valley stage function does, is iterate through each platform, strip them of the transparent pixels, smooth scales them based on each platform then add them to the platforms dictionary using their name with a postfix of

the stage name for easy access. Of course, the progress counter is also incremented for the loading screen.

A new getter for platform assets will have to be written:

```

class PlayMenuScene(scene):
    def __init__(self, manager, asset
    super().__init__(manager, ass

    self.background = self.assets
    self.backgroundRect= self.bac

    practise, practiseG = self.assets.getButton("practise")
    local, localG = self.assets.getButton("local")
    join, joinG = self.assets.getButton("join")
    host, hostG = self.assets.getButton("host")
    back, backG = self.assets.getButton("back")
    self.practiseButton = Button(practise,practiseG, lambda:
        self.manager.change_scene(PractiseScene, self.manager, self.assets, self.sSize, "volcano_valley", "a"),
        center=(self.sSize[0]/2, self.sSize[1]*2.5/8))
    self.localButton = Button(local, localG, lambda:print("local"), center=(self.sSize[0]/2, self.sSize[1]*3.5/8))

```

The play menu scene practise button function has been changed to change to the practise scene when clicked. Volcano valley is the default for now.

```
def getPlatform(self, stage, n):  
    try:  
        p = self.platforms[f"platform{n}_{stage}"]  
        return p  
    except:  
        return "not found"
```

Also stops the game from crashing if a platform is not found.

The naming convention will be the same for all platforms so they can be easily fetched like this! A try except block has been used so that , when loading the actual stage, I can loop through and get all the platforms using their numbers. Then, stop if "not found" is returned because of an error in fetching (it doesn't exist).

Now, I can test the initial assets class code and practise scene code, to ensure that all the menus still work and the new features are compatible.

Test #1

What I'm testing	Input data	Why am I testing this?	Expected outcome	Test results
Does the volcano valley stage and platform show when I press practise	Button press on practise	To get an initial working practise arena to build on	Volcano valley stage is meant to show on screen	Failed to load the game.
Are the Volcano valley platforms the right size	Button press on practise	To get an initial working practise arena to build on	Volcano valley stage and platforms are meant to be correctly scaled.	Failed to load the game.
Do all the menus still function as before	Load up the game. Mouse press on all buttons.	To make sure I haven't broken any of the code, I have written so far.	All buttons should function as normal	Failed to load the game.

Image not found error:

```
File "c:\Users\Basaa\code\NEA1\assets.py", line 127, in load_volcano_valley
    platform = pygame.image.load(os.path.join(self.stages_path, file)).convert_alpha()
FileNotFoundError: No such file or directory: 'c:\Users\Basaa\code\NEA1\assets\stages\platform1.png'.
```

I carefully checked this error to ensure that it's pointing to the right directory and it's not. It should be stages\volcano_valley\.. I returned to the line of the error and realised that it should be using the path specified earlier in the function.

```
def load_volcano_valley(self):
    #The path for this function is the volcano valley folder inside the stages folder
    path = os.path.join(self.stages_path, "volcano_valley")

    #Goes through each platform
    for file in os.listdir(path):
        #Removes file extension for easy storing
        name = file[:file.find(".")]
        platform = pygame.image.load(os.path.join(self.stages_path, file)).convert_alpha()
        platform = platform.subsurface(platform.get_bounding_rect()) #removes transparent pixels

        if name == "platform1":
            ratio = 2.09 #width by height

            platform = pygame.transform.smoothscale(platform, (self.screenSize[1]/2/ratio, self.screenSize[1]/2)) #half the screen

    #Stores each platform in dictionary
    self.platforms[name+"_volcano_valley"] = platform

    self.progressCounter+=1 #Increases loaded images counter by 1
```

Should be path.

Test #2

What I'm testing	Input data	Why am I testing this?	Expected outcome	Test results
Does the volcano valley stage and platform show when I press practise	Button press on practise	To get an initial working practise arena to build on	Volcano valley stage is meant to show on screen	As expected.
Are the Volcano valley platforms the right size	Button press on practise	To get an initial working practise arena to build on	Volcano valley stage and platforms are meant to be correctly scaled.	Platform appears stretched.
Do all the menus still function as before	Load up the game. Mouse press on all buttons.	To make sure I haven't broken any of the code, I have written so far.	All buttons should function as normal	As expected.

Stretch error



After looking at the load volcano function, I realized an error with working out the size of the platform. The ratio is used as a divisor when it should be used as a multiplier.



```
if name == "platform1":
    ratio = 2.09 #width by height
    platform = pygame.transform.smoothscale(platform, ([self.screenSize[1]/2*ratio, self.screenSize[1]/2])) #half the screen
```

Test #3

What I'm testing	Input data	Why am I testing this?	Expected outcome	Test results
Are the Volcano valley platforms the right size	Button press on practise	To get an initial working practise arena to build on	Volcano valley stage and platforms are meant to be correctly scaled.	As expected.



Generalizing stages

While writing the code, I had an idea to just store the platforms under a stage key in the platforms dictionary. This will simplify the platform fetching process and make the code more efficient as the get platform function can now just fetch all platforms under a specific stage name to return.

This change will also allow me to make a generalized stage class, that all stages will inherit from, the differences between them will be the platform sizing and positioning, which will require a specialized constructor for each stage.

Firstly, I will simplify the platform storing process with a small change:

```

def load_volcano_valley(self):
    #The path for this function is the volcano valley folder inside the stages folder
    path = os.path.join(self.stages_path, "volcano_valley")

    #Goes through each platform
    for file in os.listdir(path):
        #Removes file extension for easy storing
        name = file[:file.find(".")]
        platform = pygame.image.load(os.path.join(path, file)).convert_alpha()
        platform = platform.subsurface(platform.get_bounding_rect()) #removes transparent pixels

        if name == "platform1":
            ratio = 2.09 #width by height
            platform = pygame.transform.smoothscale(platform, (self.screenSize[1]/2*ratio, self.screenSize[1]/2)) #half the screen

        #Stores each platform in dictionary
        self.platforms["volcano_valley"][name] = platform

    self.progressCounter+=1 #Increases loaded images counter by 1

```

Rest of the code has been explained

The platforms are now stored under the name of the stage as a key.

In the assets class, the get platform function has been changed into:

```
def getPlatforms(self, name):  
    return [self.platforms[name][key] for key in self.platforms[name]]
```

Name of the stage

List comprehension allows me to quickly fetch all items under a certain key in a dictionary.

I can now make a stage class which all stages will inherit from:

Although I could pass in the images in the parameters. Abstracting it out into the class is cleaner

```
class stage:
    def __init__(self, name, assets):
        #Name/assets needed for retrieval of background/platforms
        self.name = name
        self.assets = assets

        #Each stage has a background
        self.background = self.assets.getBackground(self.name)
        self.backgroundR = self.background.get_frect(topleft = (0,0))

        #Each stage has platforms which are contained in a list for easy access
        self.platforms = self.assets.getPlatforms(self.name)
        #Rect for each of platforms, position determined in children classes
        self.platformsR = [x.get_frect() for x in self.platforms]

    def draw(self, screen):
        screen.blit(self.background, self.backgroundR)
        #Draws all platforms in the list
        #Assumes platforms and the list of rects hve same length
        for i in range(len(self.platforms)):
            screen.blit(self.platforms[i], self.platformsR[i])
```

Only one background but the platforms are stored in a list, in case there's more than one. It also allows for easy access and drawing to the screen.

This function will hopefully be extended as more stages are added. It will be used just to automatically return the correct stage object, given the name.

```
def StageFactory(name, assets, sSize):  
    match name:  
        case "volcano_valley":  
            return VolcanoValley(assets, sSize)  
  
class VolcanoValley(stage):  
    def __init__(self, assets, sSize):  
        super().__init__("volcano_valley", assets)  
        self.platformsR[0].midbottom = (sSize[0]/2, sSize[1])
```

This is an example of a stage inheriting from the parent stage class. This proves the reusability of the stage parent class as it can be used and still customize each stage's platforms' positions.

```

class PractiseScene(scene):
    def __init__(self, manager, assets, sSize, stage, character):
        super().__init__(manager, assets, sSize)
        self.stage = StageFactory(stage, self.assets, self.sSize)

    def update(self, dt, mpos, mbut, mbut_j, keys, keys_j):
        pass

    def draw(self, screen):
        self.stage.draw(screen)

```

A single line initializes the stage ready for drawing to the screen.

The draw function is called which is inherited from the stage parent class, to draw all platforms and background.

Test #4

What I'm testing	Input data	Why am I testing this?	Expected outcome	Test results
Does the practise arena still work?	Button press on practise	To get an initial working practise arena to build on	Volcano valley stage and platforms are meant to be correctly scaled.	As expected.



Creating the character and animation classes

Now that I have finished creating the stage class and optimizing it for future stage additions, I can work on implementing my first character. Of course, I must start by editing the assets class to include loading characters.

However, unlike previously, characters have many images because of the animations. Depending on where I get my assets from, character animations may be in a different format. For example: the animation for run in a particular character has different frames that are each a separate image. They also may all be in one long image that I'm meant to cut up. This is why I need to make a function that can cut up the correct frames, no matter the format. This saves time that would be wasted making a specific function for every character I want to add to the game. The two screen shots below are in the same function:

```

def load_character(self, name, fSize, oSize):
    #name of the character
    #fSize = size of each frame in the spritesheets
    #oSize = size that the character should get scaled to

    path = os.path.join(self.characters_path, name)
    #inits character and its animations in the dict
    self.characters[name] = {}
    self.characters[name]["animations"] = {}

    #for each type of animation (should have its own folder)
    for folder in os.listdir(path):
        #sets up a list for each animation (idle, attack etc)
        self.characters[name]["animations"][folder] = []
        #links to the actual folder
        animation = os.path.join(path, folder)

        #just incase there are multiple spritesheets seperate images for each frame
        #a for loop is used
        for spritesheet in os.listdir(animation):
            #loads the spritesheet after joining paths
            ss = pygame.image.load(os.path.join(animation, spritesheet))
            #retrieves size for splicing
            size = ss.get_size()

            #checks how many rows of frames exist
            for j in range(size[1]//fSize[1]):
                #y is the row
                y = j*fSize[1]

                #checks how many columns of frames exist
                for i in range(size[0]//fSize[0]):
                    #x is the column
                    x = i*fSize[0]

                    #splicing and storing
                    frame = ss.subsurface((x,y), fSize)
                    cutout = frame.subsurface(frame.get_bounding_rect())
                    scaled = pygame.transform.smoothscale(cutout, oSize)
                    self.characters[name]["animations"][folder].append(scaled)

```

The path to the character images is set up based on the name. The dictionary is initialized to store the animations. In case, I want to add weapons in the future, or styles. I have made a separate key for animations

Each folder in the character's directory will be named after an animation and will contain all the animation's frames inside of it. The loop will loop through each animation and store the frames under a key (named after the animation).

The name of the character is used to identify where the frames are stored to load. The frame size is very important, no matter what format the sprite sheets are in, each frame should have the exact same frame size. This function calculates the frames in one picture based of the frame size and loads in each one. The oSize (output size), is used to scale the character appropriately.

For each sprite sheet or image, it retrieves the size (width, height). It then checks how many frames there are across by dividing the image width by frame width. Then it will go through each row and iterate over each frame by multiplying the current frame number by frame size.

It will slice out each frame then remove the transparent pixels. Then it will scale it to the output size and append it to the list for use later.

```
def getAnimations(self, name):  
    #returns all animations of specified character  
    return self.characters[name]["animations"]  
  
def load_characters(self):  
    #knight asset has frame size of 96x84, test out oSizes  
    self.load_character("knight", (96,84), (96,84))
```

The load characters function will be responsible for loading all the characters and will be called in the main load function. The get animations will be needed in the character class to return all the images for each animation and make an animation object based off of them.

A draft of the character class is explained below:

```
def move(self, platforms, dt, mpos, mbut, mbut_j, keys, keys_j):
    #To pull a player down after jump
    self.gravity+=10 #-----needs to be tested-----
```

Gravity will increase as time goes on, to mimic the real-life effects of acceleration and terminal velocity (max_fall_speed). This will abstract out real-life physics with a simpler version.

```

    #Player can move if not recovering, attacking or knocked back
    #Cleaner code, less messy indentations so returns if any are true
    if self.recovery == True or self.attacking == True or self.knockback == True:
        self.rect.centery += min(self.max_fall_speed, self.gravity) * dt
        self.CollideVertical(platforms)
        return
```

If the character can't move, gravity will still take place, so then vertical collisions will need to be checked after movement has taken place. Then it will return as the player shouldn't be able to move

```

    #keys not keys_j → movement keys can be held
    #-----NEEDS TO BE CHANGED, DIFF keys-----
```

```

    if keys[pygame.K_d]:
        self.direction = pygame.math.Vector2(1,0)
        self.facing_left = False #Character moving right
        self.animation_state = "run"
    elif keys[pygame.K_a]:
        self.direction = pygame.math.Vector2(-1,0)
        self.facing_left = True #Character moving left
        self.animation_state = "run"
    else:
        self.direction = pygame.math.Vector2(0,0)
        self.animation_state = "idle"
```

I have added a comment in capitals to remind myself to change this once the settings menu is added to account for the user changing key binds.

It will set the direction and whether the character is facing left based on which direction the user wants to move in. The character will be in idle if the user doesn't move.

```

    #jump logic
    if keys[pygame.K_w] and self.on_ground == True:
        #----needs to be tested----
        self.gravity = -150 #upward force for jumping
        self.animation_state = "jump" #start jump animation
        self.on_ground = False
```

The user should only be allowed to jump if they are on the ground (no flying!). It will set the gravity to a negative number, so the character moves upwards and set the animation state to "jump".

```

    #applies horizontal movement and gravity
    self.rect.centerx += self.direction * self.speed * dt
    self.CollideHorizontal(platforms)
    self.rect.centery += min(self.max_fall_speed, self.gravity) * dt
    self.CollideVertical(platforms)
```

For the platform collision detection, the horizontal collisions will be checked first (order doesn't matter), after applying the horizontal movements BUT before applying the vertical movements, to avoid weird teleportation issues (as I have seen with previous games). After ensuring that horizontal movement is limited if the player collides, vertical collisions will be checked in the same way.

```
def draw(self, screen):
    #Draws character to screen
    screen.blit(self.image, self.rect)
```

Draws the character to the screen, for players to see.

#THIS FUNCTION SHOULD BE OVERRIDED AND OPTIMISED FOR EACH CHARACTER

```
def load_animations(self,):
    animations = self.assets.getAnimations(self.name)
    self.animations = {}
    #Gets the standard animations from the assets
    #Creates an animation object based off each one
    #Values should be overridden
    self.animations["idle"] = animation(animations["idle"], 0.5, True)
    self.animations["jump"] = animation(animations["jump"], 1.25, False)
    self.animations["run"] = animation(animations["run"], 1, True)
    self.animations["attack1"] = animation(animations["attack1"], 1, False)
    self.animations["attack2"] = animation(animations["attack2"], 1, False)
```

Retrieves the animations, sets a dictionary attribute that will be used later, then extracts image lists and creates an animation object of them to store in the dictionary.

As it says on the top, each character will have a different number of frames for each animation and so each character should be adjusted slightly differently.

```
def CollideHorizontal(self, platforms):
    #for each platform
    for platform in platforms:
        #if player collides with platform
        if self.rect.colliderect(platform):
            if self.direction[0]>0: #moving right
                self.rect.right = platform.left
            if self.direction[0]<0: #moving left
                self.rect.left = platform.right
```

If the player moves horizontally and collides, then they must have collided with the right or left of the platform, so the y coordinate is adjusted to the right or the left of the platform, to not allow it to move further.

```
def CollideVertical(self, platforms):
    #for each platform
    for platform in platforms:
        #if player collides with platform
        if self.rect.colliderect(platform):
            if self.gravity > 0: #moving down
                self.rect.bottom = platform.top
                self.on_ground = True # player hit ground
            if self.gravity < 0: #moving up
                self.rect.top = platform.bottom
```

If the player moves vertically and collides, then they must have collided with the top or bottom of the platform, so the y coordinate is adjusted to the top or the bottom of the platform.

Right or left return x coordinates.
Top or bottom return y coordinates.


```
def getFrame(self):
    return self.frames[self.index]
```

Returns the current image that should be displayed.

```
def update(self, dt):
    #if animation finished, return immediately
    if self.done == True:
        return "finished"

    #increment stopwatch by time elapsed since last frame
    self.stopwatch += dt

    if self.stopwatch >= self.frame_duration:
        #reset stopwatch
        self.stopwatch = 0

        #move to next frame
        self.index += 1

        #if index is past the last frame
        if self.index >= len(self.frames):
            #if animation should loop, return to 0
            if self.loop == True:
                self.index = 0
            else:
                self.index = len(self.frames)-1
                #animation finished
                self.done = True
```

If the animation is done, it returns finished, so that the animation state can be changed.

Increments by dt which is the time elapsed since last frame. Then checks if the time spent is greater than how long an animation frame should last on the screen for. If it is, then it resets the stopwatch, increments index and checks if the animation is done or if it should loop. If it should loop, then index is set to 0 again, otherwise its set to the last animation frame and done is set to true. Then it returns whether the current frame is a iFrame, rFrame or aFrame in a list for easy access.

```
#when i'm looking at frames to check whether they should be
#recovery or attack or invincible, i don't auto zero-index
#so self.index+1 to account for that
return [self.index+1 in self.iFrames, self.index+1 in self.rFrames, self.index+1 in self.aFrames]
```

```
class animation:
    def __init__(self, frames, animation_time, loop, iFrames=[0], rFrames=[0], aFrames=[0]):
        # iFrames = frames that character should be invincible in
        # rFrames = frames that character can't attack in
        # aFrames = frames that should damage the enemy if in contact
        self.frames = frames
        self.frame_duration = animation_time/len(self.frames)
        self.loop = loop

        self.index = 0
        self.stopwatch = 0
        self.done = False

        self.iFrames = iFrames
        self.rFrames = rFrames
        self.aFrames = aFrames

    def reset(self):
        self.index = 0
        self.stopwatch = 0
        self.done = False
```

The animation will need the number of frames and animation time to calculate how long each frame should stay on the screen for. Loop is a Boolean set to determine whether the animation can be looped like idle and run animations. Attacks shouldn't loop. The index is the current frame and stop watch is used to workout whether index should be incremented. Done is set to true if the animation is finished.

iFrames, rFrames and aFrames are lists containing which frames should be invincibility, recovery and attack frames.

To be efficient, if an animation ends and doesn't loop then it will reset. So, another animation object doesn't have to be created. It can just be reused.

There are a few new additions to the stage class

```
class VolcanoValley(stage):
    def __init__(self, assets, sSize):
        super().__init__("volcano_valley", assets)
        self.platformsR[0].midbottom = (sSize[0]/2, sSize[1])

        #Sets the spawn points for volcano valley
        mainPlatSize = self.platforms[0].get_size()
        width = mainPlatSize[0]
        height = mainPlatSize[1]
        self.spawnPoint1 = (2*width/8, self.platformsR[0].midtop[1])
        self.spawnPoint2 = (6*width/8, self.platformsR[0].midtop[1])

    def getSpawnPoint(self, n):
        if n == 1:
            return self.spawnPoint1
        elif n == 2:
            return self.spawnPoint2
```

There is only one platform in volcano valley, this just sets the default stage spawn points which should be different for player 1 and 2. A getter has been made to return the spawn point. This will be used at the start of any match to initialize positions.

Test #5

I have made a knight in the practise scene that I will be testing, this knight's update and draw functions are called in the scene's update and draw functions with the suitable parameters.

```
class PractiseScene(scene):
    def __init__(self, manager, assets, sSize, stage, character):
        super().__init__(manager, assets, sSize)
        self.stage = StageFactory(stage, self.assets, self.sSize)
        self.Player1 = Character("knight", self.assets, self.stage.getSpawnPoint(1), 1)

    def update(self, dt, mpos, mbut, mbut_j, keys, keys_j):
        self.Player1.update(self.stage.getPlatforms(), dt, mpos, mbut, mbut_j, keys, keys_j)

    def draw(self, screen):
        self.stage.draw(screen)
        self.Player1.draw(screen)
```

What I'm testing	Input data	Why am I testing this?	Expected outcome	Test results
------------------	------------	------------------------	------------------	--------------

Is the character drawn onto the screen in the appropriate position?	Pressing the practise arena button.	The user needs to spawn in a suitable position to play the game.	Character is drawn on top of the platform.	Failed to load the game.
Does moving work?	"A", "D"	So players can move in matches and in the practise arena.	Character should respond to A and D by moving left and right	Failed to load the game.
Does the run animation work?	"A", "D"	Allows the user to see their character moving.	The character should switch between frames in order to display a running animation.	Failed to load the game.
Is gravity being simulated correctly	Practise arena button being pressed	Allows players to move around with simulated gravity/collision detection and don't just fly.	Player's speed should increase as they fall.	Failed to load the game.
Does collision detection work correctly?	Movement	So that players can move around with simulated collision detection and don't clip platforms.	If a player, approaches the platform rect, they should be pushed back and not go inside it.	Failed to load the game.

Error #1

```

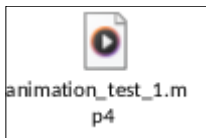
self.Player1.update(self.stage.getPlatforms(), dt, mpos, mbut, mbut_j, keys, k
#####
File "c:\Users\Basaa\code\NEA1\characters.py", line 38, in update
self.move(platforms, dt, mpos, mbut, mbut_j, keys, keys_j)
#####
File "c:\Users\Basaa\code\NEA1\characters.py", line 86, in move
self.rect.centerx += self.direction * self.speed * dt
TypeError: unsupported operand type(s) for +=: 'float' and 'pygame.math.Vector2'

```

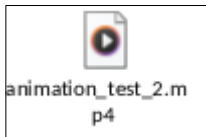
I forgot that the vector needs to automatically add to both x and y positions. So I changed the line from centerx to center.

Error #2

After solving this issue, something very weird occurs, that I can only show below in a video:



I decided to move more in the second video, before the character falls off screen.



Debugging

From these two videos I can say:

What I'm testing	Input data	Why am I testing this?	Expected outcome	Test results
Is the character drawn onto the screen in the appropriate position?	Pressing the practise arena button.	The user needs to spawn in a suitable position to play the game.	Character is drawn on top of the platform.	Spawn point is too far.
Does moving work?	"A", "D"	So players can move in matches and in the practise arena.	Character should respond to A and D by moving left and right	As, expected.
Does the run animation work?	"A", "D"	Allows the user to see their character moving.	The character should switch between frames in order to display a running animation.	Facing left doesn't work. Its being flipped the wrong way.
Is gravity being simulated correctly	Practise arena button being pressed	Allows players to move around with simulated gravity/collision detection and don't just fly.	Player's speed should increase as they fall.	Player is falling at a constant rate, when it should be increasing.
Does collision detection work correctly?	Movement	So that players can move around with simulated collision detection and don't clip platforms.	If a player, approaches the platform rect, they should be pushed back and not go inside it.	User stopped at the platform rect, which is expected.

Testing assets

First, I want to make sure that the assets are being loaded in correctly, so I will set up a separate file, where I can just display the assets one by one onto the screen.

I cleared the main file and set up a small rig with one function and the assets function initialized:

```

assets = assets((1280,720), (96,96))
assets.load()
animations = assets.getAnimations("knight")

type = 0
frame = 0

```

```

def type_changer(keys_j):
    global type
    global frame
    match type:
        case 0 :
            t = "idle"
        case 1:
            t = "run"
        case 2:
            t = "jump"
        case 3:
            t = "attack1"
        case 4:
            t = "attack2"

```

```

if keys_j[pygame.K_e]:
    type +=1
    frame = 0
    if type == 5:
        type = 0
if keys_j[pygame.K_q]:
    frame = (frame+1)%len(animations[t])

```

```

screen.blit(animations[t][frame], animations[t][frame].get_frect(center=(1280/2, 720/2)))

```

This code was written quickly just to visually see if the assets are not flipped weirdly and are all okay. This lets me change the animation type with e and frame with q. The frame is incremented with q but with the mod function so that it wraps around automatically to 0 if it goes over that animation's total frames. If the type is changed, then the frame is set to 0 to prevent any items not existing errors (different total number of frames)

Results of this test



As you can see from the video, all the assets are loaded correctly. The only issue is that the character is quite blurry. I fixed this by removing the smooth scale and replacing with a normal scale.

The issue is that smooth scale uses a bilinear filter: which calculates the average of the nearest 4 pixels' colours. This is suitable to use when scaling images down, but not when the output is the same size or larger as it can lead to a blurry image. This is why it works perfectly with the buttons, as they are being scaled down from a large res that imagen gave to around 90px.

Now that I know assets are being loaded correctly, this means the facing left issues are with the actual code itself. After reviewing the code, I have concluded that it is probably because of how the images are flipped:

```

self.image = pygame.transform.flip(self.image, False,True ) if self.facing_left else self.image
self.rect = self.image.get_frect(center = self.rect.center)

```

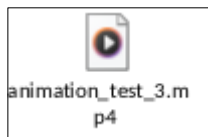
This line is in the update function which runs every frame. If an animation frame stays for more than one frame then it will be flipped more than once, because the image is saved each time to the same attribute. I also noticed that the Boolean values should be swapped to ensure that it is being swapped in the right axis.

To fix these problems, I decided to remove the line completely and instead pass in facing left into the get frame function from the animations class:

```
def getFrame(self, facing_left):
    return pygame.transform.flip(self.frames[self.index], facing_left, False)
```

This way, nothing is saved permanently. If it is facing left, then that will be true so it will flip it parallel to the y axis.

The result is looking much better:



What I can tell from this video:

What I'm testing	Input data	Why am I testing this?	Expected outcome	Test results
Is the character drawn onto the screen in the appropriate position?	Pressing the practise arena button.	The user needs to spawn in a suitable position to play the game.	Character is drawn on top of the platform.	Spawn point is still too far.
Does the run animation work?	"A", "D"	Allows the user to see their character moving.	The character should switch between frames in order to display a running animation.	The animation has a stretch during the first few frames, but they are working.
Is gravity being simulated correctly	Practise arena button being pressed	Allows players to move around with simulated gravity/collision detection and don't just fly.	Player's speed should increase as they fall.	Player is falling at a constant rate, when it should be increasing.

To fix the spawn point being too far, I will review the stage code for setting the spawn point:

```
#Sets the spawn points for volcano valley
mainPlatSize = self.platforms[0].get_size()
width = mainPlatSize[0]
height = mainPlatSize[1]
self.spawnPoint1 = (2*width/8, self.platformsR[0].midtop[1])
self.spawnPoint2 = (6*width/8, self.platformsR[0].midtop[1])
```

The problem is with setting the x coordinates, they are current just set to a fraction of the width when they should be relative to the platform's x coordinates too:

```
#Sets the spawn points for volcano valley
mainPlatSize = self.platforms[0].get_size()
width = mainPlatSize[0]
height = mainPlatSize[1]
self.spawnPoint1 = (self.platformsR[0].centerx-width/4, self.platformsR[0].midtop[1])
self.spawnPoint2 = (self.platformsR[0].centerx+width/4, self.platformsR[0].midtop[1])
```

The change makes it so that the spawn points are evenly spaced from the middle by a quarter of the total platform width.

To fix the knight being stretched in the first few frames, I first made sure that the assets are not stretched, which they aren't. Then, I looked at the assets load characters function to see if I'm scaling correctly and realized that I'm not. Rather than specifying a fixed size for the character to be scaled to that could easily not be the right size in terms of scale, I specified a scale factor. Then I just scaled the cutout to that scale factor:

```
#splicing and storing
frame = ss.subsurface((x,y), fSize)
cutout = frame.subsurface(frame.get_bounding_rect())
#scale by takes 2 inputs, image and scale factor ( 1 value )
scaled = pygame.transform.scale_by(cutout, sf)
self.characters[name]["animations"][folder].append(scaled)
```

I also changed this line in the character update function:

```

def update(self, platforms, dt, mpos, mbut, mbut_j, keys, keys_j):
    #move function checks movement/attack keys and changes animations accordingly
    self.move(platforms, dt, mpos, mbut, mbut_j, keys, keys_j)

    #increments the current animation frame
    values = self.current_animation.update(dt)
    if values != "finished":
        self.invincible = values[0]
        self.recovery = values[1]
        self.attacking = values[2]
    else:
        self.invincible = False
        self.recovery = False
        self.attacking = False

    #needs to be updated after every move function call
    self.current_animation = self.animations[self.animation_state]
    self.image = self.current_animation.getFrame(self.facing_left)

    self.rect = self.image.get_frect(midbottom = self.rect.midbottom)

```

A common issue in assets is that the center is misaligned, so I decided to change it to set the midbottom to the previous frame's midbottom, to avoid clipping issues.

I need to also fix the gravity issue by playing around with max fall speed, gravity increase each frame and the jump. After testing some values, max fall speed is now set to 800, gravity increase is $800 \cdot dt$ (all movements need to be multiplied by dt , which I forgot to do) and the jump gravity is set to -400.

Test #6

What I'm testing	Input data	Why am I testing this?	Expected outcome	Test results
Is the character drawn onto the screen in the appropriate position?	Pressing the practise arena button.	The user needs to see their character to play the game.	Character is drawn on top of the platform.	As, expected.
Is gravity being simulated correctly	Practise arena button being pressed	Allows players to move around with simulated gravity/collision detection and don't just fly.	Player's speed should increase as they fall.	As, expected.

Adding attacks and jumps!

Firstly, fixing those bugs made me realise that the done attribute in animations class is useless. It waits a frame before telling the character update function that the animation is done. I will start by changing this first, so that it doesn't cause any bugs with attacks waiting a frame before they are finished displaying.

```

def getFrame(self):
    return self.frames[self.index]

def update(self, dt):
    #if animation finished, return immediately
    if self.done == True:
        return "finished"

    #increment stopwatch by time elapsed since last frame
    self.stopwatch += dt

    if self.stopwatch >= self.frame_duration:
        #reset stopwatch
        self.stopwatch = 0

        #move to next frame
        self.index += 1

        #if index is past the last frame
        if self.index >= len(self.frames):
            #if animation should loop, return to 0
            if self.loop == True:
                self.index = 0
            else:
                self.index = len(self.frames)-1
                #animation finished
                self.done = True

    #when i'm looking at frames to check whether they should be
    #recovery or attack or invincible, i don't auto zero-index
    #so self.index+1 to account for that
    return [self.index+1 in self.iFrames, self.index+1 in self.rFrames, self.index+1 in self.aFrames]

def update(self, dt):

    #increment stopwatch by time elapsed since last frame
    self.stopwatch += dt

    if self.stopwatch >= self.frame_duration:
        #reset stopwatch
        self.stopwatch = 0

        #move to next frame
        self.index += 1

        #if index is past the last frame
        if self.index >= len(self.frames):
            #if animation should loop, return to 0
            if self.loop == True:
                self.index = 0
            else:
                self.index = len(self.frames)-1
                self.reset()
                return "finished"

    #when i'm looking at frames to check whether they should be
    #recovery or attack or invincible, i don't auto zero-index
    #so self.index+1 to account for that
    return [self.index+1 in self.iFrames, self.index+1 in self.rFrames, self.index+1 in self.aFrames]

```

If the index is past the frame, that means it has already finished the entire animation, including showing the last frame. Therefore, it should just return finished right there and then. It should also reset, so it can be reused later with the current frame being 0.

Clarifying frame status, attacking and jumping

Before I continue further, I need to clarify exactly what each of frame statuses are and tidy up the code to reflect that. Now that I'm in a position where I have written the code for some animations, I understand how they should fit in together.

Status effects:

- Certain frames grant invincibility to allow the user to attack without getting damaged (allows users to think of clever tactics rather than just be afraid to attack)
- Recovery frames will happen after an attack and stops users from spamming attacks
- Knockback will push users back and stun them for a short period of time (not allowing them to move or attack), the more attacks they land on the opponent, the higher the chance of knockback.
- Attacking frames will be part of the actual attack animation and will damage the opponent if the hit lands within those frames.
- Busy frames will lock the animation and not allow the user to perform any other animation unless they are knocked back.

Due to recovery frames happening after an animation and busy frames happening during an animation, I have changed the parameters in the animation class and removed looping as they are useless with busy frames:

```
def __init__(self, frames, animation_time, bFrames=[], iFrames=[], aFrames=[]):
    # iFrames = frames that character should be invincible in
    # bFrames = busy frames, animation is LOCKED
    # aFrames = frames that should damage the enemy if in contact
    self.frames = frames
    self.frame_duration = animation_time/len(self.frames)

    self.index = 0
    self.stopwatch = 0

    self.iFrames = iFrames
    self.bFrames = bFrames
    self.aFrames = aFrames
```

Jumps are animations that need to be locked, meaning that they shouldn't be able to start any other animation while jumping. For example, running while jumping should not happen. However, players should be able to move while jumping, just not start any animation.

Attacks should be partially locked, meaning that before a certain frame (usually before the frames that damaging the opponent) they should be able to cancel the attack. This can encourage smart interesting gameplay where a player should think about faking an attack to gain an advantage, which will make the game more fun.

Before I can implement these features, I want to neaten up my code. I feel that the character and animation functions are becoming too large and convoluted, so I have decided to make a few more functions for each task, and get rid of the list that the animation update returns, which is this:

```
#when i'm looking at frames to check whether they should be
#recovery or attack or invincible, i don't auto zero-index
#so self.index+1 to account for that
return [self.index+1 in self.iFrames, self.index+1 in self.rFrames, self.index+1 in self.aFrames]
```

It doesn't look very intuitive to use as I would have to memorise the placement of each Boolean value.

I also want to remove this large if statement that accommodates this list:

```
#increments the current animation frame
values = self.current_animation.update(dt)
if values != "finished":
    self.invincible = values[0]
    self.recovery = values[1]
    self.attacking = values[2]
else:
    self.invincible = False
    self.recovery = False
    self.attacking = False
```

To remove these, I first changed the animations class update function into:

```

def update(self, dt):
    #increment stopwatch by time elapsed since last frame
    self.stopwatch += dt

    if self.stopwatch ≥ self.frame_duration:
        #reset stopwatch
        self.stopwatch = 0

        #move to next frame
        self.index +=1

        #if animation is finished, reset to be reused
        # and return true
        if self.index ≥ len(self.frames):
            self.reset()
            return True

    return False

```

I removed the previous return finished and use of the loop attribute. If I need to say that an animation is finished, I can simply return true. It doesn't matter if it can loop, as it will just be called again, if the play doesn't move (idle animation is looped for example). Returning true means I can set busy to False, as it has finished the animation. Returning False is not needed but included anyways to say it hasn't finished.

So now, to still send the information about invincibility, busy frames (not recovery anymore) and attacking frames, I will simply create getter functions. These are very simple and the same as before so don't need explanation:

```

def getIstatus(self):
    '''returns whether or not the player
    should be invincible'''
    return (self.index+1 in self.iFrames)

def getBstatus(self):
    '''returns whether or not, the player should
    be able to cancel their attack in the
    current frame'''
    return (self.index+1 in self.bFrames)

def getAstatus(self):
    '''returns whether or not, the player
    should actually damage the enemy'''
    return (self.index+1 in self.aFrames)

```

In the character class, anything to do with updating status can be placed in this function:

```
def updateStatus(self, dt):  
    #get frame status  
    self.invincible = self.current_animation.getIstatus()  
    self.attacking = self.current_animation.getAstatus()  
  
    #if animation is finished, character is not busy  
    if self.current_animation.update(dt) == True:  
        self.busy = False  
        self.animation_state = "idle"  
    else:  
        #otherwise get frame status to check  
        self.busy = self.current_animation.getBstatus()  
  
    #Allows the player to recover from attacks  
    self.Recover()
```

Retrieves whether the player should be invincible or attacking from the animation class.

No matter what animation it is, if it is finished then the player will no longer be busy and should be set to the default idle. If not, then it will check via the getter function.

This recover function is explained below.

These set recover functions are called after an attack has been initiated. They are meant to be overridable for each character, for easy editing.

```
def SetRecover1(self,):  
    #How many recovery frames for attack 1  
    self.recovery = 3  
def SetRecover2(self):  
    #How many recovery frames for attack 2  
    self.recovery = 500  
def Recover(self):  
    #if the player is not currently attacking, they can recover  
    if self.animation_state != "attack1" and self.animation_state != "attack2":  
        self.recovery = max(0, self.recovery-1)
```

This function will be called in the move function to allow the character to recover when they are not attacking.

The recovery attribute is better as a number as it will indicate the number of frames that the player shouldn't be able to attack right after attacking. So, if the user is not currently attacking (animation state is not attack1 or 2) then it should lower recovery. Once the recovery attribute hits 0 (meaning they have fully recovered) then they can attack. So in the recover function, recovery is decremented each frame until 0. I have used a max function to limit the variable to nothing below 0 while decrementing to save space.

I have made an attack function that uses the code explained previously:

```
def attack(self, keys_j):  
    #if player has recovered  
    if self.recovery == 0 and self.on_ground == True and not self.busy:  
        if keys_j[pygame.K_f]:  
            #attack, lock animation and set recovery  
            self.animation_state = "attack1"  
            self.SetRecover1()  
        elif keys_j[pygame.K_r]:  
            self.animation_state = "attack2"  
            self.SetRecover2()
```

If the user doesn't have to recover and is on the ground and not animation locked, they should be able to attack. After attacking the set recover functions are called.

I have made set functions for recover, because they are easily overridden for customisability.

Finally, the character update function looks much better now:

```
def update(self, platforms, dt, mpos, mbut, mbut_j, keys, keys_j):
    #move function checks movement/attack keys and changes animations accordingly
    self.move(platforms, dt, mpos, mbut, mbut_j, keys, keys_j)
    self.attack(keys_j)

    #updates animation and statuses
    self.updateStatus(dt)

    #needs to be updated after every move function call
    self.current_animation = self.animations[self.animation_state]
    #updates the image and rect
    self.image = self.current_animation.getFrame(self.facing_left)
    self.rect = self.image.get_frect(midbottom = self.rect.midbottom)
```

The move function is called, then the attack function and then the update status function. Then, the current animation is updated before the image and rect are too.

```
#THIS FUNCTION SHOULD BE OVERRIDED AND OPTIMISED FOR EACH CHARACTER
def load_animations(self,):
    animations = self.assets.getAnimations(self.name)
    self.animations = {}
    #Gets the standard animations from the assets
    #Creates an animation object based off each one
    #Values should be overridden
    self.animations["idle"] = animation(animations["idle"], 0.5, )
    self.animations["jump"] = animation(animations["jump"], 1.25, [i for i in range (20)])
    self.animations["run"] = animation(animations["run"], 1, )
    self.animations["attack1"] = animation(animations["attack1"], 1, [i for i in range (20)])
    self.animations["attack2"] = animation(animations["attack2"], 1, [i for i in range (20)])
```

For now, I will specify that all attack frames are busy frames through the parameters in the function below. I will workout which frames should and shouldn't be later.

Testing attacks and jumps

What I'm testing	Input data	Why am I testing this?	Expected outcome	Test results
Does jumping and animation work, are they locked off?	"w"	This will allow the user to jump and see the animation of their character jumping.	Shows the character jumping animation, doesn't allow other animations to start while jumping.	The character jumps, but no jumping animation works.
Do attacks work, are they locked off?	"f" "r"	Allows the player to use attacks to test out characters	Plays the attacking animation of the character,	Attacks don't respond to button presses.

			doesn't allow other animations to start while jumping.	
Are recovery frames working, do they stop spamming attacks?	Repeated "r"	Stops users from spamming attacks.	The player should not be able to instantly start another attack after the animation has ended.	Attacks don't respond to button presses.



animation_test_4.m
p4

Fixing logic errors

Since the jump works but the animation doesn't, I have a suspicion that the error is to do with the new busy attribute. After thinking about the function below for a while:

```
def update(self, platforms, dt, mpos, mbut, mbut_j, keys, keys_j):
    #move function checks movement/attack keys and changes animations accordingly
    self.move(platforms, dt, mpos, mbut, mbut_j, keys, keys_j)
    self.attack(keys_j)

    #updates animation and statuses
    self.updateStatus(dt)

    #needs to be updated after every move function call
    self.current_animation = self.animations[self.animation_state]
    #updates the image and rect
    self.image = self.current_animation.getFrame(self.facing_left)
    self.rect = self.image.get_frect(midbottom = self.rect.midbottom)
```

I realised that I didn't listen to my own comments. Updating the current animation should go before updating the statuses.

This is because after pressing the jump button, busy is set to true. However, the animation is still set to idle which doesn't have any busy frames, so it is reset. Then because while jumping the player doesn't do anything. The animation state is reverted to idle.

After moving that line:



animation_test_5.m
p4

What I'm testing	Input data	Why am I testing this?	Expected outcome	Test results
Does jumping and animation work, are they locked off?	"w"	This will allow the user to jump and see the animation of their character jumping.	Shows the character jumping animation, doesn't allow other animations to start while jumping.	As, expected.
Do attacks work, are they locked off?	"f" "r"	Allows the player to use attacks to test out characters	Plays the attacking animation of the character, doesn't allow other animations to start while jumping.	As, expected.
Are recovery frames working, do they stop spamming attacks?	Repeated "r"	Stops users from spamming attacks.	The player should not be able to instantly start another attack after the animation has ended.	As, expected.

Moving while jumping

In the character move function, vertical movements are written twice. Whether or whether not the character is busy, vertical movements and collision checking needs to happen. Therefore, to tidy this up, I will simply move the vertical movement code to another function called moveV and rename the current function moveH, V and H signifying vertical and horizontal movements.

MoveV function:

```
def moveV(self, platforms, dt, keys):
    #To pull a player down after jump
    self.gravity+=800*dt #-----needs to be tested-----

    #jump logic
    if keys[pygame.K_w] and self.on_ground == True:
        self.gravity = -400 #upward force for jumping
        self.animation_state = "jump" #start jump animation
        self.on_ground = False #no longer on the ground
        self.busy = True #lock

    self.rect.centery += min(self.max_fall_speed, self.gravity) * dt
    self.CollideVertical(platforms)
```

To allow the player to move horizontally while jumping, I need to allow horizontal movement to happen when the character animations are locked but the animation state is jump, but it can't change the animation state. I have edited the moveH function to show this:

```
def moveH(self, platforms, dt, keys):
    #Player can move if locked or knockedback
    #returns if any are true
    if self.knockback or (self.busy and self.animation_state != "jump"):
        return

    #keys not keys_j → movement keys can be held
    #-----NEEDS TO BE CHANGED, D

    #move right
    if keys[pygame.K_d]:
        self.direction = pygame.math.Vector2(1,0)
        self.facing_left = False #Character moving right
        if self.on_ground:
            self.animation_state = "run"

    #move left
    elif keys[pygame.K_a]:
        self.direction = pygame.math.Vector2(-1,0)
        self.facing_left = True #Character moving left
        if self.on_ground:
            self.animation_state = "run"

    #otherwise idle
    else:
        self.direction = pygame.math.Vector2(0,0)
        if self.on_ground:
            self.animation_state = "idle"

    #applies horizontal movement and gravity
    self.rect.center += self.direction * self.speed * dt
    self.CollideHorizontal(platforms)
```

Horizontal movement will be skipped if the character is busy with an animation that is not jump or they have been knocked back. So, it will run if the player is jumping. However, I don't want it to change the animation state, so I added if statements that check if the player is on ground before changing the animation states.

Testing movement while jumping

What I'm testing	Input data	Why am I testing this?	Expected outcome	Test results
Whether I can move while jumping	"w" and "a" or "d"	Allows users to move while jumping. Some stage may have different platforms to jump onto.	The jumping animation should still play but I should be able to move left and right while jumping.	As, expected.

Finishing touches

Now that the practise arena is almost finished, I will clean up the practise scene code by making a character factory and adding a knight class that will inherit the current character parent class. A character factory function will work very well with a character selection menu to allow users to select which characters they want to play.

```
class knight(Character):
    def __init__(self, assets, spawn_point, number):
        super().__init__("knight", assets, spawn_point, number)

    def SetRecover1(self):
        self.recovery = 2
    def SetRecover2(self):
        self.recovery = 2

    def load_animations(self,):
        animations = self.assets.getAnimations(self.name)
        self.animations = {}
        #Gets the standard animations from the assets
        #Creates an animation object based off each one
        #Values should be overridden
        self.animations["idle"] = animation(animations["idle"], 0.75)
        self.animations["jump"] = animation(animations["jump"], 1.25, [i for i in range(20)])
        self.animations["run"] = animation(animations["run"], 0.9)
        self.animations["attack1"] = animation(animations["attack1"], 0.5, [2,3,4,5], [2,3,4])
        self.animations["attack2"] = animation(animations["attack2"], 1, [3,4,5,6], [3,4,5])

def CharacterFactory(name, assets, spawn_point, number):
    match name:
        case "knight":
            return knight(assets, spawn_point, number)
```

The default amount of recovery frames for each character will be 2 as this gives a nice balance between spamming and fast attacks. 3 felt too long in testing.

Here I have set up the knight animations by looking at the assets and deciding which should be attack frames and busy frames by seeing how long each slash lasts for.

Based on what name is given, the character factory will return the right character with a switch statement. Very similar to the stage factory.

I also want to edit all the asset sizes to make them relative to the screen size. I have decided to leave this until now, as I am happy with the current sizes of characters, buttons and platforms, and want to make these sizes look as proportionate as they are now with any monitor resolution.

While editing the loading scene bar sizes, I realised that they were being stretched so I switched to scaling by a scale factor:

```
class LoadingScene(scene):
    def __init__(self, manager, assets, sSize):
        super().__init__(manager, assets, sSize)
        #retrieves loading screen assets
        self.LoadingScreen, self.LoadingBar, self.bar_image = self.assets.getLoadingScreen()

        self.LoadingScreenRect = self.LoadingScreen.get_frect(topleft = (0,0))

        self.LoadingBar = pygame.transform.smoothscale_by(self.LoadingBar, 0.25)
        self.LoadingBarRect = self.LoadingBar.get_frect(center = (self.sSize[0]/2, self.sSize[1]/2))

        #Bar is scaled to fit within loading bar, max width stored
        self.bar_image = pygame.transform.smoothscale_by(self.bar_image, (0.16, 0.05))
        self.BAR_MAXWIDTH = self.bar_image.get_width() #constant
        self.bar_width = self.BAR_MAXWIDTH
        self.barRect = self.bar_image.get_frect(center=(self.sSize[0]/2, self.sSize[1]/2+4)) #slight offset
```

Scale by functions support two values, scale parallel to x and y. This is needed with the bar as the asset is just a gold bar. It needs to be separately purposely stretched to fit within the bigger holder bar.

This made the loading bar look much better:



After changing the loading bar, I decided to switch to scaling everything by scale factor. Thinking about it, there is nothing to lose and everything to gain. I don't have to estimate ratios by looking at assets which drastically lowers the chances of incorrect scaling, also, a scale factor is just one value that must be changed rather than an x and y.

In the assets class:

```
#development was in 1280x720
#scale factor for any screen size
self.SF = min(self.screenSize[0]/1280, self.screenSize[1]/720)
```

By calculating the minimum, I am ensuring that the asset fits on the screen.

```
def load_characters(self):
    #knight asset has frame size of 96x84, test out oSizes
    #multiply by sf to scale up to any screen size
    self.load_character("knight", (96,84), 2*self.SF)
```

```
def load_buttons(self):
    for file in os.listdir(self.buttons_path):
        #All buttons are png
        #Allows transparent pixels, then removes them
        image = pygame.image.load(os.path.join(self.buttons_path, file)).convert_alpha()
        image = image.subsurface(image.get_bounding_rect())
        #Scales using button size
        image = pygame.transform.smoothscale_by(image, self.buttonSF*self.SF)
```

```
class SceneManager:
    def __init__(self, network, server, sWidth, sHeight):
        self.network = network
        self.server = server
        self.assets = assets((sWidth, sHeight), 0.08)
```

A button scale factor of 0.08. I got this value by testing many times over and just seeing if the buttons were sized appropriately.

In a practise arena, the player should not be able to die by jumping off the platforms. I have made a no death function that uses the spawn point function to place the user back at the spawn point if they fall off:

```
def noDeath(self):
    #Checks whether player is out of screen
    if self.Player1.rect.bottom > self.sSize[1] or self.Player1.rect.top < 0 or self.Player1.rect.right > self.sSize[0] or self.Player1.rect.left < 0:
        self.Player1.rect.midbottom = self.stage.getSpawnPoint(1)
```

It simply checks if the character's borders are out of the screen's borders, by using the right/left and top/bottom attributes. If they are, it will place the bottom of the player on the spawn point (the bottom should be placed, as placing the centre will cause the player to be inside the platform).

As per the usability features, the practise arena should have an exit and characters button, to allow users to select a different character or to go back to the joining menu. The button and assets class makes it easy to add to the practise scene code:

```
class PractiseScene(scene):
    def __init__(self, manager, assets, sSize, stage, character):
        super().__init__(manager, assets, sSize)
        self.stage = StageFactory(stage, self.assets, self.sSize)
        self.Player1 = CharacterFactory("knight", self.assets, self.stage.getSpawnPoint(2), 2)

        #back button to joining menu, in top left corner
        exit,exitG = self.assets.getButton("exit")
        self.exitButton = Button(exit, exitG, lambda: self.manager.change_scene(PlayMenuScene, self.manager, self.assets, self),
                                topleft=(self.sSize[0]/128, self.sSize[1]/128))

        #characters button to character selection menu, topleft corner
        characters, charactersG = self.assets.getButton("characters")
        self.charactersButton = Button(characters, charactersG, lambda: print("characters scene"),
                                       topleft = (exit.get_width() + (self.sSize[0]/100), self.sSize[1]/128))

    def update(self, dt, mpos, mbut, mbut_j, keys, keys_j):
        self.Player1.update(self.stage.getPlatforms(), dt, mpos, mbut, mbut_j, keys, keys_j)
        self.exitButton.update(mpos, mbut)
        self.charactersButton.update(mpos, mbut)
        #removes deaths
        self.noDeath()

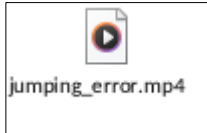
    def draw(self, screen):
        self.stage.draw(screen)
        self.Player1.draw(screen)
        self.exitButton.draw(screen)
        self.charactersButton.draw(screen)
```

What I'm testing	Input data	Why am I testing this?	Expected outcome	Test results
Are all buttons, loading bars, characters, platforms, backgrounds scaled correctly no matter the screen resolution?	Change program data width and height to multiply by 1.2	Different players may have different monitor sizes and preferences so it's important that assets don't look distorted.	All assets should be as proportionate to screen size as the default in development which is 1280x720.	As, expected.
Does falling out of the screen respawn the player?	"A" until player falls	The practise arena should allow users to test characters with no deaths.	Player should respawn	As, expected

Abnormal jumping bug

While testing character assets, I noticed a bug happening where every ,second jump, changed the animation to idle i

nstantly. A video of this is shown below:



Luckily this bug was easy to fix, since it only happens every second jump that must mean that the animation doesn't finish before the user hits the ground. So, when the user hits the ground, the animation state is changed to idle. Then on the second jump, the animation finishes nearly instantly, then no animation occurs until the user hits the ground and is no longer busy so idle animation plays.

To fix this, I just reset the jump animation myself right after the animation state is changed to idle:

```
class Character:
    def moveH(self, platforms, dt, keys):
        #otherwise idle
        else:
            self.direction = pygame.math.Vector2(0,0)
            if self.on_ground and (self.animation_state != "attack1" and self.animation_state != "attack2"):
                self.animation_state = "idle"
                self.animations["jump"].reset()
```

Jump animation is manually reset after idle is invoked.

So, right after the user hits the ground and doesn't finish the animation, it's manually reset to prevent it continuing in the next jump. This fixed the bug.

Interview with the stakeholders

Below are the success criteria, that I aimed to complete:

#	Success Criteria	Justification	Evidence for testing
1	In rounds, the players must be able to control their characters movements using A and D (left and right movement).	This is needed so that users can move their characters. A and D are very common for moving left and right across nearly all games. <i>Stakeholders have also confirmed this in the technical interview.</i>	Video of the character responding correctly to the key presses.
2	When pressing the W key in a LAN game, the character should start the jump animation and move up and then down. The animation should be playing the entire time and for a total of 1.25 seconds.	Every fighting game has a jump functionality; this will be needed to dodge attacks and traverse the stage as the stages will have different raised platforms. The W key is perfect as A and D will be used by the index and ring finger which leaves the middle finger free for the W key. <i>Stakeholders have also confirmed this in the technical interview.</i>	Video of the character responding correctly to the key presses.
3	Each character will have two attacks controlled by P and L. The heavy attack will be L while the light attack will be P.	This allows for simple controls that are also fun to use, having only one attack would be too basic and get repetitive. Players can use these attacks to strategize and execute them at the right times. <i>Stakeholders have also confirmed this in the technical interview.</i>	Video of the character responding correctly to the key presses.
7	The characters must have Animations for all kinds of movement (running,	This will add to the visual element of	Videos of the characters

	jumping, attacking) that are shown once the character performs them.	the game, making it feel more refined. Having a character that will just seem to stand still while attacking will take away from the user experience. <i>Stakeholders have also confirmed this in the technical interview.</i>	animations in practise mode.
10	The game must also include a practise arena which allows players to test out characters without any opponents or timer.	This is a great feature suggested by a stakeholder that will allow players to learn attacks and try out different characters.	Video of the practise arena being accessible from the menu and the character playing in the practise arena.

Me: I have completed most of the success criteria I aimed to complete, however I have not added changeable key binds as the settings menu is still a work in progress. I think building the practise arena first was a good decision as now, I have written the code for characters and stages which can just be reused for all the other game modes. In the next iteration, I plan to get a settings menu working where changeable key binds also work game wide. With that, I also want to build the local (2-player) mode which will need a character and stage selection menu. I also plan to add new characters to increase variety of characters that players can pick from. Networking can be added at the end, as it may be quite complex, so I don't want to waste too much time on it at the start before most of the easier features have been implemented.

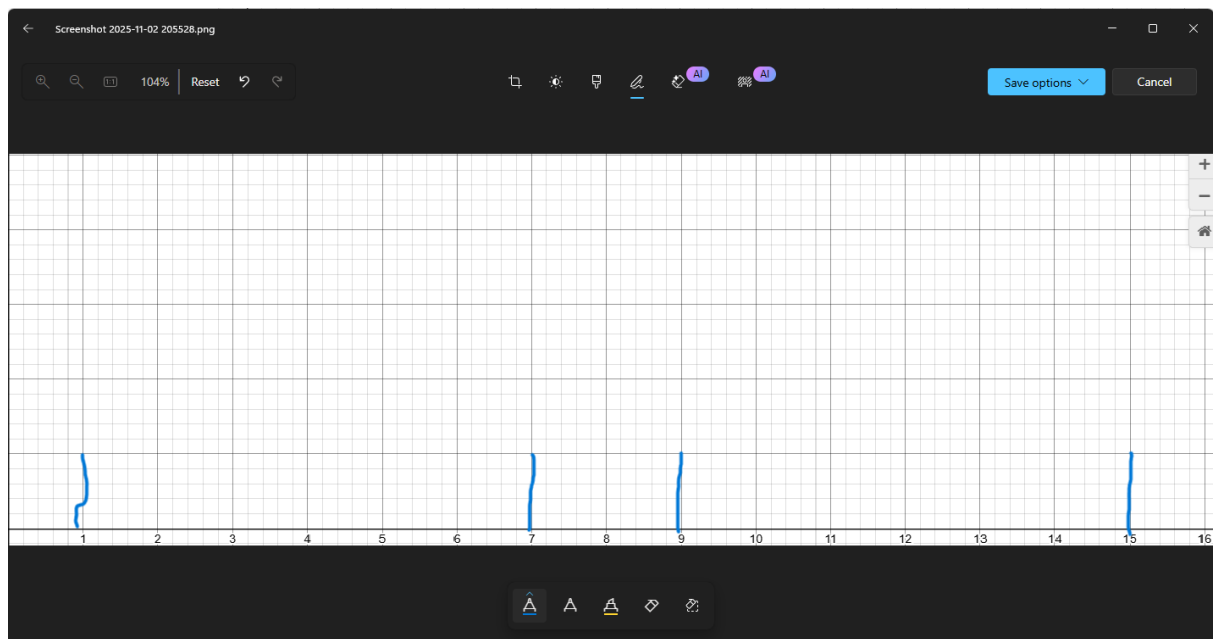
Summary of what the stakeholders said:

Excellent progress so far; everything looks and feels distinct from the previous prototype. The character movement is solid, and the visual elements are working well. As for the next steps, your plan sounds solid. I strongly agree that networking should be saved for last, as it is complex and could distract from finishing the essential local features first.

Implementing a local multiplayer mode

Creating a settings menu

I will first start by making a settings menu, so that players play comfortably in key binds they can choose. To make this menu look like the usability features and have it be compatible with different screen sizes, I will divide the screen into 16 parts. To visualize this, I will take a screenshot of Desmos and edit it like so:



The first two blue lines is where the player 1 controls will be and the second two blue lines for player 2 controls. I'm only using this for horizontal, testing vertical placements is easier, as I can just push all the controls down together.

```
#Essential program data
PROGRAM_DATA = {
    "width": 1280,
    "height": 720,
    "dt": 0,
    "controls": {
        "P1 Right" : pygame.K_d,
        "P1 Left" : pygame.K_a,
        "P1 Jump" : pygame.K_w,
        "P1 Light Attack" : pygame.K_p,
        "P1 Heavy Attack" : pygame.K_l,
        "P2 Right" : pygame.K_RIGHT,
        "P2 Left" : pygame.K_LEFT,
        "P2 Jump" : pygame.K_UP,
        "P2 Light Attack" : pygame.K_m,
        "P2 Heavy Attack" : pygame.K_n,
    },
}
```

The format of each action being the same will make it easier to automate picking controls in the character class. The default controls will be as described in the success criteria. I have picked "m" and "n" to be the default player 2 light and heavy attack buttons. These make sense as they are close to the arrow keys on the keyboard which are used for player 2 movement.

```
class SceneManager:
    def __init__(self, network, server, sWidth, sHeight, controls):
        self.network = network
        self.server = server
        self.assets = assets((sWidth, sHeight), 0.08)
        self.controls = controls
```

I pass in the controls as a parameter, so that it can easily be used in any scene by accessing it from the scene manager attribute.

```
SM = SceneManager(network=n, server=s,
                  sWidth=PROGRAM_DATA["width"], sHeight=PROGRAM_DATA["height"],
                  controls=PROGRAM_DATA["controls"]
                  )
```

```
class Character:
    def __init__(self, name, assets, controls, spawn_point,
                 #Player number is used for controls
                 self.controls = controls
                 self.number = number
```

I created a controls and number attribute so that this can be used to access the right controls for each player.

```

class Character:
    def attack(self, keys_j):
        #if player has recovered
        if self.recovery == 0 and self.on_ground == True and not self.busy:
            if keys_j[self.controls[f"P{self.number} Light Attack"]]:
                #attack, lock animation and set recovery
                self.animation_state = "attack1"
                self.SetRecover1()
            elif keys_j[self.controls[f"P{self.number} Heavy Attack"]]:
                self.animation_state = "attack2"

class Character:
    def moveH(self, platforms, dt, keys):
        #move right
        if keys[self.controls[f"P{self.number} Right"]]:
            self.direction = pygame.math.Vector2(1,0)
            self.facing_left = False #Character moving right
            if self.on_ground:
                self.animation_state = "run"
        #move left
        elif keys[self.controls[f"P{self.number} Left"]]:
            self.direction = pygame.math.Vector2(-1,0)

class Character:
    def moveV(self, platforms, dt, keys_j):
        #jump logic
        if keys_j[self.controls[f"P{self.number} Jump"]] and self.on_ground:
            self.gravity = -400 #upward force for jumping
            self.animation_state = "jump" #start jump animation
            self.on_ground = False #no longer on the ground
            self.busy = True #lock

```

All the controls can now be accessed by changing the player number.

```

class SettingsScene(scene):
    def __init__(self, manager, assets, sSize):
        super().__init__(manager, assets, sSize)
        #Sets background
        self.background = self.assets.getBackground("start_menu")
        self.backgroundRect= self.background.get_frect(topleft = (0,0))

        #Transparent darker overlay, so user can focus on settings
        self.overlay= pygame.Surface(self.sSize, pygame.SRCALPHA)
        self.overlay.fill((0,0,0,150))
        self.overlayR = self.overlay.get_frect(topleft=(0,0))

        #controls is a dictionary: {"P1 Right": blah...}
        self.controls = self.manager.controls
        self.widgets = []

        #Default font for actions
        self.font = self.assets.getFont("Cinzel-Regular")

        #Sets active action and listening for handle events functi
        self.activeAction = None
        self.listening = False

```

Having a slightly transparent layer will let the user focus on the writing in the settings menu and lessen the noise from the background.

The background attributes are set then an additional overlay layer is created. This is a transparent layer that is specified by `pygame.SRCALPHA`, to allow the alpha channel (transparency). Then this is filled with 150 being the transparency and the rest of the colour channels being 0. The higher the transparency channel, the opaquer the layer will be. I set the size of this layer to be the screen size as it needs to be completely on top of the background.

The active action will contain the action that should be changed in string format which will be used in the handle events function. The listening function will be the flag that will allow the handle events function to edit the controls dictionary.

The controls attribute will contain dictionary that has all the keybinds for different actions. This is stored in the manager and can be easily changed to be used in the character class.

```

class KeybindWidget:
    def __init__(self, font, size, action, key , **pos_kwargs):
        #sets the outline box which contains the key and action
        self.bg = pygame.Surface(size)
        self.bg.fill((100,100,100))
        self.bgr = self.bg.get_frect(**pos_kwargs)

        #default font
        self.font = font
        #highlight colour
        self.orange = (222, 156, 42)

        #sets action and action position
        self.action = action
        self.actionI = self.font.render(self.action, True, (0,0,0))
        self.actionR = self.actionI.get_frect(topleft = self.bgr.topleft)

        #Key in both text and pygame code format
        self.key = key
        self.keyText = pygame.key.name(self.key)

        self.keyI = self.font.render(self.keyText, True, (0,0,0))
        self.keyR = self.keyI.get_frect(topleft = self.bgr.topleft)

```

The orange colour is saved and will be used when the mouse hovers over an action, this will make the settings menu more interactive. Also, it will help the user see which key bind is current being changed.

Each key bind widget will be an object with it's own functions. This makes it easier to edit functions and access each key bind for updating to the main controls dictionary in the scene manager class.

Storing the key in both text and pygame key code format will allow me to display to the user, what key is currently being used and update the controls dictionary for later gameplay.

The action I and R will store the image and rect for the text describing the action. For example "P1 move right".

```

def changeKey(self, key):
    #Supports both pygame key and string
    if isinstance(key, str):
        #If string then sets text first
        self.keyText = key
        #Then retrieves the code
        self.key = pygame.key.key_code(self.keyText)
    else:
        #Otherwise set code first
        self.key = key
        #Then retrieve text
        self.keyText = pygame.key.name(self.key)

    #Must rerender to change colour back
    #Each time it is set, colour changes back to white
    self.changeKeyColour((0,0,0))

```

This function will change the key attribute and is compatible with both inputting a pygame key code or text. This will make it easier to change the key based on what the user clicks.

Whenever the key is changed, it should be set back to white to signify that it is set.

```

def changeKeyColour(self, colour):
    #Changes colour of keybind
    self.keyI = self.font.render(self.keyText, True, colour)
    self.keyR = self.keyI.get_frect(topright = self.bgr.topright)

def changeActionColour(self, colour):
    #Changes the description of the action
    self.actionI = self.font.render(self.action, True, colour)
    self.actionR = self.actionI.get_frect(topleft = self.bgr.topleft)

```

These two functions will be used for changing the action and key colour. They will be called in the update function to display to the user, what key is currently being changed and what it is being changed to.

Ultimately, the function will return true if pressed and false if not, so that the listening flag can be set for the handle events function in the settings scene.

```
def update(self, mpos, mbut_j):
    #If mouse hover
    if self.bgr.collidepoint(mpos):
        #Activate orange highlight
        self.changeActionColour(self.orange)

        #If press too
        if mbut_j[0]:
            #Set key to ? to change later
            #Highlight so user can see
            self.changeKey("?")
            self.changeKeyColour(self.orange)
            #True for press
            return True

        else:
            #No highlight if no hover
            self.changeActionColour((0,0,0))

    #No press
    return False
```

If the mouse hovers over the widget, it should change the action colour to orange, to show the user it will be changed if clicked. It also adds a cool visual effect. If the user also clicks, then the key should be changed to ? to show it will be changed with the next key press, the colour should also be changed to orange permanently until it is actually changed.

If the mouse doesn't hover over widget, then the colour should be changed to white.

```
def draw(self, screen):
    #draws background, action on the left, key on the right
    screen.blit(self.bg, self.bgr)
    screen.blit(self.actionI, self.actionR)
    screen.blit(self.keyI, self.keyR)
```

Simple function, draws the grey background, action text and key text.

```

class SettingsScene(scene):
    def __init__(self, manager, assets, sSize):

        for i, (action, key) in enumerate(self.controls.items()):
            #i is used for determining y position of widgets
            #screen is divided into 16 for positions and sizes

            #size for the widget background
            xSize = self.sSize[0]/16*(6)
            ySize = self.sSize[1]/32
            size = (xSize, ySize)

            #P1 controls go on the left of the screen
            if "P1" in action:
                pos = (1/16*self.sSize[0], (i+4)*self.sSize[1]/16)

            elif "P2" in action:
                #P2 starts at index 5
                j = i-5
                #x changed
                pos = (9/16*self.sSize[0], (j+4)*self.sSize[1]/16)

            #creates and adds widget, based on calculated positions/sizes, to list
            self.widgets.append(KeybindWidget(self.font, size, action, key, topleft = pos))

        #Back button
        back, backG = self.assets.getButton("back")
        self.backButton = Button(back,backG, lambda: self.manager.change_scene(MenuScene, s
                                center = (self.sSize[0]/2, self.sSize[1]*0.9))

```

Pos is defined and used to create a widget based on the action, key, sizes and position. It adds this widget to the list attribute so it can be handled later.

This enumerate function combined with the dictionary.items will return an index with the pair of key:data. So, for the first control, it will return 0, "P1 Right", pygame.K_d. The index is needed for determining the position of the widget. The action and key are used for creating the key binds widget.

The size will be based on the screen to make it relative and fit on any monitor/screen size. Then, for each action, using the screenshot at the start of the section, it works out how many 16ths it should across and down the screen, based on the index and whether it's a P1 control or P2.

Simple back button created and placed at the bottom of the screen like the usability features.

```

def update(self, dt, mpos, mbut,mbut_j, keys, keys_j):
    for widget in self.widgets:
        #Updates each widget
        if widget.update(mpos, mbut_j):
            #If widget pressed then,
            #turns on listening
            self.activeAction = widget.action
            self.listening = True

    self.backButton.update(mpos,mbut)

def draw(self,screen):
    screen.blit(self.background, self.backgroundRect)
    screen.blit(self.overlay, self.overlayR)
    #Draws each widget
    for widget in self.widgets:
        widget.draw(screen)
    self.backButton.draw(screen)

```

The back button will also be updated to check for if the user clicks back.

This update function will be run every frame and will call the widget update. If a widget is pressed, the active action will be set to that widget's action and the listening flag will be set to true.

Simple draw function will draw the background then the overlay, then put the widgets and back button on top.


```

def handle_events(self, events):
    #Basic inherited function for all scenes to check whether user closes the w
    for event in events:
        if event.type == pygame.QUIT:
            pygame.quit()
            quit()

        #If user has pressed a key and settings is in listening mode
        if event.type == pygame.KEYDOWN and self.listening:
            #Change controls to that key
            self.manager.controls[self.activeAction] = event.key
            for widget in self.widgets:
                #Searches for the current action being changed
                if widget.action == self.activeAction:
                    #Changes key to display to user
                    widget.changeKey(event.key)
                    break #Stops loop
            #Resets listening and active action
            self.listening = False
            self.activeAction = None

```

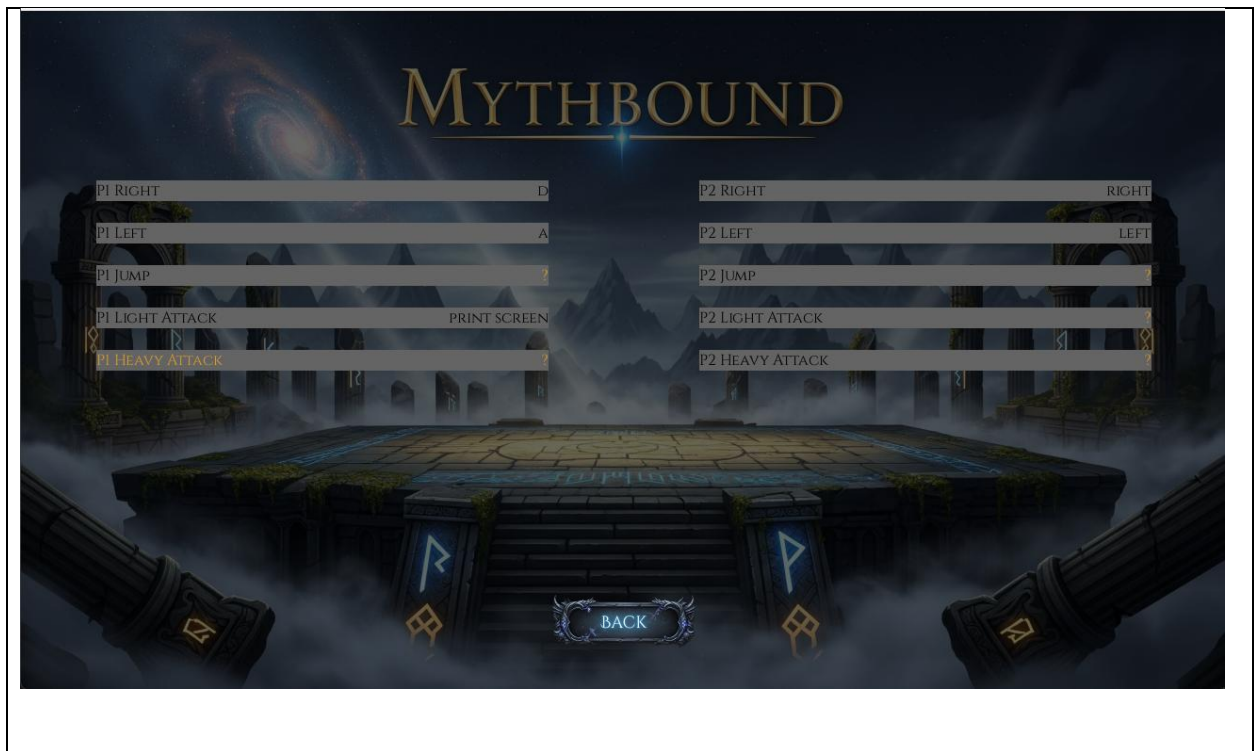
In the handle events function, if the listening flag is set to true, then it should watch out for key presses and use the active action to change the controls dictionary appropriately. To display the change to the users, it will search through the widgets to change the key to the one that is pressed. Once this is done, the listening flag should be reset, and the active Action should be none.

Testing the settings menu

What I'm testing	Input data	Why am I testing this?	Expected outcome	Test results
Do all the widgets show up as intended, all aligned with each other?	Mouse click on settings button.	Allows the user to see the settings menu clearly and systematically with player 1 on left and player 2 controls on the right.	All widgets should be drawn to the screen with player 1 controls on the left and 2 on the right.	As, expected.
Does hovering over a widget change the colour of the action text?	Mouse hover over the widget	The user should be able to see what widget is currently being selected?	When the mouse hovers over a widget, the turn orange.	As, expected
Does clicking an action lock the user into selecting a key bind for that action?	Mouse click on the widget	This will not allow the user to leave actions unbinded, so all movements/attacks can be performed in game.	User shouldn't be able to change another key bind.	Clicking doesn't lock the user into that key bind. I can click many other actions and change them to "?" too.

Does clicking the action change the key to "?" and orange to indicate to the user, which key bind is currently being changed?	Mouse click on the widget	Allows the user to see what action is currently being binded so they can press their preferred key.	Key should turn into an orange "?", until a key is pressed.	As, expected.
Does the character respond to the key presses specified by the settings menu.	"c", "v", "u", "z", "x"	Allows user to change to more comfortable key binds.	The character should respond appropriately to each key press	As, expected.

Screenshot of the settings menu below:



The problem with the code, is that it allows the player to select another key bind to edit, while already editing one. To fix the error, I need to check that the listening flag is not True before updating the widgets. The code shows this below:

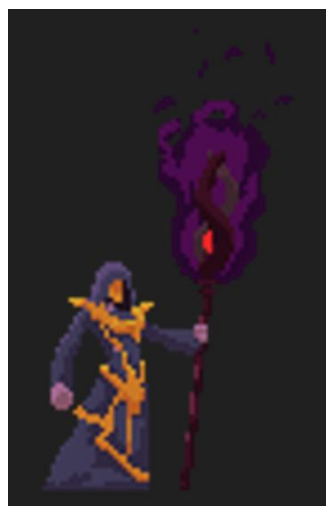
```
def update(self, dt, mpos, mbut,mbut_j, keys, keys_j):
    for widget in self.widgets:
        #Updates each widget
        if not self.listening:
            if widget.update(mpos, mbut_j):
                #If widget pressed then,
                #turns on listening
                self.activeAction = widget.action
                self.listening = True
```

What I'm testing	Input data	Why am I testing this?	Expected outcome	Test results
Does clicking an action lock the user into selecting a key bind for that action?	Mouse click on the widget	This will not allow the user to leave actions unbinded, so all movements/attacks can be performed in game.	User shouldn't be able to change another key bind.	As, expected.

Creating the 2-player mode

To finish making the core gameplay, I will first create a 2-player mode before going back to creating the stage and character selection menus.

I will start by adding another character which will be chosen later, in the character selection menus. As I have made the asset loading animation handling automatic, I can simply find a character with an idle, run, jump and two attack animations, which should work perfectly. The character I have added is called "evil wizard" and is shown below:



To implement this, I made a specific class for the character and added it to the character factory. While implementing this, I thought of incorporating the recovery frames into the parameters passed in, rather than making a separate function for them.

This would make the code neater at the expense of just two more parameters.

```
class Character:
    def __init__(self, name, assets, controls, spawn_point, number, recovery1 = 2, recovery2 = 2):
        #attack recoveries- changable
        self.recovery1 = recovery1
        self.recovery2 = recovery2
```

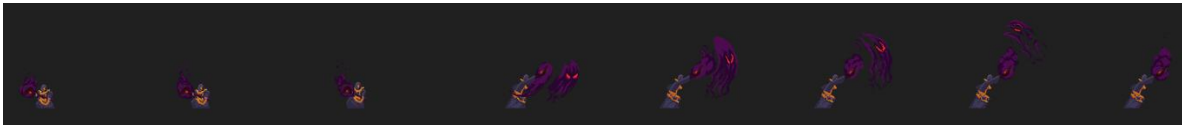
```

class evilWizard(Character):
    def __init__(self, assets, controls, spawn_point, number, recovery1, recovery2):
        super().__init__("evil_wizard", assets, controls, spawn_point, number, recovery1, recovery2)

    def load_animations(self,):
        animations = self.assets.getAnimations(self.name)
        self.animations = {}
        #Gets the standard animations from the assets
        #Creates an animation object based off each one
        #Values should be overridden
        self.animations["idle"] = animation(animations["idle"], 0.75)
        self.animations["jump"] = animation(animations["jump"], 1.25, [1,2,3,4,5,6,7,8])
        self.animations["run"] = animation(animations["run"], 0.9)
        self.animations["attack1"] = animation(animations["attack1"], 0.5, [3,4,5,6,7,8], [4,5,6,7])
        self.animations["attack2"] = animation(animations["attack2"], 1, [3,4,5,6,7,8], [5,6,7,8])

```

This code is very similar to how I added the “knight” character, I have just changed the busy and attack frames, according to the animations. As in, when a character is charging up an attack, they should not be able to damage the enemy (even if it is part of the attack animation).



Example of frames in the attack animation that should not be attacking frames

```

def CharacterFactory(name, assets, controls, spawn_point, number):
    match name:
        case "knight":
            return knight(assets, controls, spawn_point, number, 2, 2)
        case "evil_wizard":
            return evilWizard(assets, controls, spawn_point, number, 2, 2)

```

Next, I will create a 2-player local scene that will contain all the 2-player mode code:


```

class LocalModeScene(scene):
    def __init__(self, manager, assets, sSize):
        super().__init__(manager, assets, sSize)
        #stage and player's creation
        self.stage = StageFactory("volcano_valley", self.assets, sSize)
        #different spawn points and player numbers
        self.player1 = CharacterFactory("knight", self.assets, self.manager.controls, self.stage.getSpawnPoint(1), 1)
        self.player2 = CharacterFactory("evil_wizard", self.assets, self.manager.controls, self.stage.getSpawnPoint(2), 2)
        #stores number of wins for each player
        #index 0 is player 1 and index 1 is player 2
        self.wins = [0,0]

```

Different spawn points, player numbers for controls and beginning of rounds.

Wins will be stored in a list for easy access to check when rounds end.

For now, the local mode will have two players already chosen, which are the knight and evil wizard. I will add the character/stage selection menus later.

I should also mention the changes to the character class, which will allow characters to take damage from opponents attacks and reset attributes in between rounds.

```
class Character:
```

```
def take_damage(self, player):  
    #if the opponent collides  
    if player.rect.colliderect(self.rect):  
        #if not invincible and the opponent is attacking  
        if player.attacking and not self.invincible:  
            #value needs to be edited for heavy and light attacks  
            self.health -= 25
```

This attack function will allow the players to take damage from their opponents in games. It checks if they collide first, then if the opponent is attacking and the player is not invincible. If these conditions are met, then health will be deducted.
Note: This needs to be changed for heavy and light attacks.

```
def reset(self):  
    #SIMPLE RESET FUNCTION FOR ROUNDS  
    for animation in self.animations:  
        self.animations[animation].reset()  
    self.health = 100  
    self.animation_state = "idle"  
    self.current_animation = self.animations[self]  
    self.invincible = False  
    self.recovery = 0  
    self.attacking = False  
    self.knockback = False  
    self.busy = False  
    self.on_ground = True #Starts on platform  
    self.gravity = 0  
    #player 2 should face left as they will be on the right  
    self.facing_left = False if self.number == 1 else True  
    #initilizes the image and rect after the setup  
    self.image = self.current_animation.getFrame(self.facing_left)  
    self.rect = self.image.get_frect(midbottom=self.spawn_point)
```

The reset function will reset all attributes and animations, in between rounds. This is including the spawn point so players can start fresh.

I will also point the local button in the joining menu to start a new local mode scene:

```
class PlayMenuScene(scene):  
    def __init__(self, manager, assets, sSize):  
        center=(self.sSize[0]/2, self.sSize[1]*2.5/8)  
        self.localButton = Button(local, localG, lambda:self.manager.change_scene(PractiseScene, self.manager, self.assets, self.sSize)  
            , center=(self.sSize[0]/2, self.sSize[1]*3.5/8))
```

The lambda (as explained before) will allow the manger change scene function to not be called right away, it creates a callable wrapper.

```
def checkRound(self):
    #resets both players if one player is defeated
    #if player 1 is defeated
    if self.player1.health ≤ 0:
        #player 2 wins incremented
        self.wins[1] +=1
        self.player1.reset()
        self.player2.reset()
    #if player 2 is defeated
    elif self.player2.health ≤ 0:
        #player 1 wins are incremented
        self.wins[0] +=1
        self.player1.reset()
        self.player2.reset()

    #best of 3 system
    #if one player wins twice, they win
    if 2 in self.wins:
        #ending banner should be switched to here
        print(f"Player {self.wins.index(2)+1} wins!")
        #if a player wins, then the scene is switched back to the joining menu
        self.manager.change_scene(PlayMenuScene,self.manager,self.assets, self.sSize)
```

This will be checked every frame, if any player's health is 0 or below, a win will be granted to the other, and both players will be reset for the new round.

If any player gets 2 wins, then, for now, it will be outputted to the console. The scene will also be changed to the joining menu, to allow players to start a new game or exit.

```
def DeathFall(self):
    #Checks whether player is out of screen
    if self.player1.rect.bottom > self.sSize[1] or self.player1.rect.top < 0 or self.player1.rect.right > self.sSize[0] or self.player1.rect.left < 0:
        self.player1.health = 0
    if self.player2.rect.bottom > self.sSize[1] or self.player2.rect.top < 0 or self.player2.rect.right > self.sSize[0] or self.player2.rect.left < 0:
        self.player2.health = 0
```

This function is very similar to the no death function in the practise scene; however it will set player health to 0. This will be detected by the check round function which will give the win to the other player.

The update and draw functions are simple and consist of other function calls:

```
def update(self, dt, mpos, mbut,mbut_j, keys, keys_j):
    self.player1.update(self.stage.getPlatforms(), dt, mpos, mbut, mbut_j, keys, keys_j)
    self.player2.update(self.stage.getPlatforms(), dt, mpos, mbut, mbut_j, keys, keys_j)
    #take damage functions to check for attacks
    self.player1.take_damage(self.player2)
    self.player2.take_damage(self.player1)
    #check rounds on each update
    self.checkRound()
    self.DeathFall()

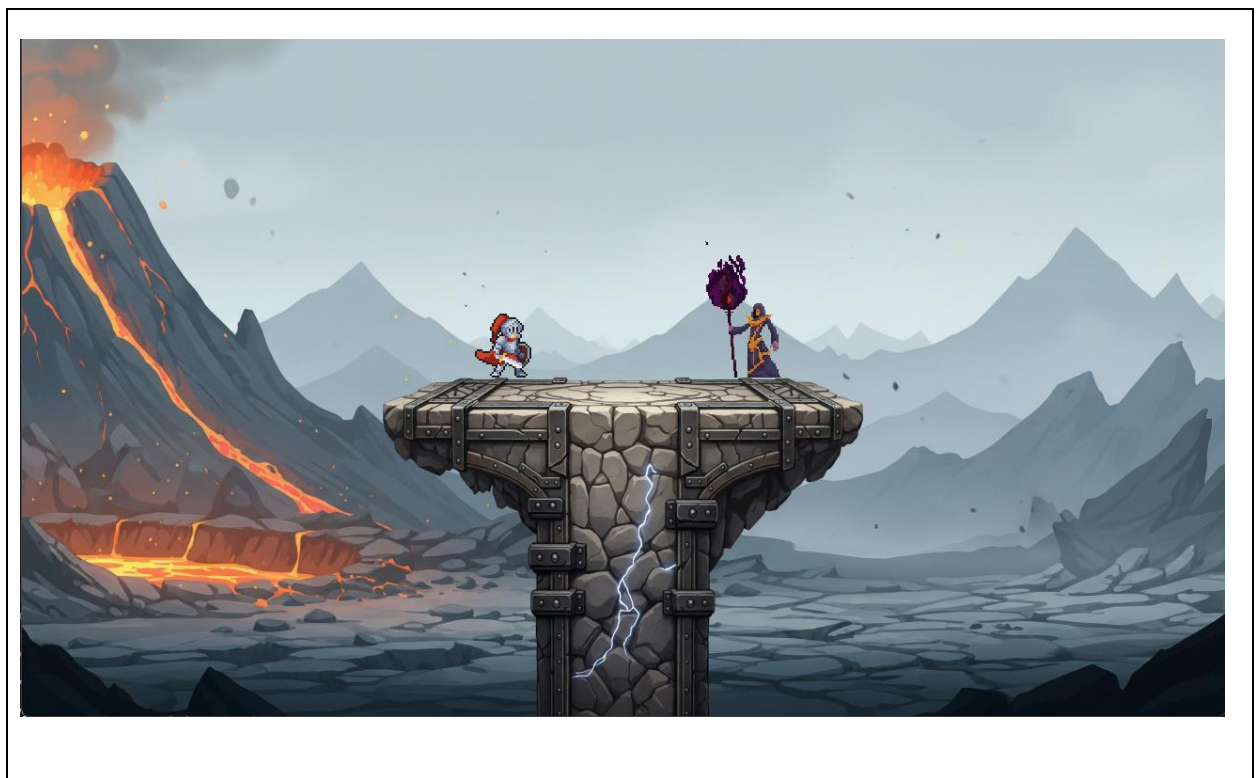
def draw(self, screen):
    self.stage.draw(screen)
    self.player1.draw(screen)
    self.player2.draw(screen)
```

Every frame, the players' update functions will be called to allow movement/attacks. The take damage functions will decide whether to deduct health and the check round and deathfall functions will enforce the rules of the rounds.

Testing the 2-player mode

What I'm testing	Input data	Why am I testing this?	Expected outcome	Test results
Do both characters show up on screen?	Mouse press on the local button.	This is essential to having a functional basic 2D fighter game.	Both players should show on the screen in their appropriate spawn points.	As, expected.
Can both characters be controlled by the key binds set in the settings?	Mouse press and key presses to change the key binds in the settings. Then using those keys.	Allows user to change to more comfortable key binds in settings.	Both characters should respond to the key binds set in the settings menu.	As, expected.
Do deaths by fall work, do they trigger a new round?	"a" until player falls off. Also, right arrow key for the second player.	This makes falling responsive and more natural. Without this, the surviving player would be stuck. This should also be a real round and unlike the practise arena where you can't die.	Falling off the platform should trigger the start of a new round with both characters resetting back to their spawn points.	As, expected.

Does falling by death twice count as a win for the other player?	"a" until player falls off. Also, right arrow key for the second player.	This makes falling responsive and more natural. Without this, the surviving player would be stuck. This should also be a real round and unlike the practise arena where you can't die.	Output the appropriate message in the console.	As, expected.
Does repeatedly attacking a character until their health reaches 0, restart the round?	"p", "l", "m", "n"	This allows players to actually attack opponents and win the game.	After attacking the opponent 4 times, a new round should start.	Attacking an opponent instantly started a new round.



Attack frames issue

As attacking an opponent instantly started a new round, it means that health was deducted more than once which is a frame issue. There are multiple attack frames within an animation that could deduct health, this is a problem as I want an attack to damage the opponent only once.

To fix this I introduced a new attribute called "attack_use" in the character class. This will be set to 1, if an attack is started and 0 if the player is in an attacking frame where they collide with the opponent. This will use up the charge and now allow more frame to damage the opponent:

```
class Character:
    def __init__(self, name, assets, controls, spawn_point, number, recovery1 = 2, recovery2 = 2):
        self.attack_use = 0
```

This is initialised to 0 and only set to 1 if an attack has started:

```
def attack(self, keys_j):
    #if player has recovered
    if self.recovery == 0 and self.on_ground == True and not self.busy:
        if keys_j[self.controls[f"P{self.number} Light Attack"]]:
            #attack, lock animation and set recovery
            self.animation_state = "attack1"
            self.attack_use = 1
            self.SetRecover1()
        elif keys_j[self.controls[f"P{self.number} Heavy Attack"]]:
            self.animation_state = "attack2"
            self.attack_use = 1
            self.SetRecover2()
```

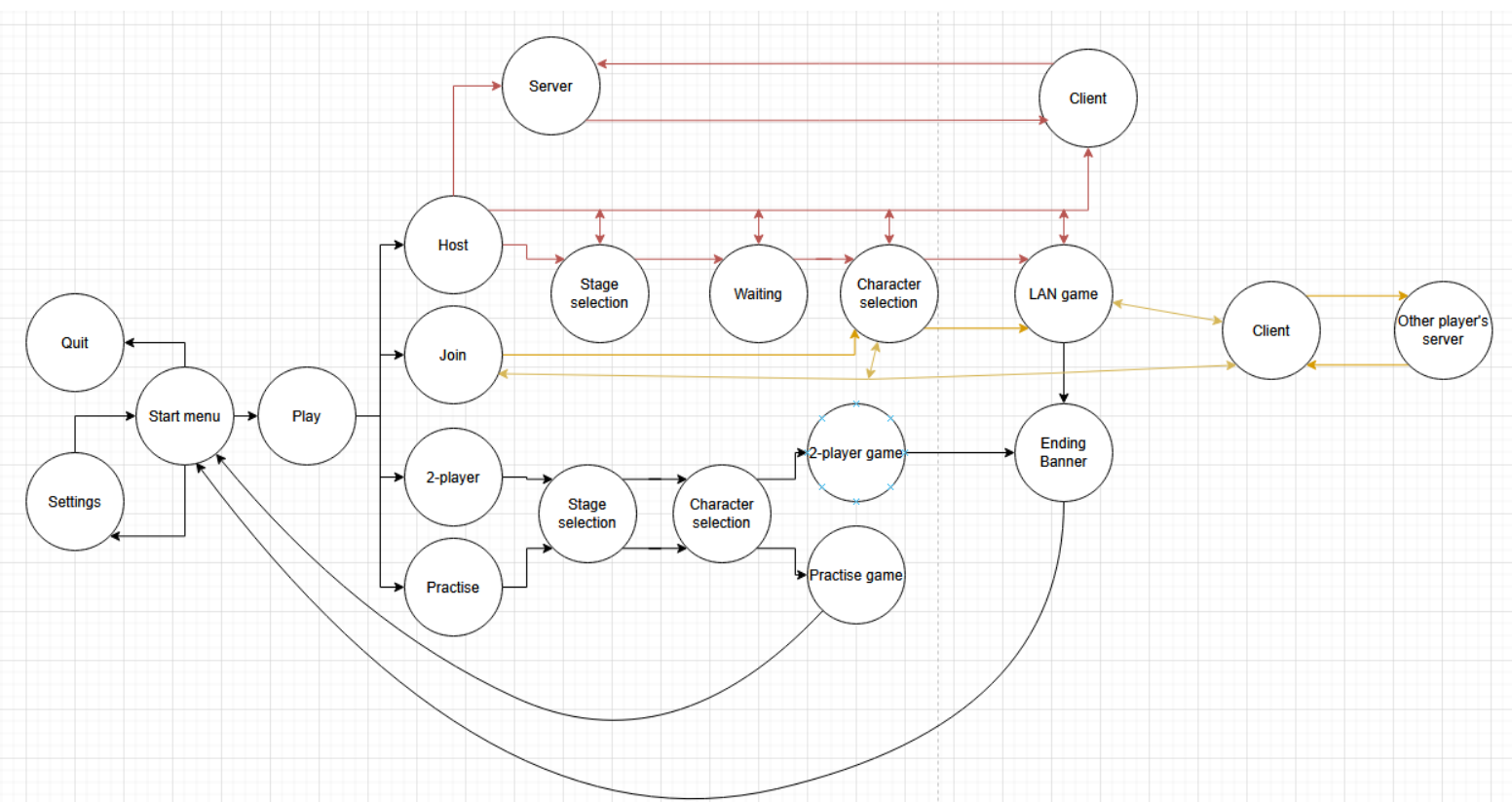
```
def take_damage(self, player):
    #if the opponent collides
    if player.rect.colliderect(self.rect):
        #if not invincible and the opponent
        if player.attack_use == 1:
            if player.attacking and not self.invincible:
                #value needs to be edited for heavy and light attacks
                self.health -= 25
                player.attack_use = 0
```

Only damages if the opponent's attack use is set to 1. Once damaged, the attack use is set back to 0.

What I'm testing	Input data	Why am I testing this?	Expected outcome	Test results
Does repeatedly attacking a character until their health reaches 0, restart the round?	"p", "l", "m", "n"	This allows players to actually attack opponents and win the game.	After attacking the opponent 4 times, a new round should start.	As, expected.

Stage selection menu

The stage selection menu should be shown only when the play mode is practise, local or host. Joining another player shouldn't allow the player to select the stage as the host should do that. This will make it a bit harder to program this menu, however it can be done with parameter that defines which menu it should switch to after the stage selection. Looking back on the data flow diagram made in design:



As we can see, stage selection after host should take the player to the waiting scene, but the 2-player and practise modes should take the player to the character selection. Looking ahead, these will still need to be handled differently in the character selection menu since 2 players will need to select their characters rather than 1 in practise.

Using this information, I can create a class for the stage selection scene:

I have also scaled the stage backgrounds down to display them as a preview for the player to see. I have chosen the scale to be 0.25 as it makes sense for a background preview to be a quarter of actual screen size.

```
def __init__(self, manager, assets, sSize, mode):
    super().__init__(manager, assets, sSize)
    #Sets background
    self.background = self.assets.getBackground("start_menu")
    self.backgroundRect= self.background.get_frect(topleft = (0,0))

    #Transparent darker overlay, so user can focus on settings
    self.overlay= pygame.Surface(self.sSize, pygame.SRCALPHA)
    self.overlay.fill((0,0,0,150))
    self.overlayR = self.overlay.get_frect(topleft=(0,0))

    self.backgrounds = [pygame.transform.scale_by(self.assets.getBackground("volcano_valley"), 0.25)]
    self.rects = [x.get_frect(center = (self.sSize[0]/2, self.sSize[1]/2)) for x in self.backgrounds]
    self.names = ["Volcano Valley"]
    self.IDs = ["volcano_valley"]

    #mode for switching scenes
    self.mode = mode

    #font may be changed
    self.font = self.assets.getFont("PressStart2P")
    self.text = [self.font.render(text, True, (0,0,0)) for text in self.names]
    #all text on top of the stages
    self.textR = [x.get_frect(midbottom = self.rects[self.text.index(x)].midtop) for x in self.text]
```

The game background is used with an overlay on top to let the user focus on selecting a stage. The code for an overlay was taken from the previous settings scene class.

Mode is made an attribute to determine which scene the game should switch to after the stage is selected.

All text will be centred on top of each stage; this is using the fact that each element at each index corresponds to the same stage.

These 4 lists should have the same number of elements and be ordered in a way where elements at each index correspond to the same stage. This will make it easier to handle all the selections on screen if I add more stages in the future. The IDs will be passed on to the next scene if selected while the names will be drawn to show the user that the stage is called as well as the background of the stage.


```
def switchScene(self, ID):
    match self.mode:
        case "practise":
            #should be changed to character selection once made
            self.manager.change_scene(PractiseScene,self.manager, self.assets, self.sSize, ID, "a")
        case "local":
            #should be changed to character selection once made
            self.manager.change_scene(LocalModeScene, self.manager, self.assets, self.sSize, ID, "a")
        case "host":
            pass
```

These a's are meant to be the chosen characters but they will be changed once the character selection menu is made

As you can see, I will be using a switch statement on mode to determine which scene the function should switch to.
No matter the scene, ID will hold the stage that is selected and should be passed into the function and the later scenes.

```
def update(self, dt, mpos, mbut,mbut_j, keys, keys_j):
    for i, rect in enumerate(self.rects):
        if rect.collidepoint(mpos):
            if mbut[0]:
                self.switchScene(self.IDs[i])
```

The enumerate function will return the element as well as its index, this will allow me to check whether the user has clicked a stage and call the switch the scene function with the associated ID using its index.

```
def draw(self, screen):
    screen.blit(self.background, self.backgroundRect)
    screen.blit(self.overlay, self.overlayR)
    for i, background in enumerate(self.backgrounds):
        screen.blit(background, self.rects[i])
    for i, text in enumerate(self.text):
        screen.blit(text, self.textR[i])
```

This function simply draws the background, overlay, text and stages to the

I also rerouted the buttons in the play menu scene to take the player to the stage selection scene once practise and local are pressed. I will reroute the join and host appropriately later.

```
class PlayMenuScene(scene):
    def __init__(self, manager, assets, sSize):
        join, joinG = self.assets.getButton("join")
        host, hostG = self.assets.getButton("host")
        back, backG = self.assets.getButton("back")
        self.practiseButton = Button(practise, practiseG, lambda:
            self.manager.change_scene(StageSelectScene, self.manager, self.assets, self.sSize, "practise"),
            center=(self.sSize[0]/2, self.sSize[1]*2.5/8))
        self.localButton = Button(local, localG, lambda:
            self.manager.change_scene(StageSelectScene, self.manager, self.assets, self.sSize, "local"),
            center=(self.sSize[0]/2, self.sSize[1]*3.5/8))
        self.joinButton = Button(join, joinG, lambda: print("join"), center=(self.sSize[0]/2, self.sSize[1]*4.5/8))
```

The correct modes are selected, "practise" and "local".

Testing the stage selection menu

What I'm testing	Input data	Why am I testing this?	Expected outcome	Test results
Is the stage selection menu drawn to the screen when practise or local is selected?	Mouse press on practise and local button.	To have an interactive and working menu and allow the user to select a stage before playing.	The stage selection menu should be drawn with the overlay and volcano valley background and text.	It is not being drawn when I press the local button.
Does pressing a stage load up the correct stage in practise?	Mouse press on volcano valley.	To have an interactive and working menu and allow the user to	Practise should be loaded up exactly as before with the	As, expected,

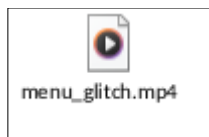
		select a stage before playing.	stage being shown.	
Does pressing a stage load up the correct stage in local mode?	Mouse press on volcano valley.	To have an interactive and working menu and allow the user to select a stage before playing.	Practise should be loaded up exactly as before with the stage being shown.	As, expected.

Menu glitches

While clicking through menus I have noticed some bugs, one is shown in the video below:



Although it looks like I may have not routed it properly, it is actually just skipping through the menus quickly. This might be shown better in this video:



When I press the top of the play button, it skips to stage selection. When I press the bottom of the play button, it skips to a local mode game. This is because the game registers mouse button presses when they are simply held in a frame and not when clicked. This makes holding the mouse button register as a click.

To fix this bug, I will pass in `mbutton_j` instead of `mbutton` into the button update function, for all buttons across all the scenes. An example is shown below:

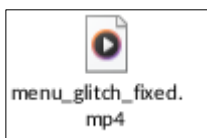
```
def update(self, dt, mpos, mbutton, mbutton_j, keys, keys_j):
    #Checks button press/hover
    self.playButton.update(mpos, mbutton)
    self.settingsButton.update(mpos, mbutton)
    self.quitButton.update(mpos, mbutton)
```

To

```
def update(self, dt, mpos, mbut,mbut_j, keys, keys_j):
    #Checks button press/hover
    self.playButton.update(mpos, mbut_j)
    self.settingsButton.update(mpos, mbut_j)
    self.quitButton.update(mpos, mbut_j)
```

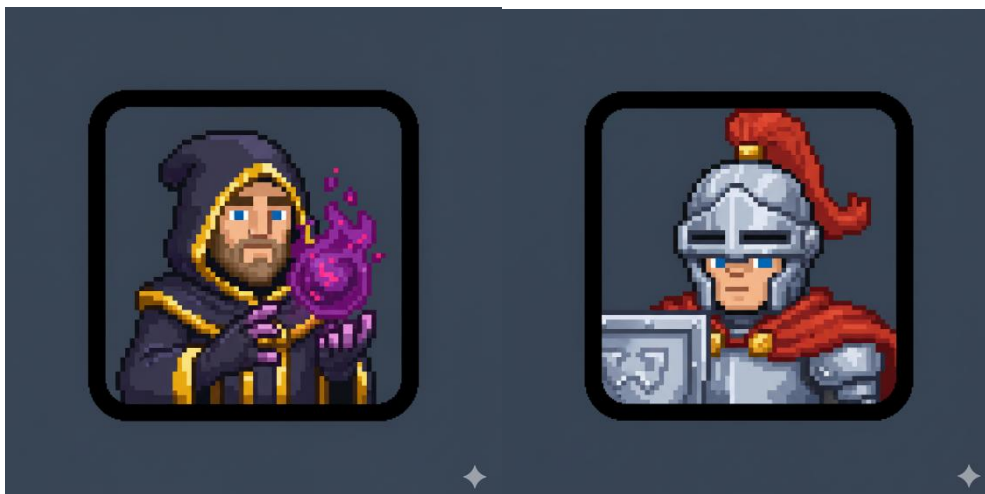
What I'm testing	Input data	Why am I testing this?	Expected outcome	Test results
Is the stage selection menu drawn to the screen when practise or local is selected?	Mouse press on practise and local button.	To have an interactive and working menu and allow the user to select a stage before playing.	The stage selection menu should be drawn with the overlay and volcano valley background and text.	As, expected.

The menu glitches are fixed now:



Creating a character selection menu

In order to create the character icons, I will paste their idle animations into google's nano banana and tell it to create icons based off the idle animations.



Next, I will edit these in photopea to remove the background and begin on writing the code.

I will start by editing the assets class to support loading in character icons and returning in the character selection menu.

```
class assets:
    def __init__(self, screenSize, buttonSF):
        self.path = os.path.join(self.base, "assets")

        #Backgrounds,buttons folders path
        self.backgrounds_path = os.path.join(self.path, "backgrounds")
        self.buttons_path = os.path.join(self.path, "buttons")
        self.loading_path = os.path.join(self.path, "loading")
        self.fonts_path = os.path.join(self.path, "fonts")
        self.stages_path = os.path.join(self.path, "stages")
        self.characters_path = os.path.join(self.path, "characters")
        self.icons_path = os.path.join(self.path, "icons")

        #Specifies screen size and button size for image scaling
        self.screenSize = screenSize
        self.buttonSF = buttonSF

        #development was in 1280x720
        #scale factor for any screen size
        self.SF = min(self.screenSize[0]/1280, self.screenSize[1]/720)

        #assets dictionaries seperated by type eg backgrounds
        self.backgrounds = {}
        self.buttons = {}
        self.fonts = {}
        self.platforms = {}
        self.characters = {}
        self.icons = {}
```

These are the new additions. The icons path will allow the load function to access the folder which stores all the character icons. The icons dictionary will hold the character icons separated by name.

```
def load_icons(self):
    #for all icons in the folder
    for file in os.listdir(self.icons_path):
        #all icons are png
        icon = pygame.image.load(os.path.join(self.icons_path, file)).convert_alpha()
        #*by SF to make icon sizes suitable for all screen sizes
        icon = pygame.transform.scale_by(icon, 0.2*self.SF)

        name = file[:file.find(".")]
        self.icons[name] = icon
```

This code is very similar to the code explained before, it will simply search through the icons folder and load in any icons, then store them in the dictionary under their name. They are also sized down by 0.2 multiplied by SF, so the icons are correctly proportionate on all screen sizes.

```
def getIcon(self, name):
    return self.icons[name]
```

Getter function that returns the character icon based on the name.

Next, I will need to create a character icon class. This is necessary to separate each character icon in the menu so that they can be easily handled by the character selection scene class.

```
class CharacterIcon:
    def __init__(self, name, assets, sSize, **pos_kwargs):
        self.name = name

        #icon files have a "_icon" at the end
        self.icon = assets.getIcon(name+"_icon")
        self.iconR = self.icon.get_frect(**pos_kwargs)

        #first frame of the idle animation will be a preview
        self.preview = assets.getAnimations(name)["idle"][0]
        self.x1 = sSize[0]/3
        #reflects around the middle of the screen of p2
        self.x2 = sSize[0] - self.x1
        #default is p1
        self.previewR = self.preview.get_frect(midbottom = (self.x1, sSize[1]))
```

The x1 and x2 constants will be based off the screen size. Player 1 will be a third of the way along the screen and player 2 previews will be reflected for perfect symmetry and alignment, like the usability features.

Uses assets to get the icon image, the pos keyword arguments are passed in for more accurate placements

The preview will be the first frame of the idle animation as it makes the most sense since they won't be moving.

The mid bottom will be used for placement, as it's much easier than trying to align the centre perfectly above the bottom of the screen. I can just place the bottom of the character on the bottom of the screen.

```
def hover(self, mpos, mbut_j):
    #returns if the mouse is hovering over the icon
    #so that the preview can change
    if self.iconR.collidepoint(mpos):
        return True

def isPressed(self, mpos, mbut_j):
    #returns if the button is pressed
    if self.iconR.collidepoint(mpos):
        if mbut_j[0]:
            return True
```

These two functions will simply return if the mouse is hovering to be used for drawing previews and if it is pressed, for confirming the character.

```
def draw(self, screen):
    screen.blit(self.icon, self.iconR)

def drawPreview(self, screen, preview):
    #if the preview is for player 1
    if preview == 1:
        #change x to left side and blit
        self.previewR.x = self.x1
        screen.blit(self.preview, self.previewR)
    #if the preview is for player 2
    elif preview == 2:
        self.previewR.x = self.x2
        screen.blit(self.preview, self.previewR)
```

One icon may draw two previews onto the screen. This draw preview function may be called more than once for that purpose, which is why the position of the preview is adjusted in the function.

Note: using this multiple times in one frame doesn't mean the previous drawing will change positions. It will just draw two copies onto the screen,

Now I will make the character selection menu class using the character icons.


```

class CharacterSelectScene(scene):
    def __init__(self, manager, assets, sSize, mode, stage):
        super().__init__(manager, assets, sSize)
        #Sets background
        self.background = self.assets.getBackground("start_menu")
        self.backgroundRect= self.background.get_frect(topleft = (0,0))

        #Transparent darker overlay, so user can focus on settings
        self.overlay= pygame.Surface(self.sSize, pygame.SRCALPHA)
        self.overlay.fill((0,0,0,150))
        self.overlayR = self.overlay.get_frect(topleft=(0,0))

        #stores the mode and stage for switch scene
        self.stage = stage
        self.mode = mode

        #icons are manually made for each character
        self.icons = [CharacterIcon("knight", assets, sSize, center = (sSize[0]/3, sSize[1]/4)),
                      CharacterIcon("evil_wizard", assets, sSize, center = (sSize[0]*2/3, sSize[1]/4))
                      ]

        self.turn = 0
        #list for easy access
        #2 values, for player 1 and 2
        self.selected = [None, None]
        #confirmed if the player presses on the icon
        self.confirmed = [None, None]

```

This background and overlay code will be reused to allow the player to focus on choosing a character.

Stage and mode made attributes as they will be used in switching scenes once choices are confirmed. Mode is crucial as, in practise, the player should only pick one character

The icons are manually made and placed thirds across the screen. This will make the placement compatible with any screen size. They are added to a list for easy access to loop through and check for hovers/button presses.

Turn will be used to check if the second player should select, once the first player is done confirming. Selected and confirmed will be used to store the current icons for player 1 and 2.

This switch scene function below is very similar to the one in the stage selection menu and will only be called once both players selections are confirmed, or one player (if in practise mode).

The draw function, this time, is more complicated as a preview should not be drawn if a choice is confirmed.

```

def draw(self, screen ):
    screen.blit(self.background, self.backgroundRect)
    screen.blit(self.overlay, self.overlayR)
    for icon in self.icons:
        icon.draw(screen)

    #will only draw the selected if a there is no confirmed yet
    #lists are 0-indexed
    if self.confirmed[0] ≠ None:
        self.confirmed[0].drawPreview(screen, 1)
    elif self.selected[0] ≠ None:
        self.selected[0].drawPreview(screen, 1)

    if self.confirmed[1] ≠ None:
        self.confirmed[1].drawPreview(screen, 2)
    elif self.selected[1] ≠ None:
        self.selected[1].drawPreview(screen, 2)

```

First, it will check the confirmed for each player. If that is picked, then it will draw that and skip to the next player. However, if there's no confirmed, it will check the selected to draw that preview.

```

class PractiseScene(scene):
    def __init__(self, manager, assets, sSize, stage, character):
        super().__init__(manager, assets, sSize)
        self.stage = StageFactory(stage, self.assets, self.sSize)
        self.Player1 = CharacterFactory(character, self.assets, self.manager.controls)

```

```

class LocalModeScene(scene):
    def __init__(self, manager, assets, sSize, stage, c1, c2):
        super().__init__(manager, assets, sSize)
        #stage and player's creation
        self.stage = StageFactory(stage, self.assets, sSize)
        #different spawn points and player numbers
        self.player1 = CharacterFactory(c1, self.assets, self.manager.controls, self.stage.getSpawnPoint(1), 1)
        self.player2 = CharacterFactory(c2, self.assets, self.manager.controls, self.stage.getSpawnPoint(2), 2)
        #stores number of wins for each player
        #index 0 is player 1 and index 1 is player 2

```

I have changed the practise and local mode scenes slightly so they use the characters specified in the parameters, which will be passed in by the new character selection scene.

```
class StageSelectScene(scene):
    def __init__(self, manager, assets, sSize, mode):
        #mode for selecting scene
        self.mode = mode

        #font may be changed
        self.font = self.assets.getFont("PressStart2P")
        self.text = [self.font.render(text, True, (0,0,0)) for text in self.names]
        #all text on top of the stages
        self.textR = [x.get_frect(midbottom = self.rects[self.text.index(x)].midtop) for x in self.text]

    def switchScene(self, ID):
        match self.mode:
            case "practise":
                #should be changed to character selection once made
                self.manager.change_scene(CharacterSelectScene, self.manager)
            case "local":
                #should be changed to character selection once made
                self.manager.change_scene(CharacterSelectScene, self.manager)
            case "host":
                pass
```

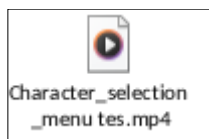
I have rerouted the stage selection scenes to point to the character select scene, in the cases of a practise or local game.

Testing the character selection menu

What I'm testing	Input data	Why am I testing this?	Expected outcome	Test results
Is the character selection menu drawn as described in the code when pressing local and practise?	Mouse press on local and practise buttons.	Allows the user to select a character to play the game.	Character selection menu with icons and previews should be drawn.	As, expected.
Do characters show up as previews if the mouse is	Mouse hover over character icons.	Allows the user to see what the character looks like before selecting it.	Previews should be drawn on the left hand side	As, expected.

hovering over them?			for player 1 and the right for player 2.	
Are the choices confirmed once icons are pressed?	Mouse press on character icons.	This allows the game to start, once the characters have been selected.	Practise mode should play, or local mode if local button is pressed previously.	As, expected
Are icons and previews in the right position and size?	Mouse press local or practise buttons.	Allows the user to see the previews and icons and select them.	All icons and previews should be appropriate size, which I will judge.	The icons are too big and the previews are too small. The character for player 2 should be flipped as its on the right side. The characters are also not centered properly.

A video of the results are shown below:



Firstly, I will change the sizes of the characters and icons. I will halve the size of the icons and triple the size of the characters as it roughly looks, from that video, suitable.

```
def load_icons(self):
    #for all icons in the folder
    for file in os.listdir(self.icons_path):
        #all icons are png
        icon = pygame.image.load(os.path.join(self.icons_path, file)).convert_alpha()
        #*by SF to make icon sizes suitable for all screen sizes
        icon = pygame.transform.scale_by(icon, 0.1*self.SF)

        name = file[:file.find(".")]
```

```

class CharacterIcon:
    def __init__(self, name, assets, sSize, **pos_kwargs):
        self.name = name

        #icon files have a "_icon" at the end
        self.icon = assets.getIcon(name+"_icon")
        self.iconR = self.icon.get_frect(**pos_kwargs)

        #first frame of the idle animation will be a preview
        self.preview = assets.getAnimations(name)["idle"][0]
        self.preview = pygame.transform.scale_by(self.preview, 3*assets.SF)
        self.x1 = sSize[0]/3

```

From looking at how the previews are positioned, I realised I forgot to place the center's x position, rather than the top left:

```

def drawPreview(self, screen , preview):
    #if the preview is for player 1
    if preview == 1:
        #change x to left side and blit
        self.previewR.centerx = self.x1
        screen.blit(self.preview, self.previewR)
    #if the preview is for player 2
    elif preview ==2:
        self.previewR.centerx = self.x2
        flip = pygame.transform.flip(self.preview, True, False)
        screen.blit(flip, self.previewR)

```

I also made sure to flip the preview for player 2.

I also rerouted the characters button in the practise arena back to the character selection menu:

```

class PractiseScene(scene):
    def __init__(self, manager, assets, sSize, stage, character):
        #characters button to character selection menu, topleft corner
        characters, charactersG = self.assets.getButton("characters")
        self.charactersButton = Button(characters, charactersG, lambda:
            self.manager.change_scene(CharacterSelectScene, self.manager, self.assets, self.sSize,
            "practise", stage),
            topleft = (exit.get_width() + (self.sSize[0]/100), self.sSize[1]/128))

```

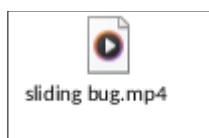
What I'm testing	Input data	Why am I testing this?	Expected outcome	Test results
Are icons and previews in the right position and size?	Mouse press local or practise buttons.	Allows the user to see the previews and icons and select them.	All icons and previews should be appropriate size, which I will judge.	As, expected.
Does the characters button change scenes to the character selection menu?	Mouse press on the characters button.	Allows users to test out all characters in practise mode		

Screenshot of the character selection menu after these changes:



Character sliding bug

While testing the character selection menu, I noticed a weird sliding bug that occurred with both characters' idle animations. I have attached an example video below:



I thought this was due to the centre points being constantly changed from getting a bounding rect of each image separately. The knight's cape waving in the air was causing the centre to shift left and misplace the mid bottom of the character every frame (like the wizard character). To confirm this, I removed the code that cut out each image in the loading assets function:

```

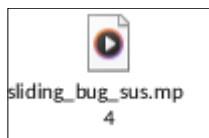
class assets:
    def load_character(self, name, fSize, sf):
        x = 1*fSize[0]

        #splicing and storing
        frame = ss.subsurface((x,y), fSize)
        #line below commented out
        #cutout = frame.subsurface(frame.get_bounding_rect())

        #scale by takes 2 inputs, image and scale factor ( 1 value )
        scaled = pygame.transform.scale_by(frame, sf)

```

This confirmed my suspicions:



To fix this issue, I made the hitbox/rect of the image constant rather than always changing based on the frame. Then, I will use the full square frame rather than the cutout and centre it according to the centre of that full square frame.

In the video that confirmed my suspicions, it was using the full square image as the hitbox which was why the characters were floating.

The full images' centre points are always perfect but producing a cut out of them offsets that centre point and causes glitches. I should have realised this when I was writing the animation code, as most games nowadays also use constant hitboxes, rather than make a new hitbox in every frame.

After ensuring that the assets are loaded as they are and not cut out, I will use a cutout in the character class initially to create the hitbox:

```

class Character:
    def __init__(self, name, assets, controls, spawn_point, number, recovery1 = 2, recovery2 = 2):
        self.recovery1 = recovery1
        self.recovery2 = recovery2

        #initilizes the image and rect after the setup
        self.image = self.current_animation.getFrame(self.facing_left)
        self.rect = self.image.get_bounding_rect()
        self.rect.midbottom = self.spawn_point

```

Then in the draw function, I will make a temp rect of the full image to align the center of it on the centre of the hitbox:


```
def draw(self, screen):
    #Draws character to screen
    pygame.draw.rect(screen, (0,0,0), self.rect, 0)

    rect = self.image.get_frect(center = self.rect.center)
    screen.blit(self.image, rect)
```

This line will draw the hitbox on the screen, for testing purposes.

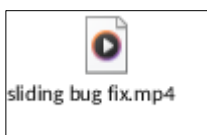
This temp rect does not matter and is only used to position the image. I have also removed the line in the update function where the rect is updated, now it is just updating the image each frame as the rect/hitbox will be constant.

```
def update(self, platforms, dt, mpos, mbut, mbut_j, keys, keys_j):
    #move function checks movement/attack keys and changes animations accordingly
    self.moveV(platforms, dt, keys_j)
    self.moveH(platforms, dt, keys)
    self.attack(keys_j)
    #needs to be updated after every move function call
    self.current_animation = self.animations[self.animation_state]

    #updates animation and statuses
    self.updateStatus(dt)

    #updates the image and rect
    self.image = self.current_animation.getFrame(self.facing_left)
```

I will now test out the code:



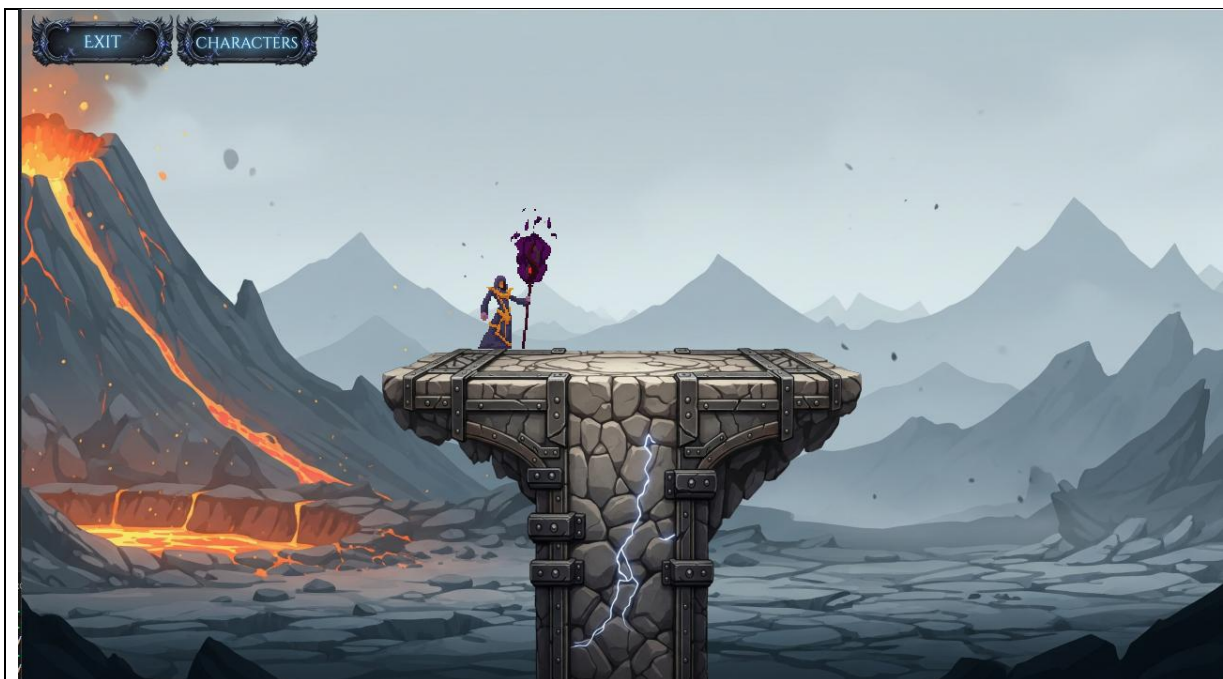
From this video I can see that, the knight character is now fixed however the wizard is positioned slightly above the hitbox because of the asset's center being slightly off. To fix this, in the wizard class, I will override and draw function and put an offset vertically.

```
class evilWizard(Character):
    def __init__(self, assets, controls, spawn_point, number, recovery1, recovery2):
        super().__init__("evil_wizard", assets, controls, spawn_point, number, recovery1, recovery2)

    def draw(self, screen):
        #Draws character to screen
        rect = self.image.get_frect(center = (self.rect.centerx, self.rect.centery+self.rect.h/15))
        screen.blit(self.image, rect)
```

I chose to put an offset based on the hitbox height so is suitable for all screen sizes as the hitbox size will be based on the image size and that assets class applies a screen size scale factor to that. I have chosen to divide the height by 15 after testing out 20.

The result:



I will also fix the weird positions in the character selection menu that were caused by the assets class no longer cutting out the characters.



The fix is simple, I will cut out the previews in the character icon class:

```
class CharacterIcon:
    def __init__(self, name, assets, sSize, **pos_kwargs):
        self.name = name

        #icon files have a "_icon" at the end
        self.icon = assets.getIcon(name+"_icon")
        self.iconR = self.icon.get_frect(**pos_kwargs)

        #first frame of the idle animation will be a preview
        self.preview = assets.getAnimations(name)["idle"][0]
        self.preview = self.preview.subsurface(self.preview.get_bounding_rect())
        self.preview = pygame.transform.scale_by(self.preview, 3*assets.SF)
        self.x1 = sSize[0]/3
        #reflects around the middle of the screen of p2
        self.x2 = sSize[0] - self.x1
        #default is p1
        self.previewR = self.preview.get_frect(midbottom = (self.x1,sSize[1]))
```

The result:



More accurate attacks

As I have now removed the cutouts feature, attacks will only work once a player is close to the other player, not taking the size of their new hitbox once their weapon has been extended.

To implement better attack registering, I will use pygame masks. Masks will create a super accurate exact cutout (not rectangle) of the character, very similar to a shadow of a person.

These are better than the previous way and are simple to use and check for collisions. Although masks are computationally expensive, I will only create one when a player is attacking and not every frame.

While I am editing the take damage function to implement masks, I will also change the damage numbers for heavy and light attacks. Light attacks will damage the opponent by 8 while the heavy attacks will damage by 15. I have chosen these numbers as the max health is 100, so, for rounds to last for a good time, these numbers make sense.

```

def take_damage(self, player):
    #if not invincible and the opponent is attacking
    if player.attacking and not self.invincible:
        if player.attack_use:
            #mask2 is the opponent
            mask2 = pygame.mask.from_surface(player.image)
            me = pygame.mask.from_surface(self.image)
            #need to center the masks perfectly on top of the images
            #the images are centered on the rects, the masks are the same size so
            #center them on rects too
            meIR = self.image.get_frect(center = self.rect.center)
            otherIR = player.image.get_frect(center = player.rect.center)
            #offset is distance from me to the player
            # AB is B-A
            offset = (otherIR.x-meIR.x, otherIR.y-meIR.y)
            #if the opponent collides
            if me.overlap(mask2, offset):
                #value needs to be edited for heavy and light attacks
                if player.animation_state == "attack1":
                    #light attacks take 8
                    self.health -= 8
                elif player.animation_state == "attack2":
                    #heavy attacks take 15
                    self.health -= 15

```

The masks need to be centred directly on top of the images for collisions to work properly. Since the images are centred on the rects, I use the rects for positioning the masks.

Offsets are needed for masks to calculate collisions. They are basically a direction vector from the player to the opponent.

If the opponent is light attacking, the animation state will be attack 1 and attack 2 for heavy, so if they 8 health should be deducted.

Attack use should be reset no matter the attack, to ensure the player is not damaged every frame.

I will test this code once I finish the hud.

Creating the HUD

To create the HUD, I will need to program some health bars and a timer for each round that will trigger the end of the round if the time runs out.

I will start by adding a max health attribute to the characters class that will be used for calculating how big the health bar should be:

```

class Character:
    def __init__(self, name, assets, controls, spawn_point, number, recovery1 = 2, recovery2 = 2):
        self.name = name
        self.assets = assets

        #Player number is used for controls
        self.controls = controls
        self.number = number

        #loads all the animation images
        #puts each one into its own class
        self.load_animations()
        self.maxHealth = 100
        self.health = 100

```

Next, I will make the HUD, initially focusing on the health bars:

```

class HUD:
    def __init__(self, sSize, timer=90):
        #sets the timer for each round
        self.timer = timer

        self.pos1 = (sSize[0]/5, 0)
        #pos 2 is based on pos 1 for easy changes
        self.pos2 = (sSize[0]- self.pos1[0], self.pos1[1])

        #initial healthbar sizes
        self.size = (sSize[0]/4, sSize[1]/64)

        #the bars that will hold the substance
        self.healthBar1 = pygame.Surface(self.size)
        self.healthBar1.fill((10, 25, 80))
        self.healthBar2 = pygame.Surface(self.size)
        self.healthBar2.fill((80, 10, 25))
        self.healthBar1R = self.healthBar1.get_frect(topleft = self.pos1)
        self.healthBar2R = self.healthBar2.get_frect(topright = self.pos2)

        #start time in seconds
        sTime = pygame.time.get_ticks()/1000

```

The sizes will be a quarter of the screen and the position of the first health bar will be a 5th along the screen. Player 2's health bar will be reflected. Player 1's health will be red, while player 2's will be blue to create a nice fitting contrast. Health bar 1's top left is set to position 1 while the top right of health bar 2 is set to position 2, this is due to the symmetry.

The timer is passed in; in case I want to add more features like custom timers later. 90 is set as the default if it is not passed in. The screen size is passed in for accurate health bar sizes and placements.

Once a HUD has been instantiated, a round would be started so the start time is instantly recorded.


```
def draw(self, screen, player1, player2):
    #Calculates ratio of health left
    health1Ratio = player1.health/player1.maxHealth
    health2Ratio = player2.health/player2.maxHealth

    #new sizes for the health
    size1 = health1Ratio*self.healthBar1.get_width()
    size2 = health2Ratio*self.healthBar2.get_width()

    #the health substance, starts at 100% size
    bar1 = pygame.Surface((size1, self.healthBar1.get_height()))
    bar1.fill((180, 20, 20))
    bar2 = pygame.Surface((size2, self.healthBar2.get_height()))
    bar2.fill((20, 80, 180))

    screen.blit(self.healthBar1, self.healthBar1R)
    screen.blit(self.healthBar2, self.healthBar2R)
    #inner bars
    screen.blit(bar1, bar1.get_frect(topleft = self.healthBar1R.topleft))
    #topright for p2 as that is on the right side of the screen
    screen.blit(bar2, bar2.get_frect(topright = self.healthBar2R.topright))
```

The actual health inside the health bars will be calculated in the draw function, with their width being a fraction of the original health bar size. The fraction will be each player's health over their max health.

The health bars will be drawn first as they are under the health indicators.

Top left and top right placement is chosen appropriately for symmetry, like in the constructor method.

As the draw function is getting slightly too long and messy, I will split the drawing the timer into a split function and call it in the main draw function.

I will need to add a function to the assets class to load in a font in a custom size. This will allow me to change the current default of 16 to a larger size, needed for the HUD.

```
def getFont(self, name):
    return self.fonts[name]

def loadFont(self, name, size):
    font = pygame.font.Font(os.path.join(self.fonts_path, name), size)
    return font
```

```
class HUD:
    def __init__(self, sSize, assets, timer=90):
        self.font = assets.loadFont("PressStart2P.ttf", 32)
```

To draw the time to the screen, I need the assets class to load the font in.

```
def update(self, ):
    #current time in seconds
    cTime = pygame.time.get_ticks()/1000

    #if round has ended, return True
    if cTime - self.sTime ≥ self.timer:
        return True
    else:
        return False

def drawTime(self, screen):
    #current time is difference between current and start
    CTIME = pygame.time.get_ticks()/1000 - self.sTime
    #text surface/rect
    cTime = self.font.render(CTIME, True, (255,255,255))
    cTimeR = cTime.get_frect(midtop = (self.sSize[0]/2, 0))
    #draw text to screen
    screen.blit(cTime, cTimeR)
```

This update function is needed to return true once the time is up. It should signal the end of a round.

It does this by checking whether the difference between the start and current time is greater than the timer set in the parameters of the constructor method.

Here, the time is rendered as text, using the font retrieved from the assets and drawn at the middle but top of the screen.

For these changes to take effect, I will need to edit the local mode scene code to include the HUD. The practise mode won't need a HUD as the player won't lose health and there is no timer.


```

class LocalModeScene(scene):
    def __init__(self, manager, assets, sSize, stage, c1, c2):
        super().__init__(manager, assets, sSize)
        #stage and player's creation
        self.stage = StageFactory(stage, self.assets, sSize)
        #different spawn points and player numbers
        self.player1 = CharacterFactory(c1, self.assets, self.manager.controls, self.stage.getSpawnPoint(1), 1)
        self.player2 = CharacterFactory(c2, self.assets, self.manager.controls, self.stage.getSpawnPoint(2), 2)
        #stores number of wins for each player
        #index 0 is player 1 and index 1 is player 2
        self.wins = [0,0]
        self.hud = HUD(self.sSize, self.assets)

```

```

def draw(self, screen):
    self.stage.draw(screen)
    self.player1.draw(screen)
    self.player2.draw(screen)
    self.hud.draw(screen, self.player1, self.player2)

```

Testing the HUD

First, I will test the health bars and new accurate attacks, and then the time.

What I'm testing	Input data	Why am I testing this?	Expected outcome	Test results
Are health bars drawn to the screen in their appropriate positions?	Mouse press on local mode button.	This will allow the players to see how much health they have left easily.	Health bars should be 5 th along the screen from either side and at the top of the screen.	As, expected.
Are the colours suitable?	Mouse press on local mode button.	Colours need to be visually pleasing.	A bright red and blue for the health bars.	The colours underneath bright colours have been flipped.
Does the health decrease once a player is hit?	"p" or "l" once close the opponent	Attacks need to be responsive and show up on the hud, so players can see how much health they have left at any point in the round.	The bright red or blue needs to get smaller to show the darker colour underneath.	As, expected.

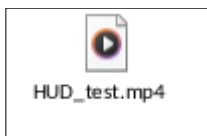
Does light attack lower the health by 8?	"p"	Attacks need to deduct health, to create a good 2D fighter game.	On the hud, the health should be lowered.	As, expected.
Does heavy attack lower health by 15?	"l"	Attacks need to deduct health, to create a good 2D fighter game.	On the hud, the health should be lowered.	As, expected.

To fix the colours being swapped, I will just swap the colours in the HUD draw function:

```
class HUD:
    def __init__(self, sSize, assets, timer=90):
        #the bars that will hold the substance
        self.healthBar1 = pygame.Surface(self.size)
        self.healthBar1.fill((80, 10, 25))
        self.healthBar2 = pygame.Surface(self.size)
        self.healthBar2.fill((10, 25, 80))
        self.healthBar1R = self.healthBar1.get_rect(to
```

What I'm testing	Input data	Why am I testing this?	Expected outcome	Test results
Are the colours suitable?	Mouse press on local mode button.	Colours need to be visually pleasing.	A bright red and blue for the health bars.	As, expected.

A video of the results is shown below:



Now I can test the time by calling the drawTime function in the HUD draw.

```

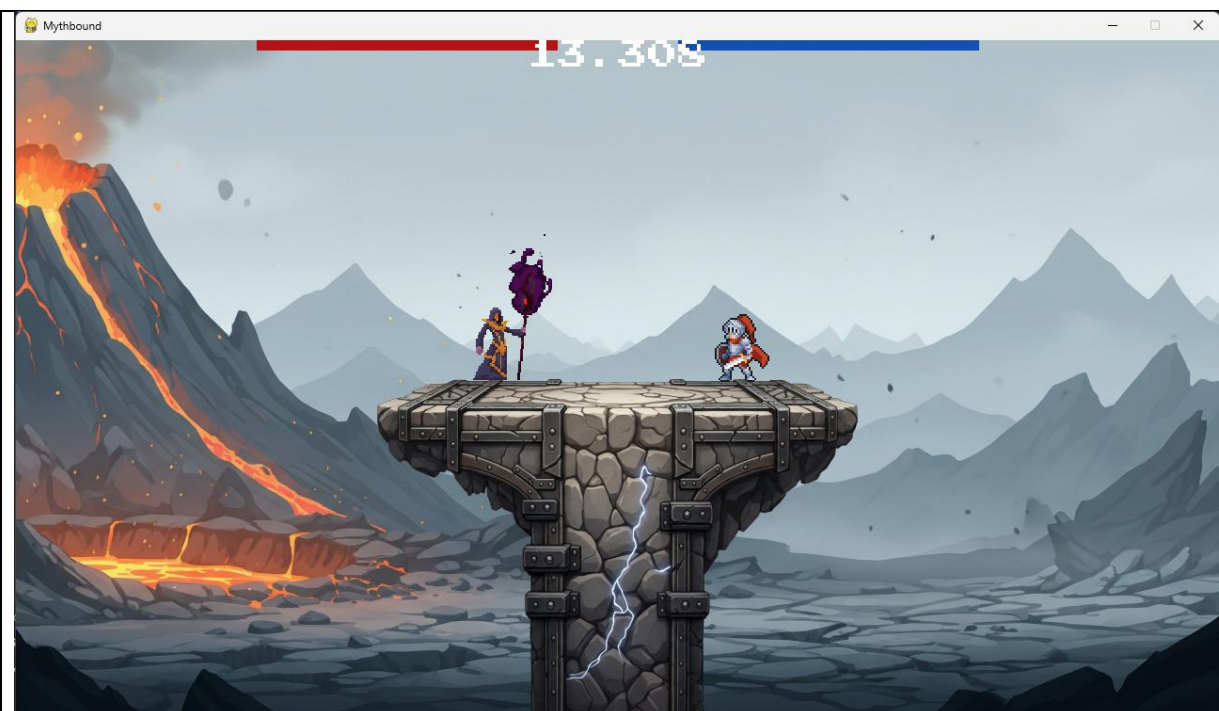
class HUD:
    def draw(self, screen, player1, player2):

        screen.blit(self.healthBar1, self.healthBar1R)
        screen.blit(self.healthBar2, self.healthBar2R)
        #inner bars
        screen.blit(bar1, bar1.get_frect(topleft = self.healthBar1R.topleft))
        #topright for p2 as that is on the right side of the screen
        screen.blit(bar2, bar2.get_frect(topright = self.healthBar2R.topright))

        self.drawTime(screen)

```

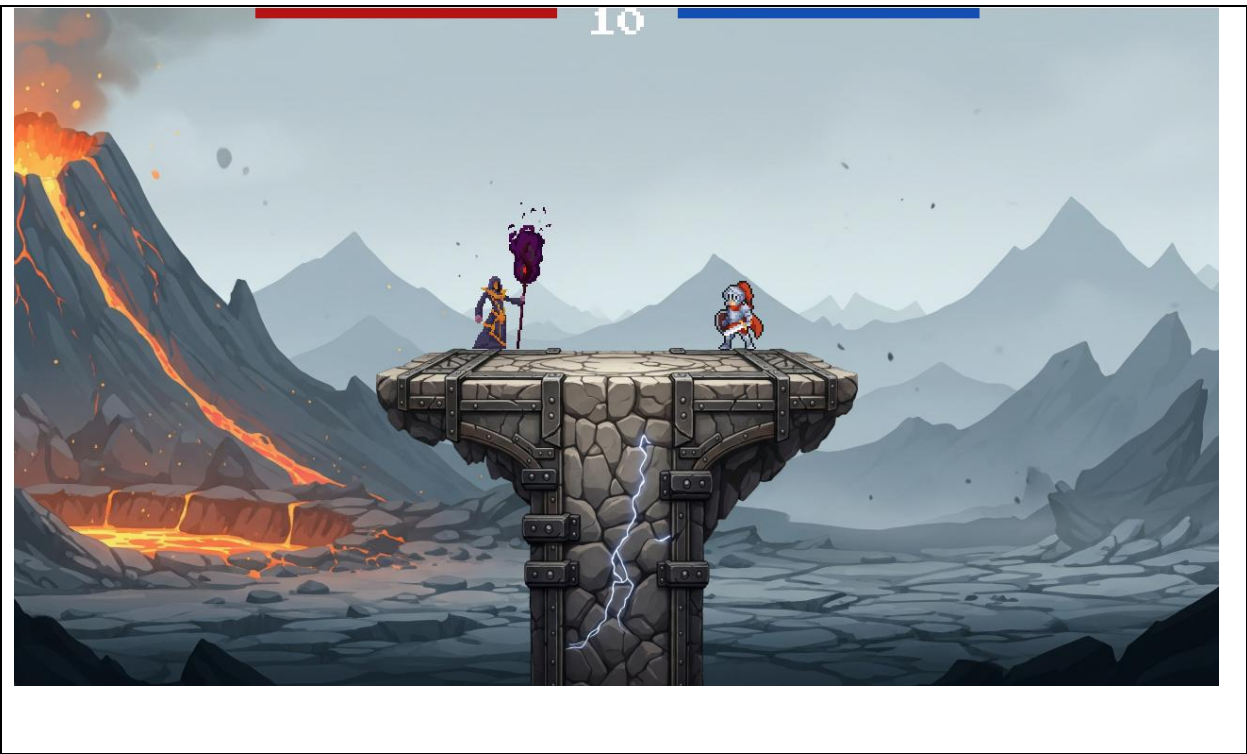
What I'm testing	Input data	Why am I testing this?	Expected outcome	Test results
Does the time show on the screen, accurately.	Mouse press on local mode button.	Drawing the time will allow players to see how much time is left in each round.	Time should be shown in the mid top of the screen, counting up in seconds.	The time is in seconds, but it also shows milliseconds. As a result of this, it is not centred properly.



I can fix this by turning in the time into an integer before rendering it:

```
def drawTime(self, screen):
    #current time is difference between current and start
    CTIME = pygame.time.get_ticks()/1000 - self.sTime
    #text surface/rect
    cTime = self.font.render(str(int(CTIME)), True, (255,255,255))
    cTimeR = cTime.get_frect(midtop = (self.sSize[0]/2, 0))
    #draw text to screen
    screen.blit(cTime, cTimeR)
```

What I'm testing	Input data	Why am I testing this?	Expected outcome	Test results
Does the time show on the screen, accurately.	Mouse press on local mode button.	Drawing the time will allow players to see how much time is left in each round.	Time should be shown in the mid top of the screen, counting up in seconds.	As, expected.



Now I can make the timer have an effect by using its update function in the local mode update:

```
class LocalModeScene(scene):
    self.hud = HUD(self.sSize, self.assets)

    def checkRound(self):
        #resets both players if one player is defeated
        #if player 1 is defeated
        if self.player1.health <= 0:
            #player 2 wins incremented
            self.wins[1] +=1
            self.player1.reset()
            self.player2.reset()
        #if player 2 is defeated
        elif self.player2.health <= 0:
            #player 1 wins are incremented
            self.wins[0] +=1
            self.player1.reset()
            self.player2.reset()
        elif self.hud.update():
            #player with the lowest health loses
            if self.player1.health > self.player2.health:
                self.wins[0] +=1
                self.player1.reset()
                self.player2.reset()
            elif self.player2.health > self.player1.health:
                self.wins[1] +=1
                self.player1.reset()
                self.player2.reset()
            #if healths are equal then no wins to either
            #round is restarted
            else:
                self.player1.reset()
                self.player2.reset()

        #best of 3 system
```

If the hud update function returns true, so the timer is up and the previous conditions were not met (no player was defeated), then it will check the player with the highest health who will win. Else, if their healths are equal, the round is restarted.

What I'm testing	Input data	Why am I testing this?	Expected outcome	Test results
Does the round end appropriately once the stopwatch is past 90?	Waiting a round in the game.	This will allow rounds to not go on forever and end once the timer is up.	Player 1/2 should be awarded the win, if end with higher health. If they are equal, then the game should go on.	As, expected.

Improving the stage selection menu

To finish off this iteration, I want to improve the stage selection menu and add more stages and characters. Doing this will be easy because of the reusability of components that I have created.

Before adding new stages and characters, I will fix the stage selection menu code as it looks quite messy right now:

```

class StageSelectScene(scene):
    def __init__(self, manager, assets, sSize, mode):
        super().__init__(manager, assets, sSize)
        #Sets background
        self.background = self.assets.getBackground("start_menu")
        self.backgroundRect= self.background.get_frect(topleft = (0,0))

        #Transparent darker overlay, so user can focus on settings
        self.overlay= pygame.Surface(self.sSize, pygame.SRCALPHA)
        self.overlay.fill((0,0,0,150))
        self.overlayR = self.overlay.get_frect(topleft=(0,0))

        self.backgrounds = [pygame.transform.scale_by(self.assets.getBackground("volcano_valley"), 0.25)]
        self.rects = [x.get_frect(center = (self.sSize[0]/2, self.sSize[1]/2)) for x in self.backgrounds]
        self.names = ["Volcano Valley"]
        self.IDs = ["volcano_valley"]

        #mode for switching scenes
        self.mode = mode

        #font may be changed
        self.font = self.assets.getFont("PressStart2P")
        self.text = [self.font.render(text, True, (0,0,0)) for text in self.names]
        #all text on top of the stages
        self.textR = [x.get_frect(midbottom = self.rects[self.text.index(x)].midtop) for x in self.text]

```

I want to create a stage icon class like the character icons, to hide the reduce the number of unnecessary lists that need to be kept indexed and ordered. This will also reduce the risk of errors and make the code more reusable.

```

class StageIcon:
    def __init__(self, name, ID, assets, highlight, sSize, **pos_kwargs):
        #backgrounds are already scaled to the screen size
        #0.25 will work on any screen
        #mini preview not large
        self.preview = pygame.transform.scale_by(assets.getBackground(ID), 0.25)
        self.rect = self.preview.get_frect(**pos_kwargs)

        #outline will light up if mouse hover
        self.outline = pygame.Surface((self.preview.get_width() + 0.05*self.preview.get_height(), 1.05* self.preview.get_height()))
        self.orect= self.outline.get_frect(**pos_kwargs)

        self.ID = ID
        self.name = name

        self.font = assets.getFont("PressStart2P")

        #Creates text surface and places it mid top of the mini preview outline
        self.text = self.font.render(self.name, True, (255,255,255))
        self.textR = self.text.get_frect(midbottom = self.orect.midtop)

        #for nice visual effect to change the outline
        self.colour = (0,0,0)
        self.highlight = highlight #constant

```

I added an outline that will be lit up once the mouse hovers over it. The name will be used for text and the ID for switching to the next scene which will retrieve the assets using this ID.

The highlight is a constant that will be used to colour the outline once the mouse hovers over the icon. Different stages will have different themes, so I can specify each highlight colour for a more fitting and pleasing look.


```
def update(self, mpos, mbut_j):
    #returns true if clicked
    if self.rect.collidepoint(mpos):
        #changes colour for hover effect
        self.colour = self.highlight
        if mbut_j[0]:
            return True
    else:
        #black if no hover
        self.colour = (0,0,0)
    return False

def draw(self, screen):
    pygame.draw.rect(screen, self.colour, self.orect, border_radius=5)
    screen.blit(self.preview, self.rect)
    screen.blit(self.text, self.textR)
```

The update function will be used to return whether the player has clicked on the stage or not, for switching to the next scene. It will also highlight the outline colour if the mouse hovers over the icon.

I used a different function to draw the outline as it allows me to have rounded corners using the border radius for a nice effect.

Now the code for the stage select scene looks much neater!

```
class StageSelectScene(scene):
    def __init__(self, manager, assets, sSize, mode):
        super().__init__(manager, assets, sSize)
        #Sets background
        self.background = self.assets.getBackground("start_menu")
        self.backgroundRect= self.background.get_frect(topleft = (0,0))

        #Transparent darker overlay, so user can focus on settings
        self.overlay= pygame.Surface(self.sSize, pygame.SRCALPHA)
        self.overlay.fill((0,0,0,150))
        self.overlayR = self.overlay.get_frect(topleft=(0,0))

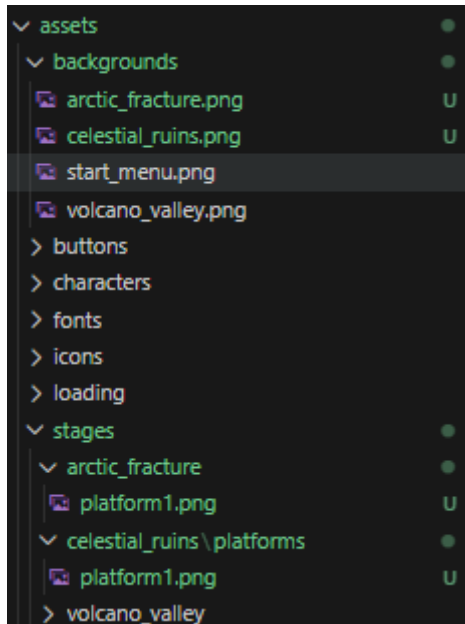
        #mode for switching scenes
        self.mode = mode

        vv = StageIcon("Volcano Valley", "volcano_valley", assets, sSize, (255, 64, 0), center = (self.sSize[0]/3, self.sSize[1]/3))
        self.icons = [vv]
```

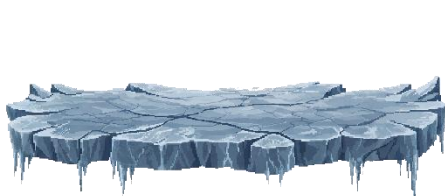
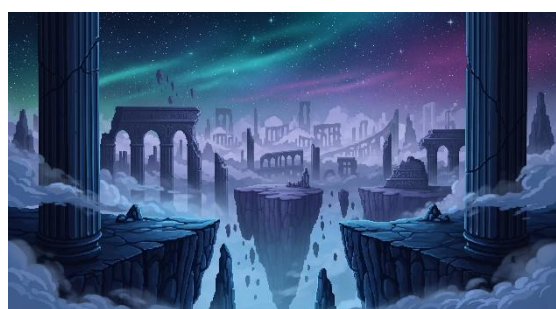
One line for each adding each stage icon, much simpler. For the colour, I thought it fitting for the highlight to be a molten red.

List of icons to loop through and update/draw each one.

To add some new stages, I prompted Google's imagen ultra 4 to create some backgrounds and platforms. I created two stages, one called "Arctic Fracture", and one called "Celestial Ruins". Then I edited these in photopea to remove the backgrounds and convert them to png. I added them to the assets folder so they can be loaded in:



The backgrounds and platforms are below:

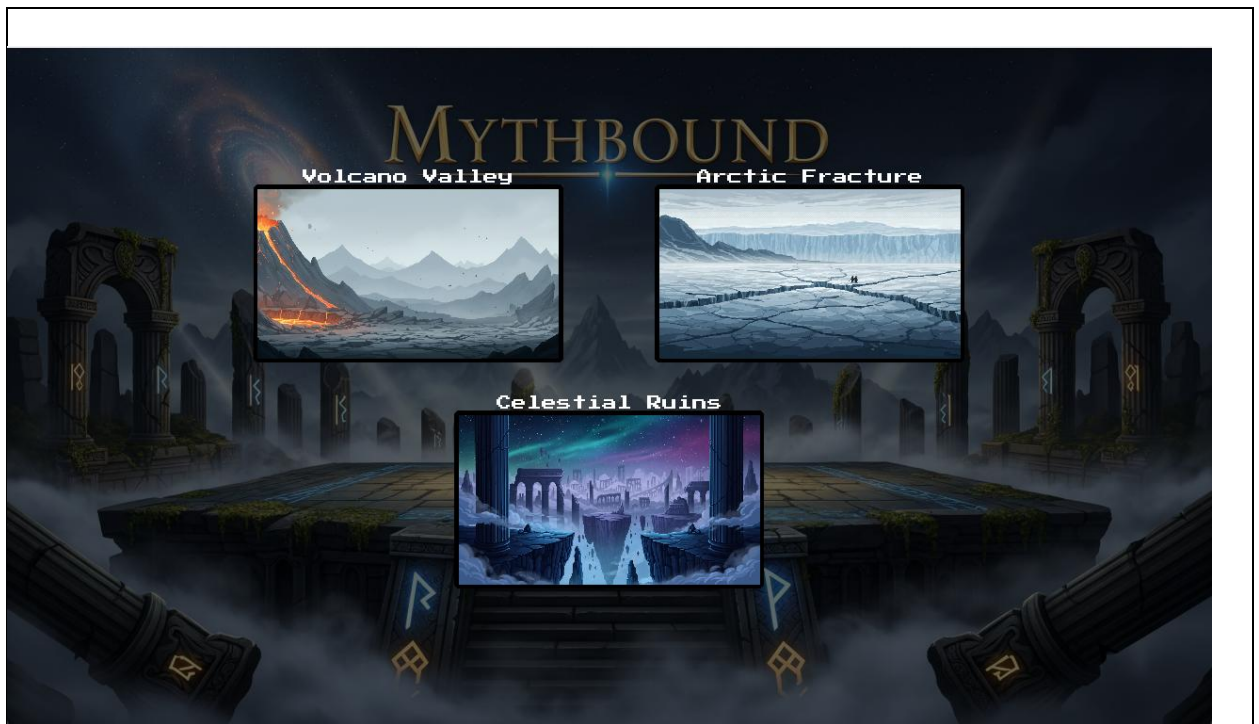


I hope the platforms will fit their backgrounds nicely, however, I have a brighter version of the celestial ruin's platform if necessary.

In the stage selection menu, I can simply add two extra lines and modify the icons list to add this right away:

```
vv = StageIcon("Volcano Valley", "volcano_valley", assets, (255, 64, 0), center = (self.sSize[0]/3, self.sSize[1]/3))
af = StageIcon("Arctic Fracture", "arctic_fracture", assets, (102, 229, 255), center = (2*self.sSize[0]/3, self.sSize[1]/3))
cr = StageIcon("Celestial Ruins", "celestial_ruins", assets, (245, 250, 255), center = (self.sSize[0]/2, 2*self.sSize[1]/3))
self.icons = [vv, af, cr]
```

The stage selection menu now looks like this:



Clicking any of these stages and proceeding to the rounds outputs this error, due their being no function to load these stages correctly in the assets class:

```

File "c:\Users\Basaa\code\WEA1\scene.py", line 438, in switchScene
self.manager.change_scene(LocalModeScene, self.manager, self.assets, self.sSize,
    self.stage, self.confirmed[0].name, self.confirmed[1].name)
File "c:\Users\Basaa\code\WEA1\scene.py", line 37, in change_scene
self.current_scene = new_scene(*args)
File "c:\Users\Basaa\code\WEA1\scene.py", line 215, in __init__
self.player1 = CharacterFactory(c1, self.assets, self.manager.controls, self.stage.getSpawnPoint(1), 1)
AttributeError: 'NoneType' object has no attribute 'getSpawnPoint'

```

To load these stages' platforms in, I will create new functions for each stage. I could just create one function for all stages, however platforms are different sizes, so it's best to create a separate function for each stage.

At first, I used the same function for the volcano valley but slightly edited it for celestial ruins, by just tweaking the names:

```

def load_celestial_ruins(self):
    #The path for this function is the celestial ruins folder inside the stages folder
    path = os.path.join(self.stages_path, "celestial_ruins")
    self.platforms["celestial_ruins"] = {}

    #Goes through each platform
    for file in os.listdir(path):
        #Removes file extension for easy storing
        name = file[:file.find(".")]
        platform = pygame.image.load(os.path.join(path, file)).convert_alpha()
        platform = platform.subsurface(platform.get_bounding_rect()) #removes transparent pixels
        size = platform.get_size()
        ratio = size[0]/size[1]

        if name == "platform1":
            platform = pygame.transform.smoothscale(platform, (self.screenSize[1]/2*ratio, self.screenSize[1]/2)) #half the screen

        #Stores each platform in dictionary
        self.platforms["celestial_ruins"][name] = platform

    self.progressCounter+=1 #Increases loaded images counter by 1

```

```
def load_stages(self):
    self.load_volcano_valley()
    self.load_celestial_ruins()
```

I also added a celestial ruins class and routed the stage factory to it:

```
def StageFactory(name, assets, sSize):
    match name:
        case "volcano_valley":
            return VolcanoValley(assets, sSize)
        case "celestial_ruins":
            return CelestialRuins(assets, sSize)
```

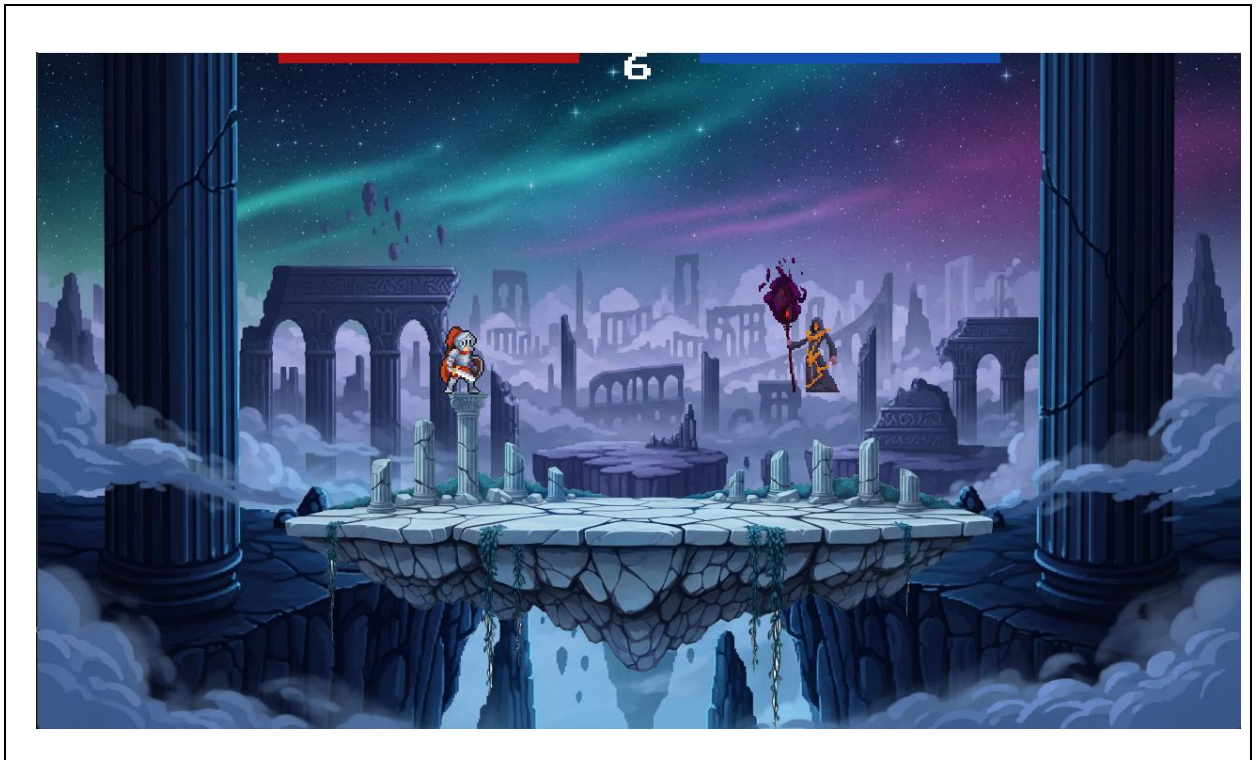
```
class CelestialRuins(stage):
    def __init__(self, assets, sSize):
        super().__init__("celestial_ruins", assets)
        self.platformsR[0].midbottom = (sSize[0]/2, sSize[1])

        #Sets the spawn points for volcano valley
        mainPlatSize = self.platforms[0].get_size()
        width = mainPlatSize[0]
        height = mainPlatSize[1]
        self.spawnPoint1 = (self.platformsR[0].centerx-width/4, self.platformsR[0].top)
        self.spawnPoint2 = (self.platformsR[0].centerx+width/4, self.platformsR[0].top)

    def getSpawnPoint(self, n):
        if n == 1:
            return self.spawnPoint1
        elif n == 2:
            return self.spawnPoint2
```

Exact same as the volcano valley, for testing.

The result is this:



The size of the platform is suitable, however, the rect for collisions is too big and the background is too noisy. I will also need to move the platform up. Although, the spawn points for volcano valley work nicely, so I will leave those.

Right now, the image is half the screen height. From looking at the screenshot above, I will try to make a rect that is half the original height. This rect will be strictly used for collisions and will be retrieved through a new getter function.

The getter function will be made in the parent class, "stage", and will be temporarily set to the normal platforms. This will be overridden in the celestial ruins class but not in the volcano valley class.


```

class stage:
    def __init__(self, name, assets):
        #Each stage has platforms which are contained in a list for easy
        self.platforms = self.assets.getPlatforms(self.name)
        #Rect for each of platforms, position determined in children class
        self.platformsR = [x.get_frect() for x in self.platforms]

        self.platformsCol = [rect.copy() for rect in self.platformsR]

    def draw(self, screen):
        screen.blit(self.background, self.backgroundR)
        #Draws all platforms in the list
        #Assumes platforms and the list of rects hve same length
        for i in range(len(self.platforms)):
            screen.blit(self.platforms[i], self.platformsR[i])

    def getPlatforms(self):
        return self.platformsR

    #for player collisions
    def getHitboxes(self):
        return self.platformsCol

```

```

class CelestialRuins(stage):
    def __init__(self, assets, sSize):
        super().__init__("celestial_ruins", assets)
        self.platformsR[0].midtop = (sSize[0]/2, sSize[1]/4)

        self.platformsCol[0].h = self.platformsCol[0].h/2
        self.platformsCol[0].center = self.platformsR[0].center

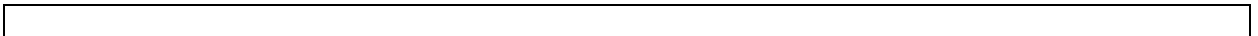
        #Sets the spawn points
        mainPlatSize = self.platformsCol[0].size
        width = mainPlatSize[0]
        height = mainPlatSize[1]
        self.spawnPoint1 = (self.platformsCol[0].centerx-width/4, self.platformsCol[0].top)
        self.spawnPoint2 = (self.platformsCol[0].centerx+width/4, self.platformsCol[0].top)

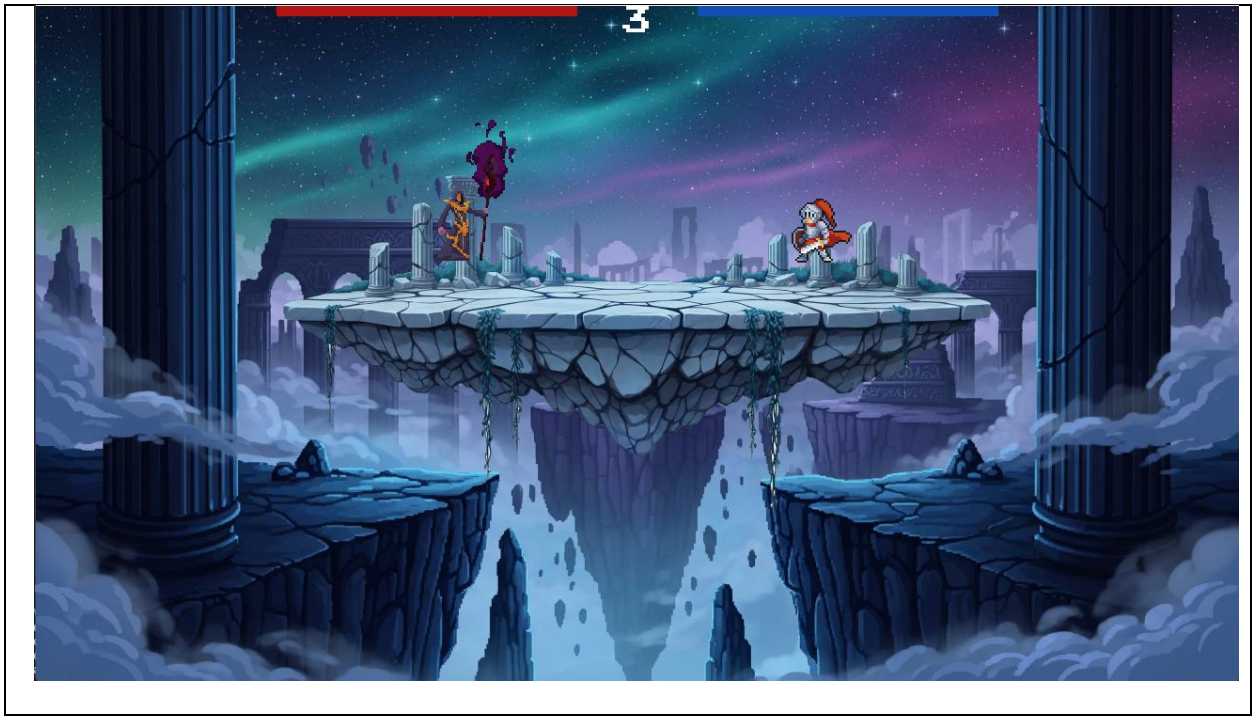
```

Instead of placing the mid bottom at the top of the screen, I have placed the mid top at a quarter of the way down the screen. I reduced the height of the collision platform by half and aligned the centers.

The spawn points are the same except, they are done with the collision platform rather than the platforms used for drawing.

The result:





It is looking much better; however, the background is still too noisy, the platform is too high up and I need to make the collision rect a bit smaller still.

I will try dividing the platform rect height by 3 and try out different placements. As for the noisy background, I will create a transparent overlay and override the draw function to draw it after drawing the background.

```

class CelestialRuins(stage):
    def __init__(self, assets, sSize):
        super().__init__("celestial_ruins", assets)
        self.platformsR[0].center = (sSize[0]/2, sSize[1]/1.75)

        self.platformsCol[0].h = self.platformsCol[0].h/3
        self.platformsCol[0].center = self.platformsR[0].center

        #overlay to reduce noise
        self.overlay= pygame.Surface(sSize, pygame.SRCALPHA)
        self.overlay.fill((0,0,0,150))
        self.overlayR = self.overlay.get_frect(topLeft=(0,0))

        #Sets the spawn points
        mainPlatSize = self.platformsCol[0].size
        width = mainPlatSize[0]
        height = mainPlatSize[1]
        self.spawnPoint1 = (self.platformsCol[0].centerx-width/4, self.platformsCol[0].top)
        self.spawnPoint2 = (self.platformsCol[0].centerx+width/4, self.platformsCol[0].top)

    def draw(self, screen):
        screen.blit(self.background, self.backgroundR)
        screen.blit(self.overlay, self.overlayR)
        #Draws all platforms in the list
        #Assumes platforms and the list of rects hve same length
        for i in range(len(self.platforms)):
            screen.blit(self.platforms[i], self.platformsR[i])

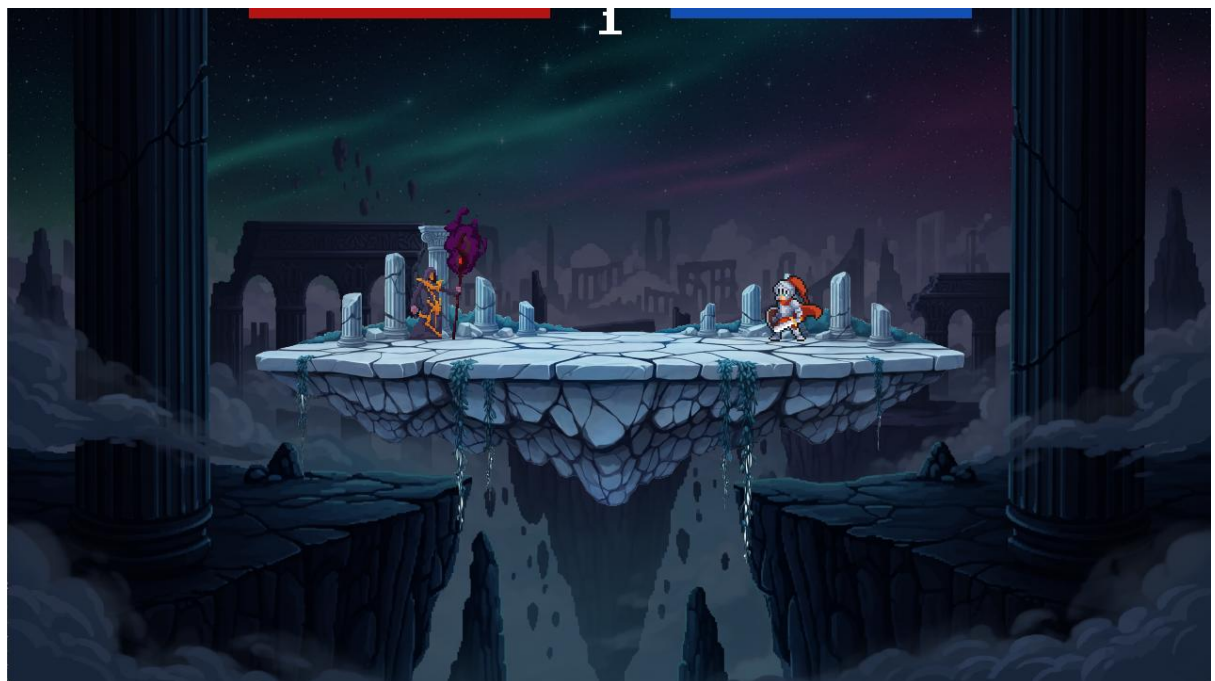
```

Settled on a value of 1.75 after testing.

Overlay code taken from settings scene.

Drawn right after the background but before the platform.

The result:



Now, I will do the same with arctic fracture. This time, I have chosen to exactly copy the celestial ruins code but change the names of everything to arctic fracture:

```
def load_arctic_fracture(self):
    #The path for this function is the celestial ruins folder inside the stages folder
    path = os.path.join(self.stages_path, "arctic_fracture")
    self.platforms["arctic_fracture"] = {}

    #Goes through each platform
    for file in os.listdir(path):
        #Removes file extension for easy storing
        name = file[:file.find(".")]
        platform = pygame.image.load(os.path.join(path, file)).convert_alpha()
        platform = platform.subsurface(platform.get_bounding_rect()) #removes transparent pixels
        size = platform.get_size()
        ratio = size[0]/size[1]

        if name == "platform1":
            platform = pygame.transform.smoothscale(platform, (self.screenSize[1]/2*ratio, self.screenSize[1]/2)) #half the screen

        #Stores each platform in dictionary
        self.platforms["arctic_fracture"][name] = platform

    self.progressCounter+=1 #Increases loaded images counter by 1
```

```
def load_stages(self):
    self.load_volcano_valley()
    self.load_celestial_ruins()
    self.load_arctic_fracture()
```

```
def StageFactory(name, assets, sSize):
    match name:
        case "volcano_valley":
            return VolcanoValley(assets, sSize)
        case "celestial_ruins":
            return CelestialRuins(assets, sSize)
        case "arctic_fracture":
            return ArcticFracture(assets, sSize)
```

```

class ArcticFracture(stage):
    def __init__(self, assets, sSize):
        super().__init__("arctic_fracture", assets)
        self.platformsR[0].center = (sSize[0]/2, sSize[1]/1.75)

        self.platformsCol[0].h = self.platformsCol[0].h/3
        self.platformsCol[0].center = self.platformsR[0].center

        #overlay to reduce noise
        self.overlay= pygame.Surface(sSize, pygame.SRCALPHA)
        self.overlay.fill((0,0,0,150))
        self.overlayR = self.overlay.get_frect(topleft=(0,0))

        #Sets the spawn points
        mainPlatSize = self.platformsCol[0].size
        width = mainPlatSize[0]
        height = mainPlatSize[1]
        self.spawnPoint1 = (self.platformsCol[0].centerx-width/4, self.platformsCol[0].top)
        self.spawnPoint2 = (self.platformsCol[0].centerx+width/4, self.platformsCol[0].top)

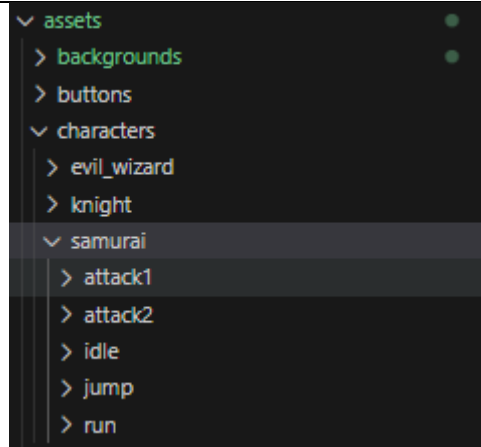
    def draw(self, screen):
        screen.blit(self.background, self.backgroundR)
        screen.blit(self.overlay, self.overlayR)
        #Draws all platforms in the list
        #Assumes platforms and the list of rects hve same length
        for i in range(len(self.platforms)):
            screen.blit(self.platforms[i], self.platformsR[i])

```

Samurai character

Now that all the stages are added, I will add just one more character before concluding the end of this iteration. This character will be a samurai, that I got the assets from on itch.io.

I have added the character assets in the appropriate format to the assets folder:



```
def load_characters(self):  
    #knight asset has frame size of 96x84, test out oSizes  
    #multiply by sf to scale up to any screen size  
    self.load_character("knight", (96,84), 2*self.SF)  
    self.load_character("evil_wizard", (250,250), 1.5*self.SF)  
    self.load_character("samurai", (200,200), 1.75*self.SF)
```

Loaded the character so it can be used. I chose 1.75 as the frame size is roughly in between the other two characters, so it makes sense that the SF is too.

```

class samurai(Character):
    def __init__(self, assets, controls, spawn_point, number, recovery1, recovery2):
        super().__init__("samurai", assets, controls, spawn_point, number, recovery1, recovery2)

    def load_animations(self,):
        animations = self.assets.getAnimations(self.name)
        self.animations = {}
        #Gets the standard animations from the assets
        #Creates an animation object based off each one
        #Values should be overridden
        self.animations["idle"] = animation(animations["idle"], 0.75)
        self.animations["jump"] = animation(animations["jump"], 1.25, [1,2])
        self.animations["run"] = animation(animations["run"], 0.9)
        self.animations["attack1"] = animation(animations["attack1"], 0.5, [2,3,4], [3])
        self.animations["attack2"] = animation(animations["attack2"], 1, [2,3,4], [3])

def CharacterFactory(name, assets, controls, spawn_point, number):
    match name:
        case "knight":
            return knight(assets, controls, spawn_point, number, 2, 2)
        case "evil_wizard":
            return evilWizard(assets, controls, spawn_point, number, 2, 2)
        case "samurai":
            return samurai(assets, controls, spawn_point, number, 2, 2)

```

Values of attack/busy frames have been chosen appropriately after looking at the animations.

I created a samurai class and added it to the character factory.

```

class CharacterSelectScene(scene):
    def __init__(self, manager, assets, sSize, mode, stage):
        super().__init__(manager, assets, sSize)
        #Sets background
        self.background = self.assets.getBackground("start_menu")
        self.backgroundRect= self.background.get_frect(topleft = (0,0))

        #Transparent darker overlay, so user can focus on settings
        self.overlay= pygame.Surface(self.sSize, pygame.SRCALPHA)
        self.overlay.fill((0,0,0,150))
        self.overlayR = self.overlay.get_frect(topleft=(0,0))

        #stores the mode and stage for switch scene
        self.stage = stage
        self.mode = mode

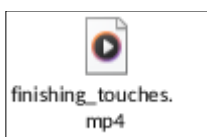
        #icons are manually made for each character
        self.icons = [CharacterIcon("knight", assets, sSize, center = (sSize[0]/4, sSize[1]/4)),
                      CharacterIcon("evil_wizard", assets, sSize, center = (sSize[0]*2/4, sSize[1]/4)),
                      CharacterIcon("samurai", assets,sSize, center = (3*sSize[0]/4, sSize[1]/4))
                      ]

        self.turn = 0
        #list for easy access
        #0 = knight, 1 = evil wizard, 2 = samurai

```

Created a new character icon and passed the name as a parameter. Replaced all the character icons by dividing the screen horizontally into 4, to fit the new samurai.

Here is a video of the results so far:



Interview with the stakeholders

Here are the success criteria that I aimed to complete within this iteration:

#	Success Criteria	Justification	Evidence for testing
8	A HUD at the top of the screen must show each player's health (100) and display a timer in the middle which is 90 seconds per round.	Placing the health bars and timer at the top is perfect as it will allow players to see how much health and time, they have left so that they can plan their next move accordingly. <i>Stakeholders have also confirmed this in the technical interview.</i>	Screenshot of a in-game round, showing the HUD at the top of the screen.
9	Before each round there must be a character selection menu for the players to select their characters.	Necessary so that the players can select a character that they wish to play.	Screenshot of the character selection menu.
11	There must be a stage selection menu for compatible modes like practise, host and local.	This feature will allow users to pick and play on the stages they want. It will make the game more fun and engaging. All solutions researched had this feature. <i>Stakeholders have also confirmed that only hosts, in online games, should be able to pick the stage in the interview.</i>	Screenshot of the stage selection menu.
14	2-player on the same device must also be supported. This should be accessible in the play menu.	This provides a fun experience for players and allows them to play with their friends on the same keyboard if they don't have a Wi-Fi connection.	A screenshot of the code and a video of the feature working.
5	There must be a settings menu accessible from the start of the game,	Changing key binds will make players feel more	A screenshot of the settings menu.

	that allows the players to change their key binds.	comfortable as they may not like the default keys. <i>Stakeholders have also confirmed this in the technical interview.</i>	
--	--	--	--

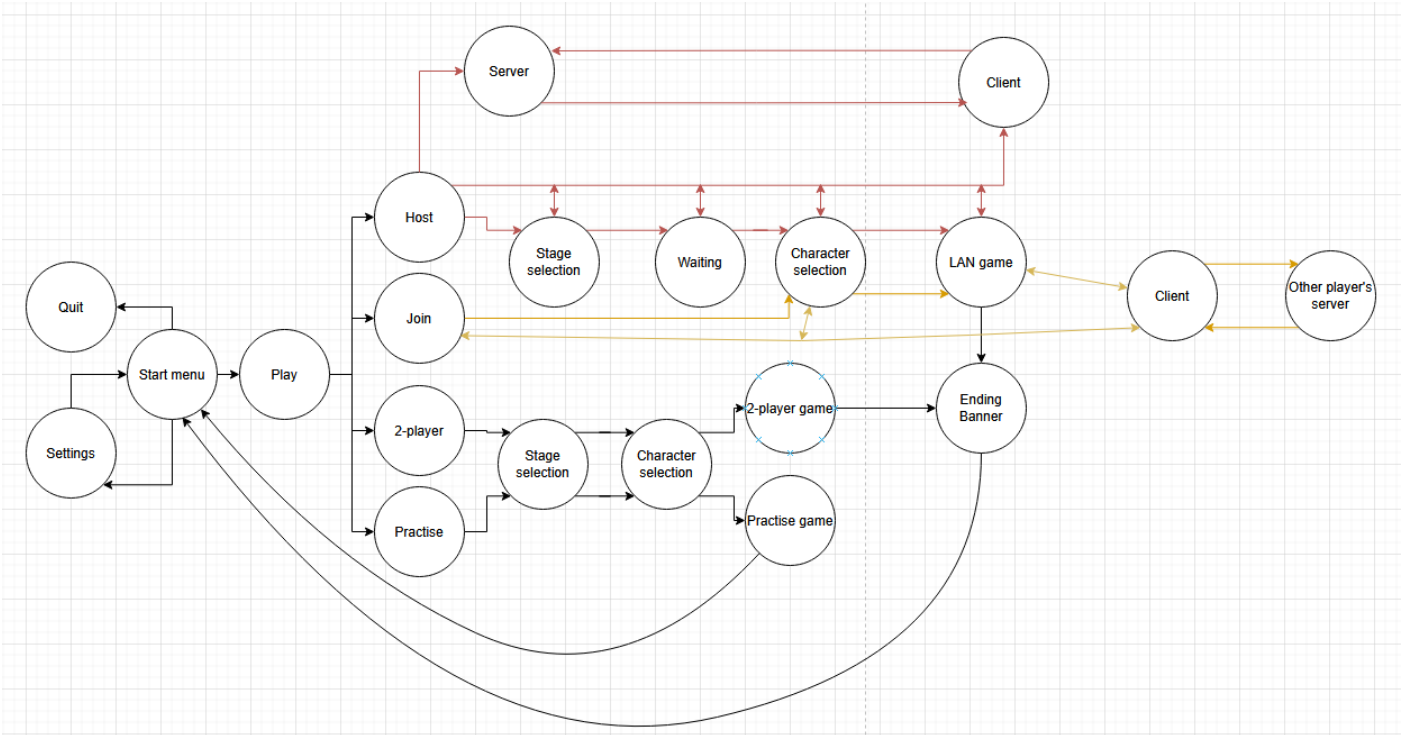
Me: I have created the settings menu this time and routed it to the controls in the local multiplayer mode. I also made the HUD from regular surfaces, not assets as I couldn't find a good substitute.

Summary of what the stakeholder said:

This iteration has really brought the game together. The 2-player mode is fully functional, and being able to customize controls via the new settings menu is a great usability improvement. The HUD is clear and functional, giving players exactly the information they need. As the local gameplay is now solid, I am happy for you to move on to the LAN multiplayer mode.

Implementing LAN mode

Before writing the code, I will review the data flow diagram made in design to help me plan my code out.



Right now, I will be focusing on creating the main Lan game with the server and client code.

I will leave the waiting, joining selection menu and customizing the character selection menu to fit LAN games later. This is to ensure that I get the core gameplay of multiplayer down first.

LAN game class

I will start by creating a LAN game scene class, to separate multiplayer code from normal local mode scene code. This will also reduce errors significantly and the chances of me breaking the local mode.

```
class LANgame(scene):
    #pNumber should be 1 for host and 2 for join
    #this will be used to identify whether to run code for the server
    #and for init
    def __init__(self, manager, assets, sSize, character, stage, pNumber):
        super().__init__(manager, assets, sSize)
        self.network = self.manager.network
        self.server = self.manager.server
        self.hud = HUD(self.sSize, self.assets)

        #if hosting, start server
        if pNumber == 1:
            self.server.start(character, stage)
            #server should be started by host already----
            self.network.connect()

        #other player number used for indexing
        self.opNumber = 1 if pNumber == 2 else 2
        self.pNumber = pNumber
        self.players = [None, None]
```

The hud will be used for both players to display the player info.

The network and server are made attributes for easy access in later functions.

The server should only be started if the player is the host. Both players should connect to this server.

The opponent player number is stored for easy access. This will be used to determine whether certain code will run later.

The players list will be used to access both players for drawing sprites, the HUD, keep track of health , game logic etc.

I will add more to this constructor later, once I have set up the network class for connecting.

Also, I will reroute the host and join buttons to this LAN game scene class.


```

class PlayMenuScene(scene):
    def __init__(self, manager, assets, sSize):
        super().__init__(manager, assets, sSize)

        self.background = self.assets.getBackgroundImage()
        self.backgroundRect = self.background.get_rect()

        practise, practiseG = self.assets.getButton("practise")
        local, localG = self.assets.getButton("local")
        join, joinG = self.assets.getButton("join")
        host, hostG = self.assets.getButton("host")

```

This code was explained earlier, I have just changed the buttons' functions to create a LANgame instance on join or host.

The join button will pass in 2 as a parameter but 1 for host button. These are the player numbers for these modes and will be used in the LANgame class to update different values and handle stage selection.

```

self.joinButton = Button(join, joinG, lambda:
    self.manager.change_scene(LANgame, self.manager, self.assets, self.sSize, "knight", None, 2),
    center=(self.sSize[0]/2, self.sSize[1]*4.5/8))
self.hostButton = Button(host, hostG, lambda:
    self.manager.change_scene(LANgame, self.manager, self.assets, self.sSize, "evil_wizard", "arctic_fracture", 1),
    center=(self.sSize[0]/2, self.sSize[1]*5.5/8))
self.backButton = Button(back, backG, lambda: self.manager.change_scene(MenuScene, self.manager, self.assets, self.sSize),
    center=(self.sSize[0]/2, self.sSize[1]*7/8))

```

I plan to route these to the character/stage selection menus appropriately later.

Whether the player is joining or hosting, I'm planning for the only differences to be the player numbers and the fact that the host will have to start a server on a separate thread on the host button press. So apart from these differences, the LAN game scene class should carry out the same tasks for both.

Network class

For the player to connect to the server, and start servers, I will use the manager attributes: network and server. Which were declared in the main.py file. I will now write the code for the class. This class should be simple and be used to send data to a server.

```

class network:
    def __init__(self):
        #init's the socket as TCP, to match the server
        self.client = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
        #local host , for now, will be changed in the future
        self.ip = "127.0.0.1"
        #random port
        self.port = 64340
        #socket address
        self.addr = (self.ip, self.port)

```

Here I have set the IP, port and socket address to the local host. AF_INET specifies the address family, which I will be using IPv4 as they are shorter, easier to type and more familiar to me. I will be using TCP which socket.SOCK_STREAM specifies, justified below.

Theoretically , if i were to change the ip to another address of a computer running this game on the same network, a lan game could work. A limitation would be the firewall as it would need to open the port on the host computer to allow incoming connections.

The actual socket used to connect to the server will be using the TCP protocol. This is a slower protocol than UDP which is used for most games, however my game will only have 2 players

communicating. Also, the data I am sending is relatively light weight and should not be slow enough that a client lags.

The connect, reset and send methods are explained below:

The connect function will be used to connect to the server, the reset function will simply shutdown and reinitialize the client attribute which holds the socket instance. The send function will just send data to the server.

```
def connect(self, ):
    try:
        if hasattr(self, "client"):
            self.reset()
            self.client.connect((self.ip, self.port))
        except Exception as e:
            print("Connection error:", e)
            return False

def reset(self):
    try:
        self.client.close()
    except Exception:
        pass
    self.client = socket.socket(socket.AF_INET, socket.SOCK_STREAM)

def send(self, data):
    #print(data)
    #try:
        self.client.send(json.dumps(data).encode())
        return json.loads(self.client.recv(1024).decode())
    #except socket.error as e:
        # print("Socket error: ", e)
```

I have put this code in a try block to gracefully handle errors. Also, if the client socket isn't already shutdown/disconnected then it should shut that connection down and start a new one. This is specifically implemented to handle weird edge cases where a host leaves a game and causes the joiner to leave. In that case, if the connection isn't fully terminated, then this function will do that before starting a new game.

The reset function shuts down the socket and removes any trace from the computer's memory, there may be an error from this as the client may already be not connected, which should just be ignored. Then, I reset the value of client.

The send function takes one data input and serializes it in json format, this is to convert any data structures into a format which can be encoded to transmit over a network. The process is reversed to receive the data.

Server class

I will make the server class before finishing off the LAN game class, as it will be more helpful to me knowing exactly how the server will respond when writing the LAN game logic to prevent any errors.

The constructor is shown below:

```
class server:
    def __init__(self):
        #ip.port needed for binding to socket
        self.ip = "127.0.0.1"
        self.port = 64340

        #specified the protocol used, I will use TCP for reliability
        self.socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
        #this allows socket to be reused within a short time frame
        #closing and opening the game should allow the player to play-
        #lan game again
        self.socket.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)

        #current connections used for passing in the
        self.current_connections = 0
        #not allow more than 2 players to join
        self.max_connections = 2

        #player 1 and 2 necessary data-
        #player1/host should always join first
        self.data = {"1": {"name": None, "stage": None, "position": (0,0), "animation_state": "idle", "frame": 0, "facing_left": False, "health": 100},
                    "2": {"name": None, "position": (0,0), "animation_state": "idle", "frame": 0, "facing_left": False, "health": 100}}
```

This line will set the value of socket.SO_REUSEADDR to 1 (true) which will allow binding to this socket, regardless if it was just closed a short period of time ago. If I want to test out the game multiple times without having to change the port, I need to allow reusing the socket.

The current connections will be used for passing the player number in, while max connections defines the max number of players to join. I have made this an attribute in case I want to increase the number of players in a mode, in a future update.

This dictionary will hold all the values that will be used in the games. The point of this is to not have the host computer calculating anything to do with the opponent. The values of the character like animation state, frame number, facing left and health should all be calculated and sent for drawing on the opponent's screen. An additional key is stored which is the stage, this will hold the name of the stage selected by the host

```
def start(self, character, stage):
    #this function is called for setting up the server
    #stage will be used in the joining menu
    #way to differentiate between diferent hosts
    self.data["1"]["name"] = character
    self.data["1"]["stage"] = stage

    #server will need to run on a seperate thread-
    #daemon means that this thread will end once main game is closed
    server = threading.Thread(target=self.accept_connections, args=(), daemon=True)
    server.start()
```

This start function will only run on the host's computer and used to set the name and stage selected by the host.

The accept connections function will be ran on a seperate thread to not freeze the game, as it will run a whilte loop to listen for incoming connections. It is shown below.

```
#this function will accept new connections from players
```

```
def accept_connections(self):
```

```
    try:
```

```
        self.socket.bind((self.ip, self.port))
```

```
    except socket.error as e:
```

```
        print(str(e))
```

```
    self.socket.listen()
```

```
    print("Waiting for a connection, Server started")
```

```
#will only accept a connection if the current_connections is
```

```
while True and self.current_connections < self.max_connection:
```

```
    conn, addr = self.socket.accept()
```

```
    #print console for confirmation of connection
```

```
    print(f"connected to{conn}:{addr}")
```

```
    self.current_connections += 1
```

```
    #current connection used to specify the player number
```

```
    client = threading.Thread(target=self.threaded_client, args=(conn, self.current_connections), daemon=True)
```

```
    client.start()
```

I have placed the socket binding in a try block, to catch any errors instead of them silently failing in this separate thread.

It will then listen for incoming connections and accept them in a while block to accept multiple repeatedly. For each new client accepted, it will run a separate thread which will be running the threaded_client function. This function will handle all requests for data on opponents.


```
def threaded_client(self, conn, n):
    try:
```

First receives the data to store in a variable and initializes the response variable with None.

The function is wrapped in a try block to catch any errors gracefully and print them out. Also, if an error occurs or the player disconnects, the try block will close the connection.

```
data = json.loads(conn.recv(1024).decode())
response = None
```

```
#the response will be the same for player 1/2 in the actual rounds
if data[0] == "game":
```

```
    #exchanges the data with the dictionary
    #player data stored in the dictionary
    self.data[f"{n}"]["position"] = data[1]
    self.data[f"{n}"]["animation_state"] = data[2]
    self.data[f"{n}"]["frame"] = data[3]
    self.data[f"{n}"]["facing_left"] = data[4]
    self.data[f"{n}"]["health"] = data[5]
```

```
    #op specifies the other player number
    op = 1 if n == 2 else 2
    print(self.data[f"{op}"])
```

```
    #makes a deepcopy to avoid race conditions
    otherPlayerData = deepcopy(self.data[f"{op}"])
    response = otherPlayerData
```

```
#if a joining player needs to init their game
elif data[0] == "initJ":
```

```
    #all they need to send is their name for init
    # the stage is already pre set and ready to send
```

```
    self.data["2"]["name"] = data[1]
```

```
    response = [self.data["1"]["name"], self.data["1"]["stage"]]
    #if , for whatever reason, the host hasn't joined at this point
    #Error host hasn't joined yet is returned
    #error handling
    if None in response:
        response = ["Error, host hasn't joined"]
```

```
elif data[0] == "initH":
```

```
    #the host doesn't need to send anything, just needs the joiner's character name
    response = self.data["2"]["name"]
```

```
    #if player 2 hasn't joined yet then that is returned
    if response == None:
        response = ["Player 2 hasn't joined yet"]
```

```
conn.send(json.dumps(response).encode())
```

```
except Exception as e:
    print(e)
```

The response is serialized and encoded for being transmitted to avoid errors.

This is the main function that will handle the client's data requests. For each client connected, a separate thread of this will be running to ensure all clients are responded to instantly.

All data received will be in list format. The data at index 0 will specify the type of request.

If the request is game data, it will store the current player's critical information and set the response to a copy of the other player's data.

If the request is an initialization request from a joiner's pov, it will send back the host character name and stage selected. If, for some reason, the joiner has joined a game without a host, it will simply return the host hasn't joined as a signal to kick the player.

Lastly, for initializing the game as a host, only the joiner character's name is needed to make an instance of the character.

Implementing the rest of the LAN game class

Now that I have finished writing the network and server class, I know exactly what information I will need to send and receive in order to have a working game.

To finish off the rest of the LAN game class, I will need to initialize the actual game using the initJ and initH requests for joining and hosting players.

```
#if join mode
#network should be connected
#INIT CHARACTER AND STAGE
if pNumber == 2:
    try:
        #initJ is init for joiners, receives character and stage info
        data = self.network.send(["initJ", character])
    except Exception as e:
        #If the network send crashes because there is no host
        print("Error, host hasn't joined")
        #Switch scenes, graceful error handling
        self.manager.change_scene(PlayMenuScene, self.manager, self.assets, self.sSize)

    #Stage should be set to the response
    #The opponent character is also created
    self.stage = StageFactory(data[1], assets, sSize)
    self.players[self.opNumber-1] = CharacterFactory(data[0], assets, self.manager.controls, self.stage.getSpawnPoint(self.opNumber), self.opNumber)

    #The player's own character is also created here
    self.players[pNumber-1] = CharacterFactory(character, assets, self.manager.controls, self.stage.getSpawnPoint(pNumber), pNumber)

    #The hud's timer is resetted
    self.hud.reset()

#hosting
elif pNumber == 1:
```

If the player is joining (therefore their number is 2), then they should send a signal to the server to get stage and opponent character info. If an error occurs with sending, that means the host hasn't joined yet, so the scene is switched back to the play menu scene.

Otherwise, if the request goes through successfully, the response is used to create a stage and players. The hud is reset as it was created earlier which starts the timer. The timer should only be started when both characters are in the game so it is restarted.

```
self.hud.reset()

#hosting
elif pNumber == 1:
    #Host doesn't need to wait before creating the stage and character
    self.stage = StageFactory(stage, assets, sSize)
    self.players[pNumber-1] = CharacterFactory(character, assets, self.manager.controls, self.stage.getSpawnPoint(pNumber), pNumber)

    #init host will only return the other player's character
    data = self.network.send(["initH"])

    if data != ["Player 2 hasn't joined yet"]:
        self.players[self.opNumber-1] = CharacterFactory(data, assets, self.manager.controls, self.stage.getSpawnPoint(self.opNumber), self.opNumber)

#wins, similar to the local game logic
self.wins = [0, 0]
```

If the player is hosting, then the stage and player is created instantly. Then, a request is sent for the joiner's details. If the player has joined, then a character instance is created using the name specified.

The wins are in a list, similar to the local game scene class. This allows for easy access and checking whether a player has won.

If the joining player has not joined yet, the host will be waiting in the game but should still be checking for a joiner. This is why i have made a method called check_p2:

```
def check_p2(self):
    #None can only be in the list if player 2 hasn't joined yet
    if self.players[1] == None:

        #init host will only return the other player's character
        data = self.network.send(["initH"])

        if data != ["Player 2 hasn't joined yet"]:
            self.players[self.opNumber-1] = CharacterFactory(data, self.assets, self.manager.controls, self.stage.getSpawnPoint(self.opNumber), self.opNumber)
            self.hud.reset()
```

If the player has not joined, then the list containing both players at index 1 will be None. This means that the function should check for a joiner. The rest of the function repeats what was written in the constructor. The hud is also reset here if there is a joining player as , at that point, both players have just joined.

Editing the character class

The general update function for this class will look like this:

```
def update(self, dt, mpos, mbut, mbut_j, keys, keys_j):
    self.check_p2()
    self.players[self.pNumber-1].update(self.stage.getHitboxes(), dt, mpos, mbut, mbut_j, keys, keys_j)
    self.set_data(dt)
```

As you can see, player 2 is checked first, then the home player is updated. The function set_data should send the “home” player’s data and receive the opponent’s data. When receiving the opponent’s data, the opponent player instance will have to be updated including updating status effects from a particular frame. Doing this will be much easier with a specialised function for setting the character’s attributes from lan data.

For this reason, I have created a set_lan_data method in the character class. Before explaining that function I will show its usage.

```
def set_data(self, dt):
    other = self.network.send(["game", self.players[self.pNumber-1].rect.midbottom,
                               self.players[self.pNumber-1].animation_state,
                               self.players[self.pNumber-1].current_animation.getFrameNumber(),
                               self.players[self.pNumber-1].facing_left,
                               self.players[self.pNumber-1].health])

    if other["name"] == None:
        print("Player 2 hasn't joined")
    else:
        self.players[self.opNumber - 1].set_lan_data(dt, other["position"], other["animation_state"], other["frame"], other["facing_left"], other["health"])
```

Firstly, a game request is sent along with the player’s data to receive the opponent’s player data.

If the other player is equal to None, that means that this is the host sending the game request and player 2 hasn’t joined. The joining player should have been kicked in the constructor. Otherwise, the opponent data will be used and passed into the set_lan_data function to set its attributes.

So, the `set_lan_data` function should just take the animation state, frame number, facing left and health to set all the attributes of the player.

Luckily, splitting the character update function into updating statuses proved to be a good idea as this `set_lan_data` function is now much easier to write.

```
def set_lan_data(self, dt, position, animation_state, frame, facing_left, health):
    self.rect.midbottom = position
    self.animation_state = animation_state

    self.current_animation = self.animations[self.animation_state]
    self.current_animation.SetFrame(frame)
    self.facing_left = facing_left
    self.image = self.current_animation.getFrame(facing_left)

    self.health = health
    self.updateStatus(dt)
```

It simply sets the animations, positions and images from the frame number. Also the health is set. Then the update status function is called to set the attacking frames which will be needed later to detect whether the opponent is attacking.

gamFinally, I can just write the draw function and test out a LAN game:

```
def draw(self, screen):
    self.stage.draw(screen)

    self.players[self.pNumber-1].draw(screen)
    if self.players[self.opNumber-1] != None:
        self.players[self.opNumber - 1].draw(screen)

    if not(None in self.players):
        self.hud.draw(screen, self.players[0], self.players[1])
```

An attempt to draw the other player only if they are in the game.

The HUD is also drawn if both players are in the game.

Testing the LAN game

I haven't implemented attacks yet, I want to make sure that everything works before adding in another feature to test. As I don't have access to another pc, I will be testing it by running two different instances of the game on my computer. These instances will be connecting to the local ip address.

What I'm testing	Input data	Why am I testing this?	Expected outcome	Test results
Can a host join a game and wait with no errors?	Mouse click on host button	The code should be able to work with only one player connecting to the server so a waiting screen can be implemented in a similar way.	Once host is clicked, one player should be drawn onto a stage, without errors until the second player joins.	Clicking host produced an error.
Does clicking the joining button while a host is active join the game and draw the opponents?	Mouse click on host	Clicking the joining button should allow you the play with another person on a LAN. It's a key feature of the game.	Both players should be drawn the same on both screens, in the same positions.	Clicking host produced an error.
Does moving the host move the host on player 2's screen?	"a", "d", "w"	Player movements should be synced across different clients.	Moving either player, should move the other on the opponent's screen.	Clicking host produced an error.
Does moving player 2 move the player 2 on the host's screen?	"a", "d", "w"	Player movements should be synced across different clients.	Moving either player, should move the other on the opponent's screen.	Clicking host produced an error.
Is the HUD synced across both players, showing the correct time?	One client clicks host while the other clicks join.	The HUD should be synced to show correct time info and not have any bugs with when a game ends.	The game should start the timer when player 2 joins and should be the same on both clients.	Clicking host produced an error.

JSON decode error

Clicking host on one of the computer produced this error:

```
Waiting for a connection, Server started
connected to<socket.socket fd=39, family=2, type=1, proto=0, laddr=('127.0.0.1', 64340), raddr=('1
27.0.0.1', 43696)>:('127.0.0.1', 43696)
Traceback (most recent call last):
  File "/home/neo/projects/mythbound/main.py", line 56, in <module>
    SM.update(PROGRAM_DATA["dt"], mpos, mbut,mbut_j, keys, keys_j)
    ~~~~~~ ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/neo/projects/mythbound/scene.py", line 45, in update
    self.current_scene.update(dt, mpos, mbut,mbut_j, keys, keys_j)
    ~~~~~~ ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/neo/projects/mythbound/scene.py", line 853, in update
    self.check_p2()
    ~~~~~~ ^^^^^
  File "/home/neo/projects/mythbound/scene.py", line 833, in check_p2
    data = self.network.send(["inith"])
  File "/home/neo/projects/mythbound/networking.py", line 152, in send
    return json.loads(self.client.recv(1024).decode())
    ~~~~~~ ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/usr/lib/python3.13/json/__init__.py", line 352, in loads
    return _default_decoder.decode(s)
    ~~~~~~ ^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/usr/lib/python3.13/json/decoder.py", line 345, in decode
    obj, end = self.raw_decode(s, idx=_w(s, 0).end())
    ~~~~~~ ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/usr/lib/python3.13/json/decoder.py", line 363, in raw_decode
    raise JSONDecodeError("Expecting value", s, err.value) from None
json.decoder.JSONDecodeError: Expecting value: line 1 column 1 (char 0)
```

This error took me a long time to figure out but eventually I noticed a missing while loop.

From this, I can see that it has reached the function check_p2, meaning the contructor was successfully completed which did involve connecting to the server so that worked. However, it says that it was expecting a value to decode meaning the server didn't send back anything on the second p2 check, so the connection was closed after the initial connection.

This was because the threaded_client function which was meant to handle the connection ended because there was no while loop. The while loop inside the try block is meant to run as long as there is no disconnections.

I added the while loop:

```

while True:
    data = json.loads(conn.recv(1024).decode())
    response = None

    #the response will be the same for player 1/2 in the actual rounds
    if data[0] == "game":
        #exchanges the data with the dictionary
        #player data stored in the dictionary
        self.data[f"{n}"]["position"] = data[1]
        self.data[f"{n}"]["animation_state"] = data[2]
        self.data[f"{n}"]["frame"] = data[3]
        self.data[f"{n}"]["facing_left"] = data[4]
        self.data[f"{n}"]["health"] = data[5]

        #op specifies the other player number
        op = 1 if n == 2 else 2
        print(self.data[f"{op}"])

        #makes a deepcopy to avoid race conditions
        otherPlayerData = deepcopy(self.data[f"{op}"])
        response = otherPlayerData

    #if a joining player needs to init their game
    elif data[0] == "initJ":
        #all they need to send is their name for init
        # the stage is already pre set and ready to send

        self.data["2"]["name"] = data[1]

        response = [self.data["1"]["name"], self.data["1"]["stage"]]
        #if ,for whatever reason, the host hasn't joined at this point
        #Error host hasn't joined yet is returned
        #error handling
        if None in response:
            response = ["Error, host hasn't joined"]

    elif data[0] == "initH":
        #the host doesn't need to send anything, just needs the joiner's character name
        response = self.data["2"]["name"]

        #if player 2 hasn't joined yet then that is returned-
        if response == None:
            response = ["Player 2 hasn't joined yet"]

    conn.send(json.dumps(response).encode())

```

After fixing that issue, I ran the tests again:

What I'm testing	Input data	Why am I testing this?	Expected outcome	Test results
Can a host join a game and wait with no errors?	Mouse click on host button	The code should be able to work with only one player connecting to the server so a waiting screen can be implemented in a similar way.	Once host is clicked, one player should be drawn onto a stage, without errors until the second player joins.	As, expected.
Does clicking the joining button while a host is active join the game and draw the oponents?	Mouse click on host	Clicking the joining button should allow you the play with another person on a LAN. It's a key feature of the game.	Both players should be drawn the same on both screens, in the same positions.	As, expected.
Does moving the host move the host on player 2's screen?	"a", "d", "w"	Player movements should be synced across different clients.	Moving either player, should move the other on the opponent's screen.	As, expected.
Does moving player 2 move the player 2 on the host's screen?	"a", "d", "w"	Player movements should be synced across different clients.	Moving either player, should move the other on the opponent's screen.	As, expected.
Is the HUD synced across both players, showing the correct time?	One client clicks host while the other clicks join.	The HUD should be synced to show correct time info and not have any bugs with when a game ends.	The game should start the timer when player 2 joins and should be the same on both clients.	As, expected.

[A video of the results is hyperlinked:](#)

Adding attacks and game logic

I will use the wins attribute specified in the constructor in the same way that I used it in the local games mode, I can just copy the code for the checkround and deathfall function like so:


```

def checkRound(self):
    # resets both players if one player is defeated
    # if player 1 is defeated
    if self.players[0].health <= 0:
        # player 2 wins incremented
        self.wins[1] += 1
        self.players[0].reset()
        self.players[1].reset()
        self.hud.reset()
    # if player 2 is defeated
    elif self.players[1].health <= 0:
        # player 1 wins are incremented
        self.wins[0] += 1
        self.players[0].reset()
        self.players[1].reset()
        self.hud.reset()
    elif self.hud.update():
        # player with the lowest health loses
        if self.players[0].health > self.players[1].health:
            self.wins[0] += 1
            self.players[0].reset()
            self.players[1].reset()
        elif self.players[1].health > self.players[0].health:
            self.wins[1] += 1
            self.players[0].reset()
            self.players[1].reset()
        # if healths are equal then no wins to either
        # round is restarted
        else:
            self.players[0].reset()
            self.players[1].reset()
            self.hud.reset()

    # best of 3 system
    # if one player wins twice, they win
    if 2 in self.wins:
        # ending banner should be switched to here
        print(f"Player {self.wins.index(2) + 1} wins!")
        # if a player wins, then the scene is switched back to the joining menu
        self.manager.change_scene(PlayMenuScene, self.manager, self.assets, self.sSize)

```

I have just changed the way I access the players by using the list.

```

def DeathFall(self):
    # Checks whether player is out of screen
    if self.players[0].rect.bottom > self.sSize[1]:
        self.players[0].health = 0
    if self.players[1].rect.bottom > self.sSize[1]:
        self.players[1].health = 0

```

Then, I can call these functions in the update function only when both players have joined like so:

```
def update(self, dt, mpos, mbut, mbut_j, keys, keys_j):
    self.check_p2()
    self.players[self.pNumber-1].update(self.stage.getHitboxes(), dt, mpos, mbut, mbut_j, keys, keys_j)
    self.set_data(dt)

    if not(None in self.players):
        self.checkRound()
        self.DeathFall()
```

Testing game logic

What I'm testing	Input data	Why am I testing this?	Expected outcome	Test results
Does falling twice on a client give the win to the other client?	"a" or "d" until fall after clicking on host and join on clients.	This will simulate a player being able to win and test out the game logic functions to see if they run as expected on LAN and sync up well.	Console should output the other player winning.	As, expected.

A screenshot of the results is shown below:

```
3, 'facing_left': True, 'health': 100}
{'name': 'evil_wizard', 'stage': 'arctic_fracture', 'position': [170, 718], 'an
imation_state': 'idle', 'frame': 7, 'facing_left': True, 'health': 100}
{'name': 'knight', 'position': [801, 381], 'animation_state': 'idle', 'frame':
3, 'facing_left': True, 'health': 100}
{'name': 'evil_wizard', 'stage': 'arctic_fracture', 'position': [170, 730], 'an
imation_state': 'idle', 'frame': 7, 'facing_left': True, 'health': 100}
{'name': 'knight', 'position': [801, 381], 'animation_state': 'idle', 'frame':
3, 'facing_left': True, 'health': 100}
Player 2 wins!
```

Adding attacks

Adding attacks will be simple as the game logic code taken from the local mode scene already factors health in when calculating which player wins. Since statuses are also already updated from the frame number in the set lan data function, it is as if the other player is playing locally. Each player will calculate their own health based on their opponent's movements and status effects which are set each frame by the set lan data function.

I only need to call the take damage function like so:

```
def update(self, dt, mpos, mbut, mbut_j, keys, keys_j):
    self.check_p2()
    self.players[self.pNumber-1].update(self.stage.getHitboxes(), dt, mpos, mbut, mbut_j, keys, keys_j)
    self.set_data(dt)

    #if both players in the game
    if not(None in self.players):
        #check if other player has attacked successfully
        self.players[self.pNumber-1].take_damage(self.players[self.opNumber-1])
        self.DeathFall()
        #check who wins
        self.checkRound()
```

The other player is passed in to check if the player is attacking and has collided.

The take damage function has been explained previously.

Testing attacks

What I'm testing	Input data	Why am I testing this?	Expected outcome	Test results
Does players attacking each other update the HUD health.	"P" and "L"	Players' attacks need to be registered accurately for a well developed 2d fighting game.	The HUD health should be lowered.	Attacks did not register.
Does a round end if the health goes to 0?	"P" or "L"	A round needs to end at the when a player's health goes to 0 so that the game is fair.	Once a player's health goes to 0 from attacks, a new round should be started.	Attacks did not register.

The video of the results is titled "lan_attacks_bug"

While debugging this error, I used the output from the host's console:

```
{'name': 'evil_wizard', 'stage': 'arctic_fracture', 'position': [478, 381], 'animation_state': 'idle', 'frame': 7, 'facing_left': False, 'health': 100}
{'name': 'knight', 'position': [529, 381], 'animation_state': 'idle', 'frame': 1, 'facing_left': True, 'health': 100}
```

Despite attacking, the health remained at 100 for both of the players. Since the health being 0 from the death fall function triggered the end of the round as shown previously, that means the actual attacks arent being registered.

To confirm this suspiscion, I will place a print statement in the take damage function:

```

def take_damage(self, player):
    #if not invincible and the opponent is attacking
    if player.attacking and not self.invincible:
        if player.attack_use:
            #mask2 is the opponent
            mask2 = pygame.mask.from_surface(player.image)
            me = pygame.mask.from_surface(self.image)
            #need to center the masks perfectly on top of the images
            #the images are centered on the rects, the masks are the same size
            #center them on rects too
            meIR = self.image.get_frect(center = self.rect.center)--
            otherIR = player.image.get_frect(center = player.rect.center)
            #offset is distance from me to the player
            # AB is B-A
            offset = (otherIR.x-meIR.x, otherIR.y-meIR.y)
            #if the opponent collides-
            if me_overlap(mask2, offset):
                print("ATTACK DETECTED")
                #value needs to be edited for heavy and light attacks
                if player.animation_state == "attack1":
                    #light attacks take 8
                    self.health -= 8
                elif player.animation_state == "attack2":
                    #heavy attacks take 15
                    self.health -= 15
                #reset attack use
                player.attack_use = 0

```

After running the game and testing the attacks, the print did not show on the console. I think that it is an issue with the `attack_use` always being 0 since the opponent does not get to click the attack button on the computer, instead all its status effects are transferred over. I can test this by moving the print statement outside the attack use if statement, like this:


```

def take_damage(self, player):
    #if not invincible and the opponent is attacking
    if player.attacking and not self.invincible:
        print("ATTACK DETECTED")
        if player.attack_use:
            #mask2 is the opponent
            mask2 = pygame.mask.from_surface(player.image)
            me = pygame.mask.from_surface(self.image)
            #need to center the masks perfectly on top of the images
            #the images are centered on the rects, the masks are the same size
            #center them on rects too
            meIR = self.image.get_frect(center = self.rect.center)--
            otherIR = player.image.get_frect(center = player.rect.center)
            #offset is distance from me to the player
            # AB is B-A
            offset = (otherIR.x-meIR.x, otherIR.y-meIR.y)
            #if the opponent collides-
            if me.overlap(mask2, offset):
                #value needs to be edited for heavy and light attacks
                if player.animation_state == "attack1":
                    #light attacks take 8
                    self.health -= 8
                elif player.animation_state == "attack2":
                    #heavy attacks take 15
                    self.health -= 15
                #reset attack use
                player.attack_use = 0

```

The console now prints:

```

ATTACK DETECTED
ATTACK DETECTED
ATTACK DETECTED
ATTACK DETECTED
ATTACK DETECTED
ATTACK DETECTED
ATTACK DETECTED
ATTACK DETECTED
ATTACK DETECTED
ATTACK DETECTED
ATTACK DETECTED
ATTACK DETECTED
ATTACK DETECTED
ATTACK DETECTED
ATTACK DETECTED
ATTACK DETECTED
ATTACK DETECTED
ATTACK DETECTED
ATTACK DETECTED
Traceback (most recent call last):

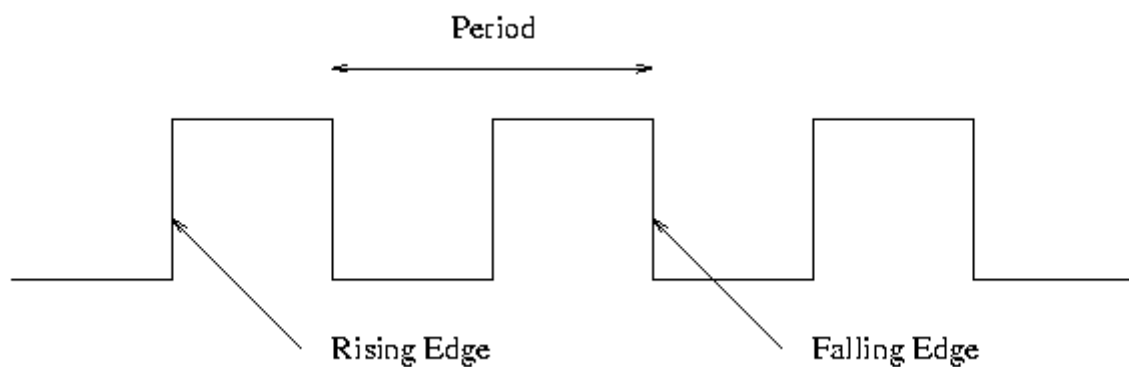
```

To fix this error, I need a way of setting the attack use to 1 on the computer as soon as the opponent attacks. I can easily do this by just adding another variable to send over which would be attack use.

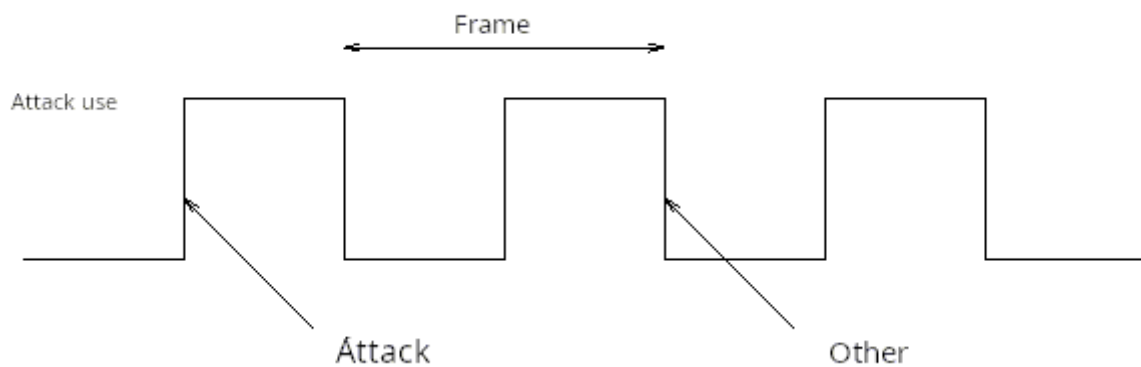
A problem with this, is that the attack use on the opponent side needs to be reset to 0 and kept at 0 if their attack lands successfully. Also, this problem should be fixed with just the animation state. Adding this data to the regular transmissions will overcomplicate things as I will need to not only send my attack use and also receive and update and my opponents attack use for them to update.

Rising attack edges

Instead, I will think about rising edges in clock pulses:



I need to modify this more to fit my 2D fighting game, this looks better:



Although this model assumes an attack will happen every frame which is not accurate, the fix to this error reminded me of it.

```
def set_lan_data(self, dt, position, animation_state, frame, facing_left, health):
    #sets all data as if a player is playing on local pc
    self.rect.midbottom = position
    self.previous_state = self.animation_state
    self.animation_state = animation_state

    #if player has JUST attacked
    if ("attack" not in self.previous_state) and ("attack" in self.animation_state):
        self.attack_use = 1

    self.current_animation = self.animations[self.animation_state]
    self.current_animation.SetFrame(frame)
```

Attack use will only be set to 1 if the player has just attacked. The opponent take damage function will reset this 0 if they collide so no further if statements are needed to set this to 0.

What I'm testing	Input data	Why am I testing this?	Expected outcome	Test results
Does players attacking each other update the HUD health.	"P" and "L"	Players' attacks need to be registered accurately for a well developed 2d fighting game.	The HUD health should be lowered.	As, expected.
Does a round end if the health goes to 0?	"P" or "L"	A round needs to end at the when a player's health goes to 0 so that the game is fair.	Once a player's health goes to 0 from attacks, a new round should be started.	The first round ends and resets the health, not the positions of the players. When the second round ends, the player who lost returns to the main menu, the player that won stays in the game.

A video of the results is titled "lan_rounds_bug"

This syncing issue can be fixed server side by moving the check round code to run on the server. This means that the server will check who wins and inform both clients, this is better as it removes any possibility of syncing issues by having two separate computers calculating the same value at possibly different speeds.

To do this, I will need to copy the check rounds function over to the server class and edit it to fit the dictionary containing all the details about the two players.

```
def checkRound(self):
    round_reset = False
    # resets both players if one player is defeated
    # if player 1 is defeated
    if self.data["1"]["health"] <= 0:
        # player 2 wins incremented
        self.wins[1] += 1
        round_reset = True
    # if player 2 is defeated
    elif self.data["2"]["health"] <= 0:
        # player 1 wins are incremented
        self.wins[0] += 1
        round_reset = True
    # Check for time out-
    elif time.time() - self.start_time >= self.time:
        round_reset = True
        # player with the lowest health loses
        if self.data["1"]["health"] > self.data["2"]["health"]:
            self.wins[0] += 1
        elif self.data["2"]["health"] > self.data["1"]["health"]:
            self.wins[1] += 1
        # if healths are equal then no wins to either
        # round is restarted

    # best of 3 system
    # if one player wins twice, they win
    if 2 in self.wins:
        # ending banner should be switched to here
        return f"victory_{self.wins.index(2) + 1}"
        # if a player wins, then the scene is switched back to the

    #
    if round_reset:
        self.start_time = time.time()
        return "new_round"
    else:
        return "active"
```

This is the same code as before, I have just changed it to use the server's collection of data on both users which has the health inside.

It will call this function each frame for both users to ensure that as soon as a round ends or there is a win, it will return an appropriate message so that the players can be informed as soon as possible.

I will also need to update the users each frame based on the function that has been written like so:

```
#the response will be the same for player 1/2 in the actual rounds
if data[0] == "game":
    #exchanges the data with the dictionary
    #player data stored in the dictionary
    self.data[f"{n}"]["position"] = data[1]
    self.data[f"{n}"]["animation_state"] = data[2]
    self.data[f"{n}"]["frame"] = data[3]
    self.data[f"{n}"]["facing_left"] = data[4]
    self.data[f"{n}"]["health"] = data[5]

    #op specifies the other player number
    op = 1 if n == 2 else 2
    print(self.data[f"{op}"])

    #makes a deepcopy to avoid race conditions
    otherPlayerData = deepcopy(self.data[f"{op}"])
    #sends the status of the game back to the players
    otherPlayerData["status"] = self.checkRound()
    otherPlayerData["wins"] = self.wins
    response = otherPlayerData
```

This is in the threaded client section that will send a response back to the user based on their request.

It will send the results of check round so that the clients can act on it appropriately.

Then I will edit the set data function to end the game or reset players depending on a status value sent back by the server.

```
def set_data(self, dt):
    other = self.network.send(["game", self.players[self.pNumber-1].rect.midbottom,
                               self.players[self.pNumber-1].animation_state,
                               self.players[self.pNumber-1].current_animation.getFrameNumber(),
                               self.players[self.pNumber-1].facing_left,
                               self.players[self.pNumber-1].health])

    if other["name"] == None:
        print("Player 2 hasn't joined")
    else:
        self.players[self.opNumber - 1].set_lan_data(dt, other["position"], other["animation_state"])
        self.wins = other["wins"]

        if other["status"] == "active":
            pass
        elif other["status"] == "new_round":
            self.players[self.pNumber-1].reset()
        elif "victory_" in other["status"]:
            pWin = "Player " + other["status"][-1] + " wins!"
            print(pWin)
            self.manager.change_scene(PlayMenuScene, self.manager, self.assets, self.sSize)
```

This new code here perform an action based on the status of the game that is checked by the server and sent back.

It will do nothing if the game is still active. It will reset its own player if a new round is started. This will automatically be done for both players as they both receive the signal and reset their own. If there is a victory, it will output who win based on the last character which should be the player number. Then it will switch scenes back to the play menu.

Testing game logic

What I'm testing	Input data	Why am I testing this?	Expected outcome	Test results
Does a round end if the health goes to 0?	"P" or "L"	A round needs to end at the when a player's health goes to 0 so that the game is fair.	Once a player's health goes to 0 from attacks, a new round should be started.	The loser crashes with an error in the console.

```

Traceback (most recent call last):
  File "/home/neo/projects/mythbound/main.py"
, line 57, in <module>
    SM.draw(screen)
    ~~~~~^~~~~~
  File "/home/neo/projects/mythbound/scene.py"
, line 49, in draw
    self.current_scene.draw(screen)
    ~~~~~^~~~~~
  File "/home/neo/projects/mythbound/scene.py"
, line 841, in draw
    self.hud.draw(screen, self.players[0], se
lf.players[1])
    ~~~~~^~~~~~
  File "/home/neo/projects/mythbound/HUD.py",
line 62, in draw
    bar1 = pygame.Surface((size1, self.health
Bar1.get_height()))
pygame.error: Invalid resolution for Surface

```

This error shows that it can't draw the hud health bar when it is 0 as a surface can't have a dimension being 0. To fix this, I will add a check to see if it is 0, and if it is, I will just replace it with a 1.

```

def draw(self, screen, player1, player2):
    #Calculates ratio of health left
    health1Ratio = player1.health/player1.maxHealth
    health2Ratio = player2.health/player2.maxHealth

    #new sizes for the health
    size1 = max([health1Ratio*self.healthBar1.get_width(), 1])
    size2 = max([health2Ratio*self.healthBar2.get_width(), 1])

    #the health
    bar1 = pygame.Surface((size1, self.healthBar1.get_height()))
    bar1.fill((1, 0, 0))
    bar2 = pygame.Surface((size2, self.healthBar2.get_height()))
    bar2.fill((20, 80, 180))

    screen.blit(self.healthBar1, self.healthBar1R)

```

Instead of a check, I used a max function as it looks neater. It will simply choose the biggest value out of the size and 1, so it will never be anything less than 1, therefore won't be 0.

What I'm testing	Input data	Why am I testing this?	Expected outcome	Test results
Does a round end if the health goes to 0?	"P" or "L"	A round needs to end at the when a player's health goes to 0 so that the game is fair.	Once a player's health goes to 0 from attacks, a new round should be started.	The game ends immediately, after one round.

The results of this test are in a video titled 'lan_game_round_end_bug'

After reviewing the check round function, the cause of this bug is the check round function running twice on seperate threads and incrementing the wins counter twice, therefore ending the game. I think the best way to fix this issue is, to check for a value of 4 in the wins list, rather than 2.

A more elegant solution would be to just run the check rounds on a seperate thread and have it update variables, but that would take too long to implement and I'm beginning to run out of time.


```

def checkRound(self):
    round_reset = False
    # resets both players if one player is defeated
    # if player 1 is defeated
    if self.data["1"]["health"] <= 0:
        # player 2 wins incremented
        self.wins[1] += 1
        round_reset = True
    # if player 2 is defeated
    elif self.data["2"]["health"] <= 0:
        # player 1 wins are incremented
        self.wins[0] += 1
        round_reset = True
    # Check for time out-
    elif time.time() - self.start_time >= self.time:
        round_reset = True
        # player with the lowest health loses
        if self.data["1"]["health"] > self.data["2"]["health"]:
            self.wins[0] += 1
        elif self.data["2"]["health"] > self.data["1"]["health"]:
            self.wins[1] += 1
        # if healths are equal then no wins to either
        # round is restarted

    # best of 3 system
    # if one player wins twice, they win
    if 4 in self.wins:
        # ending banner should be switched to here
        return f"victory_{self.wins.index(4) + 1}"
        # if a player wins, then the scene is switched back to the
    #
    if round_reset:
        self.start_time = time.time()
        return "new_round"
    else:
        return "active"

```

What I'm testing	Input data	Why am I testing this?	Expected outcome	Test results
------------------	------------	------------------------	------------------	--------------

Does a round end if the health goes to 0?	"P" or "L"	A round needs to end at the when a player's health goes to 0 so that the game is fair.	Once a player's health goes to 0 from attacks, a new round should be started.	As, expected.
---	------------	--	---	---------------

A video of the results is titled 'lan_game_fix'

More menus!

To finish off the lan game mode, I will need to implement stage selection, character selection, waiting, joining and ending banner scenes.

Stage selection

I will start off with the stage, waiting and ending banner scenes. This is because they are quite easy to do, in comparison to the character selection and joining scene.

The stage selection scene is already finished, as I just need to route the host button to it. This scene doesn't need any network compatibility as the sole decision is left to the host.

```
def switchScene(self, ID):
    match self.mode:
        case "practise":
            #should be changed to waiting scene
            self.manager.change_scene(WaitingScene, self.manager, self.assets, self.sSize, ID)
        case "host":
            #should be changed to character selection once made
            self.manager.change_scene(WaitingScene, self.manager, self.assets, self.sSize, ID)
```

This is in the stage select scene, if the mode is host, then it should switch to the waiting scene, the ID will also be passed in to let the server know what stage the host has selected.

Now I will route the host button in the play menu to point to the stage selection menu:

```
class PlayMenuScene(scene):
    def __init__(self, manager, assets, sSize):
        super().__init__(manager, assets, sSize)

        self.background = self.assets.getBackground("start_menu")
        self.backgroundRect = self.background.get_frect(topleft = (0,0))

        practise, practiseG = self.assets.getButton("practise")
        local, localG = self.assets.getButton("local")
        join, joinG = self.assets.getButton("join")
        host, hostG = self.assets.getButton("host")
        back, backG = self.assets.getButton("back")
        self.practiseButton = Button(practise, practiseG, lambda:
            self.manager.change_scene(StageSelectScene, self.manager, self.assets, self.sSize, "practise",
            center=(self.sSize[0]/2, self.sSize[1]*2.5/8))
        self.localButton = Button(local, localG, lambda:
            self.manager.change_scene(StageSelectScene, self.manager, self.assets, self.sSize, "local",
            center=(self.sSize[0]/2, self.sSize[1]*3.5/8))
        self.joinButton = Button(join, joinG, lambda:
            self.manager.change_scene(LANgame, self.manager, self.assets, self.sSize, "knight", None, 2),
            center=(self.sSize[0]/2, self.sSize[1]*4.5/8))
        self.hostButton = Button(host, hostG, lambda:
            self.manager.change_scene(StageSelectScene, self.manager, self.assets, self.sSize, "host"),
            center=(self.sSize[0]/2, self.sSize[1]*5.5/8))
```

Waiting scene

Now, to not produce any more errors, I will create the WaitingScene class which will play when a host is waiting for a player to join the game.

The background will be based on the stage selected, this will show the host a preview of the stage selected.

```
class WaitingScene(scene):
    def __init__(self, manager, assets, sSize, stageID):
        super().__init__(manager, assets, sSize)
        #sets the font for the waiting text
        self.font = self.assets.loadFont("Cinzel-Regular.otf", 67)
        #Sets background
        self.background = self.assets.getBackground(stageID)
        self.backgroundRect= self.background.get_frect(topleft = (0,0))

        #Transparent darker overlay, reduces the background noise from colours-
        self.overlay= pygame.Surface(self.sSize, pygame.SRCALPHA)
        self.overlay.fill((0,0,0,150))
        self.overlayR = self.overlay.get_frect(topleft=(0,0))

        #the text at the bottom left
        self.text = self.font.render("Waiting", True, (255,255,255))
        self.textR = self.text.get_frect(bottomleft = self.backgroundRect.bottomleft)
        #number of dots after waiting text, every second
        self.nDots = 0
```

This code is taken from the settings menu to have less of a noisy background while waiting for a player to join.

This code will form the text "Waiting..." at the bottom left of the screen. As part of the usability features, the number of dots will also alternate. I will use the nDots attribute to store the number of dots that should be drawn at any given time.

The update function will continuously check if a player has joined the game, so that it can switch scenes to the character selection scenes For it to do this, it will need to have connected to the server already.

I will copy the code for connecting to the server from the lan game scene, to the waiting scene. Also, I will call the function in the waiting scene constructor method, to connect to the server and wait for a player to join.

```
self.stageID = stageID
self.init_connection()

def init_connection(self):
    self.network = self.manager.network
    self.server = self.manager.server

    #only the stage has been picked
    self.server.start(self.stageID)
    self.network.connect()
```


Since only the stage has been selected at this point, I will need to edit the server start function to not set a character. As the character will be set in the character selection menu, once both players have joined. To make the stage easily accessible, I will create a separate attribute called stage.

```
def start(self, stage):
    #this function is called for setting up
    #stage will be used in the joining menu
    #way to differentiate between different
    self.stage = stage

    #server will need to run on a separate thread
    #daemon means that this thread will end
    server = threading.Thread(target=self.accept_connections, daemon=True)
    server.start()
```

Now, I will use the connection that I have established in the constructor to send signals to the server while waiting, in the update method

If there is any other response, that would mean there is a problem with the server, and the user should return to the play menu scene.

```
def update(self, dt, mpos, mbut, mbut_j, keys, keys_j):
    #check on whether a player has joined
    status = self.network.send(["initH"])
    if status[0] == "ready":
        print("CHARACTER SELECTION MENU")
    elif status[0] == "waiting":
        print("waiting")
    #only one of two responses should be given
    else:
        self.manager.change_scene(PlayMenuScene, self.manager, self.assets, self.sSize)

    #possible outputs are 0,1,2,3
    #time is in seconds, so it will switch through the numbers
    #every second, max of 3 dots
    self.nDots = int(time.time())%4
    #rerender the text with nDots amount of nDots
    self.text = self.font.render("Waiting" + (self.nDots*"."), True, (255,255,255))
    self.textR = self.text.get_frect(bottomleft = self.backgroundRect.bottomleft)
```

A signal of "initH" will be sent to see if a player has joined. If the server returns "ready", that means a player has joined, so the scene should switch to the character selection menu, otherwise it should wait.

In the update function, which runs every frame, the amount of dots will be changed and the text will be rendered again. The modulus function is used with time to alternate the number of dots.

I have also made a basic draw function to draw all the elements onto the screen:

```
def draw(self,screen):
    #draws all the elements
    screen.blit(self.background,self.backgroundRect)
    screen.blit(self.overlay, self.overlayR)
    screen.blit(self.text, self.textR)
```

To ensure that the update function prints character selection menu when the server is ready, and not just instantly switch to the play menu. I will need to edit the server response to init h.

Previously it was used to send the data of the other player, if available, however now, with the waiting scene in mind, that is not needed. The data of the other player such as the character selected will be chosen in the character selection menu which will play after the waiting scene.

The current connections attribute proves its use here:

```
elif data[0] == "initH":
    #if player 2 has joined then the server is ready
    if self.current_connections == 2:
        response = ["ready"]
    else:
        response = ["waiting"]

    conn.send(json.dumps(response).encode())
```

Even though the threaded client function will run on a separate thread for each player joined, it still has access to a shared attributes which is the current connections.

The code I wrote previously, which increments current connections every time a player joins, allows this to work:

```
#this function will accept new connections from players
def accept_connections(self):
    try:
        self.socket.bind((self.ip, self.port))
    except socket.error as e:
        print(str(e))

    self.socket.listen(self.max_connections)
    print("Waiting for a connection, Server started")

    #will only accept a connection if the current_connections is under max_connections
    while True and self.current_connections < self.max_connections:
        conn, addr = self.socket.accept()
        #print console for confirmation of connection
        print(f"connected to{conn}:{addr}")
        self.current_connections += 1
        #current connection used to specify the player number
        client = threading.Thread(target=self.threaded_client, args=(conn, self.current_connections), daemon=True)
        client.start()

def threaded_client(self, conn, n):
```

Testing the waiting scene

I will now test the waiting scene, to see if it will print character selection menu, once a second player joins.

What I'm testing	Input data	Why am I testing this?	Expected outcome	Test results
Does clicking the host button take the user to the waiting scene?	Mouse click on host button	The waiting scene needs to work properly so that hosts can wait for a player to join them before starting the game.	Console should output the other player winning.	As, expected.
Once a player joins, does the waiting scene print character selection menu?	Mouse click on join button from another instance of the game	To allow the players to play together over a LAN connection, the waiting scene will need to switch scenes to show the game. I will write the code to switch scenes, once I have confirmed that the if statement works correctly.	Console should output "CHARACTER SELECTION MENU".	As, expected.

Below is a screenshot of two consoles, the left is the joining player's and the right is the host.

```
pygame-ce 2.5.6 (SDL 2.32.10, Python 3.13.11)
Traceback (most recent call last):
  File "/home/neo/projects/mythbound/main.py", line 56, in <module>
    SN.update(PROGRAM_DATA["dt"], mpos, mbut,mbut_j, keys, keys_j)
    ~~~~~^~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
  File "/home/neo/projects/mythbound/scenes.py", line 46, in update
    self.current_scene.update(dt, mpos, mbut,mbut_j, keys, keys_j)
    ~~~~~^~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
  File "/home/neo/projects/mythbound/scenes.py", line 282, in update
    self.joinButton.update(mpos, mbut_j)
    ~~~~~^~~~~~
  File "/home/neo/projects/mythbound/scenes.py", line 122, in update
    self.function()
    ~~~~~^~~~~~AA
  File "/home/neo/projects/mythbound/scenes.py", line 198, in lambda*:
    self.manager.change_scene(LNGame, self.assets, self.sSize, "knight", None, 2).
    ~~~~~^~~~~~
  File "/home/neo/projects/mythbound/scenes.py", line 38, in change_scene
    self.current_scene = new_scene(*args)
    ~~~~~^~~~~~AAAAA
  File "/home/neo/projects/mythbound/scenes.py", line 752, in __init__
    self.stage = StageFactory(data[1], assets, sSize)
    ~~~~~^~~~~~AAA
IndexError: list index out of range
(neo@matrix mythbound)$
```

I can see from the output that the host is in the waiting scene, until player joins then character selection menu is printed. There is an error in the joining player's console, as I still have not set up the character selection menus.

Lan game class – note

At this point, the server responses to “initH” and soon “initJ” are different. This means that the LAN game class will not work anymore, as I wrote the code for that class expecting character names and stage names to be shared between clients.

The new stage, waiting , and character selection scenes will do most of the work in syncing stage choices, character choices, and ensuring there are two players.

I plan to edit the lan game class to facilitate these new changes. I believe that starting with the LAN game class, even though I didn't see expect the server responses to change, was still a good choice, as it was a good base to build these new menus on top of.

Character selection menu – LAN version

To simplify what I'm needing to transfer between scenes, I will use the server to store the characters and stages and retrieve them when I need them. I will only need the mode that the player is in to determine the player number for spawn points, sp that should be transferred between scenes.

In order to have a character selection menu that works with LAN, like the one drawn in the usability features, I will need to build on the previous character selection menu class to allow LAN support.

The previous character selection menu needs a stage to be passed in as a parameter. This won't be possible if a player is joining, and shouldn't be needed either way for LAN mode, as it was stored in the server in the waiting scene and can be retrieved once the actual game starts.

When inheriting from the character selection menu, I will set the stage to None. This shouldn't produce any errors, since it doesn't use it for anything other than storing.

The constructor for this class is below:


```

class CharacterSelectSceneLAN(CharacterSelectScene):
    def __init__(self, manager, assets, sSize, mode):
        super().__init__(manager, assets, sSize, mode, None)
        #The network will ALREADY be connected at this point, from the waiting scene
        self.network = self.manager.network

        #THIS SHOULD BE REMOVED AFTER JOINING MENU
        if mode == "join":
            self.network.connect()

        #sets the home player number and opponent player number
        self.pNumber = 1 if mode == "host" else 2
        self.opNumber = 2 if mode == "host" else 1

        #splitting from a list format is better
        self.selected = None
        self.confirmed = None
        #choice will be false once character is confirmed
        self.choice = True

        #stores the opponent selected/confirmed for
        self.opSelected = None
        self.opConfirmed = None

```

The player numbers are used for drawing previews. Also, the home player number needs to be passed on.

Super calls the inherited class's constructor. In the joining menu, player 2 will connect. However, since that has not been implemented yet, I will connect it in this scene.

This time, I have decided to split the lists, so it can be clearer while programming the class. The choice attribute will be used to lock the characters once they are confirmed.

I will also need a method to get the opponent's selections and confirmed characters. I have made this method separate as it's a chunky block of code that should really be separated:

```

#gets opponent selections
def getOp(self):
    #extracts the names of selected/confirmed characters
    selected = self.selected.name if self.selected != None else None
    confirmed = self.confirmed.name if self.confirmed != None else None

    #retrieves the opponent selections and confirmations
    #server should send ["CHARACTER_NAME", None]
    info = self.network.send(["request_op_character_selections", selected, confirmed])
    opConfirmed = info[0]
    opSelected = info[1]

    #one for loop for both confirmed and selected
    #more efficient
    for icon in self.icons:
        if icon.name == opSelected:
            self.opSelected = icon
        if icon.name == opConfirmed:
            self.opConfirmed = icon

```

The selected and confirmed variables get the names from the icons stored in the attributes.

New server signal, I will program the server response later. It should return the opponent's selections. I will send the home player's selections and confirmations to update the server.

After getting the opponent's selections, they need to be matched with the icons so the previews can be drawn to the screen. I use one for loop to do this, as it's more efficient than two separate loops.

The getOp method will be called in the update function. The purpose of the update function is to update both players selections and also check if the server has received both confirmations and is ready to start.

```
def update(self, dt, mpos, mbut, mbut_j, keys, keys_j):-
    #CHARACTER SELECTION FOR HOME PLAYER
    if self.choice:
        for icon in self.icons:
            #if hover then change the selected on that player's turn
            if icon.hover(mpos, mbut_j) == True:
                self.selected = icon

            #if pressed then confirm the choice
            if icon.isPressed(mpos, mbut_j) == True:
                self.confirmed = icon
                #once confirmed, the user shouldn't choose
                self.choice = False

    #get selected/confirmed from the opponent
    self.getOp()

    status = self.network.send(["request_character_status"])
    if status == ["ongoing"]:
        pass
    elif status == ["start"]:
        # player number should change how the lan game class interprets the confirmed characters
        self.manager.change_scene(LANgame, self.manager, self.assets, self.sSize, self.pNumber)
```

This code is very similar to the original scene, but I have also added an if statement to check if choice is true. It is set to false once the home player confirms their character by clicking on it. Previews are shown on hover.

Here, the getOp method retrieves and sets the opponent's selections.

The server will be programmed to respond in two ways to a "request_character_status" signal. It can either return ongoing, if, not all players have confirmed their characters, or "start" if they have. The game should be started if they have.

The characters and stage have not been passed in the change scene function above. That will produce an error, but I will change the LAN game scene function after finishing the character selection menu.

The draw function for this class is slightly more complicated. As the selections will be side to side, I want to make it clear to both players which character is theirs. This is why I have decided to draw an overlay over the opponent's characters:slight

```
def draw(self,screen ):
    screen.blit(self.background, self.backgroundRect)

    #draws the opponents selections under
    #so the players know which character
    if self.opConfirmed != None:
        self.opConfirmed.drawPreview(screen, self.opNumber)
    elif self.opSelected != None:
        self.opSelected.drawPreview(screen, self.opNumber)

    screen.blit(self.overlay, self.overlayR)
    for icon in self.icons:
        icon.draw(screen)

    if self.confirmed != None:
        self.confirmed.drawPreview(screen,self.pNumber)
    elif self.selected != None:
        self.selected.drawPreview(screen, self.pNumber)
```

The opponen't preview is drawn after the background but before the overlay.

The overlay is defined in the super constructor.

The home player's preview is drawn last.

The new server signals that I have introduced are 'request_character_status' and 'request_op_character_selections'. The server should respond to the first with either 'ongoing' or 'start' and the second, the server should respond with the opponent's selections. The code for this is places in the threaded client function and is shown below:

```
elif data[0] == "request_op_character_selections":
    self.data[f"{n}"]["selected"] = data[1]
    self.data[f"{n}"]["confirmed"] = data[2]

    response = [self.data[f"{op}"]["selected"], self.data[f"{op}"]["confirmed"]]

elif data[0] == "request_character_status":
    temp = [self.data[f"{n}"]["confirmed"], self.data[f"{op}"]["confirmed"]]

    #if one player still hasn't picked
    if None in temp:
        response = ["ongoing"]
    #else they both have picked their characters
    else:
        response = ["start"]
```

The server only checks the confirmed keys for names. If they have both picked, there should not be None in the list.

For the character selection menu to work, I need to route the join button and waiting scene to it. The waiting scene will switch to it, once player 2 has joined. The join button should switch to it instantly, as long as there is a running server.

```
def update(self, dt, mpos, mbut, mbut_
#check on whether a player has join
status = self.network.send(["init",
if status[0] == "ready":
| self.manager.change_scene(CharacterSelectSceneLAN, self.manager, self.assets, self.sSize, "host")
elif status[0] == "waiting":
| print("waiting")
#only one of two responses should be given
else:
| self.manager.change_scene(PlayMenuScene, self.manager, self.assets, self.sSize)
```

The waiting scene will switch to the character select scene once it receives the signal from the server.

```
class PlayMenuScene(scene):
def __init__(self, manager, assets, sSize):
super().__init__(manager, assets, sSize)

self.background = self.assets.getBackground("start_menu")
self.backgroundRect= self.background.get_frect(topleft = (0,0))

practise, practiseG = self.assets.getButton("practise")
local, localG = self.assets.getButton("local")
join, joinG = self.assets.getButton("join")
host, hostG = self.assets.getButton("host")
back, backG = self.assets.getButton("back")
self.practiseButton = Button(practise, practiseG, lambda:
| self.manager.change_scene(StageSelectScene, self.manager, self.assets, self.sSize, "practise")

self.localButton = Button(local, localG, lambda:
| self.manager.change_scene(StageSelectScene, self.manager, self.assets, self.sSize, "local")

self.joinButton = Button(join, joinG, lambda:
| self.manager.change_scene(CharacterSelectSceneLAN, self.manager, self.assets, self.sSize, "join"),
| center=(self.sSize[0]/2, self.sSize[1]*4.5/8))
self.hostButton = Button(host, hostG, lambda:
| self.manager.change_scene(StageSelectScene, self.manager, self.assets, self.sSize, "host"),
| center=(self.sSize[0]/2, self.sSize[1]*5.5/8))
self.backButton = Button(back, backG, lambda: self.manager.change_scene(MenuScene, self.manager, self.assets, self.sSize),
| center=(self.sSize[0]/2, self.sSize[1]*7/8))
```

Play menu scene code has been explained previously, I have changed the join button function to switch to the character select scene.

Testing the character selection menu

What I'm testing	Input data	Why am I testing this?	Expected outcome	Test results
Does clicking the join button load the character select scene?	Mouse click on join button	This will allow players to join their friends' games and play together.	The character selection menu should be shown once the join button is clicked.	As, expected.
Does the waiting scene switch to the character selection menu if player 2 joins?	Mouse click on join button from another instance of the game.	To allow the players to play together over a LAN connection, the waiting scene will need to switch scenes to show the charcter selection scene. This will allow players to select their	The character selection menu should be shown once player 2 joins.	As, expected.

		characters before starting the game.		
Is the opponent character preview less visible?	Mouse click on a character in the character selection menu.	This will make it clear to the players, which character they are going to play as. So that they can choose the correct character.	The opponent character's must be less visible in selection and confirmation in the character selection menu.	As, expected.

The results of this test are in a video titled 'lan_character_selection'

Editing the lan game class

To make the lan game class work with the new changes which are:

- The character names are stored in the server
- The stage name is stored in the server
- The waiting scene already waits until two players have joined.

Firstly, I no longer need to check whether player 2 has joined, as the waiting scene already does that, so I deleted the check_p2 function and removed the call in the update function. Then I removed the extra checks at the end of the constructor to replace with this:

```
class LANgame(scene):
    #pNumber should be 1 for host and 2 for join
    #this will be used to identify whether to run code for the server
    #and for init
    def __init__(self, manager, assets, sSize, pNumber):
        super().__init__(manager, assets, sSize)
        self.network = self.manager.network
        self.server = self.manager.server
        self.hud = HUD(self.sSize, self.assets)

        self.pNumber = pNumber
        #other player number used for indexing
        self.opNumber = 1 if pNumber == 2 else 2
        self.players = [None, None]

        #gets the stage from the server
        stage = self.network.send(["request_stage"])[0]
        self.stage = StageFactory(stage, assets, sSize)

        #should return [home player name, opponent player name]
        players = self.network.send(["request_all_character_selections"])
        self.players[self.pNumber-1] = CharacterFactory(players[0], assets, self.manager.controls, self.stage.getSpawnPoint(pNumber), pNumber)
        self.players[self.opNumber-1] = CharacterFactory(players[1], assets, self.manager.controls, self.stage.getSpawnPoint(self.opNumber), self.opNumber)

        #resets the hud, incase there was a delay
        self.hud.reset()
```

In the function above, I retrieve the stage and character names from the server using the signals 'request_stage' and 'request_all_character_selections', which I use to create instances of the stage and characters. Then I reset the hud, just to be extra sure the timer is as accurate as possible.

Below is the code for the server responses to the new signals introduced in the lan game constructor:

```
elif data[0] == "request_stage":
    response = [self.stage]

elif data[0] == "request_all_character_selections":
    #first index is always for the player requesting
    temp = [self.data[f"{n}"]["confirmed"], self.data[f"{op}"]["confirmed"]]
    self.data[f"{n}"]["name"] = self.data[f"{n}"]["confirmed"]
    response = temp
```

The request stage signal just sets the response to the stage attribute which would've been defined in the waiting scene when the server is started after the stage selection scene.

The request all character selections response returns the confirmed character of the player requesting in the first index and the opponent in the second. This makes it easier to create the instances of the characters in the lan game constructor.

The name of the requesting player is also set here, as this is when the game will begin. It is important to set the name as that is used in the lan game class set data method to set the opponent's game data and for the game logic.

Testing the lan game class

What I'm testing	Input data	Why am I testing this?	Expected outcome	Test results
Does moving the host move the host on player 2's screen?	"a", "d", "w"	Player movements should be synced across different clients.	Moving either player, should move the other on the opponent's screen.	As, expected.
Does moving player 2 move the player 2 on the host's screen?	"a", "d", "w"	Player movements should be synced across different clients.	Moving either player, should move the other on the opponent's screen.	As, expected.
Is the HUD synced across both players, showing the correct time?	One client clicks host while the other clicks join.	The HUD should be synced to show correct time info and not have any bugs with when a game ends.	The game should start the timer when player 2 joins and should be the same on both clients.	As, expected.

The results of this test are in a video titled 'lan_game_change'

Graceful error handling

To fulfill success criteria #13, the game must return to the play menu instead of crashing, in the event of network failures.

To successfully implement this, I need to look at all the possible points of failure. Naturally, this will be any line of code that sends something to the server, this is because the server could not respond or a player could leave.

In the case that the server can't respond, the network send function will produce an error. So, I need to wrap it in a try except block to catch the error and instead return a string like "error".

```
def send(self,data):
    try:
        self.client.send(json.dumps(data).encode())
        return json.loads(self.client.recv(1024).decode())
    except Exception as e:
        #in a list as all server responses are in a list
        #less likely to produce an error as I usually check the first
        #index in server response handling
        self.reset()
        return ["Error"]
```

Now, if the network has an error while sending, it will reset and I can use the value returned to return to the play menu. I need to create a similar reset function for the server.

I already have the reset function for the network class but I need to recreate it for the server class:

```
def shutdown(self):
    self.running = False
    try:
        self.socket.close()
    except:
        pass
```

This function will try to close the socket and set the running attribute to False. I added the self.running attribute to shut down the accept connections function which would be running on a separate thread. Without this, the thread would run longer or produce an unnecessary error.

I also changed the accept connections function to move the code that binds to a socket, to the start method:

I can use the error returned in the waiting scene which calls this method to return to the play menu.

```
def start(self, stage):
    #this function is called for setting up the server
    #stage will be used in the joining menu
    #way to differentiate between diferent hosts
    self.stage = stage

    #bind here to be able to return a status when starting
    try:
        self.socket.bind((self.ip, self.port))
    except Exception as e:
        return ["Error"]

    #server will need to run on a sepearte thread-
    #daemon means that this thread will end once main game is closed
    server = threading.Thread(target=self.accept_connections, args=(), daemon=True)
    server.start()

    return ["Success"]

#this function will accept new connections from players
def accept_connections(self):

    self.socket.listen(self.max_connections)
    print("Waiting for a connection, Server started")

    #will only accept a connection if the current_connections is under max_connections
    while True and self.current_connections < self.max_connections and self.running:
        conn, addr = self.socket.accept()
        #print console for confirmation of connection
        print(f"connected to{conn}:{addr}")

        self.current_connections += 1
        #current connection used to specifiy the player number
        client = threading.Thread(target=self.threaded_client, args=(conn, self.current_connections), dae
        client.start()
```

The start function will do the binding now, as that is the most likely to give errors due to a port being used , for example.

Running is used as a condition in the while loop so the loop will eventually end and the thread will be closed.

Now that I have the reset functions, I can go through my code and add checks at any point of possible failure to return the play menu.

Rather than copying the same code multiple times, I can create a function that will have the scene and response passed as parameters. This function can automate shutting down the server and changing scenes. The output from this function can be used to stop any subroutines from further executing any code:codeexecutchashutautomapeadtI

```
def checkLANErr(scene, response):
    '''This function works on these assumptions:
    -network send returns ["Error"] if there is
    -the scenes that use this function have a server, manager,
    assets, and sSize attribute'''
    if response == ["Error"]:
        #reset the server too
        scene.server.shutdown()
        scene.manager.server = server()
        #change scenes to the play menu
        scene.manager.change_scene(PlayMenuScene, scene.manager, scene.assets, scene.sSize)
        #error identified
        return True

    return False
```

I added the assumptions used at the top, as this function needs the scene passed in to have a server attribute.

It returns true if the network send function has an error. I can use this to exit out of functions and stop any more errors and crashes from happening.

Now, I can go through the scenes code and use the check lan err function in an if statement to exit out of the constructor/methods:

```
class LANgame(scene):
    #pNumber should be 1 for host and 2 for join
    #this will be used to identify whether to run code for the server
    #and for init
    def __init__(self, manager, assets, sSize, pNumber):
        super().__init__(manager, assets, sSize)
        self.network = self.manager.network
        self.server = self.manager.server
        self.hud = HUD(self.sSize, self.assets)

        self.pNumber = pNumber
        #other player number used for indexing
        self.opNumber = 1 if pNumber == 2 else 2
        self.players = [None, None]

        #gets the stage from the server
        stage = self.network.send(["request_stage"])[0]
        #check for errors to reset
        if checkLANErr(self, stage):
            return
        self.stage = StageFactory(stage, assets, sSize)

        #should return [home player name, opponent player name]
        players = self.network.send(["request_all_character_selections"])
        if checkLANErr(self, players):
            return
        self.players[self.pNumber-1] = CharacterFactory(players[0], assets, self.manager.controls, self)
        self.players[self.opNumber-1] = CharacterFactory(players[1], assets, self.manager.controls, self)

        #resets the hud, incase there was a delay
        self.hud.reset()
```

Any network send calls give a response, this is passed into the check function to return if there is an error. The return in the if statement will exit the entire constructor to stop any crashes from happening.


```

def set_data(self, dt):
    other = self.network.send(["game", self.players[self.pNumber-1].rect.midbottom,
                                self.players[self.pNumber-1].animation_state,
                                self.players[self.pNumber-1].current_animation.getFrameNumber(),
                                self.players[self.pNumber-1].facing_left,
                                self.players[self.pNumber-1].health])

    #check for LAN errors
    if checkLANErr(self, other):
        return

    self.players[self.opNumber - 1].set_lan_data(dt, other["position"], other["animation_state"], ot
    self.wins = other["wins"]

    if other["status"] == "active":
        pass
    elif other["status"] == "new_round":
        self.players[self.pNumber-1].reset()
        self.hud.reset()
    else:
        pWin = "Player " + other["status"][-1] + " wins!"
        print(pWin)
        self.manager.change_scene(PlayMenuScene, self.manager, self.assets, self.sSize)

```

In this set data function , I also removed the previous if statement that checked if a second player is in the game. That is not needed anymore, as the waiting scene does that.

```

def init_connection(self):
    self.network = self.manager.network
    self.server = self.manager.server

    #only the stage has been picked
    status = self.server.start(self.stageID)
    #if there is an error with the server start
    if status[0] == "Error":
        #reset all network attributes
        self.network.reset()
        self.server.shutdown()
        self.manager.server = server()
        self.manager.change_scene(PlayMenuScene, self.manager, self.assets, self.sSize)
        return

    self.network.connect()

```

The network reset line is unnecessary as it hasn't been connected at that point, but a good precaution.

This is the init method from the waiting scene which is meant to start the server. If there is an error with it, it resets all network attributes.

```

def update(self, dt, mpos, mbut, mbut_j, keys, keys_j):
    #check on whether a player has joined
    status = self.network.send(["initH"])
    if checkLANErr(self, status):
        return

    if status[0] == "ready":
        self.manager.change_scene(CharacterSelectSceneLAN, self.manager, self.a
    elif status[0] == "waiting":
        print("waiting")
    #only one of two responses should be given
    else:
        self.manager.change_scene(PlayMenuScene, self.manager, self.assets, sel

    #possible outputs are 0,1,2,3
    #time is in seconds, so it will switch through the numbers
    #every second, max of 3 dots
    self.nDots = int(time.time())%4
    #rerender the text with nDots amount of nDots
    self.text = self.font.render("Waiting" + (self.nDots*"."), True, (255,255,2
    self.textR = self.text.get_frect(bottomleft = self.backgroundRect.bottomle

```

```

#get selected/confirmed from the opponent
self.getOp()

status = self.network.send(["request_character_status"])
if checkLANErr(self, status):
    return

if status == ["ongoing"]:
    pass
elif status == ["start"]:
    # player number should change how the lan game class inte
    self.manager.change_scene(LANgame, self.manager, self.ass

#gets opponent selections
def getOp(self):
    #extracts the names of selected/confirmed characters
    selected = self.selected.name if self.selected != None else N
    confirmed = self.confirmed.name if self.confirmed != None els

    #retrieves the opponent selections and confirmations
    #server should send ["CHARACTER_NAME", None]
    info = self.network.send(["request_op_character_selections",
    if checkLANErr(self, info):
        return
    opConfirmed = info[0]
    opSelected = info[1]

    #one for loop for both confirmed and selected
    #more efficient
    for icon in self.icons:
        if icon.name == opSelected:
            self.opSelected = icon

```

To handle the other type of error where someone could leave, I need to implement regular checks in the character selection scene and lan game. These checks will make sure that there is still two players in the game, and if there isn't, then the player should be returned to the play menu.

To do this, I will need to send a signal to the server and request info about whether there is two players or not, then perform an appropriate action based on that.

In the character select scene's update function, I will add this:

```
def update(self, dt, mpos, mbut, mbut_j, keys, keys_j):-
    status = self.network.send(["request_lan_status"])
    if not status[0]:
        #reset all network attributes
        self.network.reset()
        self.server.shutdown()
        self.manager.server = server()
        self.manager.change_scene(PlayMenuScene, self.manager, self.assets, self.sSize)
        return

#CHARACTER SELECTION FOR HOME PLAYER
if self.choice:
    for icon in self.icons:
```

The network attributes will be reset, and the scene will be changed to the play menu, only if the server sends False to the signal 'request_lan_status'.

I will also add the same block of code to the lan game update function:

```
def update(self, dt, mpos, mbut, mbut_j, keys, keys_j):

    status = self.network.send(["request_lan_status"])
    if not status[0]:
        #reset all network attributes
        self.network.reset()
        self.server.shutdown()
        self.manager.server = server()
        self.manager.change_scene(PlayMenuScene, self.manager, self.assets, self.sSize)
        return

    self.players[self.pNumber-1].update(self.stage.getHitboxes(), dt, mpos, mbut, mbut_j, keys, keys_j)
    self.set_data(dt)
```

Now, I can program the server's response to 'request_lan_status', which is very simple:

```
elif data[0] == "request_lan_status":
    #if two players are in the game, response will be [True]
    response = [self.current_connections==2]
```

The == operator just returns true if the comparison is right.

Testing error handling

I will list the situations that will need to be tested below:

- Host leaves in the character select menu
- Host leaves in the game
- Joining player leaves character select menu
- Joining player leaves in the game

What I'm testing	Input data	Why am I testing this?	Expected outcome	Test results
Does the joining player return to the play menu if the host leaves in the character select menu.	Close window on host	Success criteria #13	Joining player return to the play menu.	As, expected.
Does the joining player return to the play menu if the host leaves in game.	Close window on host	Success criteria #13	Joining player return to the play menu.	As, expected.
Does the host return to the play menu if the joining player leaves in game.	Close window on joining player	Success criteria #13	Host return to the play menu.	As, expected.
Does the host return to the play menu if the joining player leaves the character selection menu.	Close window on joining player	Success criteria #13	Host return to the play menu.	As, expected.

Test for the host leaving the character select menu is in a video titled 'host_leaves_character_charac'host_lea

Test for the host leaving the game is in a video titled 'host_leaves_game'

Test for the joining player leaving the character select menu is in a video titled 'host_leaves_game'

Test for the joining player oinileaving the game is in a video titljoinhost_leaves_game'

The joining menu

For the joining menu to work, I will need to use the UDP protocol to broadcast the host's game's details to all devices connected to the wifi.

The algorithms used for this are already written in pseudocode in the design section, so they should be easy to write.

There will be a method in the network class which will send out game discovery requests and a method in the server class which will receive them and send the details of how to connect back.

Below is the network's method for discovering games:

```
def discover_games(self):
    #Ipv4 and UDP
    udp_sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
    #Allows broadcasting
    udp_sock.setsockopt(socket.SOL_SOCKET, socket.SO_BROADCAST, 1)

    #3 second timeout-
    #otherwise it will try to receive forever
    udp_sock.settimeout(3.0)
    #' means all interfaces
    #0 means any free port
    udp_sock.bind('', 0)

    message = "game_discovery_request_mythbound"
    discovered = []

    try:
        udp_sock.sendto(message.encode(), ('<broadcast>', 9999))
    except:
        return []

    start = time.time()
    while time.time() - start < 3:
        try:
            #receives the stage and the address
            #from the server
            stage, addr = udp_sock.recvfrom(1024)

            #appends to a list, to return
            discovered.append((stage.decode(), addr))
        except:
            pass

    udp_sock.close()
    return discovered
```

I define the socket with SOCK_DGRAM for the UDP protocol.

I set the broadcast option to true, so I can use it to discover games.

A time out is needed to stop the socket from looking for a response if it is not given in 3 seconds. Otherwise, it will run forever.

I broadcast the game discovery message here on port 9999, this will need to be listened to by the server.

The while loop will allow the socket to receive as many games as possible. Here, the udp socket will receive the stage and socket address to append to a list.

Then it will close the socket and return the list of discovered games.

I need to bind this UDP socket to all interfaces on my computer, meaning not just wifi, but also ethernet (if connected) and localhost.

Although I have written a get local ip function pseudocode, it isn't needed as the socket address (containing IP and port) is sent automatically when I send anything via UDP.

The server listen function will need to bind to all interfaces, like the discover games function, and listen on the same port as it which is 9999. The port number can be anything, I have just picked 9999 because it's a nice number.


```

def listen(self):
    #IPv4, UDP
    udp_sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)

    #recvfrom will stop after 3 seconds, if no response
    #otherwise will run forever, despite the loop
    udp_sock.settimeout(3.0)

    try:
        #' binds to all interfaces
        #9999 is the same as the discover_games
        udp_sock.bind(('', 9999))
    except Exception as e:
        print(Exception)

    while self.listening:
        #try block keeps the function from ending
        try:
            data, addr = udp_sock.recvfrom(1024)
            message = data.decode()

            if message == "game_discovery_request_mythbound":
                #send the stage back
                response = (self.stage).encode()
                #addr is the address the request came from
                udp_sock.sendto(response, addr)

        except Exception:
            pass #time out errors, shouldn't do anything

    #finally closes the socket
    udp_sock.close()

```

The listening attribute will be used to control the thread. Once it is set to true, the thread running this listen function will end.

I have wrapped this entire part in a try except block to stop any errors from exiting the while loop and the function. This part of the function needs to run as long as it takes for a second player to join.

The new attribute listening is needed to exit the thread running the listen function and stop it from running forever. It will need to be set to false in the shut down function:

```
def shutdown(self):
    #exits the threads
    self.running = False
    self.listening = False
    try:
        self.socket.close()
    except:
        pass
```

The listen function will need to be ran on a seperate thread when the server is started:

```
def start(self, stage):
    #this function is called for setting up the server
    #stage will be used in the joining menu
    #way to differentiate between diferent hosts
    self.stage = stage

    #bind here to be able to return a status when starting
    try:
        self.socket.bind((self.ip, self.port))
    except Exception as e:
        return ["Error"]

    #server will need to run on a seperate thread-
    #daemon means that this thread will end once main game is closed
    server = threading.Thread(target=self.accept_connections, args=(), daemon=True)
    server.start()

    listen = threading.Thread(target=self.listen, args=(), daemon=True)
    listen.start()

    return ["Success"]
```

It will also need to end once two players have joined, which will be when the accept connections function ends. The accept connections function ends once the while loop is exited, one of those conditions is `self.current_connections < self.max_connections`. Once that is broken (2 players join), the function will end and with it I will set listening to false:

```
#this function will accept new connections from players
def accept_connections(self):
    self.socket.listen(self.max_connections)
    print("Waiting for a connection, Server started")

    #will only accept a connection if the current_connections is under max_connections
    while True and self.current_connections < self.max_connections and self.running:
        conn, addr = self.socket.accept()
        #print console for confirmation of connection
        print(f"connected to{conn}:{addr}")

        self.current_connections += 1
        #current connection used to specifiy the player number
        client = threading.Thread(target=self.threaded_client, args=(conn, self.current_connections), da
        client.start()

    #two players have joined, so no more game discovery requests
    self.listening = False
```

Now that the backbone of the joining menu is dealt with, I need to test this by setting up a small joining menu scene:

```
class JoiningScene(scene):
    def __init__(self, manager, assets, sSize):
        super().__init__(manager, assets, sSize)-----
        #Sets background
        self.background = self.assets.getBackground("start_menu")
        self.backgroundRect= self.background.get_frect(topleft = (0,0))

        #Transparent darker overlay
        self.overlay= pygame.Surface(self.sSize, pygame.SRCALPHA)
        self.overlay.fill((0,0,0,150))
        self.overlayR = self.overlay.get_frect(topleft=(0,0))

        -----
        #network attribute for discovering games
        self.network = self.manager.network

    def update(self, dt, mpos, mbut,mbut_j, keys, keys_j):
        -----
        #prints the discovered games from the network
        print(self.network.discover_games())-

    def draw(self, screen):
        screen.blit(self.background, self.backgroundRect)
        screen.blit(self.overlay, self.overlayR)
```

I copied the code from the settings scene again to create an overlay for the background, which is the same as the start menu's background. In the update function I just print out the discovered games after calling the discover games function.

I need to route the join button to this function, to test out the discovery:

```
self.manager.change_scene(stageselectscene, self.manager, self.assets, self.sSize,
                           center=(self.sSize[0]/2, self.sSize[1]*3.5/8))
self.joinButton = Button(join, joinG, lambda:
                           self.manager.change_scene(JoiningScene, self.manager, self.assets, self.sSize),
                           center=(self.sSize[0]/2, self.sSize[1]*4.5/8))
self.hostButton = Button(host, hostG, lambda:
```

Testing the broadcasting

What I'm testing	Input data	Why am I testing this?	Expected outcome	Test results
Does the stage and socket address of the host game get printed out in the joining player's console?	Mouse click on join after hosting.	Success criteria #12	The stage and socket address to be printed to the console	As, expected.

It works nearly perfectly! Here is the output from the console:

[illegible]

The only problem with this output is 9999 which is the port used for the UDP socket but not the actual game.

To send the port of the actual game so the player can join, I will need to add it on to the stage variable like this:

```
while self.listening:
    #try block keeps the function from ending
    try:
        data,addr = udp_sock.recvfrom(1024)
        message = data.decode()

        if message == "game_discovery_request_mythbound":
            #send the stage back
            response = (self.stage+"."+str(self.port)).encode()
            #addr is the address the request came from
            udp_sock.sendto(response, addr)

    except Exception:
        pass #time out errors, shouldn't do anything
```

So that players can click on a game to join the joining menu, I will need widgets. I don't have much time to make them look nice, so a simple class (which is very similar to the `keybind widgets` class) will do:


```

class widget:
    def __init__(self, font, text, addr, size, **pos_kwargs):
        #sets the outline box which contains the key and action
        self.bg = pygame.Surface(size)
        self.bg.fill((100,100,100))
        self.bgr = self.bg.get_frect(**pos_kwargs)

        #font for text
        self.font = font
        self.addr = addr

        #sets text
        self.text = text
        self.textI = self.font.render(text, True, (0,0,0))
        self.textR = self.textI.get_frect(topleft = self.bgr.topleft)

    def update(self, mpos, mbut_j):
        #If mouse hover
        if self.bgr.collidepoint(mpos):
            #If press too
            if mbut_j[0]:
                #True for press
                return True
            #No press
            return False

    def draw(self, screen):
        #draws background, action on the left, key on the right
        screen.blit(self.bg, self.bgr)
        screen.blit(self.textI, self.textR)

```

I need to store the address so that it can be accessed for connecting to in the joining menu.

This function will just return true if the widget is clicked.

In the joining menu constructor, I will need a list to store the widgets, the widget size, a font and a back button so the user can leave the joining menu. The code for this is highlighted below:

```

class JoiningScene(scene):
    def __init__(self, manager, assets, sSize):
        super().__init__(manager, assets, sSize)
        #Sets background
        self.background = self.assets.getBackground("start_menu")
        self.backgroundRect = self.background.get_frect(topleft = (0,0))

        #Transparent darker overlay
        self.overlay = pygame.Surface(self.sSize, pygame.SRCALPHA)
        self.overlay.fill((0,0,0,150))
        self.overlayR = self.overlay.get_frect(topleft=(0,0))

        #network attribute for discovering games
        self.network = self.manager.network

        #list for storing all widgets
        self.widgets = []
        #size for each widget
        self.widgetSize = (self.sSize[0]/3, self.sSize[1]/28)

        self.font = self.assets.loadFont("Cinzel-Regular.otf", 24)

        back, backG = self.assets.getButton("back")
        self.backButton = Button(back, backG, lambda: self.manager.change_scene(PlayMenuScene, self.manager, self.assets, self.sSize),
                                center=(self.sSize[0]/2, self.sSize[1]*7/8))

```

I decided to load the font in, as its more efficient so I can pass it in to each widget, without having to load the same font for each one seperately.

For the update function to work, I will need to check when the network is finished with discovering new games before starting a new discovery thread. This can be done with a boolean that gets set to true when the discovery function starts and false when it ends.

Also, to access data from a separate thread, I will need to use a variable that is accessible to both the network class and its method, which will be the discovered variable.

Every time the discovery function ends, I will access the discovered list to extract the LAN games and present them to the user like so:

```
def update(self, dt, mpos, mbut, mbut_j, keys, keys_j):
    #if the last game discovery is over
    if not self.network.discovering:
        #retrieves the results of the last
        discovered = self.network.discovered

        self.widgets = []
        for i, game in enumerate(discovered):
            #stage name is stored in index 0-
            stage = game[0]
            addr = game[1]
            #addr[1] is always 9999
            #get the RIGHT port
            port = stage[stage.find(".")+1:]
            #keep the ip but replace the port
            addr = (addr[0], int(port))
            #make a new widget for each game
            self.widgets.append(widget(self.font, stage, addr, self.widgetSize, center = (self.sSize[0]/2, (i+4)*self.sSize[1]/8)))

        #needs to run on a separate thread to not freeze the game
        discover_games = threading.Thread(target=self.network.discover_games, args=(), daemon=True)
        discover_games.start()

    #checks if a player clicked a game
    for game in self.widgets:
        if game.update(mpos, mbut_j):
            #connect using the addr attribute in the widget
            self.network.connect(game.addr)
            #then changes scenes to the character select
            self.manager.change_scene(CharacterSelectSceneLAN, self.manager, self.assets, self.sSize, "join")
```

The discovering attribute is a boolean which is used to determine whether the network is currently searching for games, I will show the code for setting it after this.

I use the '.' to find the port which is placed after it.

After I set the widgets, I start the discovery thread again.

This checks if the widgets were clicked.

If a widget is clicked, it uses the address to connect to the server and changes scenes.

Testing the joining menu

What I'm testing	Input data	Why am I testing this?	Expected outcome	Test results
Does the stage and socket address of the host game appear as a widget in the menu.	Mouse click on join after hosting.	Success criteria #12	The stage name + port to be the text in the widget.	As, expected.
Does the character selection menu load once I click on a game?	Mouse click on the host's game.	Success criteria #12	The character selection scene should be loaded with the host.	As, expected.

The results for this test are in a video titled 'joining_menu'

The ending banner

I will be adding the ending banner here, as I forgot to add them previously.

Creating the ending banner like the usability features is very simple as most of the code is copied from other menus. This is the constructor.

```
class EndingBannerScene(scene):
    def __init__(self, manager, assets, sSize, stage, pNumber, character):
        super().__init__(manager, assets, sSize)
        #Sets background
        self.background = self.assets.getBackground(stage)
        self.backgroundRect = self.background.get_frect(topleft = (0,0))

        #Transparent darker overlay
        self.overlay = pygame.Surface(self.sSize, pygame.SRCALPHA)
        self.overlay.fill((0,0,0,150))
        self.overlayR = self.overlay.get_frect(topleft=(0,0))

        #gets the first frame of the idle animation
        self.image = self.assets.getAnimations(character)["idle"][0]
        self.preview = self.image.subsurface(self.image.get_bounding_rect())
        self.preview = pygame.transform.scale_by(self.preview, 3*assets.SF)

        #places the feet of the character at the midbottom of the screen
        self.previewR = self.preview.get_frect(midbottom = (self.sSize[0]/2, self.sSize[1]))

        #loads font for custom size
        self.font = self.assets.loadFont("PressStart2P.ttf", 32)
        #specifies the text according to player number
        text = f"Player {pNumber} wins!"
        self.text = self.font.render(text, True, (255,255,255))
        self.textR = self.text.get_frect(center = (self.sSize[0]/2, self.sSize[1]/3))

        back, backG = self.assets.getButton("back")
        #places back button in the topleft corner with a slight square offset
        self.backButton = Button(back, backG, lambda: self.manager.change_scene(PlayMenuScene, self.manager, self.assets, self.sSize),
                                topleft=(self.sSize[0]/128, self.sSize[1]/128))

        self.duration = 3
        self.start = time.time()
```

The stage will be passed in to use its background from the assets. The overlay is copied from the settings.

The preview code to size up the characters is copied from the character select scene.

Here, I just create the text using the winning player's number that is passed into the constructor.

A back button is created so the player can go back to the play menu.

The time that the banner is started is saved so the duration of 3 seconds can be followed by comparing times.

Most of the hard work is in the constructor, this is the code for the update and draw functions:


```
def update(self, dt, mpos, mbut,mbut_j, keys, keys_j):
    #if the duration is up
    if time.time() - self.start >= self.duration:
        self.manager.change_scene(PlayMenuScene, self.manager, self.assets, self.sSize)

    self.backButton.update(mpos, mbut_j)

def draw(self, screen):
    screen.blit(self.background, self.backgroundRect)
    screen.blit(self.overlay, self.overlayR)
    self.backButton.draw(screen)

    screen.blit(self.text, self.textR)
    screen.blit(self.preview, self.previewR)
```

The start time is compared with the current time so the scene can be changed once the duration of 3 seconds is up.

The backButton is also updated so the button can respond to clicks.

All the elements are drawn.

Now, I need to route the local and LAN games to switch to the the ending banner at the end:

```
#best of 3 system
#if one player wins twice, they win
if 2 in self.wins:
    #ending banner should be switched to here
    index = self.wins.index(2)
    self.manager.change_scene(EndingBannerScene, self.manager, self.assets, self.sSize, self.stage.name, index+1, self.characters[index])
```

As the image is quite small, I pasted the code here:

```
self.manager.change_scene(EndingBannerScene, self.manager, self.assets, self.sSize,
self.stage.name, index+1, self.characters[index])
```

As the stage is a stage object, I just extract the name (which is also the ID) to use in the ending banner background. The index is the index of the item that is 2 in the wins list. Index + 1 = player number. I use that to pass the player number and name to the constructor. The process is the same in the LAN game.

```
#switch scenes to the ending banner
#use the "victory_1" to get the player number
player_number = int(other["status"][-1])
#extract name
name = self.players[player_number-1].name
self.manager.change_scene(EndingBannerScene, self.manager, self.assets, self.sSize, self.stage.name, player_number, name)
```

Testing the ending banner

What I'm testing	Input data	Why am I testing this?	Expected outcome	Test results
Is the ending banner displayed at the end of a local game?	n/a	The ending banner should be displayed to tell the players who won the game. Success criteria #15	The ending banner should be displayed with the appropriate player number and character.	As, expected.
Is the ending banner displayed at the end of a LAN game?	n/a	The ending banner should be displayed to tell the players who won the game. Success criteria #15	The ending banner should be displayed with the appropriate	As, expected.

			player number and character.	
--	--	--	---------------------------------	--

A video of the results is in 'mythbound_main_test'.

Final edits

There is one slight problem with the server class, it picks the ip and port outright.

This can very easily throw an error if the port is being used or a player wants to play with someone connected to their wifi and not just local host (127.0.0.1).

The fix for this , luckily, is really easy:

```
class server:
    def __init__(self):
        #ip.port needed for binding to socket
        #"" means all interfaces
        # the server will now accept connections from
        # 192.168.1.64 or from 127.0.0.1
        self.ip = ""
        self.port = 0 #OS will assign a free port
        self.addr = (self.ip, self.port)
```

Leaving the ip as "", will select all interfaces and the port as 0 will let the OS pick a free port.

I still need to set these attributes to an actual value, once they are chosen after binding:

```
def start(self, stage):
    #this function is called for setting up the server
    #stage will be used in the joining menu
    #way to differentiate between different hosts
    self.stage = stage

    #bind here to be able to return a status when starting
    try:
        self.socket.bind(self.addr)
        #get sock name will get the ip and port picked
        #in socket address format
        self.addr = self.socket.getsockname()

        self.ip = self.addr[0]
        self.port = self.addr[1]
    except Exception as e:
```

The ip and port are used to connect the host to the server in the waiting scene, which is why they will need to be set to an actual value.

After testing the game with this, it works the same as before with no errors!

Interview with the stakeholders

Here are the success criteria, that I aimed to complete in this iteration:critera

#	Success Criteria	Justification	Evidence for testing
12	The game must support LAN multiplayer, allowing users to connect and play with each other from the same network.	Part of the essential features, to allow players to play with their friends on a local network.	A screenshot of the code and a video of the feature working.
13	In the case of network failures, the game must not crash and instead handle it gracefully by returning to the main menu.	This is good programming practise and will improve the user experience.	Video of a network failure and the game handling it gracefully
15	At the end of each game, there has to be an ending banner that displays who won, for 3 seconds.	Instead of abruptly ending the game, I can display who won the game for 3 seconds. 3 seconds is not too long to feel annoying, but not too short that you can't see who won clearly.	A screenshot of the ending banner.

Me: Implementing LAN multiplayer took a long time, but I'm pleased to say it's finished. I wasn't able to test it out using two devices, but I made sure that the game handles any potential network failures to prevent crashes and better the user experience. The testing I did was on local host which can on my own computer's ports, so theoretically it should run fine with two devices on a LAN. There's always more improvements to be made but with this, all game modes are now mostly functional and provide a good base for the rest of the game.

Summary of what the stakeholders said:

The game looks really fun. The graceful error handling is great but the LAN mode not being tested properly is concerning.

Evaluation

Final results of all tests

The main menu

What I'm testing	Input data	Why am I testing this?	Expected outcome	Test results
Button visuals/sizes	N/A	Menu button have to be readable and accessible to click and navigate easily.	For the buttons to be big enough to click and easily readable.	As, expected.
Does the play button respond to click	Mouse click on button	Menu needs be navigable for users to play.	The play button should take the user to the play menu scene which should be blank.	As, expected.
Does settings button respond	Mouse click on button	Menu needs be navigable for users to play.	Console should output "settings"	As, expected.
Does the quit button respond	Mouse click on button	Menu needs be navigable for users to play.	Game should quit	As, expected.
Assets				
Is the bar moving across the loading screen.	Starting up the game	This should work to show the user how long left until the game is started up.	Bar should be moving across to show progress in the loading screen.	As, expected.
Are the assets being loaded	Starting up the game	Without assets, the game would not work.	Assets should be loaded and look good in the menu, including glow variants of buttons.	As, expected.
Is moving in between menus instant	Starting up the game	Removing delay and lag is one of the essential features and will help users to enjoy the game.	Clicking the play button should instantly load up the join menu with no delays.	As, expected.
Play menu				
Does pressing back button work in the play menu	Mouse press on the button	It will give users and interactive menu experience.	Take user to play menu.	As, expected.

take me back to the main menu				
Does pressing the practise button print "practise" to the console	Mouse press on the button	Menu needs to be navigable for users to play.	Output "practise" to console.	As, expected.
Does pressing the local button print "local" to the console	Mouse press on the button	Menu needs to be navigable for users to play.	Output "local" to console.	As, expected.
Does pressing the join button print "join" to the console	Mouse press on the button	Menu needs to be navigable for users to play.	Output "join" to console.	As, expected.
Does pressing host button print "host" to the console	Mouse press on the button	Menu needs to be navigable for users to play.	Output "host" to console.	As, expected.
Does pressing back button change menus to the main menu	Mouse press on the button	Menu needs to be navigable for users to play.	Return back to main menu.	As, expected.

Building the practise arena pra

Does the volcano valley stage and platform show when I press practise	Button press on practise	To get an initial working practise arena to build on	Volcano valley stage is meant to show on screen	As, expected.
Are the Volcano valley platforms the right size	Button press on practise	To get an initial working practise arena to build on	Volcano valley stage and platforms are meant to be correctly scaled.	As, expected.
Do all the menus still function as before	Load up the game. Mouse press on all buttons.	To make sure I haven't broken any of the code, I have written so far.	All buttons should function as normal	As, expected.

Is the character drawn onto the screen in the appropriate position?	Pressing the practise arena button.	The user needs to spawn in a suitable position to play the game.	Character is drawn on top of the platform.	As, expected.
Does moving work?	"A", "D"	So players can move in matches and in the practise arena.	Character should respond to A and D by moving left and right	As, expected.
Does the run animation work?	"A", "D"	Allows the user to see their character moving.	The character should switch between frames in order to display a running animation.	As, expected.
Is gravity being simulated correctly	Practise arena button being pressed	Allows players to move around with simulated gravity/collision detection and don't just fly.	Player's speed should increase as they fall.	As, expected.
Does collision detection work correctly?	Movement	So that players can move around with simulated collision detection and don't clip platforms.	If a player, approaches the platform rect, they should be pushed back and not go inside it.	As, expected.
Does jumping and animation work, are they locked off?	"W"	This will allow the user to jump and see the animation of their character jumping.	Shows the character jumping animation, doesn't allow other animations to start while jumping.	As, expected.
Do attacks work, are they locked off?	"f" "r"	Allows the player to use attacks to test out characters	Plays the attacking animation of the character, doesn't allow other animations to start while jumping.	As, expected.

Are recovery frames working, do they stop spamming attacks?	Repeated "r"	Stops users from spamming attacks.	The player should not be able to instantly start another attack after the animation has ended.	As, expected.
Whether I can move while jumping	"w" and "a" or "d"	Allows users to move while jumping. Some stage may have different platforms to jump onto.	The jumping animation should still play but I should be able to move left and right while jumping.	As, expected.
Are all buttons, loading bars, characters, platforms, backgrounds scaled correctly no matter the screen resolution?	Change program data width and height to multiply by 1.2	Different players may have different monitor sizes and preferences so it's important that assets don't look distorted.	All assets should be as proportionate to screen size as the default in development which is 1280x720.	As, expected.
Does falling out of the screen respawn the player?	"A" until player falls	The practise arena should allow users to test characters with no deaths.	Player should respawn	As, expected.

Local multiplayer mode

Do all the widgets show up as intended, all aligned with each other?	Mouse click on settings button.	Allows the user to see the settings menu clearly and systematically with player 1 on left and player 2 controls on the right.	All widgets should be drawn to the screen with player 1 controls on the left and 2 on the right.	As, expected.
Does hovering over a widget change the colour of the action text?	Mouse hover over the widget	The user should be able to see what widget is currently being selected?	When the mouse hovers over a widget, the turn orange.	As, expected.
Does clicking an action lock the user	Mouse click on the widget	This will not allow the user to leave	User shouldn't be able to	As, expected.

into selecting a key bind for that action?		actions unbinded, so all movements/attacks can be performed in game.	change another key bind.	
Does clicking the action change the key to "?" and orange to indicate to the user, which key bind is currently being changed?	Mouse click on the widget	Allows the user to see what action is currently being binded so they can press their preferred key.	Key should turn into an orange "?", until a key is pressed.	As, expected.
Does the character respond to the key presses specified by the settings menu.	"C", "V", "U", "Z", "X"	Allows user to change to more comfortable key binds.	The character should respond appropriately to each key press	As, expected.
Do both characters show up on screen?	Mouse press on the local button.	This is essential to having a functional basic 2D fighter game.	Both players should show on the screen in their appropriate spawn points.	As, expected.
Can both characters be controlled by the key binds set in the settings?	Mouse press and key presses to change the key binds in the settings. Then using those keys.	Allows user to change to more comfortable key binds in settings.	Both characters should respond to the key binds set in the settings menu.	As, expected.
Do deaths by fall work, do they trigger a new round?	"a" until player falls off. Also, right arrow key for the second player.	This makes falling responsive and more natural. Without this, the surviving player would be stuck. This should also be a real round and unlike the practise arena where you can't die.	Falling off the platform should trigger the start of a new round with both characters resetting back to their spawn points.	As, expected.

Does falling by death twice count as a win for the other player?	"a" until player falls off. Also, right arrow key for the second player.	This makes falling responsive and more natural. Without this, the surviving player would be stuck. This should also be a real round and unlike the practise arena where you can't die.	Output the appropriate message in the console.	As, expected.
Does repeatedly attacking a character until their health reaches 0, restart the round?	"p", "l", "m", "n"	This allows players to actually attack opponents and win the game.	After attacking the opponent 4 times, a new round should start.	As, expected.
Is the stage selection menu drawn to the screen when practise or local is selected?	Mouse press on practise and local button.	To have an interactive and working menu and allow the user to select a stage before playing.	The stage selection menu should be drawn with the overlay and volcano valley background and text.	As, expected.
Does pressing a stage load up the correct stage in practise?	Mouse press on volcano valley.	To have an interactive and working menu and allow the user to select a stage before playing.	Practise should be loaded up exactly as before with the stage being shown.	As, expected.
Does pressing a stage load up the correct stage in local mode?	Mouse press on volcano valley.	To have an interactive and working menu and allow the user to select a stage before playing.	Practise should be loaded up exactly as before with the stage being shown.	As, expected.
Is the character selection menu drawn as described in the code when pressing	Mouse press on local and practise buttons.	Allows the user to select a character to play the game.	Character selection menu with icons and previews should be drawn.	As, expected.

local and practise?				
Do characters show up as previews if the mouse is hovering over them?	Mouse hover over character icons.	Allows the user to see what the character looks like before selecting it.	Previews should be drawn on the left hand side for player 1 and the right for player 2.	As, expected.
Are the choices confirmed once icons are pressed?	Mouse press on character icons.	This allows the game to start, once the characters have been selected.	Practise mode should play, or local mode if local button is pressed previously.	As, expected.
Are icons and previews in the right position and size?	Mouse press local or practise buttons.	Allows the user to see the previews and icons and select them.	All icons and previews should be appropriate size, which I will judge.	As, expected.
Does the characters button change scenes to the character selection menu?	Mouse press on the characters button.	Allows users to test out all characters in practise mode		As, expected.
Are health bars drawn to the screen in their appropriate positions?	Mouse press on local mode button.	This will allow the players to see how much health they have left easily.	Health bars should be 5 th along the screen from either side and at the top of the screen.	As, expected.
Are the colours suitable?	Mouse press on local mode button.	Colours need to be visually pleasing.	A bright red and blue for the health bars.	As, expected.
Does the health decrease once a player is hit?	"p" or "l" once close the opponent	Attacks need to be responsive and show up on the hud, so players can see how much health they have left at any point in the round.	The bright red or blue needs to get smaller to show the darker colour underneath.	As, expected.
Does light attack lower the health by 8?	"p"	Attacks need to deduct health, to create a good	On the hud, the health should be lowered.	As, expected.

		2D fighter game.		
Does heavy attack lower health by 15?	"I"	Attacks need to deduct health, to create a good 2D fighter game.	On the hud, the health should be lowered.	As, expected.
Does the time show on the screen, accurately.	Mouse press on local mode button.	Drawing the time will allow players to see how much time is left in each round.	Time should be shown in the mid top of the screen, counting up in seconds.	As, expected.
Does the round end appropriately once the stopwatch is past 90?	Waiting a round in the game.	This will allow rounds to not go on forever and end once the timer is up.	Player 1/2 should be awarded the win, if end with higher health. If they are equal, then the game should go on.	As, expected.

LAN mode

Can a host join a game and wait with no errors?	Mouse click on host button	The code should be able to work with only one player connecting to the server so a waiting screen can be implemented in a similar way.	Once host is clicked, one player should be drawn onto a stage, without errors until the second player joins.	As, expected.
Does clicking the joining button while a host is active join the game	Mouse click on host	Clicking the joining button should allow you the play with another person on a LAN. It's a key	Both players should be drawn the same on both screens, in the same positions.	As, expected.

and draw the oponents?		feature of the game.		
Does moving the host move the host on player 2's screen?	"a", "d", "w"	Player movements should be synced across different clients.	Moving either player, should move the other on the opponent's screen.	As, expected.
Does moving player 2 move the player 2 on the host's screen?	"a", "d", "w"	Player movements should be synced across different clients.	Moving either player, should move the other on the opponent's screen.	As, expected.
Is the HUD synced across both players, showing the correct time?	One client clicks host while the other clicks join.	The HUD should be synced to show correct time info and not have any bugs with when a game ends.	The game should start the timer when player 2 joins and should be the same on both clients.	As, expected.
Does falling twice on a client give the win to the other client?	"a" or "d" until fall after clicking on host and join on clients.	This will simulate a player being able to win and test out the game logic functions to see if they run as expected on LAN and sync up well.	Console should output the other player winning.	As, expected.
Does players attacking each other update the HUD health.	"P" and "L"	Players' attacks need to be registered accurately for a well developed 2d fighting game.	The HUD health should be lowered.	As, expected.
Does a round end if the health goes to 0?	"P" or "L"	A round needs to end at the when a player's health goes to 0 so that the game is fair.	Once a player's health goes to 0 from attacks, a new round should be started.	As, expected.
Does clicking the host button take the user to	Mouse click on host button	The waiting scene needs to work properly so	Console should output the other	As, expected.

the waiting scene?		that hosts can wait for a player to join them before starting the game.	player winning.	
Once a player joins, does the waiting scene print character selection menu?	Mouse click on join button from another instance of the game	To allow the players to play together over a LAN connection, the waiting scene will need to switch scenes to show the game. I will write the code to switch scenes, once I have confirmed that the if statement works correctly.	Console should output "CHARACTER SELECTION MENU".	As, expected.
Does clicking the join button load the character select scene?	Mouse click on join button	This will allow players to join their friends' games and play together.	The character selection menu should be shown once the join button is clicked.	As, expected.
Does the waiting scene switch to the character selection menu if player 2 joins?	Mouse click on join button from another instance of the game.	To allow the players to play together over a LAN connection, the waiting scene will need to switch scenes to show the character selection scene. This will allow players to select their characters before starting the game.	The character selection menu should be shown once player 2 joins.	As, expected.
Is the opponent character	Mouse click on a character in the character	This will make it clear to the players, which character they	The opponent character's must be less	As, expected.

preview less visible?	selection menu.	are going to play as. So that they can choose the correct character.	visible in selection and confirmation in the character selection menu.	
Does moving the host move the host on player 2's screen?	"a", "d", "w"	Player movements should be synced across different clients.	Moving either player, should move the other on the opponent's screen.	As, expected.
Does moving player 2 move the player 2 on the host's screen?	"a", "d", "w"	Player movements should be synced across different clients.	Moving either player, should move the other on the opponent's screen.	As, expected.
Is the HUD synced across both players, showing the correct time?	One client clicks host while the other clicks join.	The HUD should be synced to show correct time info and not have any bugs with when a game ends.	The game should start the timer when player 2 joins and should be the same on both clients.	As, expected.
Does the joining player return to the play menu if the host leaves in the character select menu.	Close window on host	Success criteria #13	Joining player return to the play menu.	As, expected.
Does the joining player return to the play menu if the host leaves in game.	Close window on host	Success criteria #13	Joining player return to the play menu.	As, expected.
Does the host return to the play menu if the joining player leaves in game.	Close window on joining player	Success criteria #13	Host return to the play menu.	As, expected.

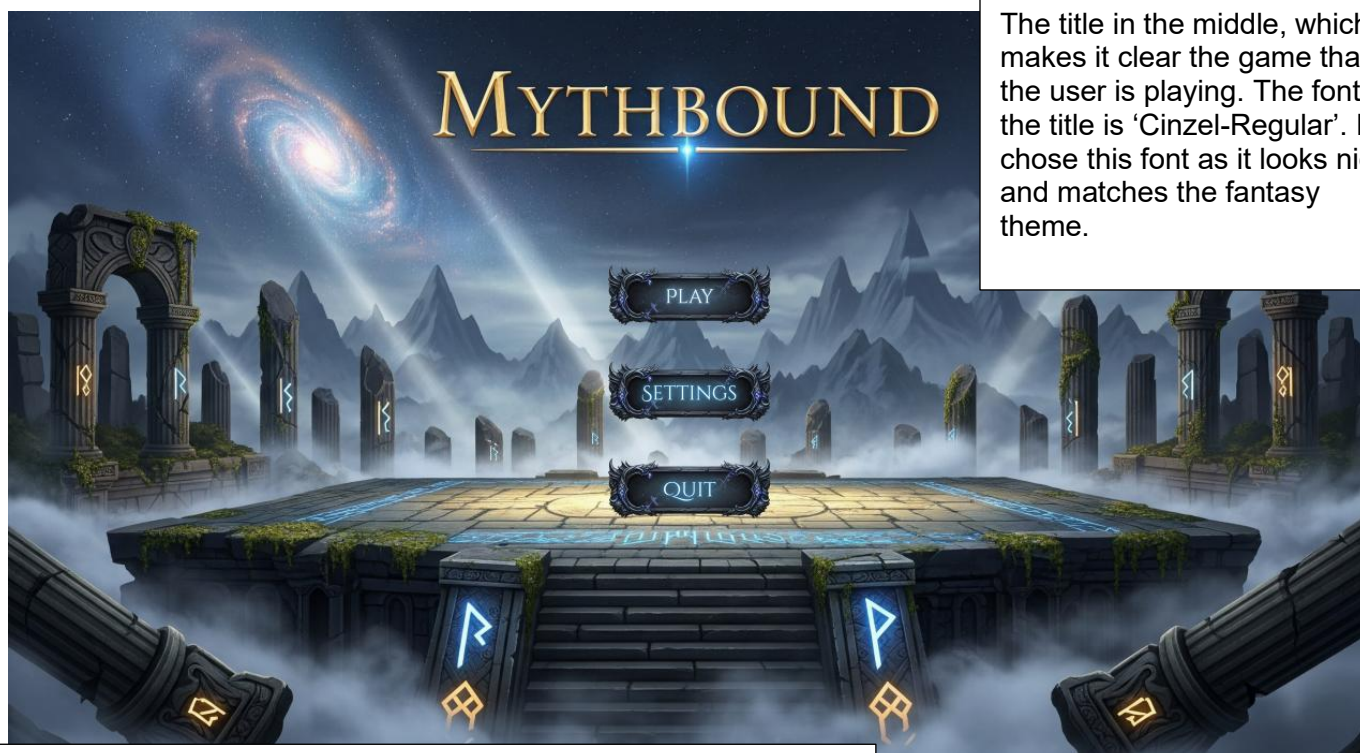
Does the host return to the play menu if the joining player leaves the character selection menu.	Close window on joining player	Success criteria #13	Host return to the play menu.	As, expected.
Does the joining player return to the play menu if the there is no host?	Mouse click on join	Success criteria #13	Joining player return to the play menu.	As, expected.
Does the stage and socket address of the host game get printed out in the joining player's console?	Mouse click on join after hosting.	Success criteria #12	The stage and socket address to be printed to the console	As, expected.
Does the stage and socket address of the host game appear as a widget in the menu.	Mouse click on join after hosting.	Success criteria #12	The stage name + port to be the text in the widget.	As, expected.
Does the character selection menu load once I click on a game?	Mouse click on the host's game.	Success criteria #12	The character selection scene should be loaded with the host.	As, expected.

Evidence

The evidence for all of these tests are combined into a single video which showcases all of the features developed in the game, titled 'mythbound_main_test'.

Usability features in the game

Below are screenshots of the usability features that I have implemented into my game:

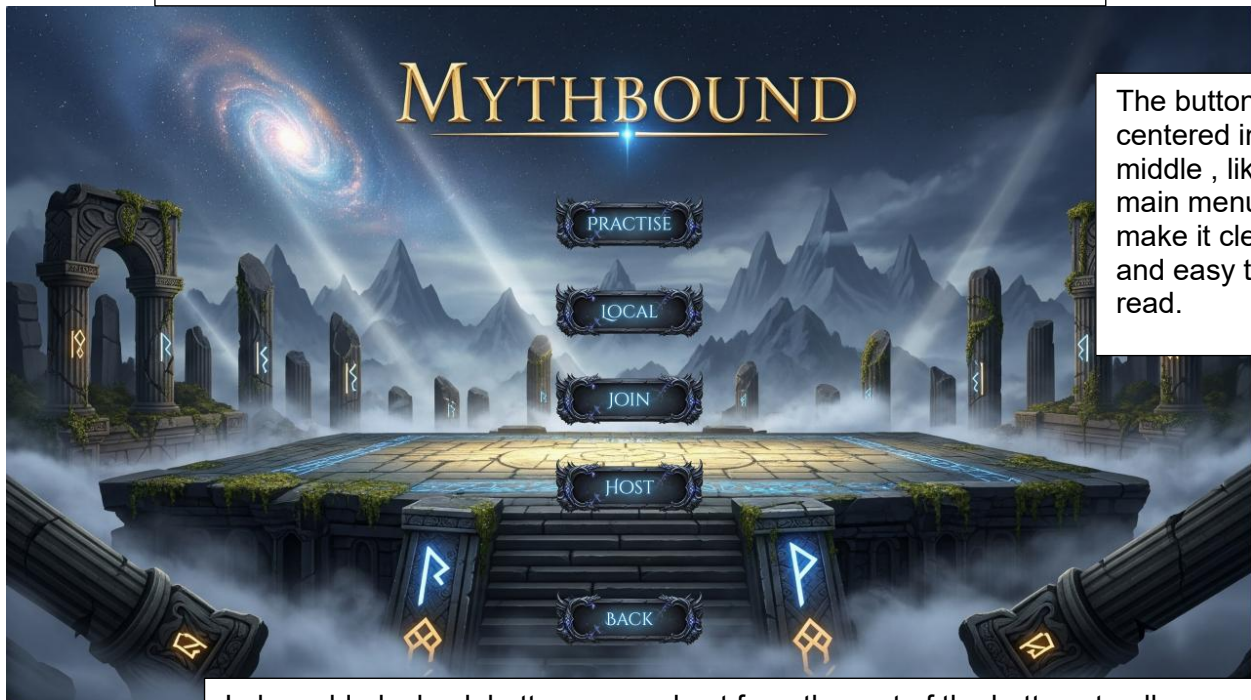


The title in the middle, which makes it clear the game that the user is playing. The font of the title is 'Cinzel-Regular'. I chose this font as it looks nice and matches the fantasy theme.

The menu has the buttons in the middle which makes it very clear how to navigate the game. The same button design is also consistently used throughout the game and all menus, to make the game have a professional look. The theme of the buttons also matches the fantasy-like theme of the game.

Rather than bombarding the player with the other different play options here like local and practise, I have split that into its own menu. This makes the game feel better organised and sectioned out, and easier for new gamers to take in.

Unlike the usability features, the title is still in the play menu. Due to constraints with AI tokens, I couldn't prompt imagen to remove the title. I'm also not very skilled with photopea, so I wasn't able to remove it myself. I don't think it looks bad.

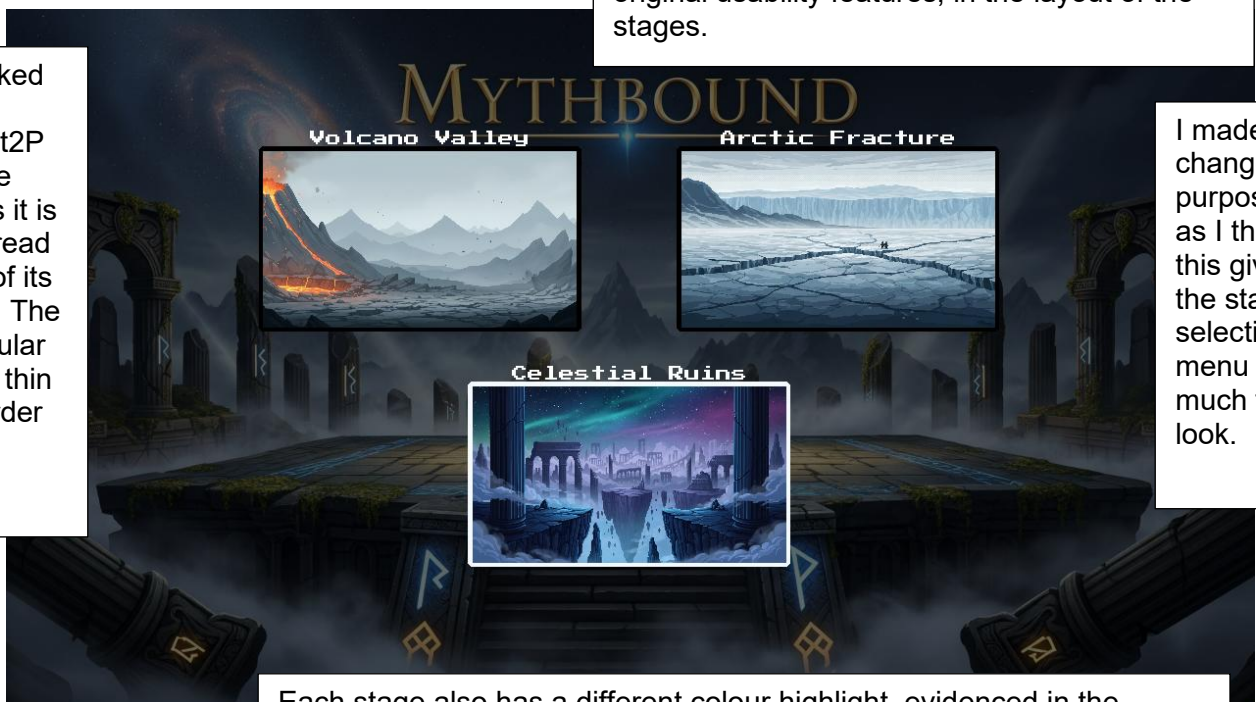


The buttons are centered in the middle, like the main menu, to make it clear and easy to read.

I also added a back button, spaced out from the rest of the buttons to allow the user to go back to the start menu. This will allow users to change controls if they forgot to, or quit the game.

The stage selection menu looks different to the original usability features, in the layout of the stages.

I have picked the PressStart2P font for the stages, as it is easier to read because of its thickness. The cinzel regular font looks thin and is harder to read.



I made this change on purpose, as I think this gives the stage selection menu a much fuller look.

Each stage also has a different colour highlight, evidenced in the videos.

The character icons are more spread out, as its easier to place them on quarters of the horizontal width of the screen. I think how I placed them in the usability features was better, as the spacing looks weird now.

I have removed the confirm buttons to have a simpler menu. Now, clicking acts as a confirm while hovering is a preview. I could add a future update where there is an option in settings that toggles between these two selection methods.

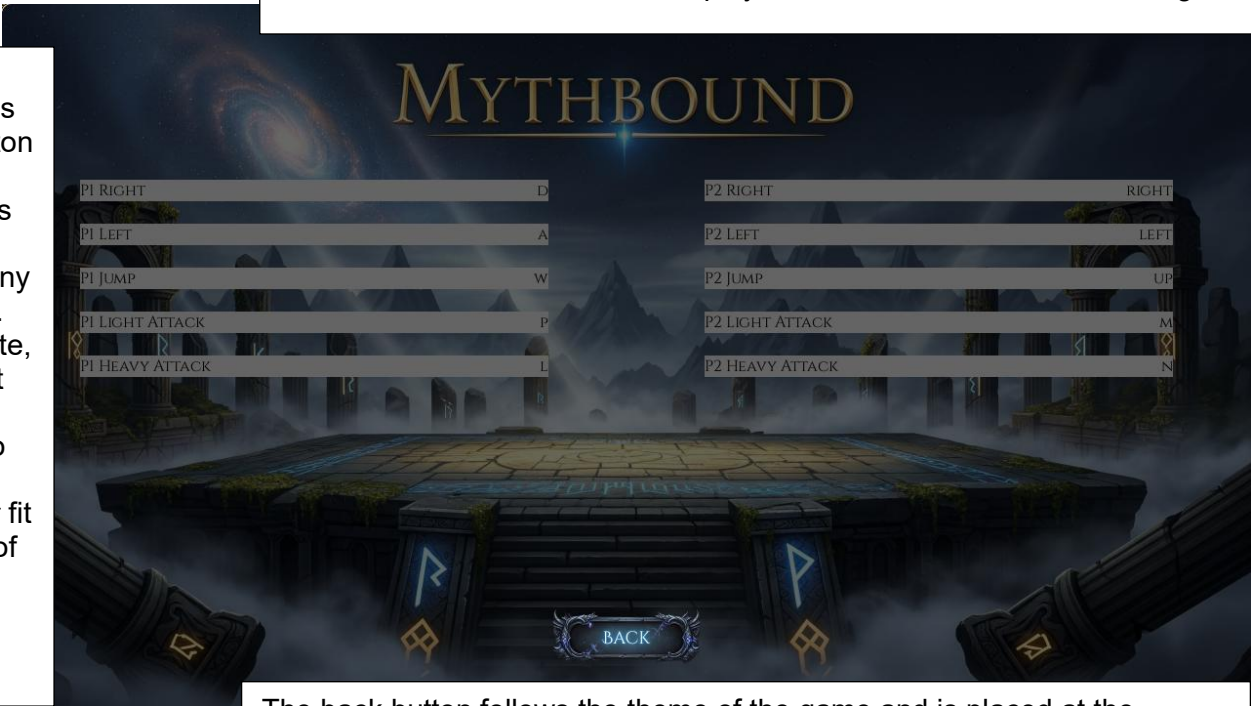
I can add more characters in future updates and remove the big gaps between icons.



Only one character is displayed now, the videos show local mode and LAN.

The player 1 and 2 controls are split vertically, like the usability features. This distinction makes it clear to players which set of controls to change.

The setting menu design is unlike the button and theme of the game. This is because I couldn't find any similar assets. In future update, I could prompt imagen to customize it to make the widgets better fit the aesthetic of the game.



The back button follows the theme of the game and is placed at the bottom of the screen so I can add more settings to toggle in future updates, like the character selection methods. It also makes it easy to see and click once changes have been made to the controls.

I do think that the score should also be placed in the hud, but I didn't get to add that feature, due to time constraints. It can easily be added in future updates.

Across all of the stages, the main platform is placed around the middle of the screen. I have also customized the platform's hitboxes for the characters to stand in a more realistic way on top of it.

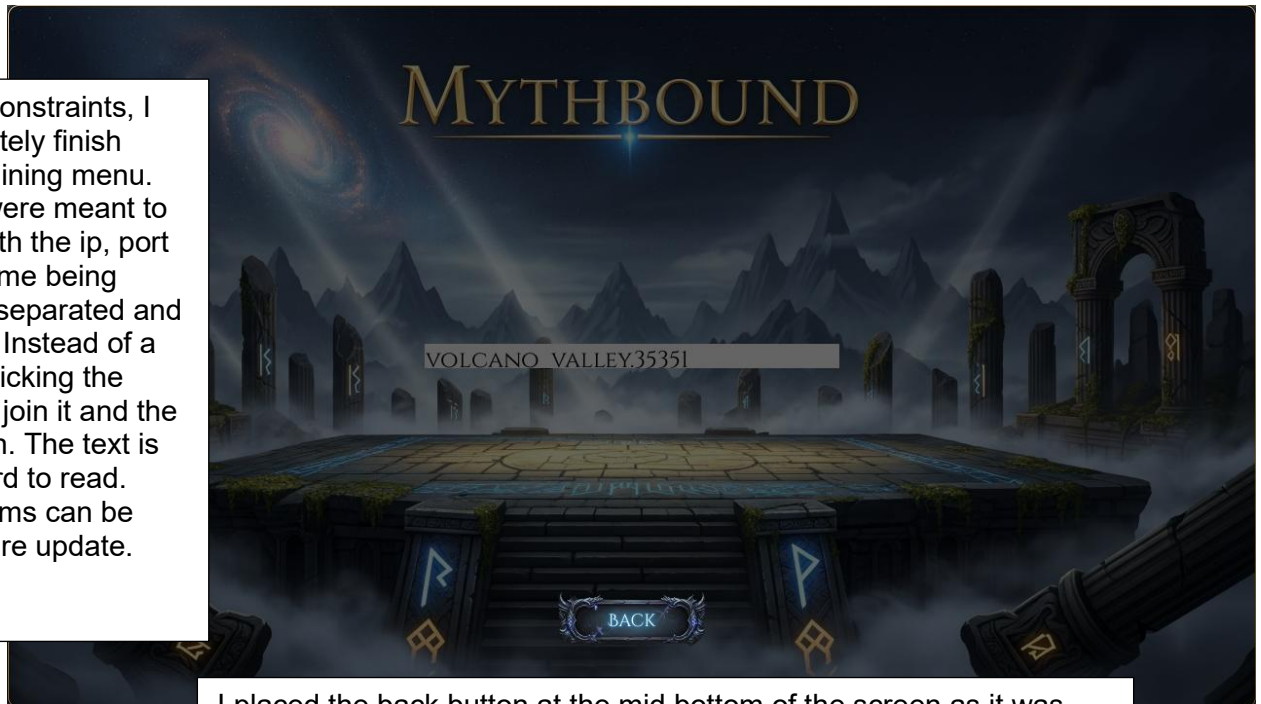


Although the waiting screen runs for 3 seconds, incase a player wants to leave quickly, a back button has been placed which takes the players to the play menu.

The text is near the centre of the screen to make it easy for the players to see who won, along with the character of the winning player being in mid bottom of the screen.



Due to time constraints, I didn't completely finish making the joining menu. The games were meant to be laid out with the ip, port and stage name being more clearly separated and a join button. Instead of a join button, clicking the game let you join it and the IP isn't shown. The text is also quite hard to read. These problems can be fixed in a future update.



I placed the back button at the mid bottom of the screen as it was easier to do in the time I had, using the screen size attributes.

Final interview with the stakeholders

I will now conduct an interview with the stakeholders to find out whether my solution was successful. Prior to the interview, stakeholders will also test the game out so that they can better respond to questions asked in the interview.

The questions and responses are presented below:

- 1) How responsive were the character movement and jump mechanics?

Jamie: The gravity simulation feels realistic; the character speeds up as they fall rather than floating.

Sarah: The animations look great. I liked that the running animation plays smoothly when I hold 'A' or 'D', making the movement feel fluid.

Alex: Controls are responsive. I tried moving while jumping to see if I could change direction mid-air, and it worked exactly as I expected.

- 2) What was your experience using the attacks (Light/Heavy)?

Jamie: The hit detection is accurate. I noticed the health bar drops by different amounts depending on if I used 'P' (light) or 'L' (heavy).

Sarah: The default keys (P and L) felt a bit far apart from the movement keys, which was uncomfortable. I had to stretch my hand a bit.

Alex: The attacks work well, but having P and L so close to the second player's controls (M and N) caused us to accidentally hit each other's hands on one keyboard.

3) Did the settings menu solve any control issues?

Jamie: Yes, the menu is clear. I tested rebinding the keys to see if the game remembered them, and it worked perfectly in the next round.

Sarah: Since I didn't like the default P/L keys, being able to change them in the settings was a lifesaver. The '?' indicator made it easy to know when to press a new key.

Alex: It works, though it wasn't immediately obvious which column belonged to Player 1 and which to Player 2 until I tested it.

4) How easy was it to use the character selection menu?

Jamie: Very intuitive. Hovering over the icons updates the large preview instantly, so I knew exactly who I was picking.

Sarah: The layout is nice. I tried clicking a character to confirm, and the game immediately loaded the match, which felt snappy.

Alex: I like that Player 1 is on the left and Player 2 is on the right. It makes it clear who is picking which character.

5) Did the stage selection menu function correctly?

Jamie: Yes, I selected 'Volcano Valley' and it loaded the correct assets and platforms. The preview images are a good size.

Sarah: The layout is visually pleasing. Using the triangle layout for the stages makes them look much better than a standard list.

Alex: I tested loading different stages in both Practise and Local modes, and the correct background loaded every time without errors.

6) Is the HUD clear and helpful during gameplay?

Jamie: The bright red and blue bars stand out well against the background. It's easy to see when I'm about to lose.

Sarah: The timer is in a good spot. I let the time run out to see what would happen, and the round ended correctly at 90 seconds.

Alex: The health updates instantly when I get hit. I like that the colored bar shrinks to reveal the darker color underneath.

7) How was the local multiplayer mode?

Jamie: Both characters spawned in the right spots. We played a full 'best of 3' set and the round resets worked perfectly.

Sarah: It's really fun playing on the same screen. The game didn't lag even with both of us spamming attacks at the same time.

Alex: The win condition works. When I knocked Jamie off the stage twice, the console correctly output that I had won the game.

8) How does the game handle errors or disconnections?

Jamie: There were no errors that I noticed from my testing.

Sarah: It feels stable. I didn't experience any crashes even when switching rapidly between menus.

Alex: I tried falling off the map repeatedly to see if I could break the respawn logic, but it just reset me every time.

9) Was the practise arena useful for learning the game?

Jamie: It's good for testing controls. Since there's no timer or death limit, I could practice the jump distances on the platforms.

Sarah: It works as intended, but I would love to see an AI opponent in the future. Hitting a stationary dummy gets a bit boring after a while.

Alex: Essential for checking the animations. I used it to see how long the recovery frames were after a heavy attack.

10) Is there anything you would like me to add to the game in a future update?

Jamie: The physics and collisions are solid, but the practice arena is a bit too static. I would really like to see a basic **AI opponent in the practise mode**. Having a computer-controlled character that moves and attacks back would help me test the defensive mechanics and hitboxes much more effectively than a stationary dummy. Sound would also be great, like sound effects from hits and background music.

Sarah: The visuals are lovely, but the game is currently silent. I think you should add background music and distinct impact **sounds to make the game feel more immersive**. Hearing a loud 'thud' when I land a heavy attack versus a quick 'whoosh' for a light attack would give the player much better feedback.

Alex: Since the local multiplayer is the focus, I'd love to see **taunts** added. It adds a fun psychological layer to the match if I can press a key to show off after knocking the other player down. It would make the competitive experience feel more complete.

Evaluating the success criteria

#	Success Criteria	Justification	Met
1	In rounds, the players must be able to control their characters movements using A and D (left and right movement).	This is needed so that users can move their characters. A and D are very common for moving left and right across nearly all games. <i>Stakeholders have also confirmed this in the technical interview.</i>	This success criterion has been met. In the videos you can see the player moving in response to the keys being pressed. Also, the default keys can be changed in the settings menu in case players want to customize their key bindings. Testing for this can be seen in the 'Building the practise arena' section of development.
2	When pressing the W key in a LAN game, the character should start the jump animation and move up and then down. The animation should be playing the entire	Every fighting game has a jump functionality; this will be needed to dodge attacks and traverse the stage as the stages will have different raised platforms. The W key	This success criterion has been met. While writing the code, I aimed to make adding future characters to be as easy as possible. This also led to customizing animations being really easy. Right now, all characters jump animations are

	time and for a total of 1.25 seconds.	is perfect as A and D will be used by the index and ring finger which leaves the middle finger free for the W key. <i>Stakeholders have also confirmed this in the technical interview.</i>	set to 1.25, but these can be easily changed from character to character. I wrote the code this way, in case I wanted to have more realistic physics in the future. I can make some character's weaker/heavier than others and therefore have different jump heights and animation durations. Testing for this can be seen in the 'Building the practise arena' section of development.
3	Each character will have two attacks controlled by P and L. The heavy attack will be L while the light attack will be P.	This allows for simple controls that are also fun to use, having only one attack would be too basic and get repetitive. Players can use these attacks to strategise and execute them at the right times. <i>Stakeholders have also confirmed this in the technical interview.</i>	This success criterion has been met. However, P and L are quite far away from the default movement controls of A and D. They can also hinder the second player's ability to press their attack controls of M and N as P and L are quite close. While playing local mode with a friend, it was quite uncomfortable to reach for the keys. I should probably change the default controls in a future update, however the settings does allow players to change them to any key they like. Testing for this can be seen in the 'Building the practice arena' section of development.
4	There must be a start menu with clickable buttons: 'Play', 'Settings' and 'Quit'.	This will allow the users to navigate between the different menus to play the game. <i>Stakeholders have also confirmed this in the technical interview.</i>	This success criterion has been met. Creating the button class made this criterion easy to meet, as after the initial button class code was written, I simply instantiated a button object for each type of button. By using OOP principles in this way, it will be easy to add buttons in the future in updates. Testing for this can be seen in the 'Constructing the main menu' section of development.
5	There must be a settings menu accessible from the start of the game, that allows the players to change their key binds.	Changing key binds will make players feel more comfortable as they may not like the default keys. <i>Stakeholders have also confirmed this in the technical interview.</i>	This success criterion has been met. The settings scene changes player 1 and 2's controls which are used by the character class to determine what controls the player. Players that are joining LAN games are automatically assigned number 2, so if a player wanted to change their controls in LAN mode, they would need to change player 2's controls and not player 1. This isn't properly shown to the players, but I can add some extra text above each column of controls in the setting to guide the players. Testing for this can be seen in the 'Implementing a local multiplayer mode' section of development.

6	There must be a playing menu that allows users to access and play the different play modes. The buttons should include, "Practise", "2-player", "join" and "host".	This will allow the users to navigate between the different menus to play the game. <i>Stakeholders have also confirmed this in the technical interview.</i>	This success criterion has been met. While making the menu, I didn't have a specialized background image to use. So, I just used the regular main screen one with an overlay applied to reduce the noise. The overlay makes it a little less obvious that I've used the same main menu background. I could use another in the future, but for now it looks good enough. Testing for this can be seen in the 'Constructing the main menu' section of development.
7	The characters must have Animations for all kinds of movement (running, jumping, attacking) that are shown once the character performs them.	This will add to the visual element of the game, making it feel more refined. Having a character that will just seem to stand still while attacking will take away from the user experience. <i>Stakeholders have also confirmed this in the technical interview.</i>	This success criterion has been met. Due to how versatile the animation class, it can hold any animation sequence and can easily be played with a key. This is great for maintenance and updates in the future, as I can add new taunts to be played in game or the ending banner. I would just need to keep the LAN connection running for longer to transfer taunt frame data. Testing for this can be seen in the 'Building the practise arena' section of development.
8	A HUD at the top of the screen must show each player's health (100) and display a timer in the middle which is 90 seconds per round.	Placing the health bars and timer at the top is perfect as it will allow players to see how much health and time, they have left so that they can plan their next move accordingly. <i>Stakeholders have also confirmed this in the technical interview.</i>	This success criterion has been met. I didn't think of adding a way to see the points earned through rounds, however, this feature would be great to add as it will help players keep track of how many rounds they have won/lost. This could be done by passing the wins list as parameter into the draw function and processing this based on the index of the points to draw them correctly to the screen. Testing for this can be seen in the 'Implementing a local multiplayer mode' section of development.
9	Before each round there must be a character selection menu for the players to select their characters.	Necessary so that the players can select a character that they wish to play.	This success criterion has been met. While creating the character selection menu, I thought to remove the confirm buttons to make it simple for both the player and me to write the code for. Now, hovering over a character triggers a preview and clicking a character confirms it. Which method is better is subjective, so I could add a toggle in the settings menu to switch between these two methods of selection. Testing for this can be seen in the 'Implementing a local multiplayer mode' section of development.
10	The game must also include a practise arena which allows players to test out	This is a great feature suggested by a stakeholder that will allow players to learn	This success criterion has been met. I can make the practise arena better by adding an option to train with AI enemies. Coding an AI enemy could

	characters without any opponents or timer.	attacks and try out different characters.	be hard to do but I can handle the movement with an A* search algorithm and the attacks to trigger whenever the AI is close enough. This feature will be good for players that may not have a person to play the game with, or just players that want to get better at the game in a more interactive way. I could also add a tutorial section to teach players how to play the game. This may seem unnecessary but could be very helpful for new gamers. Testing for this can be seen in the 'Building the practise arena' section of development.
11	There must be a stage selection menu for compatible modes like practise, host and local.	This feature will allow users to pick and play on the stages they want. It will make the game more fun and engaging. All solutions researched had this feature. <i>Stakeholders have also confirmed that only hosts, in online games, should be able to pick the stage in the interview.</i>	This success criterion has been met. The stage selection menu differs slightly from the original drawn in the usability features. The difference is how the different stages are laid out. I think the new menu looks better as the actual stages take up more of the screen. Instead of them being squashed together at the top, they are laid out like a triangle which allows the icons to be bigger and clearer for the players to see the stage previews. Testing for this can be seen in the 'Implementing a local multiplayer mode' section of development.
12	The game must support LAN multiplayer, allowing users to connect and play with each other from the same network.	Part of the essential features, to allow players to play with their friends on a local network.	This success criterion has been met. As I couldn't test LAN on two separate devices at once, I had to use the localhost ip to test it out. That means it was using my computer's network to send the game data. Theoretically, it since worked on localhost, there's no reason why it wouldn't work on the actual wifi. A possible limitation would be that the user needs to allow python to have access to the network through their firewall (if the user has one installed). I could explain this in a tutorial section of the game, that I could add in a future update. Testing for this can be seen in the 'Implementing LAN mode' section of development.
13	In the case of network failures, the game must not crash and instead handle it gracefully by returning to the main menu.	This is good programming practise and will improve the user experience.	This criterion has been met. By logically placing python try catch statements, I've ensured that any errors happening while connecting will be caught and cause the scene to switch to the joining menu. I have tested all types of errors where each player leaves in turn during each scene that has an active connection. In all cases, the error was caught, and the scene was changed to the joining menu.

			Testing for this can be seen in the 'Implementing LAN mode' section of development.
14	2-player on the same device must also be supported. This should be accessible in the play menu.	This provides a fun experience for players and allows them to play with their friends on the same keyboard if they don't have a Wi-Fi connection.	This criterion has been met; I have made the controls changeable for each player (1 and 2) in the settings. The video testing shows two players being controlled by me. I have also tested the running out of time, and a player dying all ends a round. The ending of a round can be seen in the video, when both players teleport to the starting point and their health gets reset. Testing for this can be seen in the 'Implementing a local multiplayer mode' section of development.
15	At the end of each game, there has to be an ending banner that displays who won, for 3 seconds.	Instead of abruptly ending the game, I can display who won the game for 3 seconds. 3 seconds is not too long to feel annoying, but not too short that you can't see who won clearly.	This criterion has been met. I can improve on this, by having the option for characters to dance/taunt. This could be hard to do as I would have to draw these taunts for each character myself or use a program that separates joints which allows me to easily animate each character myself. I can then add the animations to the character classes and map them to a key in the ending banner. Testing for this can be seen in the 'Implementing LAN mode' section of development.

Maintenance

There are a lot of features that I could add to the game in future updates. I have combined these into a table below:

Feature	Justification	How can I implement this?
A toggle in the settings that can allow you to select characters with a confirmation button and mouse presses for triggering previews.	This will allow players to select their own preferred way of choosing characters in the character selection menu. Having a button to press can make it clearer how to select a character.	I can link the toggle to a boolean which will be used by the character icons to determine whether or not to show the confirmation buttons and what they return in their update functions based on if those buttons are clicked.
Improve the joining menu with a scroll bar and a join button.	Right now, the joining menu can't support many games being displayed, as they may start to appear off screen. Having a scroll bar will fix that issue as the user can move the off screen games into view easily. A join button will also make the joining process clearer, as the players will now select a game by clicking on it and join it by clicking the button.	By introducing a list of games, I can use a bar's position on screen to move all the games up or down appropriately. The bar's position can also be limited to not move off screen.

An AI mode in the practise arena	Fighting an AI enemy is more interactive than testing characters alone in the practise arena. The enemy can simulate a real player and help a person improve more than if they were to play alone. It can also be played by players that may not have a person to play the game with.	I can use the A* search algorithm to control the enemy's movement and attacks that will trigger when the AI is close enough to a player.
Addition of more characters and stages.	Now, the game only has 3 characters. Adding more characters will give players more options to choose from. It can also make the game more attractive to play as the wide range of characters guarantees that a player will find their match.	Using itch.io, I can find more characters to add to the game. Each character will need to have an idle, jump, run and two attacks. I can also use AI to create a character icon based off the idle animation to use in the character selection menu.
Remove the large spacing between character icons in the character selection menu.	The character selection menu currently looks weird because of the spacing between icons. Also, the way I'm placing the icons isn't compatible with adding more characters. I will need to switch to less spacing to fit more character icons on the screen. Less spacing will also help players compare characters faster because of the less distance that the mouse will have to move to see the previews.	By using the screen size attribute, I can make the icon placing compatible and proportionally look the same on all screen sizes. I could also use a fraction of the icon size inbetween the icons. This can all be automated with a for loop that will determine the position based on the index of the icon in the icons list.
Taunts/dances in the ending banner	The ending scene looks very static now, adding taunts/dances to characters will make the ending scene look less boring and more interactive.	I can use a program called smack studio to seperate joints of each character and animate the characters myself for taunts. I can then add the animations to the character classes and bind them to a key in the ending banner. This could take a long time as I don't have much art experience.

Limitations

Limitation	Why is this a limitation?	How can this be fixed?
The game can't be ran as an exe file	This is a limitation because not all players know how to run a python file, some gamers just want to start a game and play.	I can use a 3rd party compiler to compile all of my code into a single executable file. This will make running the game a smooth and easy process for players.
The LAN functionality hasn't been fully tested	As I wasn't able to test out the game on two devices, I don't know how exactly the game will run on an actual LAN, so this feature may need more tweaks as it may not work.	I can invite other people to test the game and see what they think. I could also then gather their feedback to fix the game. This could also be implemented as a button to report errors in game. The button to report

		errors could be linked to microsoft form to fill out with error details.
Firewall blocking ports.	Initially, LAN mode may not work at all due to firewalls on player's computers blocking ports by default. This can be a problem as it will stop the python program from binding any sockets. Also, port 9999 will also likely be blocked from using UDP so discovering games with UDP won't work as I coded it to use port 9999.	I will need to use a python library that can prompt the user to access to certain ports or stop the firewall from blocking them. Of course, I won't be trying to get unauthorised access to computers which is why the library should just prompt the user with an explanation as to why the game needs certain ports.
The game doesn't run on mobile phones.	Most fighter games nowadays can be played on phones as the player base has the potential to grow more than on PC. This is naturally due to more people having phones than laptops/PCs.	I can port my game over to IOS or android by remaking my game in kotlin/swift. I will also need to ensure that the same assets are used so it does not look different to how the game looks on PC.
Networking	Currently, my LAN implementation is based on the user getting data about the opponent from the server in every frame. It is susceptible to lag in the future if a network is busy or the latency is too high. Rather, I can implement client-side prediction; this will predict the movements of the opponent and only if wrong will the server send a response back to rewind time. Also, if I want to include more players in the future, I will need to implement this, otherwise waiting for more than one user to send their data to be processed by the server will surely slow the game down.	I can set up a temporary buffer to store data about a few past frames. That way, in case the prediction of the user's computer is wrong, it will roll back to the necessary frame and continue from there with data from the server.
Hard-coded character logic	Currently, the process of creating a new character for users to play involves making a new class and manually adjusting a function the sets up the animation times. This increases the chances of breaking something in the code due to a mis format etc.	I can use an external JSON or XML file, as its contents can easily be edited

Summary

The main aim of 'Mythbound' was to create a easy to learn, accessible 2D fighter for low-end hardware. Based on the evaluation of the success criteria, I have fully implemented 14 of the 15 objectives. The game provides a function multiplayer experience which fulfils the needs of my stakeholders, being a suitable free alternative for more expensive titles.

Criteria #12 was particularly difficult to write the code for, as I had to combine my knowledge of concurrency and networking. Another unexpected use of concurrency was the asset loader which proved to be a very good solution for the game being unresponsive while loading assets. I separated my game into more classes than expected during development to make future maintenance easier, although it was also to not repeat code unnecessarily.

Final stakeholder testing was overwhelmingly positive; however, the main criticisms were a lack of audio, AI opponents and taunts. Sound was a big feature that I didn't include in my game due to a lack of time and resources as most free assets don't come with audio. All of which can be added in a future update, if I spend more time on creating custom sound effects.

Bibliography

Brawlhalla Wiki (n.d.) *Movement Mechanics*. Available at:
<https://brawlhalla.fandom.com/wiki/Movement>.

Docs.google.com (n.d.) *Fighting Game Frame Data Comparison Sheet*. Available at:
https://docs.google.com/spreadsheets/d/1pnXI_TvDJPeU0ayFbJzvYf7AmrkUcBOUuDegJ5kP7zk/edit?usp=sharing.

Ultimate Frame Data (n.d.) *Super Smash Bros. Ultimate - Character Frame Data*. Available at:
<https://ultimateframedata.com>.

Video Standards Council (VSC) Rating Board (n.d.) *What do the labels mean?*. Available at:
<https://gamesratingauthority.org.uk/RatingBoard/ratings-system>.

VGSales Wiki (n.d.) *Super Smash Bros. Series Sales Figures*. Available at:
https://vg-sales.fandom.com/wiki/Super_Smash_Bros.

Wikipedia (2024) *Brawlhalla*. Available at: <https://en.wikipedia.org/wiki/Brawlhalla>